



FOOL 2012 : 19th International Workshop on
Foundations of Object-Oriented Languages

John Boyland, editor

October 22, 2012

October 2, 2012

Preface

This report contains the papers to be presented at FOOL 2012: 19th International Workshop on Foundations of Object-Oriented Languages held on October 22nd, 2012 in Tucson, Arizona, USA.

The search for sound principles for object-oriented languages has given rise to much work during the past two decades, leading to a better understanding of the key concepts of object-oriented languages and to important developments in type theory, semantics, program verification, and program development. FOOL 2012 will be held in Tucson, Arizona, USA on Monday, 22 October 2012 during the workshop days at the beginning of SPLASH.

There were 16 submissions. Each submission was reviewed by at least 3 programme committee members. The committee decided to accept 10 papers. I thank the providers of EasyChair which made chairing the workshop easier. Thanks also to the programme committee members who put in a lot of work in a very tight reviewing schedule, and for the productive round-the-globe email discussion.

John Boyland
University of Wisconsin-Milwaukee

October 1, 2012

Organization

Program Committee

Jonathan Aldrich, Carnegie Mellon University, PA, USA
John Boyland, University of Wisconsin-Milwaukee, USA, **chair**
Nicholas Cameron, Mozilla Corporation
Dave Clarke, Katholieke Universiteit Leuven, Belgium
William R. Cook, University of Texas at Austin, USA
Stephen Freund, Williams College, MA, USA
Ronald Garcia, University of British Columbia, Vancouver, Canada
Simon Gay, University of Glasgow, UK
Peter Mller, ETH Zurich, Switzerland
Uday S. Reddy, University of Birmingham, UK
Sukyoung Ryu, KAIST, Korea
Marco Servetto, Victoria University of Wellington, New Zealand
Scott F. Smith, The Johns Hopkins University, MD, USA

External Reviewers

Hyunik Na
James Wilcox

Organizers

Jonathan Aldrich
John Boyland
Jeremy Siek

Table of Contents

Session 1. 8:30–10:00

Invited Talk

Dependent Object Types.....	1
<i>Nada Amin, Adriaan Moors, Martin Odersky</i>	

Session 2. 10:30-12:00

A Type System for Dynamic Layer Composition	13
<i>Atsushi Igarashi, Robert Hirschfeld, Hidehiko Masuhara</i>	
A Practical, Typed Variant Object Model.....	25
<i>Pottayil Harisanker Menon, Zachary Palmer, Scott Smith, Alexander Rozen- shteyn</i>	
Semantics and Types for Objects with First-Class Member Names.....	37
<i>Joe Gibbs Politz, Arjun Guha, Shriram Krishnamurthi</i>	

Session 3. 13:30-15:00

Selective Ownership: Combining Object and Type Hierarchies for Flexible Sharing.....	49
<i>Stephanie Balzer, Thomas Gross, Peter Müller</i>	
Sheep Cloning with Ownership Types.....	59
<i>Paley Li, Nicholas Cameron, James Noble</i>	
ParaSail: Pointer-Free Path to Object-Oriented Parallel Programming.....	71
<i>S. Tucker Taft</i>	

Session 4. 15:30-17:00

Inferring AJ Types for Concurrent Libraries	82
<i>Wei Huang, Ana Milanova</i>	
Dataflow and Type-based Formulations for Reference Immutability	89
<i>Ana Milanova, Wei Huang</i>	
SAFE: Formal Specification and Implementation of a Scalable Analysis Framework for ECMAScript.....	96
<i>Hongki Lee, Sooncheol Won, Joonho Jin, Junhee Cho, Sukyoung Ryu</i>	

Dependent Object Types

Towards a foundation for Scala's type system

Nada Amin Adriaan Moors Martin Odersky

EPFL

first.last@epfl.ch

Abstract

We propose a new type-theoretic foundation of Scala and languages like it: the Dependent Object Types (DOT) calculus. DOT models Scala's path-dependent types, abstract type members and its mixture of nominal and structural typing through the use of refinement types. The core formalism makes no attempt to model inheritance and mixin composition. DOT normalizes Scala's type system by unifying the constructs for type members and by providing classical intersection and union types which simplify greatest lower bound and least upper bound computations.

In this paper, we present the DOT calculus, both formally and informally. We also discuss our work-in-progress to prove type-safety of the calculus.

Categories and Subject Descriptors D.3.3 [Language Constructs and Features]: Abstract data types, Classes and objects, polymorphism; D.3.1 [Formal Definitions and Theory]: Syntax, Semantics; F.3.1 [Specifying and Verifying and Reasoning about Programs]; F.3.3 [Studies of Program Constructs]: Object-oriented constructs, type structure; F.3.2 [Semantics or Programming Languages]: Operational semantics

General Terms Languages, Theory, Verification

Keywords calculus, objects, dependent types

1. Introduction

A scalable programming language is one in which the same concepts can describe small as well as large parts. Towards this goal, Scala unifies concepts from object and module systems. An essential ingredient of this unification is objects with type members. Given a stable path to an object, its type members can be accessed as types, called path-dependent types.

This paper presents Dependent Object Types (DOT), a small object calculus with path-dependent types. In addition to path-dependent types, types in DOT are built from refinements, intersections and unions. A refinement extends a type by (re-)declaring members, which can be types, values or methods.

We propose DOT as a new type-theoretic foundation of Scala and languages like it. The properties we are interested in modeling are Scala's path-dependent types and abstract type members, as

well as its mixture of nominal and structural typing through the use of refinement types. Compared to previous approaches [5, 14], we make no attempt to model inheritance or mixin composition. Indeed we will argue that such concepts are better modeled in a different setting.

The DOT calculus does not precisely describe what's currently in Scala. It is more normative than descriptive. The main point of deviation concerns the difference between Scala's compound type formation using **with** and classical type intersection, as it is modeled in the calculus. Scala, and the previous calculi attempting to model it, conflates the concepts of compound types (which inherit the members of several parent types) and mixin composition (which build classes from other classes and traits). At first glance, this offers an economy of concepts. However, it is problematic because mixin composition and intersection types have quite different properties. In the case of several inherited members with the same name, mixin composition has to pick one which overrides the others. It uses for that the concept of linearization of a trait hierarchy. Typically, given two independent traits T_1 and T_2 with a common method m , the mixin composition T_1 **with** T_2 would pick the m in T_2 , whereas the member in T_1 would be available via a super-call. All this makes sense from an implementation standpoint. From a typing standpoint it is more awkward, because it breaks commutativity and with it several monotonicity properties.

In the present calculus, we replace Scala's compound types by classical intersection types, which are commutative. We also complement this by classical union types. Intersections and unions form a lattice wrt subtyping. This addresses another problematic feature of Scala: In Scala's current type system, least upper bounds and greatest lower bounds do not always exist. Here is an example: given two traits A and B, where each declares an abstract upper-bounded type member T,

```
trait A { type T <: A }  
trait B { type T <: B }
```

the greatest lower bound of A and B is approximated by the infinite sequence

```
A with B { type T <: A with B { type T <: A with B {  
  type T <: ...  
}}}
```

The limit of this sequence does not exist as a type in Scala.

This is problematic because greatest lower bounds and least upper bounds play a central role in Scala's type inference. For example, in order to infer the type of an **if** expression such as

```
if (cond) ((a: A) => c: C) else ((b: B) => d: D)
```

type inference tries to compute the greatest lower bound of A and B and the least upper bound of C and D. The absence of universal greatest lower bounds and least upper bounds makes type inference more brittle and more unpredictable.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

FOOL '12 October 22, 2012, Tucson, AZ, USA.

Copyright © 2012 ACM [to be supplied]...\$10.00

Compared to Scala, DOT also simplifies type members. In DOT, a type member is declared by its lower and upper bounds. Here is an example inspired by [11]:

```
trait Food
trait Animal {
  type Meal < Food
  def eat(meal: Meal) {}
}
```

Meal is a type member with a lower bound of $\perp$ and an upper bound of Food. It is possible to instantiate an Animal without further specifying Meal, though it would not be possible to feed it. Now, we define Cow by refining Animal.

```
trait Grass extends Food
trait Cow extends Animal {
  type Meal = Grass
}
```

In Scala, the type alias `type Meal = Grass` defines a concrete binding for Meal. In DOT, such a type alias is declared by giving it identical lower and upper bounds. Now, we can instantiate Cows and feed them Grass.

DOT has a notion of concrete vs abstract type members, which is used to distinguish which types members are instantiable. In the example above, all types introduced by `trait` would be concrete type members in DOT with lower bounds of $\perp$ and upper bounds describing the `extends` clause: $\top$ for Food, a refinement of $\top$ with type member Meal and method member eat for Animal, Food for Grass and a refinement of Animal with type member Meal for Cow. Concrete type members typically have a lower bound of $\perp$ so that they are purely nominal: e.g. one needs to directly or indirectly instantiate a Cow to have an object of type Cow.

We propose DOT as a core calculus for path-dependent types. We present the calculus formally in section 2 and through examples in section 3. In section 4, we show that the calculus does not satisfy the standard theorem of preservation, also known as subject reduction. However, we still believe that the calculus is type-safe, as explained in section 5. In section 6, we discuss choices and variants of the calculus, as well as related work, and conclude in section 7.

2. The DOT Calculus

The DOT calculus is a small system of dependent object-types. Figure 1 gives its syntax, reduction rules, and type assignment rules.

2.1 Notation

We use standard notational conventions for sets. The notation $\overline{X}$ denotes a set of elements X . Given such a set $\overline{X}$ in a typing rule, X_i denotes an arbitrary element of X . We use an abbreviation for preconditions in typing judgements. Given an environment Γ and some predicates P and Q , the condition $\Gamma \vdash P, Q$ is a shorthand for the two conditions $\Gamma \vdash P$ and $\Gamma \vdash Q$.

2.2 Syntax

There are four alphabets: Variable names x, y, z are freely alpha-renamable. They occur as parameters of methods, as binders for objects created by new-expressions, and as self references in refinements. Value labels l denote fields in objects, which are bound to values at run-time. Similarly, method labels m denote methods in objects. Type labels L denote type members of objects. Type labels are further separated into labels for abstract types L_a and labels for classes L_c . It is assumed that in each program every class label L_c is declared at most once.

We assume that the label alphabets l, m and L are finite. This is not a restriction in practice, because one can include in these alphabets every label occurring in a given program.

The terms t in DOT consist of variables x, y, z , field selections $t.l$, method invocations $t.m(t)$ and object creation expressions `val  $y = \text{new } c; t'$` where c is a constructor $T_c \{ \overline{l} = \overline{v} \mid \overline{m}(x) = t' \}$. The latter binds a variable y to a new instance of type T_c with fields $\overline{l}$ initialized to values $\overline{v}$ and methods $\overline{m}$ initialized to methods of one parameter $\overline{x}$ and body $\overline{t}$. The scope of y extends through the term t' .

Two sub-sorts of terms are values and paths. Values v consist of just variables. Paths p consist of just variables and field selections.

The types in DOT are denoted by letters S, T, U, V , or W . They consist of the following:

- Type selections $p.L$, which denote the type member L of path p .
- Refinement types $T \{z \Rightarrow \overline{D}\}$, which refine a type T by a set of declarations D . The variable z refers to the “self”-reference of the type. Declarations can refer to other declarations in the same type by selecting from z .
- Type intersections $T \wedge T'$, which carry the declarations of members present in either T or T' .
- Type unions $T \vee T'$, which carry only the declarations of members present in both T and T' .
- A top type $\top$, which corresponds to an empty object.
- A bottom type $\perp$, which represents a non-terminating computation.

A subset of types T_c are called *concrete types*. These are type selections $p.L_c$ of class labels, the top type $\top$, intersections of concrete types, and refinements $T_c \{z \Rightarrow \overline{D}\}$ of concrete types. Only concrete types are allowed in constructors c .

There are only three forms of declarations in DOT, which are all part of refinement types. A value declaration $l : T$ introduces a field with type T . A method declaration $m : S \rightarrow U$ introduces a method with parameter of type S and result of type U . A type declaration $L : S..U$ introduces a type member L with a lower bound type S and an upper bound type U . There are no type aliases, but a type alias can be simulated by a type declaration $L : T..T$ where the lower bound and the upper bound are the same type T .

Every value, method or type label can be declared only once in a set of declarations $\overline{D}$. A set of declarations can hence be seen as a map from labels to their declarations. Meets $\wedge$ and joins $\vee$ on sets of declarations are defined in Figure 2.

2.3 Reduction rules

Reduction rules $t \mid s \rightarrow t' \mid s'$ in DOT rewrite pairs of terms t and stores s , where stores map variables to constructors. There are three main reduction rules: Rule (MSEL) rewrites a method invocation $y.m_i(v)$ by retrieving the corresponding method definition from the store, and performing a substitution of the argument for the parameter in the body. Rule (SEL) rewrites a field selection $x.l$ by retrieving the corresponding value from the store. Rule (NEW) rewrites an object creation `val  $x = \text{new } c; t$` by placing the binding of variable x to constructor c in the store and continuing with term t . These reduction rules can be applied anywhere in a term where the hole $[]$ of an evaluation context e can be situated.

2.4 Type assignment rules

The last part of Figure 1 presents rules for type assignment.

Rules (SEL) and (MSEL) type field selections and method invocations by means of an auxiliary membership relation $\ni$, which determines whether a given term contains a given declaration as one of its members. The membership relation is defined in Figure 3 and is further explained in section 2.5.

Syntax

x, y, z	Variable	$L ::=$	Type label
l	Value label	L_c	class label
m	Method label	L_a	abstract type label
$v ::=$	Value	$S, T, U, V, W ::=$	Type
x	variable	$p.L$	type selection
$t ::=$	Term	$T \{z \Rightarrow \overline{D}\}$	refinement
v	value	$T \wedge T$	intersection type
$\mathbf{val} \ x = \mathbf{new} \ c; \ t$	new instance	$T \vee T$	union type
$t.l$	field selection	$\top$	top type
$t.m(t)$	method invocation	$\perp$	bottom type
$p ::=$	Path	$S_c, T_c ::=$	Concrete type
x	variable	$p.L_c \mid T_c \{z \Rightarrow \overline{D}\} \mid T_c \wedge T_c \mid \top$	
$p.l$	selection	$D ::=$	Declaration
$c ::= T_c \{\overline{d}\}$	Constructor	$L : S..U$	type declaration
$d ::=$	Initialization	$l : T$	value declaration
$l = v$	field initialization	$m : S \rightarrow U$	method declaration
$m(x) = t$	method initialization		
$s ::= \overline{x} \mapsto c$	Store	$\Gamma ::= \overline{x : T}$	Environment

Reduction

$$t \mid s \rightarrow t' \mid s'$$

$\frac{y \mapsto T_c \{ \overline{l = v' \ m(x) = t} \} \in s}{y.m_i(v) \mid s \rightarrow [v/x_i]t_i \mid s} \quad (\text{MSEL})$	$\mathbf{val} \ x = \mathbf{new} \ c; \ t \mid s \rightarrow t \mid s, x \mapsto c \quad (\text{NEW})$
$\frac{y \mapsto T_c \{ \overline{l = v \ m(x) = t} \} \in s}{y.l_i \mid s \rightarrow v_i \mid s} \quad (\text{SEL})$	$\frac{t \mid s \rightarrow t' \mid s'}{e[t] \mid s \rightarrow e[t'] \mid s'} \quad (\text{CONTEXT})$
where evaluation context $e ::= [] \mid e.m(t) \mid v.m(e) \mid e.l$	

Type Assignment

$$\Gamma \vdash t : T$$

$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \quad (\text{VAR})$	$\frac{\Gamma \vdash t \ni l : T'}{\Gamma \vdash t.l : T'} \quad (\text{SEL})$
$\frac{\Gamma \vdash t \ni m : S \rightarrow T}{\Gamma \vdash t.m(t') : T} \quad (\text{MSEL})$	$\frac{\begin{array}{c} y \notin \text{fn}(T') \\ \Gamma \vdash T_c \mathbf{wfe}, T_c \prec_y \overline{L : S..U}, \overline{D} \\ \Gamma, y : T_c \vdash \overline{S} <: \overline{U}, \overline{d} : \overline{D}, t' : T' \end{array}}{\Gamma \vdash \mathbf{val} \ y = \mathbf{new} \ T_c \{ \overline{d} \}; \ t' : T'} \quad (\text{NEW})$

Declaration Assignment

$$\Gamma \vdash d : D$$

$\frac{\Gamma \vdash v : V', V' <: V}{\Gamma \vdash (l = v) : (l : V)} \quad (\text{VDECL})$	$\frac{\begin{array}{c} \Gamma \vdash S \mathbf{wfe} \\ \Gamma, x : S \vdash t : T', T' <: T \end{array}}{\Gamma \vdash (m(x) = t) : (m : S \rightarrow T)} \quad (\text{MDECL})$
--	---

Figure 1. The DOT Calculus : Syntax, Reduction, Type / Declaration Assignment

$dom(\overline{D} \wedge \overline{D}')$	$=$	$dom(\overline{D}) \cup dom(\overline{D}')$	
$dom(\overline{D} \vee \overline{D}')$	$=$	$dom(\overline{D}) \cap dom(\overline{D}')$	
$(D \wedge D')(L)$	$=$	$L : (S \vee S') .. (U \wedge U')$	if $(L : S .. U) \in \overline{D}$ and $(L : S' .. U') \in \overline{D}'$
	$=$	$D(L)$	if $L \notin dom(\overline{D}')$
	$=$	$D'(L)$	if $L \notin dom(\overline{D})$
$(D \wedge D')(m)$	$=$	$m : (S \vee S') \rightarrow (U \wedge U')$	if $(m : S \rightarrow U) \in \overline{D}$ and $(m : S' \rightarrow U') \in \overline{D}'$
	$=$	$D(m)$	if $m \notin dom(\overline{D}')$
	$=$	$D'(m)$	if $m \notin dom(\overline{D})$
$(D \wedge D')(l)$	$=$	$l : T \wedge T'$	if $(l : T) \in \overline{D}$ and $(l : T') \in \overline{D}'$
	$=$	$D(l)$	if $l \notin dom(\overline{D}')$
	$=$	$D'(l)$	if $l \notin dom(\overline{D})$
$(D \vee D')(L)$	$=$	$L : (S \wedge S') .. (U \vee U')$	if $(L : S .. U) \in \overline{D}$ and $(L : S' .. U') \in \overline{D}'$
$(D \vee D')(m)$	$=$	$m : (S \wedge S') \rightarrow (U \vee U')$	if $(m : S \rightarrow U) \in \overline{D}$ and $(m : S' \rightarrow U') \in \overline{D}'$
$(D \vee D')(l)$	$=$	$l : T \vee T'$	if $(l : T) \in \overline{D}$ and $(l : T') \in \overline{D}'$

Sets of declarations form a lattice with the given meet $\wedge$ and join $\vee$, the empty set of declarations as the top element, and the bottom element $\overline{D}_\perp$. Here $\overline{D}_\perp$ is the set of declarations that contains for every term label l the declaration $l : \perp$, for every type label L the declaration $L : \top .. \perp$ and for every method label m the declaration $m : \top \rightarrow \perp$.

Figure 2. The DOT Calculus : Declaration Lattice

The last rule, (NEW), assigns types to object creation expressions. It is the most complex of DOT's typing rules. To type-check an object creation **val** $y = \mathbf{new} T_c \{ \overline{l} = v \ \overline{m}(x) = t \}$; t' , one verifies first that the type T_c is well-formed (see Figure 5 for a definition of well-formedness). One then determines the set of all declarations that this type carries, using the expansion relation $\prec$ defined in Figure 3. Every type declaration $L : S .. U$ in this set must be realizable, i.e. its lower bound S must be a subtype of its upper bound U . Every field declaration $l : V$ in this set must have a corresponding initializing value of v of type V . These checks are made in an environment which is extended by the binding $y : T_c$. In particular this allows field values that recurse on "self" by referring to the bound variable x . Similarly, every method declaration $m : T \rightarrow W$ must have a corresponding initializing method definition $m(x) = t$. The parameter type T must be **wfe** (well-formed and expanding; see Figure 5), and the body t must type check to W in an environment extended by the bindings $y : T_c$ and $x : T$.

Instead of adding a separate subsumption rule, subtyping is expressed by preconditions in rules (MSEL) and (NEW).

2.5 Membership

Figure 3 presents typing rules for membership and expansion. The membership judgement $\Gamma \vdash t \ni D$ states that in environment Γ a term t has a declaration D as a member. The membership rules rely on expansion. There are two rules, one for paths (PATH- $\ni$) and one for general terms (TERM- $\ni$). For general terms, the "self"-reference of the type must not occur in the resulting declaration D , since, to guarantee syntactic validity, we can only substitute a path for the "self"-reference.

2.6 Expansion

The expansion judgement $\Gamma \vdash T \prec_z \overline{D}$ "flattens" all the declarations of a type: it relates a type T to a set of declarations that describe the type structurally. Expansion is precise and unique, though it doesn't always exist. See section 3.2 for examples.

The expansion relation $\prec$ is needed to type-check the complete set of declarations carried by a concrete type that is used in a **new**-expression. Expansion is also used by the membership rules and in subtyping refinements on the right (see Figure 4).

Rule (RFN- $\prec$) states that a refinement type $T \prec_z \overline{D}$ expands to the conjunction of the expansion $\overline{D}'$ of T and the newly added declarations $\overline{D}$. Rule (TSEL- $\prec$) states that a type selection $p.L$ carries the same declarations as the upper bound U of L in T . Rules ($\wedge$ - $\prec$) and ($\vee$ - $\prec$) states that expansion distributes through meets and joins. Rule ($\top$ - $\prec$) states that the top type $\top$ expands to the empty set. Rule ($\perp$ - $\prec$) states that the bottom type $\perp$ expands to the bottom element $\overline{D}_\perp$ of the lattice of sets of declarations (recall Figure 2).

2.7 Subtyping

Figure 4 defines the subtyping judgement $\Gamma \vdash S <: T$ which states that in environment Γ type S is a subtype of type T . Subtyping is regular wrt **wfe**: if type S is a subtype of type T , then S and T are well-formed and expanding. Though this regularity limits our calculus to **wfe**-types, this limitation allows us to show that subtyping is transitive, as discussed in section 6.2.1.

2.8 Declaration Subsumption

The declaration subsumption judgement $\Gamma \vdash D <: D'$ in Figure 4 states that in environment Γ the declaration D subsumes the declaration D' . There are three rules, one for each kind (type, value, method) of declarations. Rule (TDECL- $<$) states that a type declaration $L : S .. U$ subsumes another type declaration $L : S' .. U'$ if S' is a subtype of S and U is a subtype of U' . In other words, the set of types between S and U is contained in the set of types between S' and U' . Rule (VDECL- $<$) states that a value declaration $l : T$ subsumes another value declaration $l : T'$ if T is a subtype of T' . Rule (MDECL- $<$) is similar to (TDECL- $<$), as the parameter type varies contravariantly and the return type covariantly.

Declaration subsumption is extended to a binary relation between sequences of declarations: $\overline{D} <: \overline{D}'$ iff $\forall D'_i, \exists D_j. D_j <: D'_i$.

2.9 Well-formedness

The well-formedness judgement $\Gamma \vdash T \mathbf{wf}$ in Figure 5 states that in environment Γ the type T is well-formed.

A refinement type $T \{z \Rightarrow \overline{D}\}$ is well-formed if the parent type T is well-formed and every declaration in $\overline{D}$ is well-formed in an environment augmented by the binding of the self-reference z to the refinement type itself (RFN-WF).

Membership		$\boxed{\Gamma \vdash t \ni D}$
$\frac{\Gamma \vdash p : T, T \prec_z \bar{D}}{\Gamma \vdash p \ni [p/z]D_i}$	(PATH- $\ni$)	$\frac{z \notin \text{fn}(D_i) \quad \Gamma \vdash t : T, T \prec_z \bar{D}}{\Gamma \vdash t \ni D_i}$ (TERM- $\ni$)
Expansion		$\boxed{\Gamma \vdash T \prec_z \bar{D}}$
$\frac{\Gamma \vdash T \prec_z \bar{D}'}{\Gamma \vdash T \{z \Rightarrow \bar{D}\} \prec_z \bar{D}' \wedge \bar{D}}$	(RFN- $\prec$)	$\frac{\Gamma \vdash p \ni L : S..U, U \prec_z \bar{D}}{\Gamma \vdash p.L \prec_z \bar{D}}$ (TSEL- $\prec$)
$\frac{\Gamma \vdash T_1 \prec_z \bar{D}_1, T_2 \prec_z \bar{D}_2}{\Gamma \vdash T_1 \wedge T_2 \prec_z \bar{D}_1 \wedge \bar{D}_2}$	($\wedge$ - $\prec$)	$\frac{\Gamma \vdash T_1 \prec_z \bar{D}_1, T_2 \prec_z \bar{D}_2}{\Gamma \vdash T_1 \vee T_2 \prec_z \bar{D}_1 \vee \bar{D}_2}$ ($\vee$ - $\prec$)
$\Gamma \vdash \top \prec_z \{\}$	($\top$ - $\prec$)	$\Gamma \vdash \perp \prec_z \bar{D}_\perp$ ($\perp$ - $\prec$)
Figure 3. The DOT Calculus : Membership and Expansion		

Subtyping		$\boxed{\Gamma \vdash S <: T}$
$\frac{\Gamma \vdash T \text{ wfe}}{\Gamma \vdash T <: T}$	(REFL)	$\frac{\Gamma \vdash T \text{ wfe}}{\Gamma \vdash \perp <: T}$ ($\perp$ - $<$)
$\frac{\Gamma \vdash T \{z \Rightarrow \bar{D}\} \text{ wfe}, S <: T, S \prec_z \bar{D}'}{\Gamma, z : S \vdash \bar{D}' <: \bar{D}} \quad \frac{}{\Gamma \vdash S <: T \{z \Rightarrow \bar{D}\}}$	($<$ -RFN)	$\frac{\Gamma \vdash T \{z \Rightarrow \bar{D}\} \text{ wfe}, T <: T'}{\Gamma \vdash T \{z \Rightarrow \bar{D}\} <: T'}$ (RFN- $<$)
$\frac{\Gamma \vdash p \ni L : S..U, S <: U, S' <: S}{\Gamma \vdash S' <: p.L}$	($<$ -TSEL)	$\frac{\Gamma \vdash p \ni L : S..U, S <: U, U <: U'}{\Gamma \vdash p.L <: U'}$ (TSEL- $<$)
$\frac{\Gamma \vdash T <: T_1, T <: T_2}{\Gamma \vdash T <: T_1 \wedge T_2}$	($<$ - $\wedge$)	$\frac{\Gamma \vdash T_1 <: T, T_2 <: T}{\Gamma \vdash T_1 \vee T_2 <: T}$ ($\vee$ - $<$)
$\frac{\Gamma \vdash T_2 \text{ wfe}, T <: T_1}{\Gamma \vdash T <: T_1 \vee T_2}$	($<$ - $\vee_1$)	$\frac{\Gamma \vdash T_2 \text{ wfe}, T_1 <: T}{\Gamma \vdash T_1 \wedge T_2 <: T}$ ($\wedge_1$ - $<$)
$\frac{\Gamma \vdash T_1 \text{ wfe}, T <: T_2}{\Gamma \vdash T <: T_1 \vee T_2}$	($<$ - $\vee_2$)	$\frac{\Gamma \vdash T_1 \text{ wfe}, T_2 <: T}{\Gamma \vdash T_1 \wedge T_2 <: T}$ ($\wedge_2$ - $<$)
$\frac{\Gamma \vdash T \text{ wfe}}{\Gamma \vdash T <: \top}$	($<$ - $\top$)	
Declaration subsumption		$\boxed{\Gamma \vdash D <: D'}$
$\frac{\Gamma \vdash S' <: S, T <: T'}{\Gamma \vdash (L : S..T) <: (L : S'..T')}$	(TDECL- $<$)	$\frac{\Gamma \vdash S' <: S, T <: T'}{\Gamma \vdash (m : S \rightarrow T) <: (m : S' \rightarrow T')}$ (MDECL- $<$)
$\frac{\Gamma \vdash T <: T'}{\Gamma \vdash (l : T) <: (l : T')}$	(VDECL- $<$)	
Figure 4. The DOT Calculus : Subtyping and Declaration Subsumption		

3.3 Functions as Sugar

Like in Scala, we can encode functions as objects with a special method. Note that the variable z must be fresh.

$$\begin{aligned} S \rightarrow_s T &\iff \top \{z \Rightarrow \text{apply} : S \rightarrow T\} \\ \text{fun } (x : S) T t &\iff \text{val } z = \text{new } S \rightarrow_s T \{ \text{apply}(x) = t \}; z \\ (\text{app } f x) &\iff f.\text{apply}(x) \\ (\text{cast } T t) &\iff (\text{app } (\text{fun } (x : T) T x) t) \end{aligned}$$

We will freely use the following sugar in the remaining of this paper. We will also sometimes use $\lambda x : S. t$ for $\text{fun } (x : S) _ t$, omitting the return type for convenience and brevity.

3.4 Class Hierarchies

A class hierarchy such as

```
object pets {
  trait Pet
  trait Cat extends Pet
  trait Dog extends Pet
  trait Poodle extends Dog
  trait Dalmatian extends Dog
}
```

can be easily represented by concrete type members, setting the upper bounds appropriately:

```
val pets = new  $\top \{z \Rightarrow$ 
   $Pet_c : \perp.. \top$ 
   $Cat_c : \perp..z.Pet_c$ 
   $Dog_c : \perp..z.Pet_c$ 
   $Poodle_c : \perp..z.Dog_c$ 
   $Dalmatian_c : \perp..z.Dog_c$ 
 $\} \{ \};$ 
```

The lower of bounds of $\perp$ ensures that these concrete types are nominal as the $<:-$ TSEL rule cannot be meaningfully applied.

3.5 Abstract Type Members

The `choices.Alt` trait takes three abstract type members: C , A , B . A and B are upper bounded by C . The intention is that the `choose` function takes an A and a B and returns one or the other.

```
object choices {
  trait Alt {
    type C
    type A <: C
    type B <: C
    val choose : A  $\Rightarrow$  B  $\Rightarrow$  C
  }
}
```

In DOT, we can state more precisely the return type of `choose`, thanks to union types:

```
val choices = new  $\top \{z \Rightarrow$ 
   $Alt_c : \perp.. \top \{a \Rightarrow$ 
     $C : \perp.. \top$ 
     $A : \perp..a.C$ 
     $B : \perp..a.C$ 
     $choose : a.A \rightarrow a.B \rightarrow_s a.A \vee a.B$ 
   $\}$ 
 $\} \{ \};$ 
```

Using lower bounds of $\perp$ for the abstract type members means they can vary covariantly. However, we wouldn't want to pass in any `pets.Petc` to a `choose` method that expects only a `pets.Dogc`. Therefore,

$$\begin{aligned} &\text{choices.Alt}_c \{a \Rightarrow C : \perp.. \text{pets.Dog}_c\} \\ <: &\text{choices.Alt}_c \{a \Rightarrow C : \perp.. \text{pets.Pet}_c\} \end{aligned}$$

but

$$\begin{aligned} &\text{choices.Alt}_c \{a \Rightarrow C : \text{pets.Dog}_c.. \text{pets.Dog}_c\} \\ \not<: &\text{choices.Alt}_c \{a \Rightarrow C : \text{pets.Pet}_c.. \text{pets.Pet}_c\} \end{aligned}$$

As expected, we cannot invoke `choose` meaningfully, unless A and B have realizable lower bounds. For example, assuming we refine the types above so that $A : C..C$ and $B : C..C$, we cannot invoke `choose` when C has a lower bound of $\perp$ but only when it has a realizable lower bound such as `pets.Dogc`.

3.6 Polymorphic Operators as Sugar

In Scala, we can implement a polymorphic operator `pickLast` which takes concrete types for C , A and B and implements a `choices.Alt` instance where the `choose` function picks the B element – note the precision of the `choose` function which has been refined to return an element of exactly type B .

```
def pickLast[Cp, Ap <: Cp, Bp <: Cp] = new Alt {
  type C = Cp
  type A = Ap
  type B = Bp
  val choose: A  $\Rightarrow$  B  $\Rightarrow$  B = a  $\Rightarrow$  b  $\Rightarrow$  b
}

val potty = new Poodle {}
val dotty = new Dalmatian {}
val picker = pickLast[Dog, Poodle, Dalmatian]
val p: picker.A = potty
val r: picker.B = picker.choose(potty)(dotty)
```

In DOT, we can implement such a polymorphic operator as sugar. Here, it is not convenient to just return a complex term like we did for functions as sugar because then invoking `choose` falls under the `TERM- $\exists$` restriction, which doesn't allow the "self" type to occur in the result of the method invocation. So the translation involves explicitly binding an object to the result of the polymorphic operator. We translate

$$\text{val } x^a = \text{pickLast}(T^C, T^A, T^B); e^a$$

to

$$\begin{aligned} \text{val } x^a &= \text{new choices.Alt}_c \{x^a \Rightarrow \\ &C : T^C..T^C \\ &A : T^A..T^A \\ &B : T^B..T^B \\ &\text{choose} : x^a.A \rightarrow x^a.B \rightarrow_s x^a.B \\ &\} \{ \text{choose}(a) = \text{fun } (b : x^a.B) x^a.B b \}; \\ &e^a \end{aligned}$$

Now, given

```
val p = new pets.Poodle_c;
val d = new pets.Dalmatian_c;
val a = pickLast(pets.Dog_c, pets.Poodle_c, pets.Dalmatian_c);
```

The type of a is a subtype of $choices.Alt_c$:

```
(cast  $\top$ 
  (cast  $choices.Alt_c$ 
    (cast  $choices.Alt_c \{a \Rightarrow C : \perp..pets.Dog_c\}$ 
      a)))
```

The type of p is a subtype of $a.A$:

```
(cast  $\top$  (cast  $a.A$   $p$ ))
```

a chooses a $pets.Dalmatian_c$:

```
(cast  $\top$  (cast  $pets.Dalmatian_c$  (app  $a.choose(p)$   $d$ )))
```

a enforces that its first argument be a $pets.Poodle_c$ and its second a $pets.Dalmatian_c$. The following does not type-check for this reason:

```
(cast  $\top$  (app  $a.choose(d)$   $p$ ))
```

3.7 F-bounded Quantification

F-bounded quantification describes an upper bound that itself contains the type being constrained: for example, `Int extends Ordered [Int]`. Here, we define `MetaAlt` to extend `choices.Alt` with `C` as an alias for `MetaAlt`.

```
trait MetaAlt extends choices.Alt {
  type C = MetaAlt
  type A = C
  type B = C
}
```

Now, we can define some `MetaAlt` instances:

```
val first = new MetaAlt {
  val choose: C  $\Rightarrow$  C  $\Rightarrow$  C = a  $\Rightarrow$  b  $\Rightarrow$  a
}
val last = new MetaAlt {
  val choose: C  $\Rightarrow$  C  $\Rightarrow$  C = a  $\Rightarrow$  b  $\Rightarrow$  b
}
val recfirst = new MetaAlt {
  val choose: C  $\Rightarrow$  C  $\Rightarrow$  C = a  $\Rightarrow$  b  $\Rightarrow$  a.choose(a)(b)
}
val reclast = new MetaAlt {
  val choose: C  $\Rightarrow$  C  $\Rightarrow$  C = a  $\Rightarrow$  b  $\Rightarrow$  b.choose(a)(b)
}
```

The equivalent in DOT is straightforward. We wrap `MetaAlt` in a namespace, so that we can refer to it.

```
val  $m$  = new  $\top \{m \Rightarrow$ 
   $MetaAlt_c : \perp..choices.Alt_c \{a \Rightarrow$ 
     $C : m.MetaAlt_c..m.MetaAlt_c$ 
     $A : a.C..a.C$ 
     $B : a.C..a.C$ 
  }
} \{ \};
```

Now, we can create the equivalent of `first` (f), `last` (l), `recfirst` (rf) and `reclast` (rl):

```
val  $f$  = new  $m.MetaAlt_c \{$ 
   $choose(a) = \text{fun } (b : m.MetaAlt_c) m.MetaAlt_c a \};$ 
val  $l$  = new  $m.MetaAlt_c \{$ 
   $choose(a) = \text{fun } (b : m.MetaAlt_c) m.MetaAlt_c b \};$ 
val  $rf$  = new  $m.MetaAlt_c \{$ 
   $choose(a) = \text{fun } (b : m.MetaAlt_c) m.MetaAlt_c$ 
     $(\text{app } a.choose(a) b) \};$ 
val  $rl$  = new  $m.MetaAlt_c \{$ 
   $choose(a) = \text{fun } (b : m.MetaAlt_c) m.MetaAlt_c$ 
     $(\text{app } b.choose(a) b) \};$ 
```

Given these definitions, here is a valid expression, which evaluates to f : $(\text{cast } \top (\text{app } rf.choose(f) l))$.

4. Counterexamples to Preservation

We first tried to prove the calculus type-safe using the standard theorems of preservation and progress [15, 16]. Unfortunately, for the calculus as presented, and any of its variants that we devised, preservation doesn't hold. In this section, we review some of the most salient counterexamples to preservation that we found. These counterexamples have been checked with PLT Redex [12].

Most of these counterexamples are related to narrowing, the phenomenon that a type can become more precise after substitution: if a method takes a parameter x of type U , then when it is invoked, any argument v of type $S <: U$ can be substituted – this is the narrowing effect: it is as if the context was changed from $x : U$ to $x : S$. Sections 4.1, 4.2 and 4.3 each present a counterexample related to narrowing.

The last counterexample, presented in section 4.4, illustrates the need to relate path-dependent types after reduction. This need for path-equality provisions in order for preservation to hold is well-known from other calculi such as Tribe [4] with path-dependent types and a small-step operational semantics.

More generally, these counterexamples illustrate that preservation doesn't hold because a term that type-checks can step to a term that does not type-check. However, these counterexamples to preservation are not counterexamples to type-safety: i.e. these programs don't get stuck – they eventually step to a value.

4.1 TERM- $\ni$ Restriction

There are two membership ($t \ni D$) rules: one for when the term t is a path, and one for an arbitrary term t . For paths, we can substitute the self-references in the declarations, but we cannot do so for arbitrary terms as the resulting types wouldn't be well-formed syntactically. Hence, the TERM- $\ni$ has the restriction that self-occurrences are not allowed. Here is a counterexample related to this restriction.

Let X be a shorthand for the type:

```
 $\top \{z \Rightarrow$ 
   $L_a : \top.. \top$ 
   $l : z.L_a$ 
}
```

Let Y be a shorthand for the type:

$$\begin{array}{c} \top \{z \Rightarrow \\ l : \top \\ \} \end{array}$$

Now, consider the term

$\text{val } u = \text{new } X \{l = u\};$
 $(\text{app } (\lambda y : \top \rightarrow Y.(\text{app } y u)) (\lambda d : \top.(\text{cast } X u)))l$

The term type-checks because the term $t = (\text{app } (\lambda y : \top \rightarrow Y.(\text{app } y u)) (\lambda d : \top.(\text{cast } X u)))$ has type Y , so we can apply $\text{TERM-}\exists$ for l . However, the term t eventually steps to $(\text{cast } X u)$ which has type X , so we cannot apply $\text{TERM-}\exists$ for l because of the self-reference $(z.L_a)$.

4.2 Expansion Lost

First, let's illustrate why expansion does not always exist.

Here is the simplest such example:

$\text{val } z = \text{new } \top \{z \Rightarrow$
 $L : \perp..z.L$
 $\} \{ \};$

The type $z.L$ is **wf** but not **wfe**. Indeed, there is no finite derivation of an expansion for $z.L$, because by the $\text{TSEL-}\prec$ rule, in order to expand $z.L$, we need to expand its upper bound, which is $z.L$! However, not that the object creation expression for z would not type-check because subtyping is regular wrt **wfe**, and so the type of a constructor in an object creation expression must have **wfe** type members, since we check that the lower bound is a subtype of the upper bound for each type member.

The example above relies on the TSEL-WF_2 , in order for the upper bound to refer to the type member being declared. Here is an example that does not rely on this rule, and that we will use below to create a counterexample to preservation. Let

$T_1 = \top \{z \Rightarrow$
 $A : \perp..z.B$
 $B : \perp..\top \quad \}$
 $T_2 = \top \{z \Rightarrow$
 $A : \perp..\top$
 $B : \perp..z.A \quad \}$

Now, consider $T_1 \wedge T_2$. The type is **wf** and **wfe**, but its members A and B are not **wfe**, because the expansion of $T_1 \wedge T_2$ with self-type z has the set of declarations $\{A : \perp..z.B, B : \perp..z.A\}$ – thus, to expand $z.A$, we need to expand $z.B$, and to expand $z.B$, we need to expand $z.A$! There is no finite derivation of expansions for the type members A and B .

Expansion is not preserved by narrowing. Here, we create two type selections that are mutually recursive in their upper bounds after narrowing: $z_0.C_2$ initially expands, but after narrowing, $z_0.C_2$ expands to what $z_0.A_2$ expands to, which expands to what $z_0.A_1$ expands to, which expands to what $z_0.A_2$ expands to, and thus we have an infinite expansion. Thus, the last new expression initially type-checks, but after narrowing, it doesn't because the precise expansion needed by **NEW** cannot be inferred.

$\text{val } x_0 = \text{new } \top \{z \Rightarrow A_1 : \perp..\top \{z \Rightarrow$
 $A_2 : \perp..\top$
 $A_3 : \perp..\top$
 $C_2 : \perp..z.A_2\} \} \{ \};$
 $\text{val } x_1 = \text{new } \top \{z \Rightarrow C_1 : \perp..\top \{z \Rightarrow A_1 : \perp..x_0.A_1\} \} \{ \};$
 $\text{val } x_2 = \text{new } x_1.C_1 \{z \Rightarrow A_1 : \perp..x_0.A_1 \{z \Rightarrow A_2 : \perp..z.A_3\} \} \{ \};$
 $\text{val } x_3 = \text{new } x_1.C_1 \{z \Rightarrow A_1 : \perp..x_0.A_1 \{z \Rightarrow A_3 : \perp..z.A_2\} \} \{ \};$
 $(\text{app } \lambda x : x_1.C_1.(\lambda z_0 : x.A_1 \wedge x_3.A_1.$
 $\quad \text{val } z = \text{new } z_0.C_2; (\text{app } (\lambda x : \top.x) z))$
 $x_2)$

4.3 Well-Formedness Lost

Even well-formedness is not preserved by narrowing. The trick is that if the lower bound of a type selection is not $\perp$, then the bounds needs to be checked for well-formedness. Here, we create two type selections that are mutually recursive in their bounds after narrowing. $y.A$ is initially well-formed, but after narrowing, it isn't because we run into an infinite derivation trying to prove the well-formedness of its bounds.

$\text{val } v = \text{new } \top \{z \Rightarrow L : \perp..\top \{z \Rightarrow A : \perp..\top, B : z.A..z.A\} \} \{ \};$
 $(\text{app } (\lambda x : \top \{z \Rightarrow L : \perp..\top \{z \Rightarrow A : \perp..\top, B : \perp..\top\} \}.$
 $\quad \text{val } z = \text{new } \top \{z \Rightarrow$
 $\quad \quad l : x.L \wedge \top \{z \Rightarrow A : z.B..z.B, B : \perp..\top\} \rightarrow \top \{ \{$
 $\quad \quad \quad l(y) = \text{fun } (a : y.A) \top a \};$
 $\quad \quad (\text{cast } \top z))$
 $v)$

4.4 Path Equality

For preservation, we need to be able to relate path-dependent types after reduction. Here is a motivating example:

$\text{val } b = \text{new } \top \{z \Rightarrow$
 $\quad \quad X : \top..\top$
 $\quad \quad \quad l : z.X \quad \} \{l = b\};$
 $\text{val } a = \text{new } \top \{z \Rightarrow i : \top \{z \Rightarrow$
 $\quad \quad \quad X : \perp..\top$
 $\quad \quad \quad \quad l : z.X \quad \} \{i = b\};$
 $(\text{app } (\lambda x : \top.x) (\text{app } (\lambda x : a.i.X.x) a.i.l))$

$a.i.l$ reduces to $b.l$. $b.l$ has type $b.X$, so we need $b.X <: a.i.X$. This cannot be established with the current rules: it is not true in general, but true here because $a.i$ reduces to b . Hence, we need to acknowledge path equality for preservation to hold.

In section 6.2.3, we discuss our failure to patch the calculus for preservation to hold.

5. Type-Safety via Logical Relations

We believe that the DOT calculus is type-safe, and are developing a proof of type-safety using step-indexed logical relations [1, 2, 10]. In this section, we briefly summarize the main theorem of type-safety, and the strategy based on step-indexed logical relations that we are using to prove it. All our development, including models in Coq [13] and PLT Redex [12], is available from <http://dot.namin.net>.

5.1 Type-Safety

Type-safety states that a well-typed program doesn't get stuck. More formally: If $\emptyset \vdash t : T$ and $t \mid \emptyset \rightarrow^* t' \mid s'$ then either t' is a value or $\exists t'', s''. t' \mid s' \rightarrow t'' \mid s''$. Note that this is stronger than the standard theorem of progress, which states that a well-type term can take a step or is a value.

5.2 Strategy based on Logical Relations

Our strategy follows the standard technique of proving type-safety using logical relations. We define a logical relation $\Gamma \models t : T$, such that $\Gamma \vdash t : T$ implies $\Gamma \models t : T$ implies type-safety.

The main logical relation $\Gamma \models t : T$ is based on a set of mutually recursive logical relations that are step-indexed in order to ensure their well-foundedness: $\mathcal{E}_{k;\Gamma;s}[[T]]$ defines the set of terms that appear to have type T when taking at most k steps, and $\mathcal{V}_{k;\Gamma;s}[[T]]$ defines the set of *values* that appear to have type T when taking at most k steps. There are also two other logical relations, one for completing the store to match the context, and one for completing the context to match the store resulting from taking some number of steps.

$\Gamma \models t : T$ is defined as $t \in \mathcal{E}_{k;\Gamma;\emptyset}[[T]]$, $\forall k. t \in \mathcal{E}_{k;\Gamma;s}[[T]]$ roughly as follows: after reducing t a number of steps $< k$, if the resulting term is irreducible, then it must be in $\mathcal{V}_{j';\Gamma';s'}[[T]]$ for appropriate j', Γ', s' . By definition, $\Gamma \models t : T$ implies that t cannot be stuck.

Then, to prove type-safety, all that needs to be proved is the fundamental theorem – or completeness – of the logical relation: $\Gamma \vdash t : T$ to $\Gamma \models t : T$. Type-safety is a straightforward corollary of this theorem, since $\Gamma \models t : T$ implies by definition that t cannot be stuck. The proof of the fundamental theorem is on induction on the derivation of $\Gamma \vdash t : T$. The logical relation approach enables us to have strong enough induction hypotheses to carry the proof through, without requiring us to strictly relate intermediate terms by types like preservation.

6. Discussion

6.1 Why No Inheritance?

In the calculus we made the deliberate choice not to model any form of inheritance. This is, first and foremost, to keep the calculus simple. Secondly, there are many different approaches to inheritance and mixin composition, so that it looks advantageous not to tie the basic calculus to a specific one. Finally, it seems that the modelization of inheritance lends itself to a different approach than the basic calculus. For the latter, we need to prove type-safety of the calculus. One might try to do this also for a calculus with inheritance, but our experience suggests that this complicates the proofs considerably. An alternative approach that might work better is to model inheritance as a form of code-reuse. Starting with an enriched type system with inheritance, and a translation to the basic calculus, one needs to show type-safety wrt the translation. This might be easier than to prove type-safety wrt reduction.

6.2 Variants of the DOT Calculus

6.2.1 Why limit the calculus to wfe-types?

Currently, the proof of type-safety via logical relations fundamentally relies on types having an expansion. However, this was not our original motivation for limiting the calculus to **wfe**-types.

Originally, subtyping was not regular wrt **wfe**. Roughly, all the **wfe** preconditions in subtyping were dropped. In this broader calculus, subtyping transitivity doesn't hold, because of the rule ($\leftarrow$ -RFN) which requires expansion of the left type.

The problem is deep, as attested by this elaborate counterexample that is not so easily patched, and directly leads to a counterexample to preservation.

Consider an environment where u is bound to:

```

 $\top \{u \Rightarrow$ 
   $Bad : \perp..u.Bad$ 
   $Good : \top \{z \Rightarrow L : \perp.. \top\} .. \top \{z \Rightarrow L : \perp.. \top\}$ 
   $Lower : u.Bad \wedge u.Good..u.Good$ 
   $Upper : u.Good..u.Bad \vee u.Good$ 
   $X : u.Lower..u.Upper$ 
 $\}$ 

```

Now, consider the types S, T, U defined in terms of u :

```

 $S = u.Bad \wedge u.Good$ 
 $T = u.Lower$ 
 $U = u.X \{z \Rightarrow L : \perp.. \top\}$ 

```

We have $S <: T$ and $T <: U$, but we cannot derive $S <: U$ because S doesn't expand.

Note that u is realizable, since each lower bound is a subtype of its upper bound. So it is straightforward to turn this counterexample to subtyping transitivity into a counterexample to preservation:

```

val  $u = \text{new } \dots;$ 
(app  $\lambda x : \top.x$ 
  (app  $\lambda f : S \rightarrow U.f$ 
    (app  $\lambda f : S \rightarrow T.f$ 
      (app  $\lambda f : S \rightarrow S.f$ 
         $\lambda x : S.x))))$ 

```

The idea is to start with a function from $S \rightarrow S$ and cast it successively to $S \rightarrow T$ then $S \rightarrow U$. To type-check the expression initially, we need to check $S <: T$ and $T <: U$. After some reduction steps, the first few casts vanish, and the reduced expression casts directly from $S \rightarrow S$ to $S \rightarrow U$, so we need to check $S <: U$.

6.2.2 Why not include the lambda-calculus instead of methods?

Originally, the DOT calculus included the lambda-calculus, and explicit methods were not needed since they could be represented by a value label with a function type. However, the expansion of the function type was defined to be the empty set of declarations (like for $\top$), which caused a real breach of type-safety.

A concrete object could be a subtype of a function type without a function ever being defined. Consider:

```

val  $u = \text{new } \top \{z \Rightarrow C : \top \rightarrow \top.. \top \rightarrow \top\} \{ \};$ 
val  $f = \text{new } u.C \{ \};$ 
 $\dots$ 

```

Now, f was a subtype of $\top \rightarrow \top$, but (**app** $f (\lambda x : \top.x)$) was stuck (and, rightfully, didn't type-check). But we could use narrowing to create a counterexample to type-safety: (**app** $(\lambda g : \top \rightarrow \top. (\text{app } g () \lambda x : \top.x)) f$).

Because of this complication, we decided to drop the lambda-calculus from DOT, and instead introduce methods with one parameter. Like in Scala, functions are then just sugar for objects with a special method.

An alternative design would have been to change the expansion of the function type to have a declaration for a special label that

either prevents instantiation or requires an implementation for the function.

6.2.3 Why not patch the DOT calculus for preservation to hold?

We tried! However, the resulting calculi were not elegant, and furthermore, we still found issues with preservation.

Here is a summary of one attempt, which is further detailed below. Because many of the counterexamples to preservation are related to narrowing, we tried to make widening an explicit operation and change rules with implicit relaxations (MSEL and NEW) to be strict. From a typing perspective, the change was straightforward, but reduction became more complicated and dependent on typing because the type information in widenings needed to be propagated correctly. We added path equality provisions in the subtyping rules, in the same spirit as the Tribe calculus [4]. Unfortunately, these two patches interacted badly, and we were left with a disturbing counterexample to type-safety.

In any case, this attempt resulted in a patched calculus that was not as elegant as the original one, in addition to being unsound. We're open to ideas!

In the DOT calculus as presented, the APP and NEW typing rules have implicit relaxations. For instance, in APP, the argument type may be a subtype of the declared parameter type. In order to deal with all the preservation counterexamples due to narrowing, we tried making widening an explicit operation and changing those rules to be strict by replacing those relaxed subtyping judgments with equality judgments. Two types S and T are judged to be equal if $S <: T$ and $T <: S$.

Syntactically, we add a widening term: $t : T$, and extend values with a case for widening: $v : T$. The typing rule for widening, WID, is then the only one admitting a subtyping relaxation: $\Gamma \vdash (t : T) : T$ if $\Gamma \vdash t : T'$ and $\Gamma \vdash T' <: T$.

The reduction rules become more complicated because the type information in the widening needed to be propagated correctly. We will motivate this informally with examples.

```
val v = new  $\top \{z \Rightarrow L_a : \perp.. \top, l : \top \{z \Rightarrow L_a : \perp.. \top\}\}$ 
    { $l = v : \top \{z \Rightarrow L_a : \perp.. \top\}$ };
(app ( $\lambda x : \top.x$ ) ( $v : \top \{z \Rightarrow l : \top\}$ )).l
```

The term $(v : \top \{z \Rightarrow l : \top\}).l$ first widens v so that the label has type $\top$ instead of $\top \{z \Rightarrow L_a : \perp.. \top\}$.

How should reduction proceed? We cannot just strip the widening and then reduce, because then the strict function application would not accept the reduced term. In short, we need to do some type manipulations during reduction, by using the membership and expansion judgments. This is a bit unfortunate, because it means that reduction now needs to know about typing.

Next, we look at path equality provisions. These are even more essential now in the presence of explicit widening. Consider this example:

```
val b = new  $\top \{z \Rightarrow X : \top.. \top, l : z.X\}$  { $l = b : b.X$ };
val a = new  $\top \{z \Rightarrow i : \top \{z \Rightarrow X : \perp.. \top, l : z.X\}$ 
    { $i = b : \top \{z \Rightarrow X : \perp.. \top, l : z.X\}$ };
a.i.l :  $\top$ 
```

$a.i.l$ reduces to $b : \top \{z \Rightarrow X : \perp.. \top, l : z.X\}$. Now, how can we continue? $b.l$ reduces to $b : b.X$ which has bounds $\top.. \top$, but $(b : \top \{z \Rightarrow X : \perp.. \top, l : z.X\}).l$ has bounds $\perp.. \top$, so without some provision for path equality, we cannot widen $b.l$ to $(b : \top \{z \Rightarrow X : \perp.. \top, l : z.X\}).l$.

We add the path equality provisions to the subtyping rules.

Let's first ignore the extension of the calculus requiring explicit widenings. Then, we need to add one intuitive rule to the subtyping judgment: $<:-\text{PATH}$. If p (path-)reduces to q , and $T <: q.L$, then $T <: p.L$. Path reduction is a simplified form of reduction involving only paths. However, this means that the subtyping judgment, and indirectly, all the typing-related judgments, now need to carry the store in addition to the context so that path reductions can be calculated.

Now, let's see how path equality provisions and explicit widening can fit together.

First, path reduction is not isomorphic to reduction anymore, since we want to actually skip over widenings, as motivated by the example above.

In addition, we now also need a dual rule, $\text{PATH-}<:$ if p (path-dually-)reduces to q , and $q.L <: T$ then $p.L <: T$. This is because when we have a widening on an object on which a method is called, we have to upcast the argument to the parameter type expected by the original method. Here is a motivating example.

Let T_c be a shorthand for the type:

```
 $\top \{z \Rightarrow$ 
   $A : \top \{z \Rightarrow m : \perp \rightarrow \top\} .. \top$ 
   $B : \top.. \top$ 
   $m : z.A \rightarrow \top$ 
 $\}$ 
```

Let T be a shorthand for the type:

```
 $\top \{z \Rightarrow$ 
   $A : \top \{z \Rightarrow m : \perp \rightarrow \top\} .. \top$ 
   $B : \top.. \top$ 
   $m : z.A \{z \Rightarrow B : \top.. \top\} \rightarrow \top$ 
 $\}$ 
```

Now, consider the term:

```
val v = new  $T_c \{m(x) = x : \top\}$ ;
(v :  $T$ ).m(v : ((v :  $T$ )).A {z ⇒ B :  $\top.. \top$ }))
```

When we evaluate the method invocation, we need to cast $v : ((v : T).A \{z \Rightarrow B : \top.. \top\})$ to $v.A$, and for this, we need the newly introduced $\text{PATH-}<:$ rule.

Note that the path dual reduction can be a bit stricter with casts than the path reduction. In any case, introducing this $\text{PATH-}<:$ rule into the subtyping judgment is problematic: it is now possible to say $p.L <: T$, even though T can do more than what $p.L$ is defined to do. Here is an example, where we construct an object, with $T = \perp$. (The convolution in the example is due to the requirement that concrete types be only mentioned once.)

```
val a = new  $\top \{z \Rightarrow C : \perp.. \top \{z \Rightarrow D : \perp.. z.X, X : \perp.. \top\}\}$ ;
val b = new  $a.C \{z \Rightarrow X : \perp.. \perp\}$ ;
val c = new  $a.C$ ;
val d = new  $(b : a.C).D$ ;
(app ( $\lambda x : \perp.x.foo$ ) d)
```

Notice that d has type $\perp$ if you ignore the cast on b . This example doesn't type-check initially because $\text{PATH-}<:$ only applies when objects are in the store, so the application is not well-typed. But if we start preservation in a store which has a, b, c and d then the application type-checks, because, through $\text{PATH-}<:$, we can find that the type of d is a subtype of $\perp$. Now, of course, when we get to $d.foo$, reduction fails.

So the preservation theorem as defined on a small-step semantics (where we start with an arbitrary well-formed environment) fails when we add the `PATH-<` rule.

6.3 Related Work

In addition to Scala’s previous models [5, 14], several calculi present some form of path-dependent types.

The `vc` calculus [7] models virtual classes with path-dependent types. `vc` restricts paths to start with “this”, though it provides a way (“out”) to refer to the enclosing object.

The Tribe calculus [4] builds an ownership types system [3] on top of a core calculus which models virtual classes. The soundness proof for the core calculus seems to be tied to the ownership types system.

Some ML-style module systems [8, 9] have a form of stratified path-dependent types. Because of the stratification, recursion is not allowed. In MixML [6] like in Scala, this restriction is lifted.

7. Conclusion

We have presented DOT, a calculus aimed as a new foundation of Scala and languages like it. DOT features path-dependent types, refinement types, and abstract type members.

Proving the DOT calculus type-safe has been an interesting adventure. We have shown that DOT does not satisfy preservation (also known as subject-reduction), because a well-typed term can step to an intermediate term that is not well-typed. In any case, the standard theorems of preservation and progress are just one way to prove type safety, which states that a well-typed term cannot be stuck – a weaker statement that does not require type-checking intermediate steps. We are developing a proof of type-safety using logical relations.

Acknowledgments

We thank Amal Ahmed for many discussions and insights about applying logical relations to DOT. We thank Donna Malayeri and Geoffrey Washburn for preliminary work on DOT. We thank Tiark Rompf and Viktor Kuncak for helpful comments.

References

- [1] A. J. Ahmed. *Semantics of types for mutable state*. PhD thesis, Princeton University, 2004.
- [2] A. J. Ahmed. Step-indexed syntactic logical relations for recursive and quantified types. In *ESOP*, pages 69–83, 2006.
- [3] N. R. Cameron, J. Noble, and T. Wrigstad. Tribal ownership. In *OOPSLA*, pages 618–633, 2010.
- [4] D. Clarke, S. Drossopoulou, J. Noble, and T. Wrigstad. Tribe: a simple virtual class calculus. In *AOSD*, pages 121–134, 2007.
- [5] V. Cremet, F. Garillot, S. Lenglet, and M. Odersky. A core calculus for Scala type checking. In *MFCs*, pages 1–23, 2006.
- [6] D. Dreyer and A. Rossberg. Mixin’ up the ML module system. In *ICFP*, pages 307–320, 2008.
- [7] E. Ernst, K. Ostermann, and W. R. Cook. A virtual class calculus. In *POPL*, pages 270–282, 2006.
- [8] R. Harper and M. Lillibridge. A type-theoretic approach to higher-order modules with sharing. In *POPL*, pages 123–137, 1994.
- [9] T. Hirschowitz and X. Leroy. Mixin modules in a call-by-value setting. In *ESOP*, pages 6–20, 2002.
- [10] C. Hritcu and J. Schwinghammer. A step-indexed semantics of imperative objects. *Logical Methods in Computer Science*, 5(4), 2009.
- [11] A. Igarashi and B. C. Pierce. Foundations for virtual types. *Inf. Comput.*, 175(1):34–49, 2002.
- [12] C. Klein, J. Clements, C. Dimoulas, C. Eastlund, M. Felleisen, M. Flatt, J. A. McCarthy, J. Rafkind, S. Tobin-Hochstadt, and R. B. Findler. Run your research: on the effectiveness of lightweight mechanization. In *POPL*, pages 285–296, 2012.
- [13] The Coq development team. *The Coq proof assistant reference manual*. LogiCal Project, 2010. URL <http://coq.inria.fr>. Version 8.3.
- [14] M. Odersky, V. Cremet, C. Röckl, and M. Zenger. A nominal theory of objects with dependent types. In *ECOOP*, pages 201–224, 2003.
- [15] B. C. Pierce. *Types and programming languages*. MIT Press, 2002. ISBN 978-0-262-16209-8.
- [16] A. K. Wright and M. Felleisen. A syntactic approach to type soundness. *Inf. Comput.*, 115(1):38–94, 1994.

A Type System for Dynamic Layer Composition

Atsushi Igarashi

Kyoto University
igarashi@kuis.kyoto-u.ac.jp

Robert Hirschfeld

Hasso-Plattner-Institut Potsdam
hirschfeld@hpi.uni-potsdam.de

Hidehiko Masuhara

The University of Tokyo
masuhara@acm.org

Abstract

Dynamic layer composition is one of the key features in context-oriented programming (COP), an approach to improving modularity of behavioral variations that depend on the dynamic context of the execution environment. It allows a layer—a set of new or overriding methods that can belong to several classes—to be added to or removed from existing objects in a disciplined way. We develop a type system for dynamic layer composition, which may change the interfaces of objects at run time, based on a variant of ContextFJ, a core calculus for COP, and prove its soundness.

Categories and Subject Descriptors D.3.1 [*Programming Languages*]: Formal Definitions and Theory; D.3.3 [*Programming Languages*]: Language Constructs and Features

General Terms Language, Theory

Keywords Context-oriented programming, dynamic layer composition, type systems

1. Introduction

Context-oriented programming (COP) is an approach to improving modularity of behavioral variations that depend on dynamic properties of the execution environment [10]. In traditional programming paradigms, such behavioral variations tend to be scattered over several modules, and system architectures that support their dynamic composition are often complicated.

Many COP extensions including those designed on top of Java [2], Smalltalk [9], Common Lisp [6], and JavaScript [15], are based on object-oriented programming languages and introduce *layers of partial methods* for defining and organizing behavioral variations and *layer activation mechanisms* for layer selection and composition. A partial method in a layer is, in many cases, a method that can run before, after, or around a (partial) method with the same name and signature defined in a different layer or a class, but it can also be a new method that does not exist in a class yet. A layer groups related partial methods and can be (de)activated at run-time. It so contributes to the specific behavior of a set of objects in response to messages sent and received.

In this paper, we develop a simple type system for such dynamic composition of layers. Although what the type system guarantees is just the absence of “no such method” errors (including the failure of `proceed` calls in around-type partial methods), the existence of partial methods that introduce new behavior to existing

classes makes the problem interesting, because layer (de)activation changes the interface of objects at run time. A key idea of our development is the introduction of an explicitly declared inter-layer dependency relation, which plays a role similar to required methods in type systems for mixins [4, 8, 14]. With the help of this dependency, the type system will estimate layers activated at each program point and use this estimation to look up the signature of an invoked method. We formalize the type system for a variant of ContextFJ [11], a COP extension of Featherweight Java [12], and prove its soundness. ContextFJ supports (around-type) partial methods, block-structured dynamic activation of layers, and `proceed` and `super` calls.

We also discuss a few variations of layer activation mechanisms (although we formalize only one of them). Because it turns out that our type system seems *not* to work for the layer activation mechanism used in many COP languages, we prove type soundness for “a variant” of ContextFJ. Nevertheless, we believe that this work is of some value as a clarification about how our simple typing scheme interacts with the design of layer activation mechanisms.

This paper is a continuation of our work [11], in which ContextFJ is formalized. There, an even simpler type system for ContextFJ is discussed, but it is very restrictive because it prohibits layers from adding new methods to existing classes.

The rest of the paper is organized as follows. We first start with reviewing the language mechanisms for COP in Section 2. Section 3 reviews the syntax and operational semantics of ContextFJ and Section 4 defines the type system and proves its soundness. We discuss related and future work in Section 5.

2. Language Constructs for COP

We briefly overview basic constructs along with their usage. Our example is a simplified telecom simulation¹ in which customers make, accept, and terminate phone calls.

2.1 The Base Layer

The base layer consists of standard Java classes and methods, which is always active. The telecom example has `Customer` and `Connection` to represent customers and phone calls between customers, respectively.

```
class Customer { ... }
class Connection {
  Connection(Customer a, Customer b) { ... }
  void complete() { ... }
  void drop() { ... }
}
```

The following method demonstrates a usage of those classes.

```
Connection simulate() {
```

Copyright is held by the author/owner(s).
FOOL2012 October 22, 2012, Tucson, AZ.
ACM.

¹ This simulation is based on an example distributed along with the AspectJ compiler [21].

```

Customer atsushi = ..., hidehiko = ...;
Connection c =
    new Connection(atsushi,hidehiko);
                                // Atsushi calls Hidehiko
c.complete();                  // Hidehiko accepts
c.drop();                      // Hidehiko hangs up
return c;
}

```

2.2 Layers and Partial Method Definitions

A layer is a collection of methods and fields² that are specific to a certain context. Syntactically, they are written as Java classes enclosed by the `layer` construct³. Below, the `Timing` layer defines a feature that measures the duration of phone calls. (A COP layer is usually used to group partial methods of more than one class, but as an illustrating example for `ContextFJ`, partial methods of one class will suffice.)

```

layer Timing {
    class Connection {
        Timer timer;
        void complete() { proceed(); timer.start(); }
        void drop() { timer.stop(); proceed(); }
        int getTime() { return timer.getTime(); }
    }
}

```

When a layer is active (as explained in Section 2.3), the methods defined in that layer, so called *partial methods*, override those in the base layer. In the above example, `complete` and `drop` are partial methods.

Unlike other COP languages, we also allow a layer to define a method that does not exist in the base layer, which we call a *layer-introduced base method*. In the above example, `getTime` is such a layer-introduced base method.

`proceed(...)` is similar to `super` as it delegates behavior to overridden methods. Whereas `super` changes the starting point of the method lookup to the superclass of the class the (partial) method was defined in, `proceed(...)` will try first to find the next partial or base-level definition of the same method in the same (current) class. If `proceed(...)` cannot find such a partial method in the current receiver class or the active layers associated with it, lookup continues in the superclass of the current lookup class.

Existence of layer-introduced base methods and `proceed` make a type system, which statically guarantees success of `proceed`, complicated. We will show this in Section 4.

2.3 Layer Activation

Many COP languages offer `with` for layer activation and `without` for layer deactivation. In this work, we consider the `ensure` construct, which is similar to but different from `with`, to activate a layer (their differences are described in Section 2.4) but no construct for deactivation.

When we simply call `simulate()`, it merely executes the methods in the base layer because *no layers are activated*.

By using the `ensure` construct, we can activate a layer during the evaluation of its body statement if not already done so. The following example simulates a phone call with the `Timing` layer activated.

```

ensure Timing {

```

```

    Connection c = simulate();
    System.out.println(c.getTime());
}

```

When simulating calls with the `Timing` layer activated, `complete` and `drop`, the partial methods defined in `Timing`, run instead of the ones in the base layer.

Note that activation of a layer also allows to call layer-introduced base methods. The above example calls `getTime` on the returned `Connection` object inside the `ensure` body, which is not possible without activating `Timing`.

As in most COP language extensions and also in ours, layer compositions are effective for the *dynamic extent* of the execution of the code block enclosed by their corresponding `ensure` statement⁴. So, layers form a stack and are (de)activated in the FIFO manner.

2.4 Order and Dependency between Layers

When we activate a layer while other layers are active, the partial methods in the last-activated layer override those in the earlier-activated layers. The following example explains this order of partial methods. The `Billing` layer below adds a feature that calculates and charges the cost of a phone call when the call ends.

```

layer Billing requires Timing {
    class Connection {
        void charge() {
            int cost = ...getTime();
            ...charge the cost on the caller... }
        void drop() { proceed(); charge(); }
    }
}

```

When we activate the `Timing` and `Billing` layers in this order, the partial method `drop` in `Billing` overrides the one in `Timing` because `Billing` is the most recently activated layer.

```

ensure Timing {
    ensure Billing {
        simulate();
    }
}

```

Since both partial methods have `proceed`, a call to `drop` in `simulate` will stop the timer, perform the base layer's behavior, and then calculate the cost of the call based on the duration of the call obtained through `getTime`.

Note that `Billing` depends on `Timing` as the `charge` method in `Billing` calls `getTime`, which is a layer-introduced base method by `Timing`. We declare this dependency by using the `requires` modifier in the layer declaration. In other words, the following statement, which activates `Billing` without activating `Timing` is incorrect and must be rejected statically. Our type system will detect such an erroneous layer activation.

```

ensure Billing { simulate(); } // incorrect

```

One might wonder why `ensure Billing` activates `Timing` at the same time because it is apparent that `Billing` requires `Timing`. Actually, one layer can depend on more than one layer, in which case it is not always clear in which order they should be activated.

2.5 Comparing `ensure` and `with`

As we mentioned above, many COP languages have a layer activation construct called `with`, which will make sure that the activated layer is always the first layer for which a method is searched. The

² The formal model omits fields defined in layers for simplicity. However, as far as type safety is concerned, supporting fields does not cause significant problems.

³ It is also possible to place those additional definitions in each class to be added, which is the so-called layer-in-class style.

⁴ Variants of COP languages allow to manage layer compositions on a per-instance basis [9, 13], which is left as future work in the paper.

difference becomes clear when the same layer is to be activated for the second time—the second activation will move the designated layer to the top of the stack of layers. For example,

```
with Timing {
  with Billing {
    with Timing { simulate(); }
  }
}
```

will invoke the partial method `drop` in `Timing` first⁵. Also, the effect of the outer `with Timing` is disabled until the body of the inner activation finishes. So, `proceed` from `Billing` calls a method in a base class. On the other hand, `ensure` will just make sure the existence of `Timing` without changing the order of already activated layers. So, the code where `with` is replaced by `ensure` is the same as the code without the inner activation of `Timing` (namely, the second code snippet in the last subsection).

The rearrangement of layers caused by `with`, however, destroys the layer ordering in which inter-layer dependency is respected. For example, there is no layer below `Billing`, which requires `Timing`, while the inner `with Timing` is executed. Similarly to double `with`, `without`, which deactivates a designated layer, destroys dependency-respecting layer ordering. Thus, for simplicity, we consider only `ensure` in this paper and leave a sound type system for `with` and `without` for future work. We propose `ensure` mainly to show that, by adopting `ensure`, simple dependency declaration is enough to design a sound type system. Comparisons of `with` and `ensure` from programmers' point of view are interesting but left for future work.

3. Syntax and Semantics of ContextFJ

In this section, we give the syntax and operational semantics of ContextFJ, which is an extension of Featherweight Java (FJ) [12] with (around-type) partial methods, `ensure` for layer activation, `proceed`, and `super`. As already mentioned, we omit fields defined in layers for simplicity. Thus, a layer is a set of partial methods; just as a set of classes is modeled as a class table—a mapping from class names to class definitions—in FJ, a set of layers will be modeled as a mapping from layer and class names to method definitions. The present version of ContextFJ replaces `with` and `without` for layer (de)activation found in the original version [11] with `ensure`. Except for the difference in the layer activation mechanism, the definitions are the same as in the original version.

3.1 Syntax

Let metavariables C, D , and E range over class names; L over layer names; f and g over field names; m over method names; and x and y over variables, which contain a special variable `this`. The abstract syntax of ContextFJ is given as follows:

CL	$::=$	<code>class C < C { $\bar{C}$ $\bar{f}$; K $\bar{M}$ }</code>	(classes)
K	$::=$	<code>C($\bar{C}$ $\bar{f}$) { super($\bar{f}$); this.$\bar{f}$ = $\bar{f}$; }</code>	(constructors)
M	$::=$	<code>C m($\bar{C}$ $\bar{x}$) { return e; }</code>	(methods)
e, d	$::=$	<code>x e.f e.m($\bar{e}$) new C($\bar{e}$)</code> <code> ensure L e</code> <code> proceed($\bar{e}$) super.m($\bar{e}$)</code> <code> new C($\bar{v}$) < C, $\bar{L}$, $\bar{L}$ > .m($\bar{e}$)</code>	(expressions)
v, w	$::=$	<code>new C($\bar{v}$)</code>	(values)

⁵ This code is admittedly artificial but in general, it is not unusual that one layer is to be activated twice in the dynamic extent of a method on code block execution. For example, it can happen that one method activates `Timing` and `Billing` in this order and then calls another method, which activates `Timing` (without knowing it has been activated already).

Following FJ, we use overlines to denote sequences: so, $\bar{f}$ stands for a possibly empty sequence $f_1, \dots, f_n$ and similarly for $\bar{C}$, $\bar{x}$, $\bar{e}$, and so on. Layers in a sequence are separated by semicolons. The empty sequence is denoted by $\bullet$. We also abbreviate pairs of sequences, writing “ $\bar{C}$ $\bar{f}$ ” for “ $C_1 f_1, \dots, C_n f_n$ ”, where n is the length of $\bar{C}$ and $\bar{f}$, and similarly “ $\bar{C}$ $\bar{f}$,” as shorthand for the sequence of declarations “ $C_1 f_1; \dots; C_n f_n$,” and “`this.f= $\bar{f}$ ;`” for “`this.f1= $f_1$ ; ...; this.fn= $f_n$ ;`”. We use commas and semicolons for concatenations. Sequences of field declarations, parameter names, layer names, and method declarations are assumed to contain no duplicate names.

A class definition CL consists of its name, its superclass name, field declarations $\bar{C}$ $\bar{f}$, a constructor K , and method definitions $\bar{M}$. A constructor K is a trivial one that takes initial values of all fields and sets them to the corresponding fields. Unlike the examples in the last section, we do not provide syntax for layers; partial methods are registered in a partial method table, explained below. A method M takes $\bar{x}$ as arguments and returns the value of expression e . As ContextFJ is a functional calculus like FJ, the method body consists of a single return statement and all constructs including `ensure` return values. An expression e can be a variable, field access, method invocation, object instantiation, layer activation/deactivation, `proceed`/`super` call, or a special expression `new C( $\bar{v}$ ) < C,  $\bar{L}$ ,  $\bar{L}$  > .m( $\bar{e}$ )`, which will be explained shortly. A value is an object of the form `new C( $\bar{v}$ )`.

The expression `new C( $\bar{v}$ ) < D,  $\bar{L}'$ ,  $\bar{L}$  > .m( $\bar{e}$ )`, where $\bar{L}'$ is assumed to be a prefix of $\bar{L}$, is a special run-time expression and not supposed to appear in classes. It basically means that m is going to be invoked on `new C( $\bar{v}$ )`. The annotation $\langle D, \bar{L}', \bar{L} \rangle$, which is used to model `super` and `proceed`, indicates where method lookup should start. More concretely, the triple $\langle D, (L_1; \dots; L_i), (L_1; \dots; L_n) \rangle$ ($i \leq n$) means that the search for the method definition will start from class D of layer L_i . So, for example, the usual method invocation `new C( $\bar{v}$ ) .m( $\bar{e}$ )` (without annotation) is semantically equivalent to `new C( $\bar{v}$ ) < C,  $\bar{L}$ ,  $\bar{L}$  > .m( $\bar{e}$ )`, where $\bar{L}$ is the active layers when this invocation is to be executed. This triple also plays the role of a “cursor” in the method lookup procedure and move across layers and base classes until the method definition is found. Figure 1 illustrates how a cursor proceeds. Notice that the third element is needed when the method is not found in D in any layer including the base: the search continues to layer L_n of D 's direct superclass.

With the help of this form, we can give a semantics of `super` and `proceed` by simple substitution-based reduction. For example, consider method invocation `new C() .m( $\bar{v}$ )`. As in FJ, this expression reduces to the method body where parameters and `this` are replaced with arguments $\bar{v}$ and the receiver `new C()`, respectively. Now, what happens to `super` in the method body? It cannot be replaced with the receiver `new C()` since it would confuse `this` and `super`. Method lookup for `super` is different from usual (virtual) method lookup in that it has to start from the direct superclass of the class in which `super` appears. So, if the method body containing `super.n()` is found in class D , then the search for n has to start from the direct superclass of D . To express this fact, we replace `super` with `new C() < E, ... >` where E is the direct superclass of D . We can deal with `proceed` similarly. Suppose the method body is found in layer L_i in D . Then, `proceed( $\bar{e}$ )` is replaced with `new C() < D, (L1; ...; Li-1),  $\bar{L}$  > .m( $\bar{e}$ )`, where $L_1; \dots; L_{i-1}$ are layers activated before L_i .

A ContextFJ program (CT, PT, e) consists of a class table CT , which maps a class name to a class definition, a partial method table PT , which maps a triple C, L , and m of class, layer, and method names to a method definition, and an expression, which corresponds to the body of the main method. In what follows, we

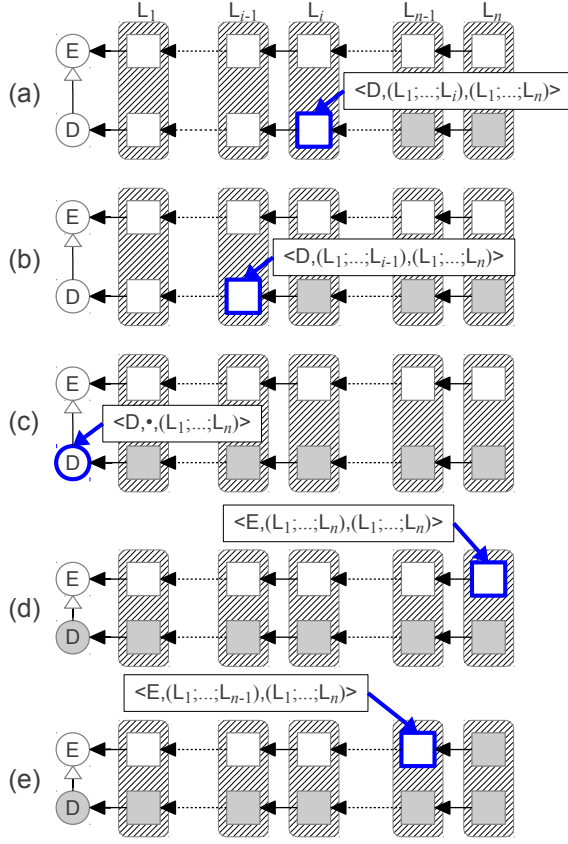


Figure 1. Method lookup in ContextFJ. A circle represents a base class and an arrow with a white head represents subclassing. A shaded round rectangle represent a layer, which contains sets of partial methods (represented by boxes) for base classes. Layers L_1 upto L_n have been activated in this order. The cursor, represented by an arrow pointing to a thick box, goes left first (subfigures (a)–(c)), goes one level up (to superclass E), and restarts lookup from the most recently activated layer L_n towards the left (subfigures (d)–(e)).

assume CT and PT to be fixed and satisfy the following sanity conditions:

1. $CT(C) = \text{class } C \dots$ for any $C \in \text{dom}(CT)$.
2. $\text{Object} \notin \text{dom}(CT)$.
3. For every class name C (except Object) appearing anywhere in CT , we have $C \in \text{dom}(CT)$;
4. There are no cycles in the transitive closure of the `extends` clauses.
5. $PT(m, C, L) = \dots m(\dots)\{\dots\}$ for any $(m, C, L) \in \text{dom}(PT)$.

We introduce dependency between layers expressed by `requires` clauses in the next section, where a type system is defined.

Lookup functions. As in FJ, we define a few auxiliary functions to look up field and method definitions. They are defined by the rules in Figure 2. The function $fields(C)$ returns a sequence $\bar{C} \bar{f}$ of pairs of a field name and its type by collecting all field declarations

$$fields(C) = \bar{C} \bar{f}$$

$$fields(\text{Object}) = \bullet \quad (\text{F-OBJECT})$$

$$\frac{\text{class } C \triangleleft D \{ \bar{C} \bar{f}; \dots \} \quad fields(D) = \bar{D} \bar{g}}{fields(C) = \bar{D} \bar{g}, \bar{C} \bar{f}} \quad (\text{F-CLASS})$$

$$mbody(m, C, \bar{L}', \bar{L}) = \bar{x}.e \text{ in } D, \bar{L}'$$

$$\frac{\text{class } C \triangleleft D \{ \dots C_0 \ m(\bar{C} \bar{x}) \{ \text{return } e; \} \dots \}}{mbody(m, C, \bullet, \bar{L}) = \bar{x}.e \text{ in } C, \bullet} \quad (\text{MB-CLASS})$$

$$\frac{PT(m, C, L_0) = C_0 \ m(\bar{C} \bar{x}) \{ \text{return } e; \} \quad mbody(m, C, (\bar{L}'; L_0), \bar{L}) = \bar{x}.e \text{ in } C, (\bar{L}'; L_0)}{mbody(m, C, \bullet, \bar{L}) = \bar{x}.e \text{ in } C, \bullet} \quad (\text{MB-LAYER})$$

$$\frac{\text{class } C \triangleleft D \{ \dots \bar{M} \} \quad m \notin \bar{M} \quad mbody(m, D, \bar{L}, \bar{L}') = \bar{x}.e \text{ in } D, \bar{L}'}{mbody(m, C, \bullet, \bar{L}) = \bar{x}.e \text{ in } D, \bar{L}'} \quad (\text{MB-SUPER})$$

$$\frac{PT(m, C, L_0) \text{ undefined} \quad mbody(m, C, \bar{L}', \bar{L}) = \bar{x}.e \text{ in } D, \bar{L}'}{mbody(m, C, (\bar{L}'; L_0), \bar{L}) = \bar{x}.e \text{ in } D, \bar{L}'} \quad (\text{MB-NEXTLAYER})$$

Figure 2. ContextFJ: Lookup functions.

from C and its superclasses. The function $mbody(m, C, \bar{L}_1, \bar{L}_2)$ returns the parameters and body $\bar{x}.e$ of method m in class C when the search starts from $\bar{L}_1$; the other layer names $\bar{L}_2$ keep track of the layers that are activated when the search initially started. It also returns the information on where the method has been found—the information will be used in reduction rules to deal with `proceed` and `super`. As we mentioned already, the method definition is searched for in class C in all activated layers and the base definition and, if there is none, then the search continues to C 's superclass. By reading the rules in a bottom-up manner, we can read off a recursive search procedure. The rule MB-CLASS means that m is found in the base class definition C (notice the third argument is $\bullet$) and the rule MB-LAYER that m is found in layer L_0 . The rule MB-SUPER, which deals with the situation where m is not found in a base class (expressed by the condition $m \notin \bar{M}$), motivates the fourth argument of $mbody$. The search goes on to C 's superclass D and has to take all activated layers into account; so, $\bar{L}$ is copied to the third argument in the premise. The rule MB-NEXTLAYER means that, if C of L_0 does not have m , then the search goes on to the next layer (in $\bar{L}'$) leaving the class name unchanged.

3.2 Operational Semantics

The operational semantics of ContextFJ is given by a reduction relation of the form $\bar{L} \vdash e \longrightarrow e'$, read “expression e reduces to e' under the activated layers $\bar{L}$ ”. Here, $\bar{L}$ do not contain duplicate names, as we noted earlier. The main rules are shown in Figure 3.

The first four rules are the main computation rules for field access and method invocation. The rule R-FIELD for field access is straightforward: $fields$ tells which argument to `new C(...)` corresponds to f_i . The next three rules are for method invocation. The rule R-INVKB is for method invocation where the cursor of the method lookup procedure has not been “initialized”; the cursor is set to be at the receiver's class and the currently activated layers. In the rule R-INVKB, the receiver is `new C( $\bar{v}$ )` and $\langle C', \bar{L}'', \bar{L}' \rangle$ is the location of the cursor. When the method body is found in the base-layer class C'' (denoted by “in $C'', \bullet$ ”), the whole expression

$$\begin{array}{c}
\frac{fields(C) = \bar{C} \ \bar{f}}{\bar{L} \vdash \text{new } C(\bar{v}) . f_i \longrightarrow v_i} \quad (\text{R-FIELD}) \\
\\
\frac{\bar{L} \vdash \text{new } C(\bar{v}) \langle C, \bar{L}, \bar{L} \rangle . m(\bar{w}) \longrightarrow e'}{\bar{L} \vdash \text{new } C(\bar{v}) . m(\bar{w}) \longrightarrow e'} \quad (\text{R-INVK}) \\
\\
\frac{mbody(m, C', \bar{L}'', \bar{L}') = \bar{x}.e_0 \text{ in } C'', \bullet \quad \text{class } C'' \triangleleft D\{\dots\}}{\bar{L} \vdash \text{new } C(\bar{v}) \langle C', \bar{L}'', \bar{L}' \rangle . m(\bar{w}) \longrightarrow} \\
\left[\begin{array}{l} \text{new } C(\bar{v}) \quad \quad \quad / \text{this,} \\ \bar{w} \quad \quad \quad \quad \quad / \bar{x}, \\ \text{new } C(\bar{v}) \langle D, \bar{L}', \bar{L}' \rangle / \text{super} \end{array} \right] e_0 \quad (\text{R-INVKB}) \\
\\
\frac{mbody(m, C', \bar{L}'', \bar{L}') = \bar{x}.e_0 \text{ in } C'', (\bar{L}'''; L_0) \quad \text{class } C'' \triangleleft D\{\dots\}}{\bar{L} \vdash \text{new } C(\bar{v}) \langle C', \bar{L}'', \bar{L}' \rangle . m(\bar{w}) \longrightarrow} \\
\left[\begin{array}{l} \text{new } C(\bar{v}) \quad \quad \quad / \text{this,} \\ \bar{w} \quad \quad \quad \quad \quad / \bar{x}, \\ \text{new } C(\bar{v}) \langle C'', \bar{L}'', \bar{L}' \rangle . m / \text{proceed,} \\ \text{new } C(\bar{v}) \langle D, \bar{L}', \bar{L}' \rangle \quad \quad \quad / \text{super} \end{array} \right] e_0 \quad (\text{R-INVKP}) \\
\\
\frac{ensure(L, \bar{L}) = \bar{L}' \quad \bar{L}' \vdash e \longrightarrow e'}{\bar{L} \vdash \text{ensure } L \ e \longrightarrow \text{ensure } L \ e'} \quad (\text{RC-ENSURE}) \\
\\
\frac{}{\bar{L} \vdash \text{ensure } L \ v \longrightarrow v} \quad (\text{R-ENSUREVAL}) \\
\\
\frac{\bar{L} \vdash e_0 \longrightarrow e_0'}{\bar{L} \vdash e_0 . f \longrightarrow e_0' . f} \quad (\text{RC-FIELD}) \\
\\
\frac{\bar{L} \vdash e_0 \longrightarrow e_0'}{\bar{L} \vdash e_0 . m(\bar{e}) \longrightarrow e_0' . m(\bar{e})} \quad (\text{RC-INVKRECV}) \\
\\
\frac{\bar{L} \vdash e_i \longrightarrow e_i'}{\bar{L} \vdash e_0 . m(\dots, e_i, \dots) \longrightarrow e_0 . m(\dots, e_i', \dots)} \quad (\text{RC-INVKARG}) \\
\\
\frac{\bar{L} \vdash e_i \longrightarrow e_i'}{\bar{L} \vdash \text{new } C(\dots, e_i, \dots) \longrightarrow \text{new } C(\dots, e_i', \dots)} \quad (\text{RC-NEW}) \\
\\
\frac{\bar{L} \vdash e_i \longrightarrow e_i'}{\bar{L} \vdash \text{new } C(\bar{v}) \langle C', \bar{L}', \bar{L}' \rangle . m(\dots, e_i, \dots) \longrightarrow} \\
\text{new } C(\bar{v}) \langle C', \bar{L}', \bar{L}' \rangle . m(\dots, e_i', \dots) \quad (\text{RC-INVKAARG})
\end{array}$$

Figure 3. ContextFJ: Reduction rules.

reduces to the method body where the formal parameters $\bar{x}$ and this are replaced by the actual arguments $\bar{w}$ and the receiver, respectively. Furthermore, super is replaced by the receiver with the cursor pointing to the superclass of C'' . The rule R-INVKB, which is similar to R-INVKB, deals with the case where the method body is found in layer L_0 in class C'' . In this case, proceed in the method body is replaced with the invocation of the same method, where the receiver's cursor points to the next layer $\bar{L}'''$ (dropping L_0). Since the meaning of the annotated invocation is not affected by the layers in the context (note that $\bar{L}$ are not significant in these rules), the substitution for super and proceed also means that their meaning is the same throughout a given method body, even when they appear inside ensure . Note that, unlike FJ, reduction in ContextFJ is call-by-value, requiring receivers and arguments to be values. This

evaluation strategy reflects the fact that arguments should be evaluated under the caller-side context.

The following rules are related to context manipulation. The rule RC-ENSURE means that e in $\text{ensure } L \ e$ should be executing by activating L . The auxiliary function $ensure(L, \bar{L})$, defined by:

$$\begin{aligned}
ensure(L, \bar{L}) &= \bar{L} \quad (\text{if } L \in \bar{L}) \\
ensure(L, \bar{L}) &= \bar{L}; L \quad (\text{otherwise})
\end{aligned}$$

adds L to the end of $\bar{L}$ if L is not in $\bar{L}$ (or returns $\bar{L}$ otherwise). It only ensures the existence of L without changing the order of already activated layers. As we have already discussed, this is in contrast to with statements, which other COP languages usually provide. A with statement activates a layer but, if the layer is already activated in the middle of the layer stack, it will be moved to the top, changing the order of activated layers.

The next rule R-ENSUREVAL means that, once the evaluation of the body of ensure is finished, it returns the value of the body.

There are other trivial congruence rules to allow subexpressions to reduce. Note that ContextFJ reduction is call by value, but the order of reduction of subexpressions is unspecified.

4. Type System

In this section, we give a type system for ContextFJ. As usual, the role of a type system is to guarantee type soundness, namely, to prevent statically field-not-found and method-not-found errors from happening at run time. In ContextFJ, it also means that a type system should ensure that every $\text{proceed}()$ or $\text{super}()$ call succeeds.

A key idea in this type system is to keep track of an (under-) approximation of layers activated at each program point. Such approximation gives information on what methods are made available by layer activation (in addition to those defined in the base layer). Roughly speaking, a type judgment for an expression is of the form $\Lambda; \Gamma \vdash e : C$, where Γ is a type environment, which records types of variables, and a set Λ of layers that are assumed to be activated when e is evaluated. This approximated layer information Λ will be used to typecheck a method invocation expression. For example, the call to getTime in partial method charge in Billing in Section 2 is valid because layer Timing provides getTime . It could be represented by a type judgment

$$\{\text{Timing}\}; \text{this} : \text{Connection} \vdash \text{this.getTime}() : \text{int}$$

where $\bullet$ is the empty type environment. On the other hand,

$$\emptyset; \text{this} : \text{Connection} \vdash \text{this.getTime}() : \text{int}$$

is not a valid type judgment because no layer is assumed (represented by $\emptyset$) and the base definition of Connection does not give method getTime .

As we have already discussed, a program will be typechecked with information on dependency between layers. Let $\mathcal{R}$ be a binary relation on layer names; $(L_1, L_2) \in \mathcal{R}$ intuitively means that layer L_1 requires L_2 , that is, when L_1 is to be activated, L_2 has to have been activated already. The dependency relation $\mathcal{R}$ is used in typechecking the entry point of each partial method: when a partial method in layer L is typechecked, all the layers related to L by $\mathcal{R}$ are assumed in a type judgment for the body of the partial method. In what follows, we assume a fixed dependency relation and write $L \text{ req } \Lambda$, read “layer L requires layers Λ ”, when $\Lambda = \{L' \mid (L, L') \in \mathcal{R}\}$.

The type system also has to guarantee that the layers assumed in type judgments are really activated at run time. It is guaranteed by the typing rule for $\text{ensure } L$ below:

$$\frac{L \text{ req } \Lambda' \quad \Lambda' \subseteq \Lambda \quad \Lambda \cup \{L\}; \Gamma \vdash e_0 : C_0}{\Lambda; \Gamma \vdash \text{ensure } L \ e_0 : C_0}$$

First, the third premise means that layer L (which is to be activated) can be assumed in addition to the already activated layers Λ when typechecking the body e_0 of `ensure`. Second, the first two premises guarantee that layers Λ' that L requires have been already activated (that is, are included in Λ) when L is activated. Since such “activated-before” relation is preserved (remember that layers are always manipulated in the FIFO manner) during program execution, all calls (including `proceed`) from L will succeed.

To summarize key technical points, (1) a type judgment is augmented with approximation of activated layers; (2) the method type lookup function takes activated layers into account; and (3) the typing rule for `ensure` guarantees the assumed layers are really activated at run time. Keeping these in mind, we proceed to a formal type system.

4.1 Subtyping

The subtyping relation $C \prec D$, which is the same as that in FJ, is defined as the reflexive and transitive closure of the `extends` clauses.

$$\begin{array}{c} \overline{C \prec C} \quad (\text{S-REFL}) \\[10pt] \frac{C \prec D \quad D \prec E}{C \prec E} \quad (\text{S-TRANS}) \\[10pt] \frac{\text{class } C \prec D \{ \dots \}}{C \prec D} \quad (\text{S-EXTENDS}) \end{array}$$

4.2 Method type lookup

We define another auxiliary function to look up the type of a method. The function $mtype(m, C, \Lambda_1, \Lambda_2)$, defined by the rules below, takes a method name m , a class name C , and two sets Λ_1 and Λ_2 of layer names and returns a pair, written $\bar{C} \rightarrow C_0$, of argument types $\bar{C}$ and a return type C_0 . The sets Λ_1 and Λ_2 stand for statically known activated layers, in which m is looked for. The first set is used to look up m in C , whereas the second is used when m is not found in C and the search continues to C ’s superclass. Although these two sets are the same in most uses of $mtype$ (in fact, we write $mtype(m, C, \Lambda)$ for $mtype(m, C, \Lambda, \Lambda)$), we need to distinguish them for typing `proceed`, because `proceed` cannot proceed to where it is executed, but may proceed to a method of the same name in a superclass in the same layer.

$$\begin{array}{c} \frac{\text{class } C \prec D \{ \dots C_0 \ m(\bar{C} \ \bar{x}) \{ \text{return } e; \} \dots \}}{mtype(m, C, \Lambda_1, \Lambda_2) = \bar{C} \rightarrow C_0} \quad (\text{MT-CLASS}) \\[10pt] \frac{L \in \Lambda_1 \quad PT(m, C, L) = C_0 \ m(\bar{C} \ \bar{x}) \{ \text{return } e; \}}{mtype(m, C, \Lambda_1, \Lambda_2) = \bar{C} \rightarrow C_0} \quad (\text{MT-PMETHOD}) \\[10pt] \frac{\text{class } C \prec D \{ \dots \bar{M} \} \quad m \notin \bar{M} \quad \forall L \in \Lambda_1. PT(m, C, L) \text{ undefined} \quad mtype(m, D, \Lambda_2, \Lambda_2) = \bar{C} \rightarrow C_0}{mtype(m, C, \Lambda_1, \Lambda_2) = \bar{C} \rightarrow C_0} \quad (\text{MT-SUPER}) \end{array}$$

The rule MT-CLASS is used when m is defined in the base layer; the rule MT-PMETHOD is used when m is defined in one of the activated layers; and the rule MT-SUPER is used when m is not defined in class C . Notice that, in the premise of MT-SUPER, both third and fourth arguments to $mtype$ are Λ_2 .

Remark. Note that these rules by themselves do *not* define $mtype$ as a (set-theoretic) function on $(m, C, \Lambda_1, \Lambda_2)$ in the sense that it may be the case that $mtype(m, C, \Lambda_1, \Lambda_2) = \bar{C} \rightarrow C_0$ and $mtype(m, C, \Lambda_1, \Lambda_2) = \bar{D} \rightarrow D_0$ are derived but $\bar{C}, C_0 \neq \bar{D}, D_0$. We

later enforce the signature of a method in a class to be the same in every layer (including the base) by a typing rule for a program.

4.3 Typing

A *type environment*, denoted by Γ , is a finite mapping from variables to class names, which are also types in ContextFJ. We write $\bar{x} : \bar{C}$ for a type environment Γ such that $dom(\Gamma) = \{\bar{x}\}$ and $\Gamma(\bar{x}_i) = C_i$ for any i . We use $\mathcal{L}$ to stand for a *location*, which is either $\bullet$ (the main expression), $C.m$ (the body of method m in class C in the base layer), or $L.C.m$ (the body of method m in class C in layer L). The typing rules for expressions, methods, classes, and programs are shown in Figure 4.

Expression typing. A type judgment for expressions is of the form $\mathcal{L}; \Lambda; \Gamma \vdash e : C$, read “expression e is given type C under context Γ , location $\mathcal{L}$, and activated layers Λ .” Activated layers Λ are supposed to be a subset of layers actually activated when the expression is evaluated at run time. Also note that Λ is a set rather than a sequence; it means that the type system does not know in what order layers are activated.

The first four rules T-VAR for variables, T-INVK for method invocation, T-FIELD for field access, T-NEW for object instantiation are mostly straightforward adaptations of those of FJ. Note that, in T-INVK, Λ is used to look up the type of method m in C . As discussed already, the rule T-ENSURE for `ensure` requires that layers Λ' that the newly activated layer L requires be activated; the body e is checked under the assumption in which L is added to the set of activated layers.

The next three rules are concerned about `super` and `proceed` calls. The rule T-SUPERB is for `super` in a method in a base class C , as represented in the location of the type judgment. The method type is retrieved as if the receiver type is E , which is the direct superclass of C , where the present expression appears. The set of activated layers passed to $mtype$ is assumed to be empty because base classes cannot assume any layers to be activated. Note that it is always empty no matter how many `ensures` surround this `super` call. This corresponds to the operational semantics that the behavior of `super` is not affected by `ensures` in the callee. In the other two rules, the sets of layers given to $mtype$ have also nothing to do with that in type judgments. The rule T-SUPERP for `super` in a partial method defined in $L.C$ is similar. The only difference is that it can assume the existence of layers Λ' that L requires and also L itself. The rule T-PROCEED for `proceed` is also similar; the method name to be looked up is taken from the location $L.C.m$. Note that the third argument to $mtype$ is just Λ' , which means that a `proceed` call cannot proceed to the same method recursively (but can proceed to a method of the same name in a superclass of the same layer).

We defer the typing rule for method invocation `new C( $\bar{v}$ ) <D,  $\bar{L}$ ,  $\bar{L}'$ >.m( $\bar{w}$ )` on an object with a cursor to the discussion about type soundness.

Method/class/program typing. A type judgment for methods is of the form $M \text{ ok in } C$ (for methods in a base class) or $M \text{ ok in } L.C$ (for partial methods), read “method M is well formed in C (or $L.C$, respectively).” The typing rules T-METHOD and T-PMETHOD are straightforward. Both rules check that the method body is well typed under the type environment that formal parameters $\bar{x}$ are given declared types $\bar{C}$ and `this` is given the name of the class name where the method appears. The type of the method body has to be a subtype of the declared return type. For methods in a base class, the method body has to be well typed without assuming any activated layers, whereas, for partial methods, layers (Λ) that the current layer (L) requires can be assumed (as well as the current layer itself). Unlike FJ, valid method overriding is not checked here because it requires the whole program to check.

Expression typing: $\boxed{\mathcal{L}; \Lambda; \Gamma \vdash e : C}$

$$\frac{(\Gamma = \bar{x} : \bar{C})}{\mathcal{L}; \Lambda; \Gamma \vdash x_i : C_i} \quad (\text{T-VAR})$$

$$\frac{\mathcal{L}; \Lambda; \Gamma \vdash e_0 : C_0 \quad \text{fields}(C_0) = \bar{C} \ \bar{f}}{\mathcal{L}; \Lambda; \Gamma \vdash e_0.f_i : C_i} \quad (\text{T-FIELD})$$

$$\frac{\mathcal{L}; \Lambda; \Gamma \vdash e_0 : C_0 \quad \text{mtype}(\mathbf{m}, C_0, \Lambda) = \bar{D} \rightarrow D_0 \quad \mathcal{L}; \Lambda; \Gamma \vdash \bar{e} : \bar{E} \quad \bar{E} <: \bar{D}}{\mathcal{L}; \Lambda; \Gamma \vdash e_0.\mathbf{m}(\bar{e}) : D_0} \quad (\text{T-INVK})$$

$$\frac{\text{fields}(C_0) = \bar{D} \ \bar{f} \quad \mathcal{L}; \Lambda; \Gamma \vdash \bar{e} : \bar{C} \quad \bar{C} <: \bar{D}}{\mathcal{L}; \Lambda; \Gamma \vdash \text{new } C_0(\bar{e}) : C_0} \quad (\text{T-NEW})$$

$$\frac{\text{L req } \Lambda' \quad \Lambda' \subseteq \Lambda \quad \mathcal{L}; \Lambda \cup \{\text{L}\}; \Gamma \vdash e_0 : C_0}{\mathcal{L}; \Lambda; \Gamma \vdash \text{ensure L } e_0 : C_0} \quad (\text{T-ENSURE})$$

$$\frac{\text{class C} \triangleleft \text{E} \{ \dots \} \quad \text{mtype}(\mathbf{m}', \text{E}, \emptyset) = \bar{D} \rightarrow D_0 \quad \text{C.m}; \Lambda; \Gamma \vdash \bar{e} : \bar{E} \quad \bar{E} <: \bar{D}}{\text{C.m}; \Lambda; \Gamma \vdash \text{super.m}'(\bar{e}) : D_0} \quad (\text{T-SUPERB})$$

$$\frac{\text{class C} \triangleleft \text{E} \{ \dots \} \quad \text{L req } \Lambda' \quad \text{mtype}(\mathbf{m}', \text{E}, \Lambda' \cup \{\text{L}\}) = \bar{D} \rightarrow D_0 \quad \text{L.C.m}; \Lambda; \Gamma \vdash \bar{e} : \bar{E} \quad \bar{E} <: \bar{D}}{\text{L.C.m}; \Lambda; \Gamma \vdash \text{super.m}'(\bar{e}) : D_0} \quad (\text{T-SUPERP})$$

$$\frac{\text{L req } \Lambda' \quad \text{mtype}(\mathbf{m}, \text{C}, \Lambda', \Lambda' \cup \{\text{L}\}) = \bar{D} \rightarrow D_0 \quad \text{L.C.m}; \Lambda; \Gamma \vdash \bar{e} : \bar{E} \quad \bar{E} <: \bar{D}}{\text{L.C.m}; \Lambda; \Gamma \vdash \text{proceed}(\bar{e}) : D_0} \quad (\text{T-PROCEED})$$

Method/class typing: $\boxed{\text{M ok in C}}$

$$\frac{\text{C.m}; \emptyset; \bar{x} : \bar{C}, \text{this} : \text{C} \vdash e_0 : D_0 \quad D_0 <: C_0}{C_0 \text{ m}(\bar{C} \ \bar{x}) \{ \text{return } e_0; \} \text{ ok in C}} \quad (\text{T-METHOD})$$

$\boxed{\text{M ok in L.C}}$

$$\frac{\text{L req } \Lambda \quad \text{L.C.m}; \Lambda \cup \{\text{L}\}; \bar{x} : \bar{C}, \text{this} : \text{C} \vdash e_0 : D_0 \quad D_0 <: C_0}{C_0 \text{ m}(\bar{C} \ \bar{x}) \{ \text{return } e_0; \} \text{ ok in L.C}} \quad (\text{T-PMETHOD})$$

$\boxed{\text{CL ok}}$

$$\frac{K = \text{C}(\bar{D} \ \bar{g}, \bar{C} \ \bar{f}) \{ \text{super}(\bar{g}); \text{this}.\bar{f} = \bar{f}; \} \quad \text{fields}(D) = \bar{D} \ \bar{g} \quad \bar{\text{M}} \text{ ok in C}}{\text{class C} \triangleleft \text{D} \{ \bar{C} \ \bar{f}; K \ \bar{\text{M}} \} \text{ ok}} \quad (\text{T-CLASS})$$

Valid overriding: $\boxed{\text{noconflict}(\text{L}_1, \text{L}_2)} \quad \boxed{\text{override}^h(\text{L}, \text{C})} \quad \boxed{\text{override}^v(\text{C}, \text{D})}$

$$\frac{\forall \mathbf{m}, \text{C}. PT(\mathbf{m}, \text{C}, \text{L}_1) = C_0 \text{ m}(\bar{C} \ \bar{x}) \{ \dots \} \text{ and } PT(\mathbf{m}, \text{C}, \text{L}_2) = D_0 \text{ m}(\bar{D} \ \bar{y}) \{ \dots \}, \text{ then } \bar{C}, C_0 = \bar{D}, D_0}{\text{noconflict}(\text{L}_1, \text{L}_2)}$$

$$\frac{\forall \mathbf{m}. \text{ if } CT(\text{C}) = \text{class C} \triangleleft \text{D} \{ \dots C_0 \text{ m}(\bar{C} \ \bar{x}) \{ \dots \} \dots \} \text{ and } PT(\mathbf{m}, \text{C}, \text{L}) = D_0 \text{ m}(\bar{D} \ \bar{y}) \{ \dots \}, \text{ then } \bar{C}, C_0 = \bar{D}, D_0}{\text{override}^h(\text{L}, \text{C})}$$

$$\frac{\forall \mathbf{m}. \text{ if } \text{mtype}(\mathbf{m}, \text{C}, \text{dom}(PT), \text{dom}(PT)) = \bar{C} \rightarrow C_0 \text{ and } \text{mtype}(\mathbf{m}, \text{D}, \text{dom}(PT), \text{dom}(PT)) = \bar{D} \rightarrow D_0 \text{ and } \bar{C} <: \bar{D}, \text{ then } \bar{C} = \bar{D} \text{ and } C_0 <: D_0}{\text{override}^v(\text{C}, \text{D})}$$

Program typing: $\boxed{\vdash (CT, PT, \mathbf{e}) : C}$

$$\frac{\begin{array}{l} \forall \text{C} \in \text{dom}(CT). CT(\text{C}) \text{ ok} \quad \forall (\mathbf{m}, \text{C}, \text{L}) \in \text{dom}(PT). PT(\mathbf{m}, \text{C}, \text{L}) \text{ ok in L.C} \\ \bullet; \emptyset; \bullet \vdash \mathbf{e} : C \\ \forall \text{L}_1, \text{L}_2 \in \text{dom}(PT). \text{noconflict}(\text{L}_1, \text{L}_2) \\ \forall \text{C} \in \text{dom}(CT), \text{L} \in \text{dom}(PT). \text{override}^h(\text{L}, \text{C}) \quad \forall \text{C}, \text{D} \in \text{dom}(CT). \text{override}^v(\text{C}, \text{D}) \end{array}}{\vdash (CT, PT, \mathbf{e}) : C} \quad (\text{T-PROG})$$

Figure 4. ContextFJ: Typing rules.

A class is well formed (written CL OK) when the constructor matches the field declarations and all methods are well formed.

Finally, a program is well formed when all classes are well formed, all partial methods are well formed, and the main expression is well typed under the empty assumption. The other conditions mean that no two layers provide conflicting methods and all method overriding (by a subclass or a partial method in a layer) is valid. The predicate $\text{noconflict}(L_1, L_2)$ means that there are no conflicting methods in L_1 and L_2 ; the predicate $\text{override}^h(L, C)$ (h stands for “horizontally”) mean that all overriding partial methods in L have the same signatures as the corresponding methods in C ; and the predicate $\text{override}^v(C, D)$ (v stands for “vertically”) mean that method override by a subclass C of D is valid. Note that when noconflict and override^h hold for any combination of layers and classes, mtype is a (set-theoretic) function.

It is interesting to see that covariant overriding of the return type is allowed only by a based method in a subclass. In fact, we cannot allow covariant overriding by a partial method because the order of layer composition varies at run time. The last premise checks that a method in a subclass correctly overrides a method of the same name in a superclass; we can allow covariant overriding of the return type here.

Note that, unlike noconflict and override^h , checking override^v for a given pair of classes needs type information on *all* the layers, as $\text{dom}(PT)$ is used for the third and fourth argument to mtype . We need to take all the layers into account in case a subclass C defines m , which is not present in its superclass D , and some layer adds (not overrides) m to D .

4.4 Type Soundness

This type system is sound with respect to the operational semantics given in the last section. Type soundness is shown via subject reduction and progress properties [16, 22]. In order to state these properties, though, we need to formalize the condition when a statically assumed layer set matches a run-time layer configuration. We write $\bar{L} \text{ wf}$, read “a run-time layer configuration $\bar{L}$ is well formed”, which is defined as follows:

$$\frac{\bullet \text{ wf} \quad \bar{L} \text{ wf} \quad L \text{ req } \Lambda \quad \Lambda \subseteq \{\bar{L}\}}{\bar{L}; L \text{ wf}}$$

The first rule means that the empty sequence of layers is well formed and the second that a sequence $\bar{L}; L$ is well formed if the prefix $\bar{L}$ is well formed and the layers Λ that L requires have already been activated ($\Lambda \subseteq \{\bar{L}\}$).⁶

We also need to give a typing rule for expressions that appear only at run time, i.e., method invocation on an object with a cursor.

$$\frac{\begin{array}{l} \bar{L}' \text{ is a prefix of } \bar{L}'' \quad \bar{L}'' \text{ wf} \\ \text{fields}(C_0) = \bar{D} \quad \bar{L}; \Lambda; \Gamma \vdash \bar{v} : \bar{C} \quad \bar{C} <: \bar{D} \\ C_0 <: D \quad \text{mtype}(m, D, \{\bar{L}'\}, \{\bar{L}''\}) = \bar{F} \rightarrow F_0 \\ \bar{L}; \Lambda; \Gamma \vdash \bar{e} : \bar{E} \quad \bar{E} <: \bar{F} \end{array}}{\bar{L}; \Lambda; \Gamma \vdash \text{new } C_0(\bar{v}) <D, \bar{L}', \bar{L}''> . m(\bar{e}) : F_0} \quad (\text{T-INVKA})$$

This rule is basically thought as a combination of T-INVK and T-NEW. One notable point is that the cursor information D , $\bar{L}'$, and $\bar{L}''$ is used to look up the type of m (instead of the receiver’s run-time class C_0 and the assumed set of activated layers Λ).

Now, the soundness theorem is stated below. Proofs of subject reduction and progress are given in Appendix A; type soundness follows easily from them.

⁶ The notation $\{\bar{L}\}$ is used to ignore the order in a sequence; formally, it denotes the set consisting of all elements of $\bar{L}$.

THEOREM 1 (Subject Reduction). *Suppose given class and partial method tables are well-formed. If $\bullet; \{\bar{L}\}; \Gamma \vdash e : C$ and $\bar{L} \text{ wf}$ and $\bar{L} \vdash e \longrightarrow e'$, then $\bullet; \{\bar{L}\}; \Gamma \vdash e' : D$ for some D such that $D <: C$.*

THEOREM 2 (Progress). *Suppose given class and partial method tables are well-formed. If $\bullet; \{\bar{L}\}; \bullet \vdash e : C$ and $\bar{L} \text{ wf}$, then either e is a value or $\bar{L} \vdash e \longrightarrow e'$ for some e' .*

THEOREM 3 (Type Soundness). *If $\vdash (CT, PT, e) : C$ and e reduces to a normal form, then e is $\text{new } D(\bar{v})$ for some $\bar{v}$ and D such that $D <: C$.*

5. Discussion

We have formalized a type system for dynamic layer composition and proved its soundness. One key idea is to approximate activated layers at each program point with the help of explicitly declared dependencies between layers. Our result shows that such a dependency relation is sufficient for a particular layer activation mechanism, namely *ensure*, which does not change the order of already activated layers.

We discuss other possible layer (de)activation mechanisms. Many COP languages have *with* $L \{ \dots \}$, which always activates L as the first layer to be executed by *changing the order of layers* when L has been already activated, and *without* $L \{ \dots \}$, which temporarily deactivates L during the execution of the body. One motivation for *with* is that a programmer may want to ensure partial methods in the activated layer are executed first. The present type system is not sufficient for such order-changing layer manipulation because it statically estimates only a *lower bound* of activated layers (whose ordering is lost). For example, consider *without* $L_1 \{ \dots \}$ when Λ in the type judgment for this expression is $\{L_1, L_2\}$. Since $\{L_1, L_2\}$ gives only a lower-bound, the run-time layer configuration can be $L_1; L_2$, $L_2; L_1$, or even $L_2; L_1; L_3$. It is unsafe, however, to remove L_1 from $L_2; L_1; L_3$ if L_3 requires L_1 . Similarly, *with* L_1 may cause trouble when the run-time configuration is $L_2; L_1; L_3$ and L_3 requires L_1 : because it will move L_1 and change the configuration to $L_2; L_3; L_1$, where L_3 ’s requirement is no longer satisfied. In short, *with* and *without* are difficult because they may break the well-formedness condition on a run-time layer configuration.

The present type system works if layer manipulation constructs do not break well-formedness. One compromise between *ensure* and *with* could be to activate the designated layer always as the first one and *leave the already activated one as it is*, resulting in two copies of the same layer in the run-time layer configuration. However, it may cause a partial method in a layer to run twice for one method call, leaving programmers surprised (especially when layers have side effects, which is the case in real COP language extensions).

Related Work A layer in COP languages is essentially a set of mixins (or a mixin layer [19]), which can be composed or decomposed at run time. An idea similar to our *requires* clauses can be found in type systems for mixins [4, 8, 14], where a mixin specifies the interface of classes to be composed. Our *require* clauses can be considered an extension of this idea to a set of interfaces. In a language with mixins, however, once an object is instantiated, composed mixins are never “deactivated”. Nevertheless, as this paper shows, a similar idea works—to some extent—even for dynamic (de)composition. Our *requires* clauses are not very flexible, because one has to specify a *single* set of layers, which are tied to specific implementations. So, one cannot express dependency like “this layer requires either L_1 or L_2 ” or even “this layer requires any layer that provides a method of this signature.” It would not be hard to extend our type system so that dependency can be specified via a

set of method signatures [4, 14] or Java-like interfaces adapted for layers.

In fact, Clarke and Sergey [5] independently formalize a core language (also called ContextFJ) for context-oriented programming (with both `with` and `without` but no inheritance) and develop such a type system. In their type system, each partial/base method (rather than a layer) is equipped with a set of the signatures of the methods that it may call as dependency information, which is very fine-grained. However, their type system turns out to be unsound because it does not handle removal of layers (caused by `without`) properly (personal communication with Clarke and Sergey).

Feature-oriented programming (FOP) [3] and delta-oriented programming (DOP) [17] also advocate the use of layers or delta modules respectively to describe behavioral variations. In both approaches, however, composition with base classes is only *static*, namely, happens at compile time. Their type systems [1, 7, 18] also use explicitly declared dependencies, often called feature models, for modular typechecking. The languages to specify dependencies between layers or delta modules are richer than ours, which is just a set of `requires` clauses. It is interesting future work to incorporate feature models in our type system.

Typestate checking [20] is a technique to keep track of state transition of computational resources (such as files and sockets) during program execution statically. Our type system, which might be considered a kind of typestate checking of layer configurations, is simpler than typestate checking in that there is only one global resource but inexact because only an approximation of a layer configuration can be obtained. Note that typestate checking is not directly applicable because it is usually based on a finite state transition system, whereas the state space of layer configurations is infinite.

Acknowledgments. Comments from anonymous reviewers of FOOL2012 helped us improve the presentation of the paper. We thank Dave Clarke and Ilya Sergey for answering questions on their work and members of the Kumiki project for fruitful discussions on this subject. This work was supported in part by Grant-in-Aid for Scientific Research No. 22240002 (Igarashi and Masuhara).

References

- [1] Sven Apel, Christian Kästner, and Christian Lengauer. Feature Featherweight Java: a calculus for feature-oriented programming and stepwise refinement. In *Proc. of GPCE*, 2008. doi:10.1145/1449913.1449931.
- [2] Malte Appeltauer, Robert Hirschfeld, Michael Haupt, and Hidehiko Masuhara. ContextJ: Context-oriented programming with Java. *Computer Software*, 28(1):272–292, January 2011.
- [3] Don Batory, Jacob Neal Sarvela, and Axel Rauschmayer. Scaling stepwise refinement. *IEEE Transactions on Software Engineering*, 2004. doi:10.1109/TSE.2004.23.
- [4] Viviana Bono, Amit Patel, Vitaly Shmatikov, and John C. Mitchell. A core calculus of classes and mixins. In *Proc. of ECOOP’99*, 1999.
- [5] Dave Clarke and Ilya Sergey. A semantics for context-oriented programming with layers. In *Proc. of 1st International Workshop on Context-Oriented Programming (COP’09)*, Genova, Italy, 2009.
- [6] Pascal Costanza and Robert Hirschfeld. Language constructs for context-oriented programming - an overview of ContextL. In *Proc. of DLS*, 2005. doi:10.1145/1146841.1146842.
- [7] Benjamin Delaware, William Cook, and Don Batory. A machine-checked model of safe composition. In *Proc. of International Workshop on Foundations of Aspect-Oriented Languages*, 2009. doi:10.1145/1509837.1509846.
- [8] Matthew Flatt, Shriram Krishnamurthi, and Matthias Felleisen. Classes and mixins. In *Proc. of ACM POPL*, pages 171–183, 1998.
- [9] Robert Hirschfeld, Pascal Costanza, and Michael Haupt. An introduction to context-oriented programming with ContextS. In *Proc. of GTTSE*, 2008.
- [10] Robert Hirschfeld, Pascal Costanza, and Oscar Nierstrasz. Context-oriented programming. *Journal of Object Technology*, 2008.
- [11] Robert Hirschfeld, Atsushi Igarashi, and Hidehiko Masuhara. ContextFJ: A minimal core calculus for context-oriented programming. In *Proc. of International Workshop on Foundations of Aspect-Oriented Languages (FOAL2011)*, Pernambuco, Brazil, March 2011.
- [12] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight Java: A minimal core calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems*, 2001. doi:10.1145/503502.503505.
- [13] Tetsuo Kamina, Tomoyuki Aotani, and Hidehiko Masuhara. EventCJ: A context-oriented programming language with declarative event-based context transition. In *Proc. of AOSD*, 2011.
- [14] Tetsuo Kamina and Tetsuo Tamai. A core calculus for mixin-types. In *Proc. of FOOL11*, 2004.
- [15] Jens Lincke, Malte Appeltauer, Bastian Steinert, and Robert Hirschfeld. An open implementation for context-oriented layer composition in ContextJS. *Science of Computer Programming, Special Issue on Software Evolution*, 2010. doi:10.1016/j.scico.2010.11.013.
- [16] Benjamin C. Pierce. *Types and Programming Languages*. The MIT Press, 2001.
- [17] Ina Schaefer, Lorenzo Bettini, Viviana Bono, Ferruccio Damiani, and Nico Tanzarella. Delta-oriented programming of software product lines. In *Proc. of Software Product Line Conference*, 2010.
- [18] Ina Schaefer, Lorenzo Bettini, and Ferruccio Damiani. Compositional type-checking for delta-oriented programming. In *Proc. of AOSD*, 2011.
- [19] Yannis Smaragdakis and Don Batory. Implementing layered designs with mixin layers. In *Proc. of ECOOP’98*, pages 550–570, 1998.
- [20] Robert E. Strom and Shaula Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, SE-12(1):157–171, January 1986.
- [21] The AspectJ Team. The AspectJ programming guide. <http://www.eclipse.org/aspectj/doc/released/progguide/index.html>. Site visited Aug. 12, 2012.
- [22] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, November 1994.

A. Proofs

LEMMA 1 (Weakening).

1. If $\mathcal{L}; \Lambda; \Gamma \vdash e : C$, then $\mathcal{L}; \Lambda; \Gamma, x : D \vdash e : C$.
2. If $\mathcal{L}; \Lambda; \Gamma \vdash e : C$, then $\mathcal{L}; \Lambda \cup \{L\}; \Gamma \vdash e : C$.

Proof: By straightforward induction on $\mathcal{L}; \Lambda; \Gamma \vdash e : C$. $\square$

LEMMA 2 (Strengthening for values). If $\mathcal{L}; \Lambda; \Gamma \vdash v : C$, then $\mathcal{L}'; \Lambda'; \Gamma' \vdash v : C$.

Proof: By straightforward induction on $\mathcal{L}; \Lambda; \Gamma \vdash v : C$. $\square$

LEMMA 3. If $\text{fields}(C) = \bar{C} \ \bar{f}$ and $D <: C$, then $\text{fields}(D) = \bar{C} \ \bar{f}, \bar{D} \ \bar{g}$ for some $\bar{D}$ and $\bar{g}$.

Proof: By straightforward induction on $D <: C$. $\square$

LEMMA 4. If $\text{mtype}(m, C, \Lambda) = \bar{D} \rightarrow D_0$ and $D <: C$, then $\text{mtype}(m, D, \Lambda) = \bar{D} \rightarrow E_0$ and $E_0 <: D_0$ for some E_0 .

Proof: By induction on $D <: C$. $\square$

LEMMA 5 (Substitution). If $\mathcal{L}; \Lambda; \Gamma, \bar{x} : \bar{C} \vdash e_0 : C_0$ and $\mathcal{L}; \Lambda; \Gamma \vdash \bar{v} : \bar{D}$ and $\bar{D} <: \bar{C}$, then $\mathcal{L}; \Lambda; \Gamma \vdash [\bar{v}/\bar{x}]e_0 : D_0$ and $D_0 <: C_0$ for some D_0 .

Proof: By induction on $\mathcal{L}; \Lambda; \Gamma, \bar{x} : \bar{C} \vdash e_0 : C_0$. $\square$

LEMMA 6 (Substitution for super and proceed).

1. If $L \text{ req } \Lambda'$ and $\Lambda' \subseteq \{\bar{L}'\}$ and $\bar{L}'; L$ is a prefix of $\bar{L}$ and $\bar{L}$ wf and $L.C.m; \Lambda; \Gamma \vdash e_0 : C_0$ and $D_0 <: C$ and $\text{fields}(D_0) = \bar{D} \ \bar{f}$ and $\bullet; \{\bar{L}\}; \Gamma \vdash \bar{v} : \bar{E}$ and $\bar{E} <: \bar{D}$ and $\text{class } C \triangleleft D$, then $\bullet; \{\bar{L}\}; \Gamma \vdash S e_0 : C_0$ where

$$S = \left[\begin{array}{l} \text{new } D_0(\bar{v}) < C, \bar{L}', \bar{L} > .m / \text{proceed}, \\ \text{new } D_0(\bar{v}) < D, \bar{L}, \bar{L} > / \text{super} \end{array} \right].$$

2. If $\bar{L}$ wf and $C.m; \Lambda; \Gamma \vdash e_0 : C_0$ and $D_0 <: C$ and $\text{fields}(D_0) = \bar{D} \ \bar{f}$ and $\bullet; \{\bar{L}\}; \Gamma \vdash \bar{v} : \bar{E}$ and $\bar{E} <: \bar{D}$ and $\text{class } C \triangleleft D$, then $\bullet; \{\bar{L}\}; \Gamma \vdash [\text{new } D_0(\bar{v}) < D, \bar{L}, \bar{L} > / \text{super}] e_0 : C_0$.

Proof: 1. By induction on $L.C.m; \Lambda; \Gamma \vdash e_0 : C_0$ with case analysis on the last typing rule used. We show only main cases below.

Case T-SUPERP: $e_0 = \text{super}.m'(\bar{e})$
 $\text{mtype}(m', D, \Lambda' \cup \{\bar{L}\}) = \bar{F} \rightarrow C_0$
 $L.C.m; \Lambda; \Gamma \vdash \bar{e} : \bar{G}$
 $\bar{G} <: \bar{F}$

Since $S e_0 = \text{new } D_0(\bar{v}) < D, \bar{L}', \bar{L} > .m'(S \bar{e})$, it suffices to show that $\bullet; \{\bar{L}\}; \Gamma \vdash \text{new } D_0(\bar{v}) < D, \bar{L}, \bar{L} > .m'(S \bar{e}) : C_0$. By the induction hypothesis, we have $\bullet; \{\bar{L}\}; \Gamma \vdash S \bar{e} : \bar{G}$. Since $D_0 <: C$ and $\text{class } C \triangleleft D$, we have $D_0 <: D$. Then, T-INVKA finishes the case.

Case T-PROCEED: $e_0 = \text{proceed}(\bar{e})$
 $\text{mtype}(m, C, \Lambda', \Lambda' \cup \{\bar{L}\}) = \bar{F} \rightarrow C_0$
 $L.C.m; \Lambda; \Gamma \vdash \bar{e} : \bar{G}$
 $\bar{G} <: \bar{F}$

Since $S e_0 = \text{new } D_0(\bar{v}) < C, \bar{L}', \bar{L} > .m(S \bar{e})$, it suffices to show that

$$\bullet; \{\bar{L}\}; \Gamma \vdash \text{new } D_0(\bar{v}) < C, \bar{L}', \bar{L} > .m(S \bar{e}) : C_0$$

but it is easy to show by T-INVKA and the induction hypothesis.

2. Similar. Note that the case T-PROCEED cannot happen. $\square$

LEMMA 7. Suppose $\bar{L}'$ is a prefix of $\bar{L}''$ and $\bar{L}''$ wf and $\text{mbody}(m, C, \bar{L}', \bar{L}'') = \bar{x}.e_0$ in $C', \bar{L}$ and $\text{mtype}(m, C, \{\bar{L}'\}, \{\bar{L}''\}) = \bar{D} \rightarrow D_0$.

1. If $\bar{L} = \bar{L}'''; L_0$, then $L_0.C'.m; \Lambda' \cup \{L_0\}; \bar{x} : \bar{D}, \text{this} : C' \vdash e_0 : E_0$ and $L_0 \text{ req } \Lambda'$ and $\Lambda' \subseteq \{\bar{L}'''\}$ and $C <: C'$ and $E_0 <: D_0$ for some E_0 and Λ' .
2. If $\bar{L} = \bullet$, then $C'.m; \emptyset; \bar{x} : \bar{D}, \text{this} : C' \vdash e_0 : E_0$ and $C <: C'$ and $E_0 <: D_0$ for some E_0 .

Proof: By induction on $\text{mbody}(m, C, \bar{L}', \bar{L}'') = \bar{x}.e_0$ in $C', \bar{L}$.

Case MB-CLASS: $\bar{L}' = \bullet$
 $\text{class } C \triangleleft D \{ \dots C_0 \ m(\bar{C} \ \bar{x}) \{ \text{return } e_0; \} \dots$
 $C' = C$
 $\bar{L} = \bullet$

By T-CLASS, T-METHOD and MT-CLASS, it must be the case that

$$\begin{aligned} C_0, \bar{C} &= D_0, \bar{D} \\ C.m; \emptyset; \bar{x} : \bar{D}, \text{this} : C &\vdash e_0 : E_0 \\ E_0 &<: D_0 \end{aligned}$$

for some E_0 , finishing the case.

Case MB-LAYER: $\bar{L}' = \bar{L}'''; L_0$
 $PT(m, C, L_0) = C_0 \ m(\bar{C} \ \bar{x}) \{ \text{return } e_0; \}$
 $C' = C$
 $\bar{L} = \bar{L}'$

By T-PMETHOD, it must be the case that

$$\begin{aligned} C_0, \bar{C} &= D_0, \bar{D} \\ L_0 \text{ req } \Lambda & \\ L_0.C.m; \Lambda \cup \{L_0\}; \bar{x} : \bar{D}, \text{this} : C &\vdash e_0 : E_0 \\ E_0 &<: D_0 \end{aligned}$$

for some E_0 . By $\bar{L}''$ wf, we have $\bar{L}'$ wf and so $\Lambda \subseteq \{\bar{L}'''\}$, finishing the case.

Case MB-SUPER: $\bar{L}' = \bullet$
 $\text{class } C \triangleleft D \{ \dots \bar{M} \}$
 $m \notin \bar{M}$
 $\text{mbody}(m, D, \bar{L}', \bar{L}'') = \bar{x}.e_0$ in $C', \bar{L}$

By MT-SUPER, it must be the case that $\text{mtype}(m, D, \{\bar{L}''\}, \{\bar{L}'''\}) = \bar{D} \rightarrow D_0$. The induction hypothesis and transitivity of subtyping finish the case.

Case MB-NEXTLAYER: $\bar{L}' = \bar{L}'''; L_0$
 $PT(m, C, L_0)$ undefined
 $\text{mbody}(m, C, \bar{L}', \bar{L}'') = \bar{x}.e_0$ in $C', \bar{L}$

The induction hypothesis finishes the case. $\square$

Proof of Theorem 1: By induction on $\bar{L} \vdash e \rightarrow e'$ with case analysis on the last reduction rule used.

Case R-FIELD: $e = \text{new } C_0(\bar{v}).f_i$ $\text{fields}(C_0) = \bar{C} \ \bar{f}$
 $e' = v_i$

By T-FIELD and T-NEW, it must be the case that

$$\bullet; \{\bar{L}\}; \Gamma \vdash \bar{v} : \bar{D} \quad \bar{D} <: \bar{C} \quad C = C_i$$

and, in particular, $\bullet; \{\bar{L}\}; \Gamma \vdash v_i : D_i$ and $D_i <: C_i$, finishing the case.

Case R-INVK: $e = \text{new } C_0(\bar{v}).m(\bar{w})$
 $\bar{L} \vdash \text{new } C(\bar{v}) < C, \bar{L}, \bar{L} > .m(\bar{w}) \rightarrow e'$

By T-INVK and T-NEW, it must be the case that

$$\begin{aligned} \bullet; \{\bar{L}\}; \Gamma \vdash \bar{v} : \bar{D} & & \text{fields}(\bar{C}) = \bar{C} \ \bar{f} & & \bar{D} <: \bar{C} \\ \text{mtype}(m, C_0) = \bar{E} \rightarrow C & & \bullet; \{\bar{L}\}; \Gamma \vdash \bar{w} : \bar{F} & & \bar{F} <: \bar{E}. \end{aligned}$$

Since $C_0 \prec C_0$, we have

$$\bullet; \{\bar{L}\}; \Gamma \vdash \text{new } C_0(\bar{v}) \langle C_0, \bar{L}, \bar{L} \rangle . m(\bar{w}) : C$$

by T-INVKA. By the induction hypothesis, $\bullet; \{\bar{L}\}; \Gamma \vdash e' : D$ for some $D \prec C$, finishing the case.

Case R-INVKP:

$$\begin{aligned} e &= \text{new } C_0(\bar{v}) \langle C', \bar{L}', \bar{L}'' \rangle . m(\bar{w}) \\ mbody(m, C', \bar{L}', \bar{L}'') &= \bar{x}.e_0 \text{ in } C', (\bar{L}''; L_0) \\ \text{class } C'' \triangleleft D \{ \dots \} \\ e' &= \left[\begin{array}{ll} \text{new } C_0(\bar{v}) & / \text{this} \\ \bar{w} & / \bar{x} \\ \text{new } C_0(\bar{v}) \langle C'', \bar{L}', \bar{L}'' \rangle . m / \text{proceed} & \\ \text{new } C_0(\bar{v}) \langle D, \bar{L}', \bar{L}'' \rangle & / \text{super} \end{array} \right] e_0 \end{aligned}$$

By T-INVKA, it must be the case that

$$\begin{aligned} \bar{L}' \text{ is a prefix of } \bar{L}'' & \quad \bar{L}'' \text{ wf} \\ fields(C_0) = \bar{C} \bar{f} & \quad mtype(m, C', \{\bar{L}'\}, \{\bar{L}''\}) = \bar{F} \rightarrow C \\ \bullet; \{\bar{L}\}; \Gamma \vdash \bar{v} : \bar{D} & \quad \bar{D} \prec \bar{C} \quad C_0 \prec C' \\ \bullet; \{\bar{L}\}; \Gamma \vdash \bar{w} : \bar{E} & \quad \bar{E} \prec \bar{F}. \end{aligned}$$

By T-NEW, $\bullet; \{\bar{L}\}; \Gamma \vdash \text{new } C_0(\bar{v}) : C_0$.

By Lemma 7,

$$L_0.C'' . m; \Lambda' \cup \{L_0\}; \bar{x} : \bar{F}, \text{this} : C'' \vdash e_0 : E_0$$

and $C' \prec C''$ and $E_0 \prec C$ and $L_0 \text{ req } \Lambda'$ and $\Lambda' \subseteq \{\bar{L}''\}$ for some E_0 and Λ' . By S-TRANS, $C_0 \prec C''$.

By Lemmas 1, 2, 5, and 6, $\bullet; \{\bar{L}\}; \Gamma \vdash e' : E_0'$ for some $E_0' \prec E_0$. By S-TRANS, $E_0' \prec C$, finishing the case.

Case R-INVKB:

Similar to the case for R-INVKP.

$$\text{Case RC-ENSURE: } e = \text{ensure } L \ e_0 \quad e' = \text{ensure } L \ e_0' \\ \bar{L} \vdash e_0 \longrightarrow e_0'$$

By T-ENSURE, it must be the case that $\bullet; \{\bar{L}\} \cup \{L\}; \Gamma \vdash e_0 : C$. By the induction hypothesis, $\bullet; \{\bar{L}\} \cup \{L\}; \Gamma \vdash e_0' : D$ for some $D \prec C$. By T-ENSURE, $\bullet; \{\bar{L}\}; \Gamma \vdash e' : D$, finishing the case.

$$\text{Case R-ENSUREVAL: } e = \text{ensure } L \ v_0 \quad e' = v_0$$

By T-ENSURE, it must be the case that $\bullet; \{\bar{L}\} \cup \{L\}; \Gamma \vdash v_0 : C$. Then, by Lemma 2, $\bullet; \{\bar{L}\}; \Gamma \vdash v_0 : C$, finishing the case.

$$\text{Case RC-FIELD: } e = e_0.f_i \quad \bar{L} \vdash e_0 \longrightarrow e_0' \\ e' = e_0'.f_i$$

By T-FIELD, it must be the case that

$$\bullet; \{\bar{L}\}; \Gamma \vdash e_0 : C_0 \quad fields(C_0) = \bar{C} \bar{f} \quad C = C_i.$$

By the induction hypothesis, $\bullet; \{\bar{L}\}; \Gamma \vdash e_0' : D_0$ for some $D_0 \prec C_0$. By Lemma 3, $fields(D_0) = \bar{C} \bar{f}, \bar{D} \bar{g}$ for some $\bar{D}$ and $\bar{g}$. By T-FIELD, $\bullet; \{\bar{L}\}; \Gamma \vdash e_0'.f_i : C_i$, finishing the case.

$$\text{Case RC-INVKRECV: } e = e_0.m(\bar{e}) \quad \bar{L} \vdash e_0 \longrightarrow e_0' \\ e' = e_0'.m(\bar{e})$$

By T-INVK, it must be the case that

$$\begin{aligned} \bullet; \{\bar{L}\}; \Gamma \vdash e_0 : C_0 & \quad mtype(m, C_0, \{\bar{L}\}) = \bar{D} \rightarrow C \\ \bullet; \{\bar{L}\}; \Gamma \vdash \bar{e} : \bar{E} & \quad \bar{E} \prec \bar{D}. \end{aligned}$$

By the induction hypothesis, $\bullet; \{\bar{L}\}; \Gamma \vdash e_0' : D_0$ for some $D_0 \prec C_0$. By Lemma 4, $mtype(m, D_0, \{\bar{L}\}) = \bar{D} \rightarrow D$ and $D \prec C$ for some D . By T-INVK, $\bullet; \{\bar{L}\}; \Gamma \vdash e_0'.m(\bar{e}) : D$, finishing the case.

$$\text{Case RC-INVKAARG: } e = e_0.m(\dots, e_i, \dots) \quad \bar{L} \vdash e_i \longrightarrow e_i' \\ e' = e_0.m(\dots, e_i', \dots)$$

By T-INVK, it must be the case that

$$\begin{aligned} \bullet; \{\bar{L}\}; \Gamma \vdash e_0 : C_0 & \quad mtype(m, C_0, \{\bar{L}\}) = \bar{D} \rightarrow C \\ \bullet; \{\bar{L}\}; \Gamma \vdash \bar{e} : \bar{E} & \quad \bar{E} \prec \bar{D}. \end{aligned}$$

By the induction hypothesis, $\bullet; \{\bar{L}\}; \Gamma \vdash e_i' : F_i$ for some $F_i \prec E_i$. By S-TRANS, $F_i \prec D_i$. So, by T-INVK, $\bullet; \{\bar{L}\}; \Gamma \vdash e' : C$, finishing the case.

Case RC-NEW, RC-INVKAARG:

Similar to the case above. $\square$

LEMMA 8. *If $mtype(m, C, \Lambda_1, \Lambda_2) = \bar{D} \rightarrow D_0$ and $\bar{L}$ is a prefix of $\bar{L}'$ and $\Lambda_1 \subseteq \{\bar{L}\}$ and $\Lambda_2 \subseteq \{\bar{L}'\}$ and $\bar{L}'$ wf, then there exist $\bar{x}$ and e_0 and $\bar{L}''$ and $C' (\neq \text{Object})$ such that $mbody(m, C, \bar{L}, \bar{L}') = \bar{x}.e_0$ in $C', \bar{L}''$ and the lengths of $\bar{x}$ and $\bar{D}$ are equal.*

Proof: By lexicographic induction on $mtype(m, C, \Lambda_1, \Lambda_2) = \bar{D} \rightarrow D_0$ and the length of $\bar{L}$.

$$\text{Case: } \bar{L} = \bullet \quad \text{class } C \triangleleft D \{ \dots C_0 \ m(\bar{C} \ \bar{x}) \{ \text{return } e_0; \} \dots \}$$

By MT-CLASS, it must be the case that $\bar{D}, D_0 = \bar{C}, C_0$ and the lengths of $\bar{C}$ and $\bar{x}$ are equal. Then, by MB-CLASS, $mbody(m, C, \bullet, \bar{L}') = \bar{x}.e_0$ in $C, \bullet$.

$$\text{Case: } \bar{L} = \bar{L}'' . L_0 \quad PT(m, C, L_0) = E_0 \ m(\bar{E} \ \bar{x}) \{ \text{return } e_0; \}$$

By T-PMETHOD, it must be the case that $\bar{E}, E_0 = \bar{D}, D_0$ and the lengths of $\bar{D}$ and $\bar{x}$ are equal. By MB-LAYER, $mbody(m, C, \bar{L}, \bar{L}') = \bar{x}.e_0$ in $C, \bar{L}$.

$$\text{Case: } \bar{L} = \bullet \quad \text{class } C \triangleleft D \{ \dots \bar{M} \} \quad m \notin \bar{M} \\ \Lambda_1 = \emptyset$$

By MT-SUPER, we have $mtype(m, D, \emptyset, \Lambda_2) = \bar{D} \rightarrow D_0$. By the induction hypothesis, there exist $\bar{x}$ and e_0 and $\bar{L}''$ and $C' (\neq \text{Object})$ such that $mbody(m, D, \bar{L}', \bar{L}') = \bar{x}.e_0$ in $C', \bar{L}''$ and the lengths of $\bar{x}$ and $\bar{D}$ are equal. By MB-SUPER, $mbody(m, C, \bullet, \bar{L}') = \bar{x}.e_0$ in $C', \bar{L}''$, finishing the case.

$$\text{Case: } \bar{L} = \bar{L}'' . L_0 \quad PT(m, C, L_0) \text{ undefined}$$

By the induction hypothesis, there exist $\bar{x}$ and e_0 and $\bar{L}''$ and $C' (\neq \text{Object})$ such that $mbody(m, C, \bar{L}'' , \bar{L}') = \bar{x}.e_0$ in $C', \bar{L}''$ and the lengths of $\bar{x}$ and $\bar{D}$ are equal. By MB-NEXTLAYER, $mbody(m, C, \bar{L}, \bar{L}') = \bar{x}.e_0$ in $C', \bar{L}''$, finishing the case. $\square$

Proof of Theorem 2: By induction on $\bullet; \Lambda; \bullet \vdash e : C$ with case analysis on the last typing rule used.

Case T-VAR, T-SUPER, T-PROCEED:

Cannot happen.

$$\text{Case T-FIELD: } e = e_0.f_i \quad \bullet; \{\bar{L}\}; \bullet \vdash e_0 : C_0 \\ fields(C_0) = \bar{C} \bar{f} \quad C = C_i$$

By the induction hypothesis, either e_0 is a value or there exists e_0' such that $\bar{L} \vdash e_0 \longrightarrow e_0'$. In the latter case, RC-FIELD finishes the case. In the former case where e_0 is a value, by T-NEW, we have

$$e_0 = \text{new } C_0(\bar{v}) \quad \bullet; \{\bar{L}\}; \bullet \vdash \bar{v} : \bar{D} \quad \bar{D} \prec \bar{C}.$$

So, we have $\bar{L} \vdash e \longrightarrow v_i$, finishing the case.

$$\text{Case T-INVK: } e = e_0.m(\bar{e}) \quad \bullet; \{\bar{L}\}; \bullet \vdash e_0 : C_0 \\ mtype(m, C_0, \{\bar{L}\}) = \bar{D} \rightarrow C \quad \bullet; \{\bar{L}\}; \bullet \vdash \bar{e} : \bar{E} \\ \bar{E} \prec \bar{D}$$

By the induction hypothesis, there exist $i \geq 0$ and e_i' such that $\bar{L} \vdash e_i \longrightarrow e_i'$, in which case RC-INVKRECV or RC-INVKAARG finishes the case, or all e_i 's are values $v_0, \bar{v}$. Then, by T-NEW, $v_0 = \text{new } C_0(\bar{w})$ for some values $\bar{w}$. By Lemma 8, there exist $\bar{x}, e_0', \bar{L}''$, and $C' (\neq \text{Object})$ such that $mbody(m, C_0, \bar{L}, \bar{L}') = \bar{x}.e_0$ in $C', \bar{L}''$ and the lengths of $\bar{x}$ and $\bar{D}$ are the same. Since $C' \neq \text{Object}$, there exists D' such that $\text{class } C' \triangleleft D' \{ \dots \}$. We have two subcases here depending on whether $\bar{L}''$ is empty or not. We will show the case where $\bar{L}''$ is not empty; the other

case is similar. Let $\bar{L}'' = \bar{L}''' ; L_0$ for some $\bar{L}'''$ and L_0 . Then, the expression

$$e' = \left[\begin{array}{l} \text{new } C_0(\bar{w}) \\ \bar{v} \\ \text{new } C_0(\bar{w}) \langle C', \bar{L}''', \bar{L} \rangle . m \\ \text{new } C_0(\bar{w}) \langle D', \bar{L}, \bar{L} \rangle \end{array} \begin{array}{l} / \text{this} \\ / \bar{x} \\ / \text{proceed} \\ / \text{super} \end{array} \right] e_0'$$

is well defined (note that the lengths of $\bar{x}$ and $\bar{v}$ are equal). Then, by R-INVKP and R-INVK, $\bar{L} \vdash e \longrightarrow e'$.

Case T-NEW: $e = \text{new } C(\bar{e})$ $\text{fields}(C) = \bar{C} \ \bar{f}$
 $\bullet; \{\bar{L}\}; \bullet \vdash \bar{e} : \bar{D}$ $\bar{D} <: \bar{C}$

By the induction hypothesis, either (1) $\bar{e}$ are all values, in which case e is also a value; or (2) there exist i and e_i' such that $\bar{L} \vdash e_i \longrightarrow e_i'$, in which case RC-NEW finishes the case.

Case T-ENSURE: $e = \text{ensure } L \ e_0$ $\bullet; \{\bar{L}\} \cup \{L\}; \bullet \vdash e_0 : C$
 $L \text{ req } \Lambda'$ $\Lambda' \subseteq \{\bar{L}\}$

By the induction hypothesis, either e_0 is a value, in which case R-ENSUREVAL finishes the case, or there exists e_0' such that $\text{ensure}(L, \bar{L}) \vdash e_0 \longrightarrow e_0'$, in which case RC-ENSURE finishes the case (notice that $\text{ensure}(L, \bar{L})$ wf).

Case T-INKVA:

Similar to (the second half of) the case for T-INVK. $\square$

A Practical, Typed Variant Object Model

Or, How to Stand On Your Head and Enjoy the View

Pottayil Harisanker Menon Zachary Palmer Alexander Rozenshteyn Scott Smith

The Johns Hopkins University

{pharisa2, zachary.palmer, arozens1, scott}@jhu.edu

Abstract

Traditionally, typed objects have been encoded as records; the fields and methods of an object are stored in the fields of a record and projected when needed. While the dual approach of using variants instead of records has been explored, it is more challenging to type: the output type of a variant case match must depend on the input value; this is a form of dependent typing.

In this paper, we construct a variant-based encoding of objects which is statically typeable and which improves on the flexibility of typed object models in several dimensions: messages can be represented as simple first-class data, object extension is more generally typeable than in previous systems and, arguably, a better integration of objects and functions is obtained.

This encoding is possible due to the features of our new core language, TinyBang, which incorporates a subtype constraint type inference system with several novel extensions. We develop a generalized notion of first-class cases – functions with an inherent pattern match that are composable – and we extend previous notions of conditional constraint types to obtain accurate typings. For added flexibility, TinyBang’s record-like structure is type-indexed, meaning data can be projected based on its type alone. Our record structure’s concatenation operator is asymmetric by default, naturally supporting object extension. Finally, we develop a refined notion of parametric polymorphism which aims to achieve a good combination of flexibility and efficiency of inference.

1. Introduction

Typed objects today sit on a foundation of records and subtyping: object members are stored in record fields and are projected on access, update, or invocation [9]. While many formalizations of typed objects are direct and not defined by a translation to records [1], they are in spirit not far from this underlying record view.

While records have been the most common form of encoding, there is a well-known duality between records and variants which means tagged variants and a match/case over them can also be used to encode objects. This *variant encoding* has been effectively used in dynamically-typed settings: an object is a function which receives a message in the form of a variant, matches on that variant, and executes the appropriate branch of code. Instead of invoking a method by projecting a `foo` field from a record and passing arguments to the result, one passes a `Foo(...)` message to the object and allows the object to invoke the appropriate function. The variant-based encoding has the advantage of representing object messages as first-class data that can be directly manipulated. This view is the object model of actors [3] and is the most natural for distributed objects since the packet data being sent across the network to a remote object is directly realized as a variant.

Unfortunately, it is well-known that the variant-based encoding of objects is difficult to type; all languages currently supporting a

variant-based view of objects are dynamically typed. The root of the typing problem is that a match expression’s cases are generally required to each produce the same type, meaning all methods of the encoded objects would have to have the same return type to typecheck (ouch!). In the record-based encoding, record fields are heterogeneously typed and so are not afflicted in this way. So while the duality is perfect in the dynamically-typed setting, it is not perfect using standard type theory.

In this paper, we present a small core language, TinyBang, in which a variant-based encoding of objects is typeable. TinyBang improves on the flexibility of typed object models in several dimensions. We can represent messages as simple first-class data; existing work on first-class message typing generally requires messages to be “frozen” under a function abstraction whereas our messages can be direct tagged data. Also, object extension is more generally typeable than in previous type theories for extensible objects; this results in an arguably better integration of objects and functions.

In order to achieve these expressiveness advantages, TinyBang contains several novel approaches to record/variant syntax and typing. The variants and records are different enough from the standard variety that we elect to use different names: *onions* are our record-like constructor, and *scapes*¹ are the variant match/case destructor. Onions and scapes combine and improve on several language design ideas, including the concepts of *first-class cases*, *dependent pattern types*, *type-indexed records*, and *asymmetric concatenation*. A high-level overview of these features is as follows.

Scapes: generalizing first-class cases Traditional case/match statements are monolithic blocks that lack an explicit composition operator on case constructs. Such a composition is key to encoding objects as variants because, if objects are case matches on messages, object extension/subclassing/mixin is case composition.

The solution is a (typed) notion of first-class cases, which has been explored previously in [6]. There, a construct $\oplus$ is defined for extending a case with an additional single clause. Our scapes generalize first-class cases by supporting composition of arbitrary case expressions, not just the addition of one clause. In the record analogy, this is like the difference between extension of a record by a single field and a generalized record append. Since their case extension is only adding case clauses to the end of an existing case, it cannot model inheritance or mixins: the former requires added methods to be higher-priority than existing ones and the latter appends of two sets of methods rather than adding a single method to a set. So, we believe our scapes offer the first high-level algebraic case composition operator which can directly support object inheritance for the variant encoding of objects.

Less related is the Pattern Calculus [13], which is a more fundamental algebra in that patterns themselves are first-class entities separate from the code to be executed in case of a match. How-

¹ A scape is the green, leafless stem of an onion that grows above the ground.

ever, variable bindings in this calculus are extremely complex; we therefore elect to make the case clause our level of abstraction.

Dependent pattern types A weak form of dependent type is necessary to address the need for heterogeneity in case branches, the most fundamental problem existing type systems have with typing the variant object encoding. A case branching on multiple variants needs to return a type based on the particular *value* of the variant that arrives at runtime, in particular based on which tag the variant has. Our *dependent pattern types* solve this problem. They are *weakly* dependent since they depend only on the tag and not more detailed information from the value; the advantage of their weakness is that type inference is still decidable.

Our approach is a generalization of the conditional constraints originating in [5] and elaborated in [18]. Conditional constraints partly capture this dependency but only locally and so lose the dependency information in the presence of side effects. Our dependent pattern types fully capture the dependency.

Onions: type-indexed records supporting asymmetric concatenation and object extension A type-indexed record is one for which contents are projected using types rather than labels [22]. For example, consider the type-indexed record `{foo = 45; bar = 22; (); 13}` which implicitly tags the untagged elements `()` and `13` with their types. Projecting `int` from this record would yield `13` (as the other integers are already labeled). Similarly, one can project unlabeled functions from a type-indexed record. Our *onions* are a form of type-indexed record. This added flexibility is handy in many situations as will be seen below.

Additionally, since (untagged) functions can be placed in our onions, we can re-use the onion record structure to hold our scape clauses and avoid the need for two different extension operations as found in [6]. As we alluded above, *asymmetric* concatenation is the key to composing scapes and thus properly defining inheritance and overriding; onions thus support asymmetric concatenation as the default. Asymmetric record concatenation was initially proposed by Wand for modeling inheritance [27], but difficulties in obtaining principal types in unification-based type inference [26] caused a switch in research focus to symmetric notions of record concatenation, which unfortunately are not amenable to modeling the overriding of methods. In standard record-based encodings of inheritance [9], there is no first-class record extension operation; inheritance requires the superclass to be known statically. This is implicitly a consequence of the difficulty of typing asymmetric record extension. Dynamically-typed OO languages have explored first-class object extenders to good benefit; the use of typed asymmetric record concatenation can bring this flexibility to the typed world.

There has been work developing type systems that support more flexible notions of object extension [8, 20] and our presented object encoding builds on this work. They define a phase transition where objects are initially open for extension but closed for messaging and can transition to *sealed* objects open for messaging but permanently closed for extension. This approach allows for more flexible programming patterns than standard class/inheritance object construction. Here we show for the first time how this approach may be generalized to support the (functional) extension of already-sealed objects, bringing fundamentally new dynamic-style object flexibility to a statically-typed domain.

Synergy Each of these features has some precedent in the literature and in each we have made improvements. The main claim of this paper is that the *combination* of this set of features yields a highly expressive language in which many paradigms of object-oriented programming can be encoded in an extremely lightweight fashion. Along with supporting the variant-based encoding of objects, it naturally supports functional object extension, mixins, as

well as new programming patterns not even imaginable when wearing the straitjacket of a fixed object model.

Subtype constraint types TinyBang uses a subtype constraint inference based type system [5, 12, 18]; in particular, it is most closely related to [18]. Compared to [18], our system does not need row types or conditional constraints to typecheck record concatenation, incorporates type-indexed records and first-class cases, and contains a more general notion of conditional type and a more general model of parametric polymorphism. Our approach to parametric polymorphism is based on flow analysis [23, 28] and improves on previous work for expressive but efficient contour sharing.

While use of subtype constraint inference is a major thrust of our broader research agenda, the concepts of this paper should not fundamentally depend on use of a constraint inference type system. We believe it should be possible to build a more standard declarative polymorphic subtype system for TinyBang syntax; it would need to include a form of dependent pattern type for scapes, as well as an asymmetric record concatenation operation for onions.

Direct objects vs encoded objects plus translucent sugar While this paper discusses how to *encode* object features in a non-OO language [9] as opposed to directly giving types and runtime semantics for objects [1], the fundamental ideas here could also be used to give an object language a direct type system and runtime semantics. We focus on encodings rather than direct semantics because, given that our encodings are extremely lightweight, we believe it is possible to expose them directly to the programmer. In other words, sugar can be directly added to the language for objects, classes, inheritance, etc, but the sugar is so simple it is *translucent*: it is possible for the programmer to look under the hood of the encoding if needed and do some cool things not possible if the hood was permanently locked.

Record vs variant encoding We believe the variant encoding has been under-appreciated outside of the actor-like language community and a main focus of this paper is showing it has potential as a viable alternative in a typed context. The typed variant encoding was previously explored by our lab in [24], but that paper only showed how primitive objects could be encoded and was not focusing on a practical encoding of all object features such as self awareness and object extension.

Outline In the next section, we give a high-level overview of how TinyBang can encode objects. Section 3 contains the formal type system for MicroBang, a subset of TinyBang containing the key features but trimmed down for technical readability. We conclude in Section 4.

2. Overview

In this Section we informally present our object encoding and make the case that it is a good one.

2.1 Variant-Based Object Encodings

We begin with a very brief review of the standard record encoding, and then we outline the main problem with the natural dual variant encoding.

Record encodings Typed object encodings today generally encode objects into some form of record: an object consists of a record with the labels representing the method names and the label values are the functions which represent the method bodies, as in:

```
1 let o = { double = (fun x -> x + x);
2       inc = (fun x -> x + 1) } in ...
```

Various different encoding methods are used for self-awareness of such records and for representing fields [9]; we momentarily

$e ::= x \mid \text{lbl } e \mid e \& e \mid e \& -\pi \mid e \& . \pi \mid e e$	<i>TinyBang expressions</i>
$\mid e \text{ op } e \mid \text{def } x = e \text{ in } e \mid x = e \text{ in } e$	
$\mid () \mid \mathbb{Z} \mid c \mid (\&) \mid \chi \rightarrow e$	<i>TinyBang literals</i>
$q ::= \epsilon \mid \text{final} \mid \text{immut}$	
$\text{op} ::= + \mid - \mid == \mid <= \mid >=$	
$x ::= (\text{alphanumeric identifiers})$	
$\text{lbl} ::= [\text{a-zA-Z0-9_}]^+$	
$c ::= [\backslash'] \mid [\backslash\backslash'] \mid [\backslash']$	<i>character literals</i>
$\chi ::= x : \chi^p \mid x \mid \chi^p$	<i>patterns</i>
$\chi^p ::= \text{tprim} \mid \text{lbl } \chi \mid (\chi^p \& \dots \& \chi^p) \mid \text{fun} \mid \text{any}$	
$\pi ::= \text{tprim} \mid \text{lbl} \mid \text{fun}$	<i>projectors</i>
$\text{tprim} ::= \text{char} \mid \text{int} \mid \text{unit}$	

Figure 1: TinyBang Syntax

ignore these issues for simplicity. Invoking a method on a record-encoded object involves projecting the function from the record and invoking that function, `o.double 4`.

Variant encodings In a variant-based encoding of objects, the object is instead a single dispatch function. Messages are passed directly to the dispatch function as variants and the dispatch function cases on the message to choose the code to execute. The following example shows a trivial variant-encoded object and a corresponding message dispatch.

```
1 let obj = fun msg -> match msg with
2   | Say_Hello -> print_string "Hello!"
3   | Calc_Double x -> x + x
4 in obj (Calc_Double 2)
```

Unfortunately, the above variant-encoded object does not type-check in most languages with a `match` or similar expression because the branches are required to have the same result type. In the above, the `Say_Hello` branch has type `unit` while the `Calculate_Double` branch has type `int`. This property of the `match` expression imposes the debilitating requirement that all methods on an object have the same return type. We solve this problem by inferring more expressive *dependent pattern types* for match expressions; we discuss these types in the following Section.

2.2 Language Features for a Typeable Encoding

Our goal is to create a concise, typeable encoding for variant-based objects. By concise, we mean that the desugared, encoded version of objects should be legible and intuitive to a programmer. TinyBang’s feature set is specifically selected to make such an encoding viable. The syntax for TinyBang appears in Figure 1. This figure should be used as a reference; we will discuss the semantics of each production as necessary throughout this section.

In order to typecheck our encoding, we will use a subtype constraint-based type system. The encoding we present is not intrinsically tied to such an approach, but subtype constraints provide a number of advantages. Principal typing, for instance, is trivial. Also, unlike when row types are used, it is not necessary to explicitly annotate type transition sites such as upcasts [21]. Most importantly, subtype constraint systems are handily modified to accommodate the more unusual language features we describe below. We defer detailed discussion of our type system until section 3; this section’s use of types remains informal for presentation clarity.

Scapes as methods We begin by considering the oversimplified case of an object with a single method and no fields. This object is not self-aware; self-awareness is discussed in Section 2.3. In the variant encoding, such an object can be represented by a function which matches on a single case. In the TinyBang grammar, note that *all* functions are written $\chi \rightarrow e$, with χ being the pattern to match against the function’s argument. Combining pattern match

with function definition is also possible in ML and Haskell, but we can go further: there is no need for any `match` syntax in TinyBang since `match` can be encoded as a pattern and its application. We call such pattern-matching functions *scapes*. For instance, consider the following object and its invocation:

```
1 def obj = 'double x -> x + x
2 in obj ('double 4)
```

The syntax `'double 4` is a label constructor similar to a OCaml polymorphic variant; as in OCaml, the expression `'double 4` has type `'double int`. The scape `'double x -> x + x` is a function which matches on any argument containing a `'double` label and binds its contents to the variable `x`. As a result, the above code evaluates to 8. Note that the expression `'double 4` represents a *first-class message*; the entire object invocation is represented with its arguments as a variant.

Unlike a traditional `match` expression, an individual scape is a function, and is only capable of matching a single pattern. To express general patterns with scapes, individual scapes are appended via our general data conjoiner, the *onion* operation `&`. This conjoiner has many uses which are discussed below; for now we only use it to append scapes together. Given two scape expressions `e1` and `e2`, the expression `(e1 & e2)` will conjoin the patterns together to make a function with the conjoined pattern, and `(e1 & e2) a` will apply the scape which has a pattern matching `a`; if both patterns match `a`, the rightmost scape (`e2`) is given priority. We can thus write a dispatch on an object with two methods simply as:

```
1 def obj = ('double x -> x + x)
2   & ('isZero x -> x == 0)
3 in obj 'double 4
```

The above shows that traditional `match` expressions can be encoded by using the `&` operator to join a number of scapes: one scape for each case. Our scape conjunction generalizes the first-class cases of [6] to support general appending of arbitrary cases; the aforementioned work only supports adding one clause to the end and so does not allow “override” of an existing clause or “mixing” of two arbitrary sets of clauses. The above work does include a construct $\ominus$ for removing a case clause containing a certain pattern and we plan to add similar functionality to our system in the near future.

Dependent pattern types Critical to our typed encoding is the fact that the scape extension is assigned a dependent pattern type which remembers the association between input types and output types. The two scapes above have the approximate² types `'double int → int` and `'isZero int → boolean`, respectively. While we could assign a type `( 'double int ) ∪ ( 'isZero int ) → int ∪ boolean`, this type is imprecise; we want to retain the fact that, if a `'double int` is passed to the scape, an `int` will always be returned.

The dependent pattern type assigned to the above scape contains this relationship and loses no information; it is `( 'double int → int ) & ( 'isZero int → boolean )`. If the scape is applied in the context where the type of message is known, the appropriate result type is inferred; the above method invocation, for instance, always has type `int` and not type `int ∪ boolean`. Because of this dependent typing, `match` expressions encoded in TinyBang may be heterogeneous; that is, each case branch may have a different type in a meaningful way. When we present our type system in Section 3, we show how these dependent pattern types extend the expressivity of conditional constraint types in a dimension critical for typing objects.

²This section uses simplified types which improve readability by avoiding constraint sets. The actual types used in TinyBang are more precise and are described in Section 3.

Onions as records We now show how our general onion data conjoiner, `&`, can act like a record constructor. For example, here is how we encode objects with methods that take multiple arguments:

```
1 def obj = ('sum ('x x & 'y y) -> x + y)
2         & ('equal ('x x & 'y y) -> x == y)
3 in obj ('sum ('x 3 & 'y 2))
```

In the last line, the object is invoked with the argument `'sum ('x 3 & 'y 2)`. As above, the `'sum` label merely wraps another value. In this case, it is an onion between two labels; this is pretty much a two-label record. This record-like onion is passed to the pattern `'x x & 'y y`; here, we use `&` to denote pattern conjunction, which requires that the value must match both subpatterns to match the overall pattern. When the argument `'sum ('x 3 & 'y 2)` is passed to the object, the formal parameters `x` and `y` are bound to the values 3 and 2. Observe from this example how there is no hard distinction in TinyBang between records and variants: there is only one class of label and a 1-ary record is the same as a 1-ary variant.

2.3 Self-Awareness and Resealable Objects

Up to this point objects have not been able to invoke their own methods, so the encoding is incomplete. To address this, each scape representing an object method now includes an additional parameter component `'self` which binds a `self` variable. For instance, we can encode a simple self-aware object as:

```
1 def obj = ('double x -> x + x)
2         & ('quad x & 'self self ->
3           self 'double (self 'double x)) in
4 ...
```

Messaging such an object requires the more cumbersome `obj ('quad 4 & 'self obj)` to send message `'quad 4` to `obj`. While this encoding is acceptable for simple uses of objects, it requires the full type of the object to be known at the call site; in a heterogeneous collection of objects, for instance, the exposed self-types are artificially forced to be the same and legal messagings may fail. This is a classic problem and we follow previous work [8] to address this issue. In [8], an object exists in one of two states: as a prototype, which can be extended but not messaged, or as a “proper” object, which can be messaged but not extended. A prototype may be “sealed” to transform it into a proper object, at which point it may never again be extended. This work was refined in [20] to increase the granularity to a per-method basis: an object can be proper in one method and prototypical in another, but a method must still be sealed before the object can receive any messages of that type, and sealed code may never again be extended.

Unlike the aforecited work, our encoding permits objects to be *resealed*: sealed objects may be extended and then sealed again. The flexibility of TinyBang allows the sharp phase distinction between prototypes and proper objects to be relaxed. All object extension below will be performed on sealed objects. The resulting language is, to our knowledge, the first static type system with general, flexible support for typing extensible objects. Object sealing in TinyBang is defined directly as a function `seal`:

```
1 def fix = f -> (g -> x -> g g x)
2             (h -> y -> f (h h) y) in
3 def seal = fix (seal -> obj ->
4               obj & (msg ->
5                 obj ('self (seal obj) & msg))) in
6 def sObj = seal obj in
7 ...
```

The `seal` function above accepts an object as an argument and returns that same object with a new message handler onioned onto its right. This message handler matches *every* argument and will

therefore be used for every message the returned object receives. We call this message handler the *self binding scape* because it adds a `'self` component to every message sent to the object using a reference to the object as it stood at the time the seal occurred. The self binding scape also ensures that the value in `'self` is a sealed object, allowing methods to message it normally. As a result of sealing the object, a `'quad 4` message sent to `sObj` would produce the same effect as a `'self sObj & 'quad 4` message sent to `obj`.

Extending previously sealed objects We now show how we can improve on the previous object extension models to support extension of previously-sealed objects. In the self binding scape, the value of `self` is onioned onto the *left* of the message rather than the right; this choice is purposeful. Because of this, any value of `'self` which exists in a message passed to a sealed object takes priority over the `'self` provided by the self binding scape. While we do not anticipate that programmers will want to manually specify a `self` value, this overriding is what permits an object to be resealed. Consider the following continuation of the previous code:

```
1 ...
2 def sixteen = sObj 'quad 4 in
3 def obj2 = sObj & ('double x -> x) in
4 def sObj2 = seal obj2 in
5 def four = sObj2 'quad 4 in
6 ...
```

When this code is executed, the variable `sixteen` will hold the value 16. Even after the `'quad 4` message has been sent to `sObj`, we may extend it; in this case, `obj2` redefines how `'double` messages are handled. `sObj2` represents the sealed version of this new object. When the `'quad 4` message is sent to `sObj2`, it is passed to `obj2` with a `'self` component; that is, `sObj2 ('quad 4)` has the same effect as `obj2 ('self sObj2 & 'quad 4)`. Because `obj2` does not change how `'quad` messages are handled, this has the same effect as `sObj ('self sObj2 & 'quad 4)`. `sObj` is also a sealed object which adds a `'self` component to the *left*; thus this has the same effect as `obj ('self sObj & 'self sObj2 & 'quad 4)`. Because any pattern match will always match the rightmost `'self`, the latter-sealed object is provided in the `'self` added to the message passed to the `'quad`-handling method and so any messages sent from that method will be dispatched to an object which includes the extensions in `sObj2`.

In summary, this encoding works because, while we “tie the knot” on `self` using `seal`, we leave open the possibility of future *overriding* of `self`; it is merely a record element and we support record field override via asymmetric concatenation. Note that we could easily define a final, non-extensible object to restrict extension for encapsulation purposes; this is achieved simply by adding `'self` to the *right* of the argument rather than the left in `seal`.

One limit of this sealing approach is that sealed objects cannot be extended to the *left*; thus, the following will not typecheck:

```
1 ...
2 def obj = ('foo _ -> 1) in
3 def sObj = seal obj in
4 def obj2 = ('bar _ -> 2) & sObj in
5 def sObj2 = seal obj2 in
6 sObj2 'bar ()
```

The last line in the above code produces a type error. This is because the `'bar` message is captured by the self-binding scape of `sObj` and is then sent only to `obj`, where it will fail to match.

For examples in the remainder of the paper, we will assume that `seal` has been defined as above.

Onioning it all together Onions also provide a natural mechanism for including fields; we simply concatenate them to the scapes that represent the methods. Consider the following object which stores and increments a counter:

<code>o.x</code>	$\approx$	<code>(('x x -> x) o</code>
<code>o.x = e₁ in e₂</code>	$\approx$	<code>(('x x -> x = e₁ in e₂) o</code>
<code>if e₁ then e₂ else e₃</code>	$\approx$	<code>((('True _ -> e₂) &</code> <code>('False _ -> e₃)) e₁</code>
<code>e₁ and e₂</code>	$\approx$	<code>((('True _ -> e₂) &</code> <code>('False _ -> 'False ())) e₁</code>

Figure 2: Some Simple Syntactic Sugar

```

1 def obj = seal (
2   ('inc _ & 'self self ->
3     ('x x -> x = x + 1 in x) self)
4   & 'x 0) in
5 obj 'inc ()

```

The above bears some description. Label construction implicitly creates a mutable cell, so the `'x` label is used to store the counter's current value. The bottom line invokes the `obj` onion with an increment message. The label `'x 0` is not considered as part of this invocation because it is not a scape; thus, the scape is considered next (and is executed because its pattern matches the message).

In this body, `self` is passed to a scape matching `'x x`. Because `seal` onions the target object onto the left of the `self` binding method, all of the labels from the unsealed object are still visible in the sealed object. Thus, `self` has the same `'x` label as `obj` and the cell within `obj`'s `'x` label is bound to the variable `x`. The code `x = x + 1 in x` is then executed; this increments the contents of the `'x` label and returns the value 1. It should be noted that, in TinyBang, variable expressions implicitly dereference cells; this is the reason that `x + 1` successfully typechecks.

It may seem unusual that `obj` is, at the top level, a heterogeneous “mash” of a record field (the `'x`) and a function (the scape which handles `'inc`). This is in fact perfectly sensible; onions are *type-indexed* [22], meaning that they use the types of the values themselves to identify data. A simple case of type indexing is `4 & ()` which denotes the “onion” between the values 4 and `()`; such an onion would have the type `int & unit`. Values are projected when matched by a pattern, but they can also be implicitly projected; for example, `(4 & ()) + 1` evaluates to 5. Note that, unlike label-indexed records, a value is equivalent to the 1-ary indexed record containing it; 5 can be viewed as both the integer five as well as the onion containing only the integer five.

In the case of overlap, the rightmost value is projected; `(7 & 3) + 1` is just 4 since the rightmost integer in the onion has precedence. Scares are a special case; instead of projecting the rightmost scape, all scares are projected and application selects the rightmost scape which matches the argument. For instance, the application `(('x 0 & (int -> 4)) 1` implicitly projects the scape from the onion. We believe this type-indexed view is useful because it leads to more concise code; it also makes it trivial to define operator overloading, as we will discuss in Section 2.5 below.

The above counter object code is quite concise considering that it defines a self-referential, mutable counter object using no syntactic sugar whatsoever in a core language with no explicit object syntax. But programmers would benefit from syntactic sugar to capture common abstractions. We define a number of sugarings in Figure 2 which we use in the examples throughout the remainder of this section. It should be observed that this sugar is still translucent in the sense we describe above: looking at desugared code is not prohibitive for programmers desiring more control.

Using this sugar, the third line of the counter object above can be more concisely expressed as `self.x = self.x + 1 in self.x`.

2.4 Typeable OO Abstractions

The previous section demonstrates a typed, variant-based encoding for objects. In this section, we focus on a typed, variant-based en-

coding for common object-oriented abstractions such as mixins and inheritance. Traditionally, these abstractions are defined in a first-order sense; inheritance, for instance, can only be expressed if the type of the parent class is statically known. In contrast, TinyBang's variant-based encoding can type higher-order abstractions.

We show our encoding in terms of objects rather than classes for simplicity; applying these concepts to classes is straightforward. We also work at the object and not class layer to show how TinyBang can easily express (functional) object extension. For clarity, we use the above-defined sugar for projection.

Mixins The following example shows how a simple two-dimensional point object can be combined with a mixin providing extra methods:

```

1 def point = seal (
2   'x 0 & 'y 0
3   & ('l1 _ & 'self self -> self.x + self.y)
4   & ('isZero _ & 'self self ->
5     self.x == 0 and self.y == 0)) in
6 def mixin = (('nearZero _ & 'self self ->
7   (self 'l1 ()) <= 4)) in
8 def mixedPoint = seal (point & mixin) in
9 mixedPoint 'nearZero ()

```

The `point` variable is our original point object. The `mixin` is merely a scape which makes demands on the value passed as `self`. Because an object's methods are just scares onioned together, onioning the `mixin` into the `point` object is sufficient to produce the resulting mixed point; the `mixedPoint` variable contains an onion with `x` and `y` fields as well as all three scares.

The above example is well-typed; parametric polymorphism is used to allow `point`, `mixin`, and `mixedPoint` to have different self-types. The `mixin` variable, the interesting part of the above code, has roughly the type `((('nearZero unit & 'self  $\alpha$ )  $\rightarrow$  boolean` where α is an object capable of receiving the `'l1` message and producing an `int`. `mixin` can be onioned with any object that satisfies these properties. If the object does not have these properties, a type error will result when the `'nearZero` message is passed. For instance, consider the following code in which the `mixin` is invoked directly:

```

1 def mixin = seal (
2   ('nearZero _ & 'self self ->
3     (self 'l1 ()) <= 4)) in
4 mixin 'nearZero ()

```

As we would expect, this code is not typeable because `mixin`, the value of `self`, does not have a scape which can handle the `'l1` message.

TinyBang mixins are first-class values; the actual mixing need not occur until runtime. For instance, the following code selects a weighting metric to mix into a point based on some runtime condition `cond`.

```

1 def cond = (runtime boolean) in
2 def point = (as above) in
3 def w1 = ('weight _ & 'self self ->
4   self.x + self.y) in
5 def w2 = ('weight _ & 'self self ->
6   self.x - self.y) in
7 def mixedPoint = seal (point &
8   (if cond then w1 else w2)) in
9 mixedPoint 'weight ()

```

Inheritance Inheritance can be encoded with similar ease. As above, we show inheritance using objects rather than classes for simplicity; applying these concepts to classes provides no significant technical challenge. We consider the case in which we define an extension of the above point object which includes a concept of brightness in a field `k`:

```

1 def point = (as above) in
2 def brightPoint = seal (
3   def super = point in
4     point & 'k 255 &
5     ('isZero _ & 'self self ->
6      self.k == 0 and
7      super ('isZero () & 'self self)))
8 in brightPoint 'isZero ()

```

As with mixins, we extend the object by onioning new scapes and fields onto the right. But recall that the onioning operator `&` is *asymmetric*; the rightmost scape matching a message is invoked. Because this extended object has a scape which handles `'isZero` messages, that scape will be invoked (as opposed to the original `'isZero`-matching scape from `point`). Because we have bound the parent object using the `super` variable, we may use it to statically invoke `point`'s scape from within our new `'isZero`-handling scape. To ensure that any messages it receives are potentially handled by `brightPoint` methods, we explicitly pass to `super` our *own* `self`.

Classes and subclasses The above examples show mixins and inheritance over objects. Classes are encoded by taking the view that they are merely objects with construction methods for other objects. The class for the point object above can be encoded as below. This class also includes a counter which tracks the number of points which have been created.

```

1 def Point = seal (
2   'created 0 &
3   ('new ('x x & 'y y) & 'self self ->
4     self.created = self.created + 1 in
5     seal (
6       'x x & 'y y &
7       ('li _ & 'self self -> self.x + self.y) &
8       ('isZero _ & 'self self ->
9         self.x == 0 and self.y == 0))) in
10 def point = Point 'new ('x 1 & 'y 2) in
11 ...

```

Subclasses of classes can be defined in direct parallel to how extension of objects was defined above.

As previously stated, a language in practice would benefit from sugar to standardize and beautify class definitions. Nonetheless, users of the class may wish to examine the desugared form of the class definitions they create; as they are first-class values, these classes may be passed as arguments to other routines, pattern-matched, and so on. The lightweight encoding we have specified in this section ensures that the desugared form of a class is legible.

2.5 Programming Patterns with Scapes and Onions

Scapes and onions as defined above are motivated by our typeable, variant-based encoding for objects. However, this expressiveness also permits us to code in patterns not strictly in line with traditional object-oriented concepts such as objects or inheritance. This section provides a few examples of the emergent expressiveness of TinyBang's onions and scapes.

Operator overloading The pattern-matching semantics of scapes also provide a natural definition of operator overloading; we merely view an operator as an onion of scapes matching against the arguments. Operator overloading generally refers to infix operators; here we avoid complicating the discussion with matters of parsing and consider overloading on functions only. We might originally define negation on the integers as

```

1 def neg = x:int -> 0 - x in ...

```

Later code could extend the definition of negation to include boolean values. Booleans in TinyBang are represented as `'True ()`

and `'False ()`. Because operator overloading assigns new meaning to an existing symbol, we redefine `neg` to include all of the behavior of the old `neg` as well as new cases for `'True` and `'False`:

```

1 ...
2 def neg = neg
3   & ('True unit -> 'False ())
4   & ('False unit -> 'True ()) in ...

```

Negation is now overloaded: `neg 4` evaluates to `-4`, and `neg 'True ()` evaluates to `'False ()` due to how scape application matches patterns. Furthermore, these operator overloadings can be defined in a lexically scoped manner; thus, operators may be defined to have different meanings in specific contexts.

The biggest problem with reasoning about operator overloading in this fashion is that it admits the same *overriding* properties that we described for objects. TinyBang can easily solve this problem by including a symmetric concatenation operator `&!` which produces a type error if its arguments overlap.

Data sharing TinyBang has two additional operations over onions: onion projection (`&.`) and onion subtraction (`&-`). Both of these operations use a projector to keep (or discard) components of an onion which match the projector. Projectors, represented in Figure 1 as π , are a shallow form of pattern with no variable bindings. For instance, consider the following:

```

1 def a = 'A 1 & 'B 2 & 'C 3 in
2 def b = a &- 'A in
3 def c = a &. 'A in
4 b.B = 4 in
5 ...

```

In the above code, `b` lacks the `'A` component of `a` but is otherwise structurally the same. `c` contains *only* the `'A` component of `a` and nothing else. Because `b` is a structural copy of `a` (including copies of all of the references in its labels), the assignment of a value to `b`'s `'B` component also affects `a`; by the end of the above code, `a` is `'A 1 & 'B 4 & 'C 3`.

The object-oriented analogue of these operations is being able to strip away or extract fields from an object. This can result in a form of data sharing. Consider the following code (in which we leave objects unsealed for brevity):

```

1 def obj1 = seal (
2   'x 0 &
3   ('take _ & 'self self ->
4     self.x = self.x + 1 in self.x)) in
5 def obj2 = seal (
6   (obj1 &. 'x) & 'y 2 &
7   ('sum _ & 'self self ->
8     self.x + self.y)) in
9 ...

```

In the above, `obj1` is a counter object increments an internal counter and returns that counter's current state. `obj2` is simply an object which sums two numbers. However, the `'x` label in `obj2` is drawn from `obj1`; they are, in essence, the same variable. This means that `obj2`'s response to the `'sum` message will change each time `obj1` is sent a `'take` message.

Exceptions While the TinyBang grammar in Figure 1 does not include a mechanism for exceptions, they are easy to incorporate. Typical exception semantics can be obtained by (1) extending the expression grammar with a `throw` expression and (2) extending the pattern grammar with an exception form. This observation is made in [7] and is a natural consequence of the first-class cases defined in [6]. TinyBang improves upon this behavior. Just as the aforementioned extensible cases can extend a case with a single clause but not generally concatenate cases, their exception handling mechanism only permits extension by one exception handler at a

time. TinyBang onions permit general concatenation whether the scapes contained within are using exception patterns or not. The following is an example of how exception handling can be written using this extension of TinyBang:

```
1 def f = z -> throw 'Bar z in
2 def handler = (n -> n)
3   & (exn 'Foo _ -> 0)
4   & (exn 'Bar x -> x) in
5 handler (f 4)
```

The above code would evaluate to 4. When `f 4` is evaluated, an exception with the payload `'Bar 4` is raised. When the argument to an application evaluates to an exception, the applied onion's exception-matching scapes are used to locate a match for the exception payload. If no such match is found, the entire application expression evaluates to the exception (which will then propagate upward). The identity function in the above `handler` is required to ensure that the argument will be passed on if no exception is raised.

2.6 Record vs variant encoding comparison

Now that we have overviewed the joys of standing on your head, let us step back to compare the objects-as-records and objects-as-variants approaches.

First, note that TinyBang's scapes and onions can also be used to build a pure record-based encoding of objects that would include support for first-class messages. Unlike previous encodings of first-class messages [17, 18], TinyBang provides a view which uses subtyping and dependent pattern types and does not require conditional constraints, row types, or a higher-kinded system. Such a record-based encoding is the true dual of the variant-based encoding presented above; the variant-encoded message `'msg arg`, for example, dualizes to `obj -> obj.msg arg` in the record-encoded system.

It is also possible to use first-class labels to form a record-based encoding of objects; first-class messages are then easily encoded with first-class labels. Fluid object types [11] are a recent system which uses this approach to infer types in scripting languages such as JavaScript. (Note that we do intend to investigate in the future whether the addition of first-class labels to TinyBang will give us additional added flexibility; they should not be theoretically challenging to add.)

We focus on the variant-based encoding of messages because we believe they are a more simple and direct syntax; for example, the variant form `'msg arg` may be directly pattern-matched. The TinyBang variant encoding also cleanly puts fields and methods into different syntactic sorts (labels and scapes, respectively). By analogy with boolean logic, it is possible to use only $\wedge$ (or only $\vee$) since DeMorgan's Laws allow one to be defined in terms of the other; in practice, however, it is far more pleasant to live on both sides of the duality. We believe the most natural view is to put fields on the record ($\wedge$) side of the duality and methods on the variant ($\vee$) side. We were motivated to switch from a record-based encoding to the variant-record hybrid presented here due to how pleasantly simple the self-reference encoding of objects becomes with the simple `seal` function. A record-based encoding which declares `seal` as a function and which supports object extensibility as outlined in Section 2.5 is either very complex or impossible to define (we tried, not very hard, and failed).

Overall, the use of variants as messages is more suitable for our uses and, more generally, is an approach that is woefully neglected in the current literature.

3. Type System

In this section, we outline the type system for TinyBang in terms of a smaller language: MicroBang. MicroBang has a simpler syntax and lacks both state and complex patterns. We have developed the

$e ::= x \mid \text{lbl } e \mid e \& e \mid e \& \pi \mid e \& . \pi$	<i>MicroBang expressions</i>
$\mid e \mid e \mid e + e \mid e - e$	
$\mid () \mid \mathbb{Z} \mid (\&) \mid \chi \rightarrow e$	<i>MicroBang literals</i>
$x ::= [\text{a-zA-Z_}][\text{a-zA-Z0-9_}]^*$	
$\text{lbl} ::= [\text{a-zA-Z0-9_}]^+$	
$v ::= () \mid \mathbb{Z} \mid \text{lbl } v \mid v \& v \mid (\&) \mid \chi \rightarrow e$	
$\mathbb{Z} ::= [0123456789]^+ \mid -[0123456789]^+$	
$\chi ::= x : \chi^p$	<i>patterns</i>
$\chi^p ::= \text{tprim} \mid \text{lbl } x \mid \text{fun} \mid \text{any}$	
$\pi ::= \text{tprim} \mid \text{lbl} \mid \text{fun}$	<i>projectors</i>

Figure 3: MicroBang Grammar

full TinyBang type rules and omit features here only for conciseness of presentation. The syntax of MicroBang appears in Figure 3.

The TinyBang type system is designed to realize an intuition of types and subtyping held by programmers accustomed to dynamically-typed languages: a method call which can be understood using the philosophy of duck typing should simply work. For this reason, we base our system on expressive polymorphic set constraint type systems [4, 10, 12, 18, 28] adding several extensions for greater expressiveness.

Parametric polymorphism is important for expressiveness and we would like to avoid the arbitrary cutoffs of polymorphism that are found with `let`-polymorphism or local type inference. In order to obtain the maximal amount of polymorphism, every scape in MicroBang is inferred a polymorphic type; this approach is inspired by flow analyses [2, 23] and has previously been ported to a type constraint context [28]. In the ideal, each call site then produces a fresh instantiation of the type variables. Unfortunately, this ideal cannot be implemented since a single program can have an unbounded number of function call sequences and so produce infinitely many instantiations. A standard solution to dealing with this case in the program analysis literature is to simply chop off call sequences at some fixed point. While arbitrary cutoffs may work for program analyses, they work less well for type systems: they make the system hard for users to understand and potentially brittle to small refactorings.

We have developed an approach here starting from our previous work on this topic [14, 15, 28] to produce polymorphism on functions which is “nearly maximal” in that common programming patterns have expressive polymorphism, suffer no arbitrary cutoffs, and are not too inefficient. In this approach, non-recursive functions are maximally polymorphic. Recursive call cycles are polymorphic only on the first traversal; type variables are reused when a recursive cycle is closed. We discuss our polymorphism model in more detail in Sections 3.3 and 3.5.

3.1 Type Grammar

We now present the mechanics of the MicroBang type system. Typechecking consists of two phases: derivation, in which the expression is assigned a type variable and a set of constraints based on its structure; and closure, during which the constraint set is deductively closed over a logical system in a monotonically increasing fashion, and then checked for consistency. We begin our discussion by presenting the grammar of types and constraints for MicroBang, shown in Figure 4. In that figure, we use the notation $[R, \dots, R]$ to denote a list of R . Throughout this paper, we use $\oplus$ to denote list concatenation.

Type derivation occurs over a fixed program. We assign each subexpression of that program a unique expression identifier; we use ι to range over all expression identifiers. Program points are named by an expression identifier ι and an index. Type variables are named by a program point and a *contour identifier* c . Contour

$\ell ::= \langle \iota, \mathbb{Z} \rangle$
$\alpha ::= \ell^c$
$\mathcal{C} ::= [\ell, \dots, \ell]$ <i>contour identifiers</i>
$\tau ::= \text{tprim} \mid \text{lbl } \alpha \mid \alpha \& \alpha \mid \alpha \& -\pi \mid \alpha \& . \pi \mid (\&)$ $\quad \mid \forall \bar{\alpha}. \tau^x \rightarrow \alpha \setminus C$
$\bar{\tau} ::= [\tau, \dots, \tau]$
$\tau^x ::= \alpha \sim \tau^{x^p}$
$\tau^{x^p} ::= \text{tprim} \mid \text{lbl } \alpha \mid \text{fun} \mid \text{any}$
$\text{tprim} ::= \text{int} \mid \text{unit}$
$c ::= \tau <: \alpha \mid \alpha <: \alpha \mid \alpha <: \alpha \rightarrow \alpha \mid \alpha \text{ op } \alpha <: \alpha \mid \frac{1}{2}$
$C ::= c^*$
$\bar{C} ::= C \mid \clubsuit$

Figure 4: MicroBang Type Grammar

identifiers are defined by lists of program points and are used for tracking polymorphism. We defer discussion of contours until Section 3.3. For now, a reader may assume that all contour identifiers are equal; indeed, this is the case until constraint closure.

The types τ in the type grammar represent concrete *lower* bounds in MicroBang's type system. They are, informally and respectively, primitives, labels of cells, the onion concatenation of two other types, the subtraction and projection of a term from an onion, the empty onion type, and polymorphic functions. Functions are associated with a set of constraints which are expanded whenever that function is applied.

MicroBang's constraint grammar is inspired by [25] in that the lower-bounding types τ are not used as upper bounds. Instead, upper bounds define uses of types; for instance, the constraint $\alpha_1 <: \alpha_2 \rightarrow \alpha_3$ describes the use of α_1 as a function where input is a subtype of α_2 and output is a supertype of α_3 . This allows MicroBang to be precise about how types are used.

The grammar uses a symbol $\clubsuit$ to represent a special “none” value which is used in the relations discussed in Section 3.4. We also include a special constraint $\frac{1}{2}$ which indicates that a contradiction has occurred and typechecking should fail.

3.2 Type Derivation

Type derivation is described in terms of a four place relation $\Gamma \vdash e : \alpha \setminus C$ which relates a type context, an expression, a type variable, and a set of constraints over that type variable.

A type context in derivation is a set Γ of mappings $x : \alpha$ from a variable name x onto a type variable α . We define a context Γ to be well formed if $\forall x : \alpha, x' : \alpha'. x = x' \implies \alpha = \alpha'$; that is, no two mappings exist which have different values but the same key. We assume that we will only operate over well-formed contexts.

Given the nature of contexts, the disjoint union $\Gamma_1 \uplus \Gamma_2$ of two well-formed contexts Γ_1 and Γ_2 is well-formed; however, the union $\Gamma_1 \cup \Gamma_2$ is not necessarily well-formed. We use the notation $\Gamma_1 \uplus \Gamma_2$ to denote $\Gamma_2 \uplus \{x : \alpha \mid x : \alpha \in \Gamma_1, \forall \alpha'. x : \alpha' \notin \Gamma_2\}$.

We use the notation ${}^n\alpha$ to select type variables as follows:

$${}^n\alpha = \ell^{\alpha} \text{ where } \ell = \langle \iota, n \rangle$$

Thus an expression with identity ι would have ${}^2\alpha = \langle \iota, 2 \rangle^{\alpha}$.

We define a distinguished type variable, α_{unit} , to represent a unit type. We take some ι_{unit} used in evaluation which is *not* associated with any expression in e and define $\alpha_{\text{unit}} = \langle \iota_{\text{unit}}, 0 \rangle^{\alpha}$. In addition to the constraints below, derivation is always assumed to include the constraint $\text{unit} <: \alpha_{\text{unit}}$.

Type derivation uses a function TPATTYPE to type patterns. That function is defined as follows:

$$\text{TPATTYPE}(x : \chi^p, \iota) = \langle \Gamma \uplus \{x : {}^3\alpha\}, {}^3\alpha \sim \tau^{x^p} \rangle$$

where $\text{TPATPRITYPE}(\chi^p, \iota) = \langle \Gamma, \tau^{x^p} \rangle$ and

INTEGER LITERAL $\frac{}{\Gamma \vdash [0-9]^+ : {}^1\alpha \setminus \{\text{int} <: {}^1\alpha\}}$	UNIT LITERAL $\frac{}{\Gamma \vdash () : {}^1\alpha \setminus \{\text{unit} <: {}^1\alpha\}}$
EMPTY UNION LITERAL $\frac{}{\Gamma \vdash (\&) : {}^1\alpha \setminus \{(\&) <: {}^1\alpha\}}$	LABEL $\frac{\Gamma \vdash e : \alpha_2 \setminus C}{\Gamma \vdash \text{lbl } e : {}^1\alpha \setminus \{\text{lbl } \alpha_2 <: {}^1\alpha\} \cup C}$
UNION $\frac{\Gamma \vdash e_1 : \alpha_1 \setminus C_1 \quad \Gamma \vdash e_2 : \alpha_2 \setminus C_2}{\Gamma \vdash e_1 \& e_2 : {}^0\alpha \setminus \{\alpha_1 \& \alpha_2 <: {}^0\alpha\} \cup C_1 \cup C_2}$	
UNION SUBTRACTION $\frac{\Gamma \vdash e : \alpha_2 \setminus C}{\Gamma \vdash e \& -\pi : {}^1\alpha \setminus \{\alpha_2 \& -\pi <: {}^1\alpha\} \cup C}$	
UNION PROJECTION $\frac{\Gamma \vdash e : \alpha_2 \setminus C}{\Gamma \vdash e \& . \pi : {}^1\alpha \setminus \{\alpha_2 \& . \pi <: {}^1\alpha\} \cup C}$	HYPOTHESIS $\frac{x : \alpha_1 \in \Gamma}{\Gamma \vdash x : \alpha_1 \setminus \square}$
SCOPE $\frac{\text{TPATTYPE}(\chi, \iota) = \langle \Gamma', \tau^x \rangle \quad \Gamma \uplus \Gamma' \vdash e : \alpha_2 \setminus C}{\Gamma \vdash \chi \rightarrow e : {}^1\alpha \setminus \{(\forall \bar{\alpha}. \tau^x \rightarrow \alpha_2 \setminus C) <: {}^1\alpha\}}$	
APPLICATION $\frac{\Gamma \vdash e_1 : \alpha_1 \setminus C_1 \quad \Gamma \vdash e_2 : \alpha_2 \setminus C_2}{\Gamma \vdash e_1 e_2 : {}^2\alpha' \setminus \{\alpha_1 <: {}^1\alpha' \rightarrow {}^2\alpha', \alpha_2 <: {}^1\alpha'\} \cup C_1 \cup C_2}$	
LAZY OPERATION $\frac{\Gamma \vdash e_1 : \alpha_1 \setminus C_1 \quad \Gamma \vdash e_2 : \alpha_2 \setminus C_2}{\Gamma \vdash e_1 \text{ op } e_2 : {}^0\alpha \setminus \{\alpha_1 \text{ op } \alpha_2 <: {}^0\alpha\} \cup C_1 \cup C_2}$	

Figure 5: MicroBang Type Derivation

$$\begin{aligned} \text{TPATPRITYPE}(\text{tprim}, \iota) &= \langle \square, \text{tprim} \rangle \\ \text{TPATPRITYPE}(\text{lbl } x, \iota) &= \langle \{x : {}^4\alpha\}, \text{lbl } {}^4\alpha \rangle \\ \text{TPATPRITYPE}(\text{fun}, \iota) &= \langle \square, \text{fun} \rangle \\ \text{TPATPRITYPE}(\text{any}, \iota) &= \langle \square, \text{any} \rangle \end{aligned}$$

Using this function, the type derivation rules appear in Figure 5. It should be noted that these rules are not subtyping judgments; as is the standard in the constraint subtyping literature [5], we need not present judgments for a subtyping relation. Instead, the subtyping relation is implicit in whether or not closure over the constraints inferred from Figure 5's derivation will produce a contradiction [10]. We defer discussion of constraint closure until Section 3.5 since it is defined in terms of a number of supporting relations.

We begin by considering the integer literal rule. In any context, an integer literal expression is assigned a type variable ${}^1\alpha$ into which the concrete lower bound int will flow. Similar rules apply for other literal forms. These are the means by which ground types enter the flow graph.

The onion derivation rule for an expression $() \& 5$ first performs derivations on the subexpressions $()$ and 5 ; it then creates a constraint which indicates that an onion of these two values flows into the type variable representing the entire expression. Using natural numbers to represent points in the program, such an expression might result in the overall type ${}^1\alpha \setminus \{\text{unit} <: {}^2\alpha, \text{int} <: {}^3\alpha, {}^2\alpha \& {}^3\alpha <: {}^1\alpha\}$.

Scope types are particularly important in MicroBang; they provide both polymorphism and dependent pattern typing. Scope types are always polymorphic; the $\bar{\alpha}$ represents those type variables which are free in the scape's body. As in [25], we polyinstantiate functions during *closure* and not derivation; thus, the application rule merely adds an upper-bounding constraint.

$$\begin{aligned}
\mathcal{V} &::= && \text{vertices} \\
\mathbf{v} &::= \{v, \dots, v\} \\
\mathcal{E} &::= \langle v, v, \ell \rangle && \text{edges} \\
\mathbf{e} &::= \{e, \dots, e\} \\
\mathcal{T} &::= \langle \mathcal{V}, \mathcal{E} \rangle && \text{contour trees}
\end{aligned}$$

Figure 6: Contour Tree Grammar

The input for a scape is represented as a pattern type τ^x ; this is critical to dependent pattern typing. When an onion of scapes is applied, each scape type includes a set of constraints representing its behavior. Because the pattern type is available when application closure occurs, we can use it to select only the scape (and corresponding constraint set) which will actually be used. MicroBang’s dependent pattern typing has the same expressiveness to conditional constraints [5, 12, 19]. But in TinyBang, dependent pattern typing also captures the constraints for state modifications; conditional constraints do not model this behavior. For instance, we statically know that the expression

```

1 ( ( 'A int & 'B x -> x = 2 in x )
2   & ( 'A unit & 'B x -> x = () in x ) )
3 ( 'A 1 & 'B 0 )

```

has the type `int` because the latter scape will never be invoked. Using dependent pattern types, the TinyBang type system can correctly infer this behavior; the constraints in the latter scape which allow `unit` to flow into the `'B` cell are never introduced into the global constraint set.

3.3 Contours and Contour Trees

We take a brief aside to discuss type contours. Each time application is processed in the constraint closure described below, the type variables in the body of the applied scape are instantiated with a new contour; in essence, the flow graph representing the function’s body is duplicated. This is the means by which the MicroBang type system achieves polymorphism; these new variables are only used to represent the flow of a specific scape application. Decidability is achieved by reusing old contours when recursion is detected.

In order to determine when and which contours are reused, we make use of a *contour tree*. A contour tree is, more precisely, a rooted multi-tree with self loops. Edges in this tree are annotated with a single program label ℓ representing the site of the application which created them; vertices are unannotated. The grammar describing this system appears in Figure 6.

A contour tree $\mathcal{T} = \langle \mathcal{V}, \mathcal{E} \rangle$ is *well-formed* if (1) it is non-empty and (2) for all $\langle v_1, v_2, \ell_1 \rangle$ and $\langle v_1, v_3, \ell_2 \rangle$ in $\mathcal{E}$, $\ell_1 \neq \ell_2$; that is, no vertex has more than one outgoing edge with a given ℓ . In the following discussion, we only describe well-formed contour trees.

Extension When new contours are created, they are represented in the contour tree by an *extension*. In Figure 7, we define a function `CEXTEND` over contour trees which ensures that a given tree contains a specified path. `CEXTEND` accepts as input a contour tree and a path in the form of a contour identifier; it produces a resulting tree containing that path.

Reduction When a contour tree contains a path which repeats a call site, the type system has detected recursion. In order to ensure termination, the contour tree is *reduced*; this creates equivalence classes of contours, effectively assigning only a single contour to each recursive call cycle. We define contour tree reduction as a relation $\otimes$; we write $\mathcal{T}_1 \otimes \mathcal{T}_2$ to indicate that $\mathcal{T}_1$ reduces to $\mathcal{T}_2$. This relation is defined in terms of a tree rewriting function `CIDENTIFY`.

The function `CIDENTIFY` shown in Figure 8 models vertex identification over contour trees; given a set of vertices and a tree, it produces the tree in which those vertices are merged. The identity

$$\begin{aligned}
\text{CEXTEND}(\mathcal{T}, c) &= \text{CEXTHelp}(\mathcal{T}, c, v) \text{ where } v \text{ is the root of } \mathcal{T} \\
\text{CEXTHelp}(\langle \mathcal{V}, \mathcal{E} \rangle, c, v) &= \begin{cases} \mathcal{T} & \text{when } c = [] \\ \text{CEXTHelp}(\langle \mathcal{V}, \mathcal{E} \rangle, c', v') & \text{when } c = [\ell] \oplus c', \langle v, v', \ell \rangle \in \mathcal{E} \\ \text{CEXTHelp}(\langle \mathcal{V} \cup \{v'\}, \mathcal{E} \cup \{v, v', \ell\} \rangle, c', v') & \text{when } c = [\ell] \oplus c', \langle v, v', \ell \rangle \notin \mathcal{E} \end{cases}
\end{aligned}$$

Figure 7: CEXTEND Definition

For a given $\mathbf{v}_{\text{MERGE}}$ and $\mathcal{T}_{\text{IN}} = \langle \mathbf{v}_{\text{IN}}, \mathcal{E}_{\text{IN}} \rangle$, let

- $\mathbf{v}_{\text{REPL}} \notin \mathbf{v}_{\text{IN}}$
- $\text{CRENAME}(v) = \begin{cases} \mathbf{v}_{\text{REPL}} & \text{when } v \in \mathbf{v}_{\text{MERGE}} \\ v & \text{otherwise} \end{cases}$
- $\mathbf{v}_{\text{OUT}} = \{\mathbf{v}_{\text{REPL}}\} \cup \mathbf{v}_{\text{IN}} - \mathbf{v}_{\text{MERGE}}$
- $\mathcal{E}_{\text{OUT}} = \{ \langle \text{CRENAME}(v_{\text{SRC}}), \text{CRENAME}(v_{\text{DST}}), \ell \rangle \mid \langle v_{\text{SRC}}, v_{\text{DST}}, \ell \rangle \in \mathcal{E}_{\text{IN}} \}$

Then $\text{CIDENTIFY}(\mathcal{T}_{\text{IN}}, \mathbf{v}_{\text{MERGE}}) = \mathcal{T}_{\text{OUT}} = \langle \mathbf{v}_{\text{OUT}}, \mathcal{E}_{\text{OUT}} \rangle$.

Figure 8: CIDENTIFY Definition

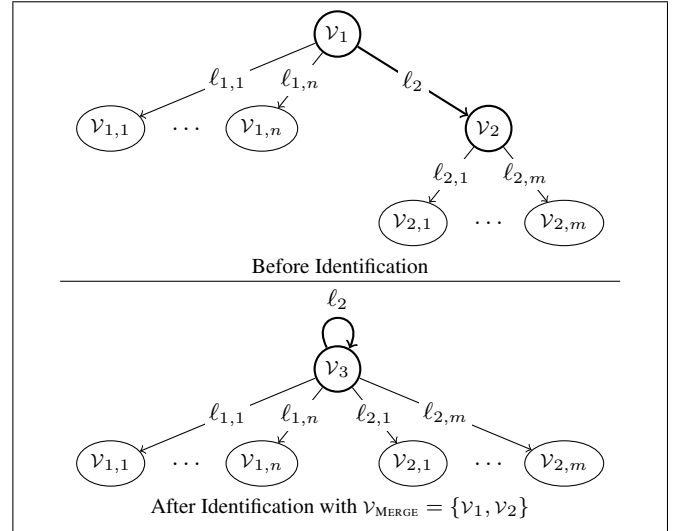


Figure 9: Vertex Identification Example

of the new vertex is the union of the old vertices’ identities. This operation does not eliminate any edges between the merged vertices; any edges between the set of vertices being merged become self-loops on the new vertex in the resulting tree.

The `CIDENTIFY` function will always output a tree when the input set $\mathbf{v}_{\text{MERGE}}$ forms a path. This is always the case in the tree reduction definition. The diagram in Figure 9 gives an example of a vertex identification between two nodes.

The reduction relation We define a relation $\otimes$ which describes reduction on contour trees; we write $\mathcal{T}_1 \otimes \mathcal{T}_2$ to indicate that $\mathcal{T}_1$ reduces to $\mathcal{T}_2$. This operation is defined as follows:

$\mathcal{T}_1 \otimes \mathcal{T}_2$ iff there exists some sequence of edges $\langle v_0, v_1, \ell_1 \rangle, \dots, \langle v_{n-1}, v_n, \ell_n \rangle$ such that $n \geq 2$, $\ell_1 = \ell_n$, and $\text{CIDENTIFY}(\mathcal{T}_1, \{v_0, \dots, v_n\}) = \mathcal{T}_2$

We use the notation $\mathcal{T}_1 \otimes \mathcal{T}_2$ to signify both of the following:

- Either $\tau_1 = \tau_2$ or there exists some τ_3 such that $\tau_1 \odot \tau_3$ and $\tau_3 \odot \tau_2$ (that is, τ_1 transitively reduces to τ_2) and
- There exists no τ_4 such that $\tau_2 \odot \tau_4$

Contour equivalence When scape applications are expanded in closure, they are initially assumed to be non-recursive. When a given application is discovered to be part of a recursive cycle, a contour tree reduction occurs. For this reduction to affect constraint closure, we must define equivalence between contours. We then use this equivalence definition to define constraint equivalence.

Because contour identifiers are lists of program labels and because edges in the contour tree are annotated with program labels, a contour identifier describes a path (or vertex) in a contour tree. Because there may exist multiple edges from one node to another and because self-loops exist, multiple paths in a given contour tree may refer to the same vertex. We define an equivalence relation $\simeq^T$ such that two contour identifiers are equivalent if, as paths from the root of a contour tree τ , they arrive at the same vertex. We write $\simeq$ when τ is evident from context. This relation is defined as follows:

For a given $\tau = \langle \nu, \mathcal{E} \rangle$, $[\ell_1, \dots, \ell_n] \simeq^T [\ell'_1, \dots, \ell'_m]$ if all of the following are true:

- $\{\langle \nu_0, \nu_1, \ell_1 \rangle, \dots, \langle \nu_{n-1}, \nu_n, \ell_n \rangle\} \subseteq \mathcal{E}$
- $\{\langle \nu_0, \nu'_1, \ell'_1 \rangle, \dots, \langle \nu_{m-1}, \nu'_m, \ell'_m \rangle\} \subseteq \mathcal{E}$
- ν_0 is the root of τ and $\nu_n = \nu'_m$

This equivalence definition allows us to define equivalence classes on contour identifiers; we use the notation $[c]^T$ to denote the equivalence class containing c under the equivalence relation $\simeq^T$. Again, we elide τ when it is evident from context.

Constraint equivalence We also overload the above equivalence relation notation with a natural homomorphism over other grammatical constructs. For instance:

$$\begin{aligned}
\alpha_1 <: \alpha_2 &\simeq \alpha_3 <: \alpha_4 && \text{iff } \alpha_1 \simeq \alpha_3 \text{ and } \alpha_2 \simeq \alpha_4 \\
\alpha_1 \& \alpha_2 &\simeq \alpha_3 \& \alpha_4 && \text{iff } \alpha_1 \simeq \alpha_3 \text{ and } \alpha_2 \simeq \alpha_4 \\
\alpha_1 \sim \tau_1^{\chi^p} &\simeq \alpha_2 \sim \tau_2^{\chi^p} && \text{iff } \alpha_1 \simeq \alpha_2 \text{ and } \tau_1^{\chi^p} \simeq \tau_2^{\chi^p} \\
\ell_{\alpha^c} &\simeq \ell_{\alpha^{c'}} && \text{iff } c \simeq c' \\
&\vdots &&
\end{aligned}$$

We likewise extend the equivalence class notation to cover other grammatical constructs. For example, if $\ell_{\alpha^{c_1}} \simeq \ell_{\alpha^{c_2}}$, then we would write $[\text{'A } \ell_{\alpha^{c_1}}]$ to denote at least the set $\{\text{'A } \ell_{\alpha^{c_1}}, \text{'A } \ell_{\alpha^{c_2}}\}$.

3.4 Closure Relations

The next step of typechecking is closure over the set of constraints obtained by type derivation. This closure is defined in terms of a number of relations, the definitions for which appear in this section. Each of these relations is implicitly indexed by a constraint set $\hat{C}$ which is typically the current global constraint set. We sometimes write constraints in place of predicates; writing the constraint c in place of a predicate is syntactic sugar for $c \in \hat{C}$.

Concretization The first and simplest relation we define is concretization; see Figure 10. This relation determines those concrete types in a constraint set which may flow to a given type variable. While similar to a transitive subtype closure rule, concretization guarantees us that only concrete lower bounds are propagated.

Projection Next, we define type-based projection; this definition appears in Figure 11. This is a three place relation between a type, a projector, and another type. The projection relation determines

$$\tau \blacktriangleleft: \alpha_n \stackrel{\text{def}}{=} \exists \alpha_1, \dots, \alpha_{n-1}. (\tau <: \alpha_1) \quad \wedge (\alpha_1 \simeq \alpha_2) \\
\quad \wedge (\alpha_2 <: \alpha_3) \quad \wedge (\alpha_3 \simeq \alpha_4) \\
\quad \wedge \dots \\
\quad \wedge (\alpha_{n-2} <: \alpha_{n-1}) \wedge (\alpha_{n-1} \simeq \alpha_n)$$

Figure 10: TinyBang Concretization Relation

PRIMITIVE PROJECTION		LABEL PROJECTION	
$\overline{\text{tprim}} \xrightarrow{\text{tprim}} [\text{tprim}]$		$\overline{\text{lbl } \alpha} \xrightarrow{\text{lbl } \alpha} [\text{lbl } \alpha]$	
ONION PROJECTION			
$\tau_1 \blacktriangleleft: \alpha_1$	$\tau_2 \blacktriangleleft: \alpha_2$	$\tau_1 \xrightarrow{\pi} \bar{\tau}_1$	$\tau_2 \xrightarrow{\pi} \bar{\tau}_2$
$\alpha_1 \& \alpha_2 \xrightarrow{\pi} \bar{\tau}_1 \oplus \bar{\tau}_2$			
ONION SUB. PROJECTION		ONION PROJ. PROJECTION	
$\pi \neq \pi'$	$\tau \blacktriangleleft: \alpha \quad \tau \xrightarrow{\pi} \bar{\tau}$	$\pi = \pi'$	$\tau \blacktriangleleft: \alpha \quad \tau \xrightarrow{\pi} \bar{\tau}$
$\alpha \& - \pi' \xrightarrow{\pi} \bar{\tau}$		$\alpha \& . \pi' \xrightarrow{\pi} \bar{\tau}$	
SCAPE PROJECTION		PRIM. FAIL	
$\overline{\forall \bar{\alpha}. \tau^X \rightarrow \alpha \setminus C} \xrightarrow{\text{fub}} [\forall \bar{\alpha}. \tau^X \rightarrow \alpha \setminus C]$		$\tau \neq \text{tprim}$	
$\tau \xrightarrow{\text{fub}} []$		$\tau \xrightarrow{\text{tprim}} []$	
LABEL FAIL		ONION SUB. FAIL	
$\tau \neq \text{lbl } \alpha$	$\pi = \pi'$	$\pi \neq \pi'$	
$\tau \xrightarrow{\text{lbl}} []$	$\alpha \& - \pi' \xrightarrow{\pi} []$	$\alpha \& . \pi' \xrightarrow{\pi} []$	
EMPTY FAIL		SCAPE FAIL	
$(\&) \xrightarrow{\pi} []$		$\tau \neq \forall \bar{\alpha}. \tau^X \rightarrow \alpha \setminus C$	
		$\tau \xrightarrow{\text{fub}} []$	

Figure 11: MicroBang Projection Rules

a priority-ordered list of ways the original type can be used as the form of type described by the type projector. For example, `int & char` $\xrightarrow{\text{int}} [\text{int}]$ but `unit` $\xrightarrow{\text{fub}} []$. Projection never produces an onion; instead, onions are concretized and explored in a fashion which gives right precedence. This is necessary due to the fact that projection of scapes must produce every available scape so that they may be pattern-matched in order.

Compatibility We define a type/pattern compatibility relation in Figure 12. This is a three-place relation between a type, a pattern type form (τ^X or τ^{χ^p}), and a set of constraints (or the $\ast$ symbol). If a set of constraints is related by compatibility with a type and a pattern type, then that type can be bound to the pattern type in question using the specified constraints. This relation is used to ensure that arguments arriving at a call site will flow correctly into the variables in the scape's pattern. If a given type and pattern type relate to $\ast$, then that type does not match the pattern in question. In the following, we use the notation $\dot{C}_1 \cup \dot{C}_2$ to refer to $C_1 \cup C_2$ (if $\dot{C}_1 = C_1$ and $\dot{C}_2 = C_2$) or to $\ast$ (if either $\dot{C}_1$ or $\dot{C}_2$ is $\ast$).

Application substitution The application substitution relation defines how function application builds new type variables for polyinstantiation. It is a three place relation written $\zeta(\ast, \bar{\alpha}, c)$ where $\ast$ is a type grammar construct such as τ or α . This relation is used to perform deep structural replacement of type variables in a given set of constraints. We thus define application substitution on each type grammar construct. Most of these definitions are simply the natural homomorphisms. For instance:

$$\begin{aligned}
\zeta(\tau <: \alpha_1, \bar{\alpha}, c) &= \zeta(\tau, \bar{\alpha}, c) <: \zeta(\alpha_1, \bar{\alpha}, c) \\
\zeta(\alpha_1 \& \alpha_2, \bar{\alpha}, c) &= \zeta(\alpha_1, \bar{\alpha}, c) \& \zeta(\alpha_2, \bar{\alpha}, c) \\
&\vdots
\end{aligned}$$

BOUND COMPATIBILITY $\frac{\tau \sim \tau^{\chi^p} \setminus \hat{C}}{\tau \sim \alpha \sim \tau^{\chi^p} \setminus \{\tau <: \alpha\} \cup \hat{C}}$	PRIMITIVE COMPATIBILITY $\frac{\tau \text{ tprim } \bar{\tau} \oplus [\text{tprim}]}{\tau \sim \text{tprim} \setminus \emptyset}$
LABEL COMPATIBILITY $\frac{\tau \text{ lbl } \bar{\tau} \oplus [\text{lbl } \alpha_1]}{\tau \sim \text{lbl } \alpha_2 \setminus \{\alpha_1 <: \alpha_2\}}$	FUNCTION COMPATIBILITY $\frac{\tau \text{ fun } \bar{\tau} \oplus [\tau']}{\tau \sim \text{fun} \setminus \emptyset}$
ANY COMPATIBILITY $\tau \sim \text{any} \setminus \emptyset$	PRIMITIVE FAIL $\frac{\tau \text{ tprim } []}{\tau \sim \text{tprim} \setminus \#}$
	LABEL FAIL $\frac{\tau \text{ lbl } []}{\tau \sim \text{lbl } \alpha \sim \tau^{\chi^p} \setminus \#}$
	FUNCTION FAIL $\frac{\tau \text{ fun } []}{\tau \sim \text{fun} \setminus \#}$

Figure 12: MicroBang Compatibility Rules

The only cases which are not natural homomorphisms are:

$$\zeta(\forall \bar{\alpha}_1. \alpha_1 \rightarrow \alpha_2 \setminus C, \bar{\alpha}_2, c) = \forall \bar{\alpha}_1. \alpha_1 \rightarrow \alpha_2 \setminus \zeta(C, \bar{\alpha}_2 - \bar{\alpha}_1, c)$$

$$\zeta(\ell^{\alpha^{c'}}, \bar{\alpha}, c) = \begin{cases} \ell^{\alpha^c} & \text{when } \ell^{\alpha^{c'}} \in \bar{\alpha} \\ \ell^{\alpha^{c'}} & \text{otherwise} \end{cases}$$

Substitution on type variables will only apply a new contour (c) if the old contour (c') is the initial contour ($[]$). This is the case in the use of ζ because it is only used on free type variables captured during derivation and such variables always use the initial contour.

Constraint set extension The constraint set extension function is used to ensure that, during constraint closure, constraints are only added to the global constraint set in each closure step if no equivalent constraint is already present. We denote constraint set extension by the symbol $\oplus$ and define it as follows:

$$C_1 \oplus C_2 = C_1 \cup \{c \mid c \in C_2 \wedge \forall c' \in C_1. c \notin [c']\}$$

3.5 Constraint Closure

Using the above relations, we now specify the constraint closure step as the relation $\langle C, \tau \rangle \xrightarrow{\zeta} \langle C, \tau \rangle$ defined in Figure 13.

We begin illustration of the constraint closure process by analyzing the Integer Addition rule. Given an expression $4 + 3$, we would acquire from derivation the type $\alpha_3 \setminus \{\text{int} <: \alpha_1, \text{int} <: \alpha_2, \alpha_1 + \alpha_2 <: \alpha_3\}$. Concretizing α_1 yields the type int which we can project using the projector int ; the same is true for α_2 . We can thus conclude that the result of the addition is int , which we then constrain as a lower bound of α_3 . The contour tree is largely unused by this rule; it merely provides a notion of constraint equivalence. Integer Equality and Integer Operation Failure work similarly.

The Application rule bears explanation; it is responsible for the usual complexity of application in a type system as well as for pattern matching and precedence. At any call site $\alpha'_1 \rightarrow \alpha'_2$, a value α_0 may arrive; this is the onion of scapes being invoked. We concretize it as τ_1 and project from it all scapes it contains. We then concretize any argument which could arrive at that call site as τ_2 . For that argument, we determine the rightmost scape which is compatible with the provided argument. The compatibility relation provides a set of constraints C' which, when added to the constraint set, cause the contents of the argument to flow into the variables in the pattern type of the scape. In this way, the input type flows into the body of the function. We also introduce a constraint, $\alpha_i <: \alpha'_2$, to connect the function body to the output of the call site.

To achieve polymorphism, we extend the contour tree using the call site to generate a new contour. To achieve termination, we

APPLICATION $\frac{\begin{array}{l} \tau_1 \blacktriangleleft: \alpha_0 \quad \alpha_0 <: \alpha'_1 \rightarrow \alpha'_2 \\ \tau_1 \xrightarrow{\text{fun}} [\forall \bar{\alpha}_1. \tau_1^{\chi} \rightarrow \alpha_1 \setminus C_1, \dots, \forall \bar{\alpha}_n. \tau_n^{\chi} \rightarrow \alpha_n \setminus C_n] \\ \tau_2 \blacktriangleleft: \alpha'_1 \quad \tau_2 \sim \tau_i^{\chi} \setminus C' \quad \forall j > i. \tau_j \sim \tau_j^{\chi} \setminus \# \\ \alpha'_2 = \ell^{\alpha^c} \quad \text{CEXTEND}(\hat{\tau}, c \oplus [\ell]) = \tau' \quad \tau' \ominus \tau \end{array}}{\langle \hat{C}, \hat{\tau} \rangle \xrightarrow{\zeta} \langle \hat{C} \oplus \zeta(C' \cup C_i \cup \{\alpha_i <: \alpha'_2\}, \bar{\alpha}_i, c), \tau \rangle}$
INTEGER ADDITION $\frac{\begin{array}{l} \alpha_1 + \alpha_2 <: \alpha_3 \\ \tau_1 \blacktriangleleft: \alpha_1 \quad \tau_2 \blacktriangleleft: \alpha_2 \quad \tau_1 \text{ int } \bar{\tau}'_1 \quad \tau_2 \text{ int } \bar{\tau}'_2 \end{array}}{\langle \hat{C}, \hat{\tau} \rangle \xrightarrow{\zeta} \langle \hat{C} \oplus \{\text{int} <: \alpha_3\}, \hat{\tau} \rangle}$
INTEGER EQUALITY $\frac{\begin{array}{l} \alpha_1 == \alpha_2 <: \alpha_3 \\ \tau_1 \blacktriangleleft: \alpha_1 \quad \tau_2 \blacktriangleleft: \alpha_2 \quad \tau_1 \text{ int } \bar{\tau}'_1 \quad \tau_2 \text{ int } \bar{\tau}'_2 \end{array}}{\langle \hat{C}, \hat{\tau} \rangle \xrightarrow{\zeta} \langle \hat{C} \oplus \{\text{'True } \alpha_{\text{unit}} <: \alpha_3, \text{'False } \alpha_{\text{unit}} <: \alpha_3\}, \hat{\tau} \rangle}$
NON-FUNCTION APPLICATION $\frac{\begin{array}{l} \tau_1 \blacktriangleleft: \alpha_0 \quad \alpha_0 <: \alpha'_1 \rightarrow \alpha'_2 \\ \tau_1 \xrightarrow{\text{fun}} [\forall \bar{\alpha}_1. \tau_1^{\chi} \rightarrow \alpha_1 \setminus C_1, \dots, \forall \bar{\alpha}_n. \tau_n^{\chi} \rightarrow \alpha_n \setminus C_n] \\ \tau_2 \blacktriangleleft: \alpha'_1 \quad \forall i. \tau_2 \sim \tau_i^{\chi} \setminus \# \end{array}}{\langle \hat{C}, \hat{\tau} \rangle \xrightarrow{\zeta} \langle \hat{C} \oplus \{\ell\}, \hat{\tau} \rangle}$
INTEGER OPERATION FAILURE $\frac{\begin{array}{l} \alpha_1 \text{ op } \alpha_2 <: \alpha_3 \quad \tau \blacktriangleleft: \alpha_i \quad i \in \{1, 2\} \quad \tau \text{ int } [] \end{array}}{\langle \hat{C}, \hat{\tau} \rangle \xrightarrow{\zeta} \langle \hat{C} \oplus \{\ell\}, \hat{\tau} \rangle}$

Figure 13: Closure Rules

then transitively reduce the resulting contour tree; this is relevant if the extension we just performed reveals a recursive call. We then use the application substitution function ζ to polyinstantiate the type variables in the set of constraints describing the input, output, and body flow, granting polymorphism. Finally, we subject this substituted set to the constraint set extension operator $\oplus$ to eliminate constraints for which equivalents already exist in $\hat{C}$; this is key to ensuring termination and preventing unnecessary exponential complexity in closure.

Typechecking Given the above definition of a constraint closure step, we can provide the following definitions:

Definition 1. Given a constraint set C_1 and a contour tree τ_1 ,

1. We write $\langle C_1, \tau_1 \rangle \xrightarrow{\zeta^*} \langle C_2, \tau_2 \rangle$ when either
 - (a) $C_1 = C_2$ and $\tau_1 = \tau_2$, or
 - (b) $\langle C_1, \tau_1 \rangle \xrightarrow{\zeta} \langle C_3, \tau_3 \rangle$ and $\langle C_3, \tau_3 \rangle \xrightarrow{\zeta^*} \langle C_2, \tau_2 \rangle$
2. We write $\langle C_1, \tau_1 \rangle \not\xrightarrow{\zeta}$ to indicate that there exists no C_2 and τ_2 such that $\langle C_1, \tau_1 \rangle \xrightarrow{\zeta} \langle C_2, \tau_2 \rangle$.

We can now define typechecking as follows:

Definition 2. Given a closed e ,

1. $\text{TYPEINFER}(e) = \langle C_n, \tau_n \rangle$ such that $\emptyset \vdash e : \alpha \setminus C_0$ for some α and C_0, τ_0 is the initial contour tree containing one vertex and no edges, $\langle C_0, \tau_0 \rangle \xrightarrow{\zeta^*} \langle C_n, \tau_n \rangle$, and $\langle C_n, \tau_n \rangle \not\xrightarrow{\zeta}$.
2. $\text{TYPECHECK}(e)$ holds if and only if $\text{TYPEINFER}(e) = \langle C, \tau \rangle$ and $\ell \notin C$.

Algorithmic properties The above typechecking algorithm is sound and decidable. We present these statements below:

Theorem 1 (Soundness). Let $e_1 \xrightarrow{S} e_2$ be a small-step evaluation relation for MicroBang; let $e_1 \xrightarrow{S^*} e_2$ be its transitive closure. Let τ_0 be the contour tree with one vertex and no edges. Then

$\forall e. e \xrightarrow{*} \not\vdash \text{ implies } (\emptyset \vdash e : \alpha \setminus C_0), \langle C_0, \tau_0 \rangle \xrightarrow{*} \langle C_n, \tau_n \rangle,$
and $\not\vdash \in C_n$.

Proof of this Theorem is not trivial; we provide here a very high-level sketch of our proof technique. We elect to use a technique different than subject-reduction since it is difficult to re-build type derivations after a single step. Instead we create a so-called *constraint evaluator* which is a formal system lying at the rough midpoint between a type constraint system and a small-step operational semantics: it has actual integer values and does no approximation, but programs are expressed as a set of constraints that are structurally very similar to type constraints. Constraints are never merged; the constraint evaluator may run forever. We can show our type system here is simulated by the constraint evaluator and the constraint evaluator is simulated by the small-step operational semantics, giving us the above Theorem by transitivity.

Theorem 2 (Decidability). *The predicate TYPECHECK is decidable.*

This proof proceeds by demonstrating that there are finitely many closure steps which may occur on any derived constraint set and that each of these closure steps is computable. The latter is achieved by analysis of the operations used in closure; the prior is achieved by demonstrating that only finitely many constraints can be added during closure. In fact, this algorithm takes polynomial time under normal circumstances; worst-case exponential complexity is possible for pathological code similar to the exponential case of let-polymorphism.

4. Conclusions

In this paper, we have shown how it is possible to define a flexible variant encoding for typed objects in a novel core language, TinyBang. Particular advantages of programming in the encoding beyond the usual OO features include support for a more general notion of typed object extension than previous works, including support of fundamentally dynamic extension and extension of objects already actively being messaged; direct expressibility of first-class messages as simple labeled data; a trivial encoding of operator overloading; and enabling of other novel patterns that break the traditional object straitjacket.

Such a flexible object model is possible due to the flexibility of TinyBang, the underlying typed language into which we encode. TinyBang does not have one “blockbuster” extension but gets its significant expressiveness from a combination of a number of improvements to the state of the art in type system design: a more general notion of first-class case, simpler typing for asymmetric record append, a more expressive dependent typing of case/match, concise syntax via type-indexed records, and a highly flexible method for inference of parametric polymorphism.

The complete TinyBang type inference algorithm and a TinyBang interpreter have been implemented in Haskell; with this implementation, we have confirmed correctness of the type inference algorithm and operational semantics on code examples. All examples in this paper typecheck and run in the current implementation³.

The larger picture In this paper we focus on fundamental questions of typing objects. It is not, however, a realistic language design proposal; TinyBang lacks syntactic sugar and other features that a real language needs. We are presently working on a realistic language design and implementation for BigBang [16]; TinyBang constitutes the proposed core for BigBang. We believe BigBang will be appealing because of the potential for great flexibility in coding, coming close to the spirit of dynamically-typed languages,

but with the advantage of full static typechecking and more efficient running times compared to dynamically-typed languages.

References

- [1] M. Abadi and L. Cardelli. *A Theory of Objects*. Springer-Verlag, 1996.
- [2] O. Agesen. The cartesian product algorithm. In *Proceedings ECOOP’95*, volume 952 of *Lecture Notes in Computer Science*, 1995.
- [3] G. Agha. *Actors: a model of concurrent computation in distributed systems*. MIT Press, 1985.
- [4] A. Aiken and E. L. Wimmers. Type inclusion constraints and type inference. In *FPCA*, pages 31–41, 1993.
- [5] A. Aiken, E. L. Wimmers, and T. K. Lakshman. Soft typing with conditional types. In *POPL 21*, pages 163–173, 1994.
- [6] M. Blume, U. A. Acar, and W. Chae. Extensible programming with first-class cases. In *ICFP ’06*, pages 239–250, 2006.
- [7] M. Blume, U. A. Acar, and W. Chae. Exception handlers as extensible cases. In *APLAS ’08*, pages 273–289. Springer-Verlag, 2008.
- [8] V. Bono and K. Fisher. An imperative, first-order calculus with object extension. In *ECOOP’98*, pages 462–497. Springer Verlag, 1998.
- [9] K. B. Bruce, L. Cardelli, and B. C. Pierce. Comparing object encodings. *Information and Computation*, 155(1-2):108 – 133, 1999.
- [10] J. Eifrig, S. Smith, and V. Trifonov. Type inference for recursively constrained types and its application to OOP. In *MFPS*, Electronic Notes in Theoretical Computer Science. Elsevier, 1995.
- [11] A. Guha, J. G. Politz, and S. Krishnamurthi. Fluid object types. Technical Report CS-11-04, Brown University, 2011.
- [12] N. Heintze. Set-based analysis of ML programs. In *LFP*, pages 306–317. ACM, 1994.
- [13] B. Jay and D. Kesner. First-class patterns. *J. Funct. Program.*, 19(2): 191–225, 2009.
- [14] A. Kulkarni, Y. D. Liu, and S. F. Smith. Task types for pervasive atomicity. In *OOPSLA*, pages 671–690. ACM, 2010.
- [15] Y. D. Liu and S. Smith. Pedigree types. In *International Workshop on Aliasing, Confinement and Ownership in object-oriented programming (IWACO)*, 2008.
- [16] P. H. Menon, Z. Palmer, A. Rozenshteyn, and S. Smith. Big Bang: Designing a statically-typed scripting language. In *International Workshop on Scripts to Programs (STOP)*, Beijing, China, 2012.
- [17] S. Nishimura. Static typing for dynamic messages. In *POPL’98*, 1998.
- [18] F. Pottier. A versatile constraint-based type inference system. *Nordic J. of Computing*, 7(4):312–347, 2000.
- [19] F. Pottier. A 3-part type inference engine. In *ESOP’00*, pages 320–335. Springer Verlag, 2000.
- [20] D. Rémy. From classes to objects via subtyping. In *ESOP*, pages 200–220. Springer-Verlag, 1998.
- [21] D. Rémy and J. Vouillon. Objective ml: a simple object-oriented extension of ml. In *Proceedings of the 24th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, POPL ’97, pages 40–53. ACM, 1997.
- [22] M. Shields and E. Meijer. Type-indexed rows. In *POPL*, pages 261–275, 2001.
- [23] O. Shivers. *Control-Flow Analysis of Higher-Order Languages*. PhD thesis, Carnegie-Mellon University, 1991. TR CMU-CS-91-145.
- [24] P. Shroff and S. F. Smith. Type inference for first-class messages with match-functions. In *FOOL 11*, 2004.
- [25] S. F. Smith and T. Wang. Polyvariant flow analysis with constrained types. In *ESOP ’00*. Springer Verlag, 2000.
- [26] M. Wand. Corrigendum: Complete type inference for simple objects. In *LICS*. IEEE Computer Society, 1988.
- [27] M. Wand. Type inference for record concatenation and multiple inheritance. *Information and Computation*, 93(1):1–15, July 1991.
- [28] T. Wang and S. F. Smith. Precise constraint-based type inference for Java. In *ECOOP’01*, pages 99–117, 2001. ISBN 3-540-42206-4.

³With the exception of the exceptions example in Section 2.5 – exceptions are currently being implemented.

Semantics and Types for Objects with First-Class Member Names

Joe Gibbs Politz

Brown University
joe@cs.brown.edu

Arjun Guha

Cornell University
arjun@cs.cornell.edu

Shriram Krishnamurthi

Brown University
sk@cs.brown.edu

Abstract

Objects in many programming languages are indexed by first-class strings, not just first-order names. We define λ_S^{ob} (“lambda sob”), an object calculus for such languages, and prove its untyped soundness using Coq. We then develop a type system for λ_S^{ob} that is built around *string pattern types*, which describe (possibly infinite) collections of members. We define subtyping over such types, extend them to handle inheritance, and discuss the relationship between the two. We enrich the type system to recognize tests for whether members are present, and briefly discuss exposed inheritance chains. The resulting language permits the ascription of meaningful types to programs that exploit first-class member names for object-relational mapping, sandboxing, dictionaries, etc. We prove that well-typed programs never signal member-not-found errors, even when they use reflection and first-class member names. We briefly discuss the implementation of these types in a prototype type-checker.

1. Introduction

In most statically-typed object-oriented languages, an object’s type or class enumerates its member names and their types. In “scripting languages”, member names are first-class strings that can be computed dynamically. In recent years, programmers using these languages have employed first-class member names to create useful abstractions that are applied broadly. Of course, this power also leads to problems, especially when combined with other features like inheritance.

This paper explores uses of first-class member names, a dynamic semantics of their runtime behavior, and a static semantics with a traditional soundness theorem for these untraditional programs. In section 2, we present examples from several languages that highlight uses of first-class members. These examples also show how these languages differ from traditional object calculi. In section 3 we present an object calculus, λ_S^{ob} , which features the essentials of objects in these languages. In section 4, we explore types for λ_S^{ob} . The challenge is to design types that can properly capture the consequences of first-class member names. We especially focus on the treatment of subtyping, inheritance, and their interaction, as well as reflective features such as tests of member presence. Finally, in section 5 we briefly discuss an implementation and its uses.

In summary, we make the following contributions:

1. extract the principle of first-class member names from existing languages;
2. provide a dynamic semantics that distills this feature;
3. identify key problems for type-checking objects in programs that employ first-class member names;
4. extend traditional record typing with sound types to describe objects that use first-class member names; and,
5. briefly discuss a prototype implementation.

We build up our type system incrementally. All elided proofs and definitions are available online:

<http://www.cs.brown.edu/research/plt/dl/fcfn/v1/>

2. Using First-Class Member Names

Most languages with objects, not only scripting languages, allow programmers to use first-class strings to index members. The syntactic overhead differs, as does the prevalence of the feature’s use within the corpus of programs in the language. This section explores first-class member names in existing languages, and highlights several of their uses.

2.1 Objects with First-Class Member Names

In Lua and JavaScript, `obj.x` is merely syntactic sugar for `obj["x"]`, so any member can be indexed by a runtime string value:

```
obj = {};  
obj["xy" + "z"] = 22;  
obj["x" + "yz"]; // evaluates to 22 in both languages
```

Python and Ruby support this pattern with only minor syntactic overhead. In Python:

```
class C(object):  
    pass  
obj = C()  
setattr(obj, "x" + "yz", 22)  
getattr(obj, "xy" + "z") # evaluates to 22
```

and in Ruby:

```
class C; end  
obj = C.new  
class << obj  
  define_method(("x" + "yz").to_sym) do; return 22; end  
end  
obj.send(("xy" + "z").to_sym) # evaluates to 22
```

In fact, even in Java, programmers are not forced to use first-order labels to refer to member names; it is merely a convenient

default. Java, for example, has `java.lang.Class.getMethod()`, which returns the method with a given name and parameter set.¹

2.2 Leveraging First-Class Member Names

Once member names are merely strings, programmers can manipulate them as mere data. The input to member lookup and update can come by concatenating strings, from configuration files, from reflected runtime values, via `Math.random()`, etc. This flexibility has been used, quite creatively, in many contexts.

Django The Python Django ORM dynamically builds classes based on database tables. In the following snippet, it adds a member `attr_name`, that represents a database column, to a class `new_class`, which it is constructing on-the-fly:²

```
attr_name = '%s_ptr' % base._meta.module_name
field = OneToOneField(base, name=attr_name,
    auto_created=True, parent_link=True)
new_class.add_to_class(attr_name, field)
```

`attr_name` concatenates `"_ptr"` to `base._meta.module_name`. It names a new member that is used in the resulting class as an accessor of another database table. For example, if the `Paper` table referenced the `Submittable` table, `Paper` instances would have a member `submittable_ptr`. Django has a number of pattern-based rules for inserting new members into classes, carefully designed to provide an expressive, object-based API to the client of the ORM. Its implementation, which is in pure Python, requires no extralingual metaprogramming tools.

Ruby on Rails When setting up a user-defined model, ActiveRecord iterates over the members of an object and only processes members that match certain patterns:³

```
attributes.each do |k, v|
  if k.include?("(")
    multi_parameter_attributes << [ k, v ]
  elsif respond_to?("#{k}=")
    send("#{k}=", v)
  else
    raise(UnknownAttributeError, "unknown attr: #{k}")
  end
end
```

The first pattern, `k.include?("(")`, checks the shape of the member name `k`, and the second pattern checks if the object has a member called `k + "="`. This is a Ruby convention for the setter of an object attribute, so this block of code invokes a setter function for each element in a key-value list. As with Django, ActiveRecord is leveraging first-class member names in order to provide an API implemented in pure Ruby that it couldn't otherwise without richer metaprogramming facilities.

Java Beans Java Beans provide a flexible component-based mechanism for composing applications. The Beans API uses reflective reasoning on canonical naming patterns to construct classes on-the-fly. For example, from `java.beans.Introspector`: "If we don't find explicit `BeanInfo` on a class, we use low-level reflection to study the methods of the class and apply standard design patterns to identify property accessors, event sources, or public methods."⁴ Properties of Beans are not known statically, so the API

¹[http://docs.oracle.com/javase/1.4.2/docs/api/java/lang/Class.html#getMethod\(java.lang.String,java.lang.Class\[\]\)](http://docs.oracle.com/javase/1.4.2/docs/api/java/lang/Class.html#getMethod(java.lang.String,java.lang.Class[]))

²<https://github.com/django/django/blob/master/django/db/models/base.py#L157>

³https://github.com/rails/rails/blob/master/activerecord/lib/active_record/base.rb#L1717

⁴<http://download.oracle.com/javase/tutorial/javabeans/introspection/index.html>

```
var banned = { "caller": true, "arguments": true, ... };

function reject_name(name) {
  return ((typeof name !== 'number' || name < 0)
    && (typeof name !== 'string'
      || name.charAt(0) === '_'
      || name.slice(-1) === '_'
      || name.charAt(0) === '-'))
    || banned[name];
}
```

Figure 1. Check for Banned Names in ADsafe

exposes a `PropertyDescriptor` class that provides methods including `getPropertyType` and `getReadMethod`, which return reflective descriptions of the types of properties of Beans.

Sandboxes JavaScript sandboxes like ADsafe [10] and Caja [24] use a combination of static and dynamic checks to ensure that untrusted programs do not access *banned fields* that may contain dangerous capabilities. To enforce this dynamically, all member-lookup expressions (`obj[name]`) in untrusted code are rewritten to check whether `name` is banned. Figure 1 is ADsafe's check; it uses a collection of ad hoc tests and also ensures that `name` is not the name of any member in the banned object, which is effectively used as a set of names. Caja goes further: it employs eight different patterns for encoding information related to members.⁵ For example, the Caja runtime adds a boolean-flagged member named `s + "_w_"` for each member `s` in an object, to denote whether it is writable or not. This lets Caja emulate a number of features that aren't found in its target language, JavaScript.

Objects as Arrays In Lua and JavaScript, built-in operations like splitting and sorting, and simple indexed `for` loops, work with any object that has numeric members. For example, in the following program, JavaScript's built-in `Array.prototype.sort` can be used on `obj`:

```
var obj = {length: 3, 0: 'def', 1: 'abc', 2: 'hij'};
// Array.prototype holds built-in methods
Array.prototype.sort.call(obj);
// evaluates to true
obj[0] == 'abc' && obj[1] == 'def' && obj[2] == 'hij'
```

In fact, the JavaScript specification states that a string P is a valid array index "if and only if `ToString(ToUint32(P))` is equal to P and `ToUint32(P)` is not equal to $2^{32} - 1$."⁶

2.3 The Perils of First-Class Member Names

Along with their great flexibility, first-class member names bring their own set of subtle error cases. First, developers must program defensively against the possibility that dereferenced members are not present (we see an example of this in the Ruby example from the last section, which explicitly uses `respond_to?` to check that the setter is present before using it). These sorts of reflective checks are common in programs that use first-class member names; a summary of such reflective operators across languages is included in figure 2. In other words, first-class member names put developers back in the era before type systems could guard against run-time *member not found* errors. Our type system (section 4) restores them to the happy state of finding these errors statically.

Second, programmers also make mistakes because they sometimes fail to respect that objects truly are *objects*, which means

⁵Personal communication with Jasvir Nagra, technical lead of Google Caja.

⁶<http://es5.github.com/#x15.4>

	Structural	
Ruby	<code>o.respond_to?(p)</code>	<code>o.methods</code>
Python	<code>hasattr(o, p)</code>	<code>dir(o)</code>
JavaScript	<code>o.hasOwnProperty(p)</code>	<code>for (x in o)</code>
Lua	<code>o[p] == nil</code>	
	Nominal	
Ruby	<code>o.is_a?(c)</code>	
Python	<code>isinstance(o, c)</code>	
JavaScript	<code>o instanceof c</code>	
Lua	<code>getmetatable(o) == c</code>	

Figure 2. Reflection APIs

they inherit members.⁷ When member names are computed at run-time, this can lead to subtle errors. For instance, Google Docs uses an object as a dictionary-like data structure internally, which uses strings from the user’s document as keys in the dictionary. Since (in most major browsers) JavaScript exposes its inheritance chain via a member called “`__proto__`”, the simple act of typing “`__proto__`” would cause Google Docs to lock up due to a misassignment to this member.⁸ In shared documents, this enables a denial-of-service attack. The patch of concatenating the lookup string with an unambiguous prefix or suffix itself requires careful reasoning (for instance, “`_`” would be a bad choice). Broadly, such examples are a case of a *member should not be found* error. Our type system protects against these, too.

The rest of this paper presents a semantics that allows these patterns and their associated problems, types that describe these objects and their uses, and a type system that explores verifying their safe use statically.

3. A Scripting Language Object Calculus

The preceding section illustrates how objects with first-class member names are used in several scripting languages. In this section, we distill the essentials of first-class member names into an object calculus called λ_S^{ob} . λ_S^{ob} has an entirely conventional account of higher-order functions and state (figure 4). However, it has an unusual object system that faithfully models the characteristic features of objects in scripting languages (figure 3). We also note some of the key differences between λ_S^{ob} and some popular scripting languages below.

- In member lookup $e_1[e_2]$, the member name e_2 is not a static string but an arbitrary expression that evaluates to a string. A programmer can thus dynamically pick the member name, as demonstrated in section 2.2. While it is possible to do this in languages like Java, it requires the use of cumbersome reflection APIs. Object calculi for Java thus do not bother modeling reflection. In contrast, scripting languages make dynamic member lookup easy.
- The expression $e_1[e_2 = e_3]$ has two meanings. If the member e_2 does not exist, it creates a new member (E-Create). If e_2 does exist, it updates its value. Therefore, members need not be declared and different instances of the same class or prototype can have different sets of members.

⁷ Arguably, the real flaw is in using the same data structure to associate both a fixed, statically-known collection of names and a dynamic, unbounded collection. Many scripting languages, however, provide only one data structure for both purposes, forcing this identification on programmers. We refrain here from moral judgment.

⁸ <http://www.google.com/support/forum/p/Google+Docs/thread?tid=0cd4a00bd4aef9e4>

- When a member is not found, λ_S^{ob} looks for the member in the parent object, which is the subscripted v_p part of the object value (E-Inherit). In Ruby and Lua this member is not directly accessible. In JavaScript and Python, it is an actual member of the object (“`__proto__`” in JavaScript, “`__class__`” in Python).
- The o **hasfield** str expression checks if the object o has a member str anywhere on its inheritance chain. This is a basic form of reflection.
- The str **matches** P expression returns **true** if the string matches the pattern P , and **false** otherwise. Scripting language programs employ a plethora of operators to pattern match and decompose strings, as shown in section 2.2. We abstract these to a single string-matching operator and representation of string patterns. The representation of patterns P is irrelevant to our core calculus. We only require that P represent some class of string-sets with decidable membership.

Given the lack of syntactic restrictions on object lookup, we can easily write a program that looks up a member that is not defined anywhere on the inheritance chain. In such cases, λ_S^{ob} signals an error (E-NotFound). Naturally, our type system will follow the classical goal of avoiding such errors.

Soundness We mechanize λ_S^{ob} with the Coq Proof Assistant, and prove a simple untyped progress theorem.

THEOREM 1 (Progress). *If σe is a closed, well-formed configuration, then either:*

- $e \in v$,
- $e = E\langle \text{err} \rangle$, or
- $\sigma e \rightarrow \sigma' e'$, where $\sigma' e'$ is a closed, well-formed configuration.

This property requires additional evaluation rules for runtime errors, which we elide from the paper.

4. Types for Objects in Scripting Languages

The rest of this paper explores typing the object calculus presented in section 3. This section addresses the structure and meaning of object types, followed by the associated type system and details of subtyping.

Typed λ_S^{ob} has explicit type annotations on variables bound by functions:

func($x:T$) { e }

Type inference is beyond the scope of this paper, so λ_S^{ob} is explicitly typed. Figure 5 specifies the full core type language. We incrementally present its significant elements, object types and string types, in the following subsections. The type language is otherwise conventional: types include base types, function types, and types for references; these are necessary to type ordinary imperative programs. We employ a top type, equirecursive μ -types, and bounded universal types to type-check object-oriented programs.

4.1 Basic Object Types

We adopt a structural type system, and begin our presentation with canonical record types, which map strings to types:

$$T = \dots \mid \{str_1 : T_1 \dots str_n : T_n\}$$

Record types are conventional and can type simple λ_S^{ob} programs:

```
let objWithToStr = ref {
  toStr: func(this:  $\mu\alpha$ .Ref { "toStr":  $\alpha \rightarrow \text{Str}$  }) { "hello" } }
in (deref objWithToStr) ["toStr"] (objWithToStr)
```

We need to work a little harder to express types for the programs in section 2. We do so in a principled way—all of our additions are conservative extensions to classical record types.

String patterns	$P = \dots$	patterns are abstract, but must have decidable membership, $str \in P$
Constants	$c = bool \mid str \mid null$	
Values	$v = c$	
	$\mid \{ str_1 : v_1 \dots str_n : v_n \}_{v_p}$	object, where $str_1 \dots str_n$ must be unique
Expressions	$e = v$	
	$\mid \{ str_1 : e_1 \dots str_n : e_n \}_e$	object expression
	$\mid e[e]$	lookup member in object, or in prototype
	$\mid e[e = e]$	update member, or create new member if needed
	$\mid e \text{ hasfield } e$	test if member is present
	$\mid e \text{ matches } P$	match a string against a pattern
	$\mid \text{err}$	runtime error

$e \hookrightarrow e$

E-GetField	$\{ \dots str : v \dots \}_{v_p} [str] \hookrightarrow v$
E-Inherit	$\{ str : v \dots \}_{l_p} [str_x] \hookrightarrow (\text{deref } l_p) [str_x], \text{ if } str_x \notin (str \dots)$
E-NotFound	$\{ str : v \dots \}_{null} [str_x] \hookrightarrow \text{err}, \text{ if } str_x \notin (str \dots)$
E-Update	$\{ str_1 : v_1 \dots str_i : v_i \dots str_n : v_n \}_{v_p} [str_i = v] \hookrightarrow \{ str_1 : v_1 \dots str_i : v \dots str_n : v_n \}_{v_p}$
E-Create	$\{ str_1 : v_1 \dots \}_{v_p} [str_x = v_x] \hookrightarrow \{ str_x : v_x, str_1 : v_1 \dots \}_{v_p} \text{ when } str_x \notin (str_1 \dots)$
E-Hasfield	$\{ \dots str : v \dots \}_{v_p} \text{ hasfield } str \hookrightarrow \text{true}$
E-HasFieldProto	$\{ \dots str : v \dots \}_{v_p} \text{ hasfield } str \hookrightarrow v_p \text{ hasfield } str, \text{ when } str' \notin (str \dots)$
E-HasNotField	$null \text{ hasfield } str \hookrightarrow \text{false}$
E-Matches	$str \text{ matches } P \hookrightarrow \text{true}, str \in P$
E-NoMatch	$str \text{ matches } P \hookrightarrow \text{false}, str \notin P$

Figure 3. Semantics of Objects and String Patterns in λ_S^{ob}

Locations	$l = \dots$	heap addresses
Heaps	$\sigma = \cdot \mid (l, v)\sigma$	
Values	$v = \dots \mid l \mid \text{func}(x) \{ e \}$	
Expressions	$e = \dots$	
	$\mid x$	identifiers
	$\mid e(e)$	function application
	$\mid e := e$	update heap
	$\mid \text{ref } e$	initialize a new heap location
	$\mid \text{deref } e$	heap lookup
	$\mid \text{if } (e_1) e_2 \text{ else } e_3$	branching
Evaluation Contexts	$E = \dots$	left-to-right, call-by-value evaluation

$e \hookrightarrow e$

β_v	$(\text{func}(x) \{ e \})(v) \hookrightarrow e[x/v]$
E-IfTrue	$\text{if } (\text{true}) e_2 \text{ else } e_3 \hookrightarrow e_2$
E-IfFalse	$\text{if } (\text{false}) e_2 \text{ else } e_3 \hookrightarrow e_3$

$\sigma e \rightarrow \sigma e$

E-Cxt	$\sigma E \langle e_1 \rangle \rightarrow \sigma E \langle e_2 \rangle, \text{ when } e_1 \hookrightarrow e_2$
E-Ref	$\sigma E \langle \text{ref } v \rangle \rightarrow \sigma(l, v) E \langle l \rangle, \text{ when } l \notin \text{dom}(\sigma)$
E-Deref	$\sigma E \langle \text{deref } l \rangle \rightarrow \sigma E \langle \sigma(l) \rangle$
E-SetRef	$\sigma E \langle l := v \rangle \rightarrow \sigma[l := v] E \langle v \rangle, \text{ when } l \in \text{dom}(\sigma)$

Figure 4. Conventional Features of λ_S^{ob}

4.2 Dynamic Member Lookups: String Patterns

The previous example is uninteresting because all member names are statically specified. Consider a Beans-inspired example, where there are potentially many members defined as "get.*" and "set.*". Beans libraries construct actual calls to methods by getting property names from reflection or configuration files, and appending them to the strings "get" and "set" before invoking. Beans also inherit useful built-in functions, like "toString" and "hashCode".

We need some way to represent all the potential get and set members on the object. Rather than a collection of singleton names, we need families of member names. To tackle this, we begin by introducing string patterns, rather than first-order labels, into our object types:

String patterns	$L = P$
String and object types	$T = \dots \mid L \mid \{ L_1 : T_1 \dots L_n : T_n \}$

Patterns, P , represent sets of strings. String literals type to singleton string sets, and subsumption is defined by set inclusion.

String patterns	L	$=$	P	extended in figure 11
Base types	b	$=$	$\text{Bool} \mid \text{Null}$	
Type variables	α	$=$	$\dots$	
Types	T	$=$	b $T_1 \rightarrow T_2$ $\mu\alpha.T$ $\text{Ref } T$ $\top$ L $\{L_1^{p_1} : T_1, \dots, L_n^{p_n} : T_n, L_A : \mathbf{abs}\}$	function types recursive types type of heap locations top type string types object types
Member presence	p	$=$	$\downarrow$ $\circ$	definitely present possibly absent
Type Environments	Γ	$=$	$\cdot \mid \Gamma, x : T$	

$\Gamma \vdash T$

$$\text{WF-Object} \frac{\Gamma \vdash T_1 \dots \Gamma \vdash T_n \quad \forall i. L_i \cap L_A = \emptyset \text{ and } \forall j \neq i. L_i \cap L_j = \emptyset}{\Gamma \vdash \{L_1^{p_1} : T_1 \dots L_n^{p_n} : T_n, L_A : \mathbf{abs}\}}$$

The well-formedness relation for types is mostly conventional. We require that string patterns in an object must not overlap (WF-Object).

Figure 5. Types for λ_S^{ob}

Our theory is parametric over the representation of patterns, as long as pattern containment, $P_1 \subseteq P_2$ is decidable, and the pattern language is closed over the following operators:

$$P_1 \cap P_2 \quad P_1 \cup P_2 \quad \overline{P} \quad P_1 \subseteq P_2 \quad P = \emptyset$$

With this notation, we can write an expressive type for our Bean-like objects, assuming Int-typed getters and setters:⁹

$$\text{IntBean} = \{ \begin{array}{l} ("get".*) : \rightarrow \text{Int}, \\ ("set".*) : \text{Int} \rightarrow \text{Null}, "toString" : \rightarrow \text{Str} \end{array} \}$$

(where $*$ is the regular expression pattern of all strings, so $"get".*$ is the set of all strings prefixed by `get`). Note, however, that `IntBean` seems to promise the presence of an infinite number of members, which no real object has. This type must therefore be interpreted to mean that a getter, say, will have `Int` if it is *present*. Since it may be absent, we can get the very “member not found” error that the type system was trying to *prevent*, resulting in a failure to conservatively extend simple record types.

In order to model members that we know are safe to look up, we add annotations to members that indicate whether they are *definitely* or *possibly* present:

$$\begin{array}{ll} \text{Member presence} & p = \downarrow \mid \circ \\ \text{Object types} & T = \dots \mid \{L_1^{p_1} : T_1 \dots L_n^{p_n} : T_n\} \end{array}$$

The notation $L^\downarrow : T$ means that for each string $\text{str} \in L$, a member with name str *must* be present on the object, and the value of the member has type T . This is the traditional meaning of a member’s annotation in simple record types. In contrast, $L^\circ : T$ means that for each string $\text{str} \in L$, if there is a member named str on the object, *then* the member’s value has type T . We would therefore write the above type as:

$$\text{IntBean} = \{ \begin{array}{l} ("get".*)^\circ : \rightarrow \text{Int}, ("set".*)^\circ : \text{Int} \rightarrow \text{Null}, \\ "toString"^\downarrow : \rightarrow \text{Str} \end{array} \}$$

As a matter of well-formedness, it does not make sense to place a definitely-present annotation ($\downarrow$) on an infinite set of strings, only on finite ones (such as the singleton set consisting of `"toString"`

above). In contrast, possibly-present annotations can be placed even on finite sets: writing `"toString"°` would indicate that the `toString` member does not need to be present.

Definitely-present members allow us to recover simple record types. However, we still cannot guarantee safe lookup within an infinite set of member names. We will return to this problem in section 4.6.

4.3 Subtyping

Once we introduce record types with string pattern names, it is natural to ask how pairs of types relate. This relationship is important in determining, for instance, when an actual argument may safely be passed to a formal parameter. This requires the definition of a subtyping relationship.

There are two well-known kinds of subsumption rules for record types, popularly called *width* and *depth* subtyping [1]. Depth subtyping allows for specialization, while width subtyping hides information. Both are useful in our setting, and we would like to understand them in the context of first-class member names. String patterns and possibly-present members introduce more cases to consider, and we must account for them all.

When determining whether one object type can be subsumed by another, we must consider whether each member can be subsumed. We will therefore treat each member name individually, iterating over all strings. Later, we will see how we can avoid iterating over this infinite set.

Figure 6 presents the initial version of our subtyping relation. For each member s , it considers the member’s annotation in each of the subtype and supertype. Each member name can have one of three relationships with a type T : definitely present in T , possibly present in T , or not mentioned at all (indicated by a dash, $-$). This naturally results in nine combinations to consider.

The column labeled *Antecedent* describes what further proof is needed to show subsumption. We use *ok* to indicate axiomatic base cases of the subtyping relation, $\mathfrak{S}$ to indicate cases where subsumption is undefined (resulting in a “subtyping” error), and $S <: T$ to indicate what is needed to show that the two members subsume. In the explanation below, we refer to table rows by the annotations on the members: e.g., $s^\downarrow s^\downarrow$ refers to the first row and $s^\circ s^-$ to the sixth.

⁹ Adding type abstraction at the member level gives more general setters and getters, but is orthogonal to our goals in this example.

Subtype	Supertype	Antecedent
$s^\downarrow : S$	$s^\downarrow : T$	$S <: T$
$s^\downarrow : S$	$s^\circ : T$	$S <: T$
$s^\downarrow : S$	$s : -$	<i>ok</i>
$s^\circ : S$	$s^\downarrow : T$	
$s^\circ : S$	$s^\circ : T$	$S <: T$
$s^\circ : S$	$s : -$	<i>ok</i>
$s : -$	$s^\downarrow : T_p$	
$s : -$	$s^\circ : T_p$	
$s : -$	$s : -$	<i>ok</i>

Figure 6. Per-String Subsumption (incomplete, see figure 7)

Depth Subtyping All the cases where further proof that $S <: T$ is needed are instances of depth subtyping: $s^\downarrow s^\downarrow$, $s^\downarrow s^\circ$, and $s^\circ s^\circ$. The $s^\circ s^\downarrow$ case is an error because a possibly-present member cannot automatically become definitely-present: e.g., substituting a value of type $\{ "x"^\circ : \text{Int} \}$ where $\{ "x"^\downarrow : \text{Int} \}$ is expected is unsound because the member may fail to exist at run-time. However, a member's presence can gain in strength through an operation that checks for the existence of the named member; we return to this point in section 4.6.

Width subtyping In the first two *ok* cases, $s^\downarrow s^-$ and $s^\circ s^-$, a member is “dropped” by being left unspecified in the supertype. This corresponds to information hiding.

The Other Three Cases We have discussed six of the cases above. Of the remaining three, $s^- s^-$ is the simple reflexive case. The other two, $s^- s^\downarrow$ and $s^- s^\circ$, must be errors because the subtype has failed to name a member (which may in fact be present), thereby attempting information hiding; if the supertype reveals this member, it would leak that information.

4.3.1 A Subtyping Parable

For a moment, let us consider the classical object type world with fixed sets of members and no presence annotations [1]. We will use patterns purely as a syntactic convenience, i.e., they will represent only a finite set of strings (which we could have written out by hand). Consider the following neutered array of booleans, which has only ten legal indices:

$$\text{DigitArray} \equiv \{ ([0-9]) : \text{Bool} \}$$

Clearly, this type can be subsumed to an even smaller one of just three indices:

$$\{ ([0-9]) : \text{Bool} \} <: \{ ([1-3]) : \text{Bool} \}$$

Now consider the following object, with a proposed type that would be ascribed by a classical type rule for object literals:

$$\begin{aligned} \text{obj} &= \{ "0" : \text{false}, "1" : \text{true} \}_{\text{null}} \\ \text{obj} &: \{ [0-1] : \text{Bool} \} \end{aligned}$$

obj clearly does not have type *DigitArray*, since it lacks eight required members. Suppose, using the more liberal type system of this paper, which permits possibly-present members, we define the following more permissive array:

$$\text{DigitArrayMaybe} \equiv \{ ([0-9])^\circ : \text{Bool} \}$$

Subtype	Supertype	Antecedent
$s^\downarrow : S$	$s^\downarrow : T$	$S <: T$
$s^\downarrow : S$	$s^\circ : T$	$S <: T$
$s^\downarrow : S$	$s : -$	<i>ok</i>
→ $s^\downarrow : S$	$s : \text{abs}$	
$s^\circ : S$	$s^\downarrow : T$	
$s^\circ : S$	$s^\circ : T$	$S <: T$
$s^\circ : S$	$s : -$	<i>ok</i>
→ $s^\circ : S$	$s : \text{abs}$	
$s : -$	$s^\downarrow : T_p$	
$s : -$	$s^\circ : T_p$	
$s : -$	$s : -$	<i>ok</i>
→ $s : -$	$s : \text{abs}$	
→ $s : \text{abs}$	$s^\downarrow : T$	
→ $s : \text{abs}$	$s^\circ : T$	<i>ok</i>
→ $s : \text{abs}$	$s : -$	<i>ok</i>
→ $s : \text{abs}$	$s : \text{abs}$	<i>ok</i>

Figure 7. Per-String Subsumption (with Absent Fields)

By the $s^- s^\circ$ case, obj's type doesn't subsume to *DigitArrayMaybe*, either, with good reason: something of its type could be hiding the member "2" containing a string, which if dereferenced (as *DigitArrayMaybe* permits) would result in a type error at run-time. (Of course, this does not tarnish the utility of maybe-present annotations, which we introduced to handle infinite sets of member names.)

However, there is information about obj that we have not captured in its type: namely that the members not listed *truly are absent*. Thus, though obj still cannot satisfy *DigitArray* (or, equivalently in our notation, $\{ ([0-9])^\downarrow : \text{Bool} \}$), it is reasonable to permit the value obj to inhabit the type *DigitArrayMaybe*, whose client understands that some members may fail to be present at run-time. We now extend our type language to permit this flexibility.

4.3.2 Absent Fields

To model absent members, we augment our object types one step further to describe the set of member names that are *definitely absent* on an object:

$$\begin{aligned} p &= \downarrow \mid \circ \\ T &= \dots \mid \{ L_1^{p_1} : T_1 \dots L_n^{p_n} : T_n, L_A : \text{abs} \} \end{aligned}$$

With this addition, there is now a fourth kind of relationship a string can have with an object type: it can be known to be absent. We must update our specification of subtyping accordingly. Figure 7 has the complete specification, where the new rows are marked with an arrow →.

Nothing subsumes *to abs* except for *abs* itself, so supertypes have a (non-strict) subset of the absent members of their subtypes. This is enforced by cases $s^\downarrow s^a$, $s^\circ s^a$, and $s^- s^a$ (where we use s^a as the abbreviation for an $s : \text{abs}$ entry).

If a string is absent on the subtype, the supertype cannot claim it is definitely present ($s^a s^\downarrow$). In the last three cases ($s^a s^\circ$, $s^a s^-$, or $s^a s^a$), an absent member in the subtype can be absent, not mentioned, or possibly-present with *any* type in the supertype,

Child	Parent	Result
$s^\downarrow : T_c$	$s^\downarrow : T_p$	$s^\downarrow : T_c$
$s^\downarrow : T_c$	$s^\circ : T_p$	$s^\downarrow : T_c$
$s^\downarrow : T_c$	$s : -$	$s^\downarrow : T_c$
$s^\downarrow : T_c$	$s : \mathbf{abs}$	$s^\downarrow : T_c$
$s^\circ : T_c$	$s^\downarrow : T_p$	$s^\downarrow : T_c \sqcup T_p$
$s^\circ : T_c$	$s^\circ : T_p$	$s^\circ : T_c \sqcup T_p$
$s^\circ : T_c$	$s : -$	$s : -$
$s^\circ : T_c$	$s : \mathbf{abs}$	$s^\circ : T_c$
$s : -$	$s^\downarrow : T_p$	$s : -$
$s : -$	$s^\circ : T_p$	$s : -$
$s : -$	$s : -$	$s : -$
$s : -$	$s : \mathbf{abs}$	$s : -$
$s : \mathbf{abs}$	$s^\downarrow : T_p$	$s^\downarrow : T_p$
$s : \mathbf{abs}$	$s^\circ : T_p$	$s^\circ : T_p$
$s : \mathbf{abs}$	$s : \mathbf{abs}$	$s : \mathbf{abs}$
$s : \mathbf{abs}$	$s : -$	$s : -$

Figure 8. Per-String Flattening

which even allows subsumption to introduce types for members that may be added in the future. To illustrate our final notion of subsumption, we use a more complex version of the earlier example (overline indicates set complement):

$\text{BoolArray} \equiv \{([0-9]^+ : \text{Bool}, \text{"length"}^\downarrow : \text{Num})\}$
 $\text{arr} \equiv \{\text{"0"} : \text{false}, \text{"1"} : \text{true}, \text{"length"} : 2\}_{\text{null}}$

Suppose we ascribe the following type to arr:

$\text{arr} : \{ \begin{array}{l} [0-1]^\downarrow : \text{Bool}, \text{"length"}^\downarrow : \text{Num}, \\ [0-1], \text{"length"} : \mathbf{abs} \end{array} \}$

This subsumes to BoolArray thus:

- The member "length" is subsumed using $s^\downarrow s^\downarrow$, with $\text{Num} < \text{Num}$.
- The members "0" and "1" subsume using $s^\downarrow s^\circ$, with $\text{Bool} < \text{Bool}$.
- The members made of digits other than "0" and "1" (as a regex, $[0-9][0-9]^+ \cup [2-9]$), subsume using $s^a s^\circ$, where T is Bool.
- The remaning members—those that aren't "length" and whose names aren't strings of digits—such as "iwishiwereml" are hidden by $s^a s^-$.

Absent members let us bootstrap from simple object types into the domain of infinite-sized collections of possibly-present members. Further, we recover simple record types when $L_A = \emptyset$.

4.3.3 Algorithmic Subtyping

Figure 7 gives a declarative specification of the per-string subtyping rules. This is clearly not an algorithm, since it iterates over pairs of an infinite number of strings. In contrast, our object types contain string patterns, which are finite representations of infinite sets. By considering the pairwise intersections of *patterns* between the two object types, an algorithmic approach presents itself. The algorithmic typing judgments must contain clauses that work at the level of

Child	Parent	Antecedent
$s^\downarrow : T_c$	$s^\downarrow : T_p$	$T_c <: T_p$
$s^\downarrow : T_c$	$s^\circ : T_p$	$T_c <: T_p$
$s^\downarrow : T_c$	$s : -$	ok
$s^\downarrow : T_c$	$s : \mathbf{abs}$	ok
$s^\circ : T_c$	$s^\downarrow : T_p$	$T_c \sqcup T_p <: T_p$ ($= T_c <: T_p$)
$s^\circ : T_c$	$s^\circ : T_p$	$T_c \sqcup T_p <: T_p$ ($= T_c <: T_p$)
$s^\circ : T_c$	$s : -$	ok
$s^\circ : T_c$	$s : \mathbf{abs}$	ok
$s : -$	$s^\downarrow : T_p$	ok
$s : -$	$s^\circ : T_p$	ok
$s : -$	$s : -$	ok
$s : -$	$s : \mathbf{abs}$	ok
$s : \mathbf{abs}$	$s^\downarrow : T_p$	$T_p <: T_p$ ($= ok$)
$s : \mathbf{abs}$	$s^\circ : T_p$	$T_p <: T_p$ ($= ok$)
$s : \mathbf{abs}$	$s : -$	ok
$s : \mathbf{abs}$	$s : \mathbf{abs}$	ok

Figure 9. Subtyping after Flattening

patterns. Thus, in deciding subtyping for

$$\{L_1^{p_1} : S_1, \dots, L_A : \mathbf{abs}\} <: \{M_1^{q_1} : T_1, \dots, M_A : \mathbf{abs}\}$$

one of several antecedents intersects all the definitely-present patterns, and checks that they are subtypes:

$$\forall i, j. (L_i \cap M_j \neq \emptyset) \wedge (p_i = q_j = \downarrow) \implies S_i <: T_i$$

A series of rules like these specify an algorithm for subtyping that is naturally derived from figure 7. The full definition of algorithmic subtyping is available online.

4.4 Inheritance

Conventional structural object types do not expose the position of members on the inheritance chain; types are “flattened” to include inherited members. A member lower in the chain “shadows” one of the same name higher in the chain, with only the lower member’s type present in the resulting record.

The same principle applies to first-class member names but, as with subtyping, we must be careful to account for all the cases. For subtyping, we related subtypes and supertypes to the proof obligation needed for their subsumption. For flattening, we will define a function that constructs a new object type out of two existing ones:

$$\text{flatten} \in T \times T \rightarrow T$$

As with subtyping, it suffices to specify what should happen for a given member s . Figure 8 shows this specification.

- When the child has a *definitely present* member, it overrides the parent ($s^\downarrow s^\downarrow$, $s^\downarrow s^\circ$, $s^\downarrow s^-$, and $s^\downarrow s^a$).

- In the cases $s^\circ s^\downarrow$ and $s^\circ s^\circ$, the child specifies a member as *possibly present* and the parent also specifies a type for the member, so a lookup may produce a value of either type. Therefore, we must join the two types (the $\sqcup$ operator).¹⁰ In case $s^\circ s^-$, the parent doesn't specify the member; it cannot be safely overridden with only a possibly-present member, so we must leave it hidden in the result. In case $s^\circ s^a$, since the member is absent on the parent, it is left possibly-present in the result.
- If the child doesn't specify a member, it hides the corresponding member in the parent ($s^-s^\downarrow$, s^-s° , s^-s^- , s^-s^a).
- If a member is absent on the child, the corresponding member on the parent is used ($s^a s^\downarrow$, $s^a s^\circ$, $s^a s^-$, $s^a s^a$).

The online material includes a flattening algorithm.

Inheritance and Subtyping It is well-known that inheritance and subtyping are different, yet not completely orthogonal concepts [9]. Figures 7 and 8 help us identify when an object inheriting from a parent is a subtype of that parent. Figure 9 presents this with three columns. The first two show the presence and type in the child and parent, respectively. In the third column, we apply *flatten* to that row's child and parent; then look up the result and the parent's type in the figure 7; and copy the resulting *Antecedent* entry. This column thus explains under exactly what condition a child that extends a parent is a subtype of it. Consider some examples:

- If a child overrides a parent's member (e.g., $s^\downarrow s^\downarrow$), it must override the member with a subtype.
- When the child hides a parent's definitely-present member ($s^-s^\downarrow$), it is not a subtype of its parent.
- Suppose a parent has a definitely-present member s , which is explicitly not present in the child. This corresponds to the $s^a s^\downarrow$ entry. Applying *flatten* to these results in $s^\downarrow : T_p$. Looking up and substituting $s^\downarrow : T_p$ for both subtype and supertype in figure 7 yields the condition $T_p <: T_p$, which is always true. Indeed, at run-time the parent's s would be accessible through the child, and (for this member) inheritance would indeed correspond to subtyping.

By relating pairs of types, this table could, for instance, determine whether a mixin—which would be represented here as an object-to-object function that constructs objects relative to a parameterized parent—obeys subtyping.

4.5 Typing Objects

Now that we are equipped with *flatten* and a notion of subsumption, we can address typing λ_S^{ob} in full. Figure 10 contains the rules for typing strings, objects, member lookup, and member update.

T-Str is straightforward: literal strings have a singleton string set L -type. T-Object is more interesting. In a literal object expression, all the members in the expression are definitely present, and have the type of the corresponding member expression; these are the pairs $str_i^\downarrow : S_i$. Fields not listed are definitely absent, represented by $\star : \mathbf{abs}$.¹¹ In addition, object expressions have an explicit parent subexpression (e_p). Using *flatten*, the type of the new object is combined with that of the parent, T_p .

A member lookup expression has the type of the member corresponding to the lookup member pattern. T-GetField ensures that

the type of the expression in lookup position is a set of definitely-present members on the object, L_i , and yields the corresponding type, S_i .

The general rule for member update, T-Update, similarly requires that the entire pattern L_i be on the object, and ensures invariance of the object type under update (T is the type of e_o and the type of the whole expression). In contrast to T-GetField, the member does *not* need to be definitely-present: assigning into possibly-present members is allowed, and maintains the object's type.

Since simple record types allow strong update by extension, our type system admits one additional form of update expression, T-UpdateStr. If the pattern in lookup position has a singleton string type str_f , then the resulting type has str_f definitely present with the type of the update argument, and removes str_f from each existing pattern.

4.6 Typing Membership Tests

Thus far, it is a type error to lookup a member that is possibly absent all along the inheritance chain: T-GetField requires that the accessed member is definitely present. For example, if `obj` is a dictionary mapping member names to numbers:

$$\{(.*)^\circ : \text{Num}, \emptyset : \mathbf{abs}\}$$

then `obj["10"]` is untypable. We can of course relax this restriction and admit a runtime error, but it would be better to give a guarantee that such an error cannot occur. A programmer might implement a lookup wrapper with a guard:

```
if (obj hasfield x) obj[x] else false
```

Our type system must account for such guards and *if-split*—it must narrow the types of `obj` and `x` appropriately in at least the then-branch. We present a single if-splitting rule that exactly matches this pattern:¹²

```
if (xobj hasfield xfld) e2 else e3
```

A special case is when the type of x_{obj} is $\{\dots L^\circ : T \dots\}$ and the type of x_{fld} is a singleton string $str \in L$. In such a case, we can narrow the type of x_{obj} to:

$$\{\dots \{str\}^\downarrow : T, L \cap \overline{\{str\}^\circ} : T \dots\}$$

A lookup on this type with the string str would then be typable with type T . However, the second argument to **hasfield** won't always type to a singleton string. In this case, we need to use a bounded type variable to represent it. We enrich string types to represent this, shown in the if-splitting rule in figure 11.¹³ For example, let $P = (.*)$. If $x : P$ and $obj : \{P^\circ : \text{Num}, \emptyset : \mathbf{abs}\}$, then the narrowed environment in the true branch is:

$$\Gamma' = \Gamma, \alpha <: P, x : \alpha, obj : \{\alpha^\downarrow : \text{Num}, P \cap \overline{\alpha^\circ} : \text{Num}, \emptyset : \mathbf{abs}\}$$

This new environment says that `x` is bound to a value that types to some subset of P , and that subset is definitely present ($\alpha^\downarrow$) on the object `obj`. Thus, a lookup `obj[x]` is guaranteed to succeed with type `Num`.

Object subtyping must now work with the extended string patterns of figure 11, introducing a dependency on the environment Γ . Instead of statements such as $L_1 \subseteq L_2$, we must now discharge $\Gamma \vdash L_1 \subseteq L_2$. We interpret these as propositions with set variables that can be discharged by existing string solvers [19].

¹⁰ A type U is the join of S and T if $S <: U$ and $T <: U$.

¹¹ In our type language, we use $\star$ to represent “all members not named in the rest of the object type”. Since string patterns are closed over union and negation, $\star$ can be expressed directly as the complement of the union of the other patterns in the object type. Therefore $\star$ is purely a syntactic convenience, and does not need to appear in the theory.

¹² There are various if-splitting techniques for typing complex conditionals and control [8, 16, 34]. We regard the details of these techniques as orthogonal.

¹³ This single typing rule is adequate for type-checking, but the proof of preservation requires auxiliary rules in the style of Typed Scheme [33].

T-Str	$\frac{}{\Sigma; \Gamma \vdash \text{str} : \{\text{str}\}}$	T-Object	$\frac{\Sigma; \Gamma \vdash e_1 : S_1 \cdots \quad \Sigma; \Gamma \vdash e_p : \text{Ref } T_p \quad T = \text{flatten}(\{\text{str}_1^\downarrow : S_1 \cdots * : \mathbf{abs}\}, T_p)}{\Sigma; \Gamma \vdash \{ \text{str}_1 : e_1 \cdots \} e_p : T}$
T-GetField	$\frac{\Sigma; \Gamma \vdash e_o : \{L_1^{p_1} : S_1, \dots, L_i^\downarrow : S_i, \dots, L_n^{p_n} : S_n, L_A : \mathbf{abs}\} \quad \Sigma; \Gamma \vdash e_f : L_i}{\Sigma; \Gamma \vdash e_o[e_f] : S_i}$		
T-Update	$\frac{\Sigma; \Gamma \vdash e_o : T \quad \Sigma; \Gamma \vdash e_f : L_i \quad \Sigma; \Gamma \vdash e_v : S_i \quad T = \{L_1^{p_1} : S_1, \dots, L_i^{p_i} : S_i, \dots, L_n^{p_n} : S_n, L_A : \mathbf{abs}\}}{\Sigma; \Gamma \vdash e_o[e_f = e_v] : T}$		
T-UpdateStr	$\frac{\Sigma; \Gamma \vdash e_f : \{\text{str}_f\} \quad \Sigma; \Gamma \vdash e_v : S \quad \Sigma; \Gamma \vdash e_o : \{L_1^{p_1} : S_1, \dots, L_A : \mathbf{abs}\}}{\Sigma; \Gamma \vdash e_o[e_f = e_v] : \{\text{str}_f^\downarrow : S, (L_1 - \text{str}_f)^{p_1} : S_1, \dots, L_A - \text{str}_f : \mathbf{abs}\}}$		

Figure 10. Typing Objects

String patterns	$L = P \mid \alpha \mid L \cap L \mid L \cup L \mid \bar{L}$	
Types	$T = \dots \mid \forall \alpha <: S. T$	bounded quantification
Type Environments	$\Gamma = \dots \mid \Gamma, \alpha <: T$	
$\Gamma(o) = \{\dots L^\circ : S \dots\}$	$\Gamma(f) = L$	$\Sigma; \Gamma, \alpha <: L, f : \alpha, o : \{\dots \alpha^\downarrow : S, L^\circ : S \dots\} \vdash e_2 : T \quad L' = L \cap \bar{\alpha} \quad \Gamma \vdash e_3 : T$
		$\Sigma; \Gamma \vdash \text{if } (o \text{ hasfield } f) \text{ } e_2 \text{ else } e_3 : T$

Figure 11. Typing Membership Tests

4.7 Accessing the Inheritance Chain

In λ_S^{ob} , the inheritance chain of an object is present in the model but not provided explicitly to the programmer. Some scripting languages, however, expose the inheritance chain through members: "`__proto__`" in JavaScript and "`__class__`" in Python (section 2). Reflecting this in the model introduces significant complexity into the type system and semantics, which need to reason about lookups and updates that conflict with this explicit parent member. We have therefore elided this feature from the presentation in this paper. In the appendix, however, we present a version of λ_S^{ob} where a member "parent" is explicitly used for inheritance. All our proofs work over this richer language.

4.8 Object Types as Intersections of Dependent Refinements

An alternate scripting semantics might encode objects as functions, with application as member lookup. Thus an object of type:

$$\{L_1^\downarrow : T_1, L_2^\downarrow : T_2, \emptyset : \mathbf{abs}\}$$

could be encoded as a function of type:

$$(\{s : \text{Str} \mid s \in L_1\} \rightarrow T_1) \wedge (\{s : \text{Str} \mid s \in L_2\} \rightarrow T_2)$$

Standard techniques for typing intersections and dependent refinements would then apply.

This trivial encoding precludes precisely typing the inheritance chain and member presence checking in general. In contrast, the extensionality of records makes reflection much easier. However, we believe that a typable encoding of objects as functions is related to the problem of typing object proxies (which are present in Ruby and Python and will soon be part of JavaScript), which are collections of functions that behave like objects but are not objects themselves [11]. Proxies are beyond the scope of this paper, but we note their relationship to dependent types.

4.9 Guarantees

The full typing rules and formal definition of subtyping are available online. The elided typing rules are a standard account of functions, mutable references, and bounded quantification. Subtyping is interpreted co-inductively, since it includes equirecursive μ -types. We prove the following properties of λ_S^{ob} :

LEMMA 1 (Decidability of Subtyping). *If the functions and predicates on string patterns are decidable, then the subtype relation is finite-state.*

THEOREM 2 (Preservation). *If:*

- $\Sigma \vdash \sigma$,
- $\Sigma; \cdot \vdash e : T$, and
- $\sigma e \rightarrow \sigma' e'$,

then there exists a Σ' , such that:

- $\Sigma \subseteq \Sigma'$,
- $\Sigma' \vdash \sigma'$, and
- $\Sigma'; \cdot \vdash e' : T$.

THEOREM 3 (Typed Progress). *If $\Sigma \vdash \sigma$ and $\Sigma; \cdot \vdash e : T$ then either $e \in v$ or there exist σ' and e' such that $\sigma e \rightarrow \sigma' e'$.*

Unlike the untyped progress theorem of section 3, this typed progress theorem does not admit runtime errors.

5. Implementation and Uses

Though our main contribution is intended to be foundational, we have built a working type-checker around these ideas, which we now discuss briefly.

Our type checker uses two representations for string patterns. In both cases, a type is represented a set of pairs, where each pair is a set of member names and their type. The difference is in the representation of member names:

```

type Array =
  typrec array :: * => * .
  typlambda a :: * . {
    /([0-9])|("(+Infinity"|("-Infinity"| "NaN"))/ : 'a,
    length : Int,
    * : _,
    __proto__: {
      __proto__: Object,
      * : _,
      // Note how the type of "this" is array<'a>. If
      // these are applied as methods, arr.map(...), then
      // the inner 'a and outer 'a will be the same.
      map: forall a . forall b . [array<'a>] ('a -> 'b)
        -> 'array<'b>,
      slice: forall a . [array<'a>] Int * Int + Undef
        -> 'array<'a>,
      concat: forall a . [array<'a>] 'array<'a>
        -> 'array<'a>,
      forEach: forall a . [array<'a>] ('a -> Any)
        -> Undef,
      filter: forall a . [array<'a>] ('a -> Bool)
        -> 'array<'a>,
      every: forall a . [array<'a>] ('a -> Bool)
        -> Bool,
      some: forall a . [array<'a>] ('a -> Bool)
        -> Bool,
      push: forall a . [array<'a>] 'a -> Undef,
      pop: forall a . [array<'a>] -> 'a,
      /* and several more methods */
    }
  }
}

```

Figure 12. JavaScript Array Type (Fragment)

1. The set of member names is a finite enumeration of strings representing either a collection of members or their complement.
2. The set of member names is represented as an automaton whose language is that set of names.

The first representation is natural when objects contain constant strings for member names. When given infinite patterns, however, our implementation parses their regular expressions and constructs automata from them.

The subtyping algorithms require us to calculate several intersections and complements. This means we might, for instance, need to compute the intersection of a finite set of strings with an automaton. In all such cases, we simply construct an automaton out of the finite set of strings and delegate the computation to the automaton representation.

For finite automata, we use the representation and decision procedure of Hooimeijer and Weimer [20]. Their implementation is fast and based on mechanically proven principles. Because our type checker internally converts between the two representations, the treatment of patterns is thus completely hidden from the user.¹⁴

Of course, a practical type checker must address more issues than just the core algorithms. We have therefore embedded these ideas in the existing prototype JavaScript type-checker of Guha, et al. [15, 16]. Their checker already handles various details of JavaScript source programs and control flow, including a rich treatment of if-splitting [16], which we can exploit. In contrast, their (undocumented) object types use simple record types that can only type trivial programs. Prior applications of that type-checker to most real-world JavaScript programs has depended on the theory in this paper.

We have applied this type system in several contexts:

¹⁴ Our actual type-checker is a functor over a signature of patterns.

• We can type variants of the examples in this paper; because we lack parsers for the different input languages, they are implemented in λ_S^{ob} . These examples are all available in our open source implementation as test cases.¹⁵

• Our type system is rich enough to provide an accurate type for complex built-in objects such as JavaScript’s arrays (an excerpt from the actual code is shown in figure 12; the top of this type uses standard type system features that we don’t address in this paper). Note that patterns enable us to accurately capture not only numeric indices but also JavaScript oddities such as the Infinity that arises from overflow.

• We have applied the system to type-check ADsafe [27]. This was impossible with the trivial object system in prior work [16], and thus used an intermediate version of the system described here. However, this work was incomplete at the time of that publication, so that published result had two weaknesses that this work addresses:

1. We were unable to type an important function, `reject_name`, which requires the pattern-handling we demonstrate in this paper.¹⁶
2. More subtly but perhaps equally important, the types we used in that verification had to hard-code collections of member names, and as such were not future-proof. In the rapidly changing world of browsers, where implementations are constantly adding new operations against which sandbox authors must remain vigilant, it is critical to instead have pattern-based whitelists and blacklists, as shown here.

Despite the use of sophisticated patterns, our type-checker is fast. It runs various examples in approximately one second on an Intel Core i5 processor laptop; even ADsafe verifies in under twenty seconds. Therefore, despite being a prototype tool, it is still practical enough to run on real systems.

6. Related Work

Our work builds on the long history of semantics and types for objects and recent work on semantics and types for scripting languages.

Semantics of Scripting Languages There are semantics for JavaScript, Ruby, and Python that model each language in detail. This paper focuses on objects, eliding many other features and details of individual scripting languages. We discuss the account of objects in various scripting semantics below.

Furr, et al. [14] tackle the complexity of Ruby programs by desugaring to an intermediate language (RIL); we follow the same approach. RIL is not accompanied by a formal semantics, but its syntax suggests a class-based semantics, unlike the record-based objects of λ_S^{ob} . Smeding’s Python semantics [30] details multiple inheritance, which our semantics elides. However, it also omits certain reflective operators (e.g., `hasattr`) that we do model in λ_S^{ob} . Maffei, et al. [22] account for JavaScript objects in detail. However, their semantics is for an abstract machine, unlike our syntactic semantics. Our semantics is closest to the JavaScript semantics of Guha, et al. [15]. Not only do we omit unnecessary details, but we also abstract the plethora of string-matching operators and

¹⁵ <https://github.com/brownplt/strobe/tree/master/tests/typable> and <https://github.com/brownplt/strobe/tree/master/tests/strobe-typable> have examples of objects as arrays, field-presence checks, and more.

¹⁶ Typing `reject_name` requires more than string patterns to capture the numeric checks and the combination of predicates, but string patterns let us reason about the `charAt` checks that it requires.

make object membership checking manifest. These features were not presented in their work, but buried in their implementation of desugaring.

Extensible Records The representation of objects in λ_S^{ob} is derived from extensible records, surveyed by Fisher and Mitchell [12] and Bruce, et al. [7]. We share similarities with ML-Art [29], which also types records with explicitly absent members. Unlike these systems, member names in λ_S^{ob} are first-class strings; λ_S^{ob} includes operators to enumerate over members and test for the presence of members. First-class member names also force us to deal with a notion of infinite-sized patterns in member names, which existing object systems don't address. ML-Art has a notion of "all the rest" of the members, but we tackle types with an arbitrary number of such patterns.

Nishimura [25] presents a type system for an object calculus where messages can be dynamically selected. In that system, the *kind* of a dynamic message specifies the finite set of messages it may resolve to at runtime. In contrast, our string types can describe potentially-infinite sets of member names. This generalization is necessary to type-check the programs in this paper where objects' members are dynamically computed from strings (section 2). Our object types can also specify a wider class of invariants with presence annotations, which allow us to also type-check common scripting patterns that employ reflection.

Types and Contracts for Untyped Languages There are various type systems retrofitted onto untyped languages. Those that support objects are discussed below.

Strongtalk [6] is a typed dialect of Smalltalk that uses protocols to describe objects. String patterns can describe more ad hoc objects than the protocols of Strongtalk, which are a finite enumeration of fixed names. Strongtalk protocols may include a brand; they are thus a mix of nominal and structural types. In contrast, our types are purely structural, though we do not anticipate any difficulty incorporating brands.

Our work shares features with various JavaScript type systems. In Anderson, et al. [5]'s type system, objects' members may be potentially present; it employs strong updates to turn these into definitely present members. Recency types [17] are more flexible, support member type-changes during initialization, and account for additional features such as prototypes. Zhao's type system [36] also allows unrestricted object extension, but omits prototypes. In contrast to these works, our object types do not support strong updates via mutation. We instead allow possibly-absent members to turn into definitely-present members via member presence checking, which they do not support. Strong updates are useful for typing initialization patterns. In these type systems, member names are first-order labels. Thiemann's [32] type system for JavaScript allows first-class strings as member names, which we generalize to member patterns.

RPython [3] compiles Python programs to efficient byte-code for the CLI and the JVM. Dynamically updating Python objects cannot be compiled. Thus, RPython stages evaluation into an interpreted initialization phase, where dynamic features are permitted, and a compiled running phase, where dynamic features are disallowed. Our types introduce no such staging restrictions.

DRuby [13] does not account for member presence checking in general. However, as a special case, An, et al. [2] build a type-checker for Rails-based Web applications that partially-evaluates dynamic operations, producing a program that DRuby can verify. In contrast, our types tackle membership presence testing directly.

System D [8] uses dependent refinements to type dynamic dictionaries. System D is a purely functional language, whereas λ_S^{ob} also accounts for inheritance and state. The authors of System D suggest integrating a string decision procedure to reason about dic-

tionary keys. We use DPRLE [20] to support exactly this style of reasoning.

Heidegger, et al. [18] present *dynamically*-checked contracts for JavaScript that use regular expressions to describe objects. Our implementation uses regular expressions for *static* checking.

Extensions to Class-based Objects In scripting languages, the shape on an object is not fully determined by its class (section 4.1). Our object types are therefore structural, but an alternative class-based system would require additional features to admit scripts. For example, Unity adds structural constraints to Java's classes [23]; class-based reasoning is employed by scripting languages but not fully investigated in this paper. Expanders in eJava [35] allow classes to be augmented, affecting all objects; scripting also allows individual objects to be customized, which structural typing admits. Fickle allows objects to change their class at runtime [4]; our types do admit class-changing (assigning to "parent"), but they do not have a direct notion of class. J& allows packages of related classes to be composed [26]. The scripting languages we consider do not support the runtime semantics of J&, but do support related mechanisms such as mixins, which we can easily model and type.

Regular Expression Types Regular tree types and regular expressions can describe the structure of XML documents (e.g., XDuce [21]) and strings (e.g., XPerl [31]). These languages verify XML-manipulating and string-processing programs. Our type system uses patterns not to describe trees of objects like XDuce, but to describe objects' member names. Our string patterns thus allow individual objects to have semi-determinate shapes. Like XPerl, member names are simply strings, but our strings are used to index objects, which are not modeled by XPerl.

7. Conclusion

We present a semantics for objects with first-class member names, which are a characteristic feature of popular scripting languages. In these languages, objects' member names are first-class strings. A program can dynamically construct new names and reflect on the dynamic set of member names in an object. We show by example how programmers use first-class member names to build several frameworks, such as ADsafe (JavaScript), Ruby on Rails, Django (Python), and even Java Beans. Unfortunately, existing type systems cannot type-check programs that use first-class member names. Even in a typed language, such as Java, misusing first-class member names causes runtime errors.

We present a type system in which well-typed programs do not signal "member not found" errors, even when they use first-class member names. Our type system uses string patterns to describe sets of members names and presence annotations to describe their position on the inheritance chain. We parameterize the type system over the representation of patterns. We only require that pattern containment is decidable and that patterns are closed over union, intersection, and negation. Our implementation represents patterns as regular expressions and (co-)finite sets, seamlessly converting between the two.

Our work leaves several problems open. Our object calculus models a common core of scripting languages. Individual scripting languages have additional features that deserve consideration. The dynamic semantics of some features, such as getters, setters, and eval, has been investigated [28]. We leave investigating more of these features (such as proxies), and extending our type system to account for them, as future work. We validate our work with an implementation for JavaScript, but have not built type-checkers for other scripting languages. A more fruitful endeavor might be to build a common type-checking backend for all these languages, if it is possible to reconcile their differences.

Acknowledgments

We thank Cormac Flanagan for his extensive feedback on earlier drafts. We are grateful to StackOverflow for unflagging attention to detail, and to Claudiu Saftoiu for serving as our low-latency interface to it. Gilad Bracha helped us understand the history of Smalltalk objects and their type systems. We thank William Cook for useful discussions and Michael Greenberg for types. Ben Lerner drove our Coq proof effort. We are grateful for support to the US National Science Foundation.

References

- [1] M. Abadi and L. Cardelli. *A Theory of Objects*. Springer-Verlag, 1996.
- [2] J. D. An, A. Chaudhuri, and J. S. Foster. Static typing for Ruby on Rails. In *IEEE International Symposium on Automated Software Engineering*, 2009.
- [3] D. Ancona, M. Ancona, A. Cuni, and N. D. Matsakis. RPython: a step towards reconciling dynamically and statically typed OO languages. In *ACM SIGPLAN Dynamic Languages Symposium*, 2007.
- [4] D. Ancona, C. Anderson, F. Damiani, S. Drossopoulou, P. Giannini, and E. Zucca. A provenly correct translation of Fickle into Java. *ACM Transactions on Programming Languages and Systems*, 2, 2007.
- [5] C. Anderson, P. Giannini, and S. Drossopoulou. Towards type inference for JavaScript. In *European Conference on Object-Oriented Programming*, 2005.
- [6] G. Bracha and D. Griswold. Strongtalk: Typechecking Smalltalk in a production environment. In *ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages & Applications*, 1993.
- [7] K. B. Bruce, L. Cardelli, and B. C. Pierce. Comparing object encodings. *Information and Computation*, 155(1–2), 1999.
- [8] R. Chugh, P. M. Rondon, and R. Jhala. Nested refinements for dynamic languages. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2012.
- [9] W. R. Cook, W. L. Hill, and P. S. Canning. Inheritance is not subtyping. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1990.
- [10] D. Crockford. ADSafe. www.adsafe.org, 2011.
- [11] T. V. Cutsem and M. S. Miller. Proxies: Design principles for robust object-oriented intercession APIs. In *ACM SIGPLAN Dynamic Languages Symposium*, 2010.
- [12] K. Fisher and J. C. Mitchell. The development of type systems for object-oriented languages. *Theory and Practice of Object Systems*, 1, 1995.
- [13] M. Furr, J. D. An, J. S. Foster, and M. Hicks. Static type inference for Ruby. In *ACM Symposium on Applied Computing*, 2009.
- [14] M. Furr, J. D. An, J. S. Foster, and M. Hicks. The Ruby Intermediate Language. In *ACM SIGPLAN Dynamic Languages Symposium*, 2009.
- [15] A. Guha, C. Saftoiu, and S. Krishnamurthi. The essence of JavaScript. In *European Conference on Object-Oriented Programming*, 2010.
- [16] A. Guha, C. Saftoiu, and S. Krishnamurthi. Typing local control and state using flow analysis. In *European Symposium on Programming*, 2011.
- [17] P. Heidegger and P. Thiemann. Recency types for dynamically-typed, object-based languages: Strong updates for JavaScript. In *ACM SIGPLAN International Workshop on Foundations of Object-Oriented Languages*, 2009.
- [18] P. Heidegger, A. Bieniusa, and P. Thiemann. Access permission contracts for scripting languages. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2012.
- [19] P. Hooimeijer and M. Veanes. An evaluation of automata algorithms for string analysis. In *International Conference on Verification, Model Checking, and Abstract Interpretation*, 2011.
- [20] P. Hooimeijer and W. Weimer. A decision procedure for subset constraints over regular languages. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2009.
- [21] H. Hosoya, J. Vouillon, and B. C. Pierce. Regular expression types for XML. *ACM Transactions on Programming Languages and Systems*, 27, 2005.
- [22] S. Maffei, J. C. Mitchell, and A. Taly. An operational semantics for JavaScript. In *Asian Symposium on Programming Languages and Systems*, 2008.
- [23] D. Malayeri and J. Aldrich. Integrating nominal and structural subtyping. In *European Conference on Object-Oriented Programming*, 2008.
- [24] M. S. Miller, M. Samuel, B. Laurie, I. Awad, and M. Stay. Caja: Safe active content in sanitized JavaScript. Technical report, Google, Inc., 2008. <http://google-caja.googlecode.com/files/caja-spec-2008-06-07.pdf>.
- [25] S. Nishimura. Static typing for dynamic messages. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1998.
- [26] N. Nystrom, X. Qi, and A. C. Myers. J&: Nested intersection for scalable software composition. In *ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages & Applications*, 2006.
- [27] J. G. Politz, S. A. Eliopoulos, A. Guha, and S. Krishnamurthi. AD-safety: Type-based verification of JavaScript sandboxing. In *USENIX Security Symposium*, 2011.
- [28] J. G. Politz, M. J. Carroll, B. S. Lerner, and S. Krishnamurthi. A tested semantics for getters, setters, and eval in JavaScript. In *ACM SIGPLAN Dynamic Languages Symposium*, 2012.
- [29] D. Rémy. Programming objects with ML-ART: an extension to ML with abstract and record types. In M. Hagiya and J. C. Mitchell, editors, *Theoretical Aspects of Computer Software*, volume 789 of *Lecture Notes in Computer Science*. Springer-Verlag, 1994.
- [30] G. J. Smeding. An executable operational semantics for Python. Master’s thesis, Utrecht University, 2009.
- [31] N. Tabuchi, E. Sumii, and A. Yonezawa. Regular expression types for strings in a text processing language. *Electronic Notes in Theoretical Computer Science*, 75, 2003.
- [32] P. Thiemann. Towards a type system for analyzing JavaScript programs. In *European Symposium on Programming*, 2005.
- [33] S. Tobin-Hochstadt and M. Felleisen. The design and implementation of Typed Scheme. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2008.
- [34] S. Tobin-Hochstadt and M. Felleisen. Logical types for untyped languages. In *ACM SIGPLAN International Conference on Functional Programming*, 2010.
- [35] A. Warth, M. Stanojević, and T. Millstein. Statically scoped object adaption with expanders. In *ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages & Applications*, 2006.
- [36] T. Zhao. Type inference for scripting languages with implicit extension. In *ACM SIGPLAN International Workshop on Foundations of Object-Oriented Languages*, 2010.

Selective Ownership: Combining Object and Type Hierarchies for Flexible Sharing

Stephanie Balzer

School of Computer Science
Carnegie Mellon University
balzers@cs.cmu.edu

Thomas R. Gross

Department of Computer Science
ETH Zurich
thomas.gross@inf.ethz.ch

Peter Müller

Department of Computer Science
ETH Zurich
peter.mueller@inf.ethz.ch

Abstract

Most ownership systems enforce a tree topology on a program's heap. The tree topology facilitates many aspects of programming such as thread synchronization, memory management, and program verification. Ownership-based verification techniques leverage the tree topology of an ownership system (and hence the fact that there exists a single owner) to restore sound modular reasoning about invariants over owned objects. However, these techniques in general restrict sharing by limiting modifying access to an owned object to the object's owner and to other objects in that owner's ownership tree. In this paper, we introduce *selective ownership*, a less rigid form of ownership. The key idea is to structure the heap in two ways, by defining an order on a program's type declarations and by imposing ownership on *selected objects*. The order on type declarations results in a stratified program heap but permits shared, modifying access to instances further "down" in the heap topology. By superimposing object ownership on selected objects in the heap, programmers can carve out partial sub-trees in the heap topology where the objects are owned. We show how selective ownership enables the modular verification of invariants over heap topologies that subsume shared, modifiable sub-structures. Selective ownership has been elaborated for our programming language Rumer, a programming language with first-class relationships, which naturally give rise to an ordering on type declarations.

Categories and Subject Descriptors D.2.4 [Software Engineering]: Software/Program Verification—Class invariants

General Terms Languages, Verification

Keywords Selective ownership, Universe types, Ownership types, Visible-state verification techniques, First-class relationships

1. Introduction

Object-oriented programs typically produce graphs of highly-interconnected objects. These graphs bear little resemblance to the programs that produced them, complicating any reasoning about them. Ownership type systems [17–23, 37, 41] have been shown to ease program reasoning by imposing a tree structure on a program's heap. For instance, Ownership type systems have been successfully

employed for program verification [30, 34, 37, 38], for guaranteeing thread safety [13], for memory management [14], and for enforcing architectural styles [2].

However, many common design patterns and programming idioms do not naturally produce a tree structure but a heap that subsumes owned and shared objects. For instance, the nodes of Java's linked list implementation are shared and manipulated by a list's head and all its iterators. Most ownership systems either disallow such implementations or provide weak guarantees. For example, the classical Ownership types [19] enforce the *owner-as-dominator* discipline [22, 24] and thus disallow direct access to the linked list both by the list's head and its iterators. Universe types [37], on the other hand, enforce the *owner-as-modifier* discipline [22, 24] and thus permit a limited form of sharing (restricting all non-owning accesses to read-only). But the verification technique built on top of Universe types restricts sharing by limiting modifying access to an owned object to the object's owner and to other objects in that owner's ownership tree.

This paper introduces *selective ownership*, a more flexible way of giving structure to a program heap. Selective ownership allows programmers to structure the heap in two ways: (i) by defining an order on a program's type declarations and (ii) by imposing ownership on selected objects. The order on type declarations results in a stratified program heap but permits shared, modifying access to instances further "down" in the heap topology. By superimposing object ownership on selected objects in the heap, programmers can carve out partial sub-trees in the heap topology where the objects are owned.

Like classical ownership, selective ownership may have several applications. In this paper, we describe how selective ownership can facilitate the modular verification of multi-object invariants in the presence of call-backs. Selective ownership-based verification leverages the type ordering for the prevention of transitive call-backs, and ownership for the declaration and verification of invariants on owned objects. We exemplify selective ownership in the context of Rumer [4, 5], a programming language we have designed to embody *first-class relationships* [1, 4, 6, 10, 33, 39, 42, 44]. In Rumer, first-class relationships naturally define an order on type declarations, making Rumer a natural fit as a host language for selective ownership.

This paper extends our earlier work [5], in which we introduced Rumer as well as a verification technique for Rumer. The described verification technique leverages the type order defined by relationship declarations (called Matryoshka Principle) as well as a relationship-specific encapsulation mechanism (called member interposition) for the modular verification of multi-object invariants. In [5], we further briefly touch on the idea to overlay the type order with ownership. This paper works out the details of this idea and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

FOOL'12 October 22, 2012, Tucson, AZ, USA
Copyright © 2012 ACM [to be supplied]...\$10.00

also provides an abstract presentation, detached from relationship-based programming languages and Rumer, in particular.

The rest of this paper is structured as follows: Section 2 introduces the main idea of selective ownership, independently of a specific host language. Section 3 then exemplifies selective ownership in the context of Rumer. Section 4 briefly discusses some consequences of the co-existence of owned and shared objects. Section 5 summarizes related work, and Section 6 concludes the paper.

2. Selective ownership in a nutshell

This section introduces the core ideas of selective ownership. Even though we have developed selective ownership in the context of the Rumer programming language, the core ideas of selective ownership are of general applicability. We thus keep the presentation abstract in this section and defer the Rumer-specific aspects of selective ownership to Section 3. In the following, we first show how a program’s heap can be structured using selective ownership and then sketch how the imposed structure facilitates the modular verification of object invariants.

2.1 Heap structure

Selective ownership provides programmers with a “mix and match” approach to giving structure to a program’s heap. At its core, the approach relies on an ordering relation on a program’s type declarations. This ordering relation gives some basic structure to the program heap. If additional structuring is required, then the resulting heap can further be shaped by superimposing ownership on selected type instances in the heap.

We illustrate the approach on Figure 1 (a) and Figure 1 (b). Figure 1 (a) displays a program heap that has been shaped by declaring a type order and Figure 1 (b) displays one that has been shaped by declaring both a type order and instance ownership. To keep the presentation abstract, the two figures are not geared towards a particular programming language. Instead, they use only the notions of a type, type instance, and type instance reference. Types are represented as dark grey boxes, which enclose the instances of the type (represented as light gray circles). Type instance references are represented as arrows with a split arrowhead. In a class-based, object-oriented setting, types correspond to classes, type instances to objects, and type instance references to object references. To increase readability, we may occasionally refer to “type instances” as “instances” or “objects” and to “type instance references” as “references”.

Figure 1 (a) shows the result of imposing a *strict partial order* on a program’s type declarations. This figure displays the ordering relation using bold arrows with filled arrowheads, which indicate the order’s transitive reduction. To give structure to a program’s heap, selective ownership confines the declaration of type instance references such that an instance o of type O can only declare a reference to an instance o' of type O' if the pair $O \mapsto O'$ is an element of the type order. As shown by Figure 1 (a), the enforcement of this requirement gives rise to a program heap that forms a *directed acyclic graph (DAG)*. Similarly to the Universe type system [20–23, 37], selective ownership leaves the declaration of read-only type instance references unconstrained. This relaxation guarantees that any modifications by means of assignments or side-effect-generating method invocations comply with a program’s type order but allows for arbitrary reads or pure method invocations. For simplicity, Figure 1 (a) omits read-only references, but the reader can think of such references as occurring between arbitrary instances. A DAG topology is more permissive than the tree topology typically enforced by Ownership type systems [17–23, 37, 41] as it allows for shared, modifying access to instances further “down” in the heap topology. For example, the type instance “e1”, in Figure 1 (a), can both be modified by the type instances

“a2” and “c1”, and the type instance “f3” by the type instances “c2”, “b1”, and “d1”.

To shape a program’s heap even further, a programmer can superimpose the type ordering with instance ownership. Figure 1 (b) shows the result of imposing ownership on *selected instances* in the program heap of Figure 1 (a). Ownership is displayed by a dotted arrow from the owner to the owned instance. In Figure 1 (b), ownership has been declared for three type instances: for the type instance “e1” with the owning type instance “c1”, for the type instance “d1” with the owning type instance “b2”, and for the type instance “d2” with the owning type instance “b3”. The ownership declaration guarantees that only the owner can declare modifying type instance references to the owned instance and, hence, modify the owned type instance. As a result, the modifying references between the instances “a2” and “e1” and the instances “b3” and “d1”, which are legal in Figure 1 (a), are illegal in Figure 1 (b) and are crossed out. The declaration of corresponding read-only references, however, would be admissible since selective ownership permits arbitrary read-only references. As opposed to the ownership enforced by Ownership type systems [17–23, 37, 41], the ownership enforced by selective ownership is not transitive. This property allows for more flexibility. For example, even though the type instances “d1” and “d2” are owned by different owners, they share modifying access to the type instance “g1”.

2.2 Invariant verification technique

Invariants provide a foundation for verifying programs [26], and various object-oriented programming and specification languages [9, 28, 35] have adopted invariants for objects. An *object invariant* captures the properties of an object that the object exhibits in its consistent states. Object invariants are central to a wealth of object-oriented verification techniques [7, 8, 27, 30, 32, 36–38, 40, 43]. These techniques establish appropriate proof obligations to verify at compile-time that an object satisfies its invariant at designated program points at run-time. To be practical, those proof obligations must be *modular*, allowing classes to be verified independently from each other.

Modular, object-oriented verification techniques face two key challenges: multi-object invariants and call-backs [25, 29]. A *multi-object invariant* declared for an object o is an invariant that depends not only on the state of o but also on the state of any object p that o refers to. The verification of such invariants is difficult since the invariant may be violated not only through modifications of o but also by modifications of any of the objects p . A *call-back*, on the other hand, happens when a method $m()$ (possibly transitively) invokes a method $n()$ on $m()$ ’s current receiver object o_m . Call-backs may complicate the adoption of a visible-state semantics for invariants. A *visible-state semantics* [25, 38] expects the current receiver object to satisfy its invariant in the initial and final states of the executing method (the so-called *visible states*) but allows temporary violations in between those states. For instance, if $m()$ (possibly transitively) calls $n()$ while the invariant of o_m is temporarily broken, then o_m would not satisfy its invariant in the initial state of $n()$ when a call-back occurs.

Ownership-based verification techniques for object invariants [30, 34, 37, 38] leverage the tree topology of an ownership system to address the above mentioned challenges. In particular, they exploit (i) that invariants can depend only on fields of the invariant-declaring object or on fields of objects it owns, (ii) that modifications of an owned object’s fields are initiated¹ by the object’s owner (“owner-as-modifier discipline” [22, 24]), and (iii) that owned objects may invoke methods only on objects with the same owner

¹ Modifications of an owned object’s field are initiated by the owner if there exists a stack frame on the call-stack with the owner as the current receiver.

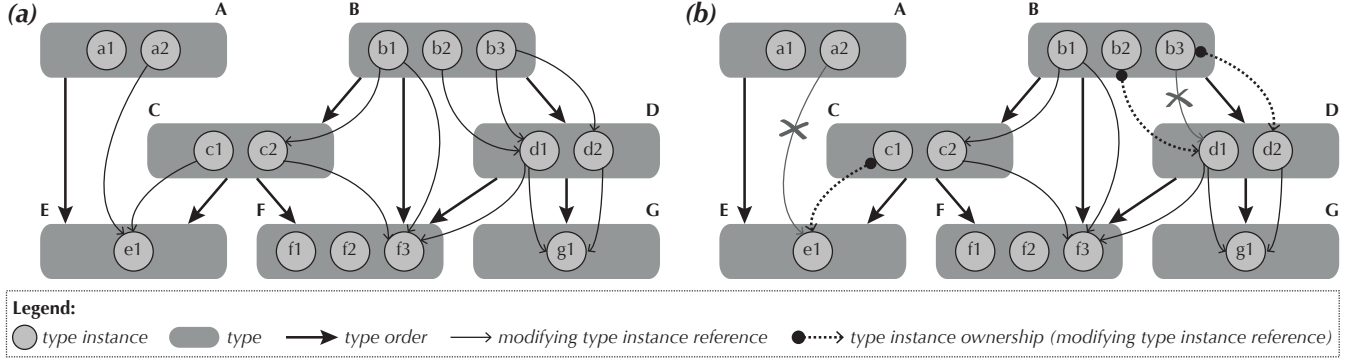


Figure 1. Abstract illustration of selective ownership: (a) valid program heap structure due to type order; (b) valid program heap structure due to type order *and* selective ownership. Illegal modifying type instance references from (a) are crossed out in (b).

or on objects they own. The first and the second property give an owner whose invariant relies on an owned object the chance to re-establish the invariant upon modifications of the owned object. The third property prevents call-backs into owners from objects within their ownership trees.

Next, we sketch how a visible-state verification technique can leverage both type ordering and object ownership to accommodate invariants over heap topologies that subsume shared, modifiable sub-structures. For simplicity, we assume that the underlying programming language allows assignments to a field f only of the form $this.f = \text{expr}$. In line with [38], we refer to this restriction as classical encapsulation. Our presentation draws from the visible-state verification technique we have developed for Rumer but generalizes its main ideas to fit the abstract description in this section. A detailed account of our verification technique, including its proof obligations as well as its soundness proof, can be found in [4].

A verification technique can leverage the type ordering entailed by selective ownership to prevent transitive call-backs. Since the type ordering forms a strict partial order, it is guaranteed to be *acyclic*. To prevent transitive call-backs, a verification technique simply needs to require that method invocations either propagate in the direction of a program’s type ordering relation or that the caller and callee of an invocation are the same object. For example, under this restriction, the type instance “d1” in Figure 1 (a) is allowed to invoke methods only on itself or on any instances of the types “G” and “F”, such as the referred-to instances “g1” and “f3”.

Given the absence of transitive call-backs, the support of a visible-state semantics for single-object invariants is straightforward: a verification technique simply needs (i) to require invariants declared for a type instance to depend only on fields of that instance and (ii) to impose the proof obligation on a method to restore the invariant of the current receiver instance in the final state of the method as well as before any invocations on the current receiver instance. For example, assume that we declare an invariant for instances of type “D” in Figure 1 (a) and we consider the execution of the method $m()$ on the type instance “d1”. The first requirement guarantees that $m()$ can violate at most the invariant of “d1” while it executes. The second requirement and the absence of transitive call-backs guarantee that any method invocation during $m()$ ’s execution encounters the callee instance in a consistent state.

The type ordering entailed by selective ownership is sufficient to prevent transitive call-backs, but not (generally²) sufficient to

support multi-object invariants. For example, if the invariant of instance “b2” in Figure 1 (a) were allowed to depend on the state of instance “d1”, a method with the receiver instance “d1” might violate the invariant of “b2”. If calls to this method are not controlled by instance “b2”, then the violation would go unnoticed during verification, making the verification technique unsound.

A way of accommodating multi-object invariants for a verification technique is to additionally impose ownership on the type instances on which a multi-object invariant depends. For example, assume that we declare an invariant for instances of type “B” in Figure 1 (b) such that the invariant not only depends on fields of the current instance but also on the fields of a referred-to instance of type “D”. Modular reasoning about such an invariant can be restored by making the instances of type “B” become the owner of the referred-to “D” instance. Given the ownership, instances of type “B” are guaranteed that any modifications of the referred-to “D” instance are solely triggered by themselves, giving them a chance to re-establish the invariant accordingly. Furthermore, thanks to the absence of transitive call-backs, instances of type “B” are guaranteed that they are not re-entered via a method invocation from their owned instances (as the instances of type “B” might be in an inconsistent state at that moment).

So far, we have shown how a verification technique can leverage type ordering to prevent transitive call-backs and selective ownership to control modifications of objects and what objects an invariant depends on. Since the ownership enforced by selective ownership is not transitive, a verification technique based on selective ownership can accommodate even invariants over heap topologies that subsume shared, modifiable sub-structures. For example, in Figure 1 (b) instances “d1” and “d2” share and may modify instance “g1”, although “d1” and “d2” have different owners. In Section 3, we show further how selective ownership can develop its full power in Rumer, a programming language supporting first-class relationships. Rumer complements the notion of an object or relationship instance with the one of an extent instance, that is, collections of instances. This combination allows for a more modular program design and facilitates the verification of invariants even in the presence of shared, modifiable sub-structures.

3. Selective ownership in Rumer

This section discusses how selective ownership can be incorporated into a programming language. We use as an example Rumer [4, 5], a programming language we have designed to embody *first-class relationships* [1, 4, 6, 10, 33, 39, 42, 44] and for which we have developed a *visible-state verification technique based on selective ownership* [4]. The section starts with a brief introduction to

²In earlier work [5], we describe a visible-state verification technique that leverages the type order defined by relationship declarations as well as an encapsulation mechanism called member interposition for the modular verification of multi-object invariants.

```

1 entity Node {
2   string info; // element field
3
4   void setInfo(string info) // element method
5   { this.info = info; }
6 }
7
8 relationship Parent
9   participants (Node child, Node parent) {
10
11   // extent method
12   extent void link(Node c, Node p) {
13     these.add(new Parent(c, p));
14   }
15 }

```

Figure 2. Simple Rumer program modeling node hierarchies.

Rumer; we introduce Rumer only as far as necessary to follow the presentation of selective ownership in this paper. An in-depth introduction to Rumer as well as a discussion of related work on first-class relationships is given in [4]. Then, based on a running example, we explain how programmers define type ordering and ownership in Rumer and we sketch the Rumer verification technique based on the example.

3.1 Introduction to Rumer

Relationship-based programming languages extend object-oriented languages with the abstraction of a *relationship* to encapsulate the relationships that naturally arise between instances of classes. In those languages, relationships are first-class citizens: relationships can be instantiated as well as declare their own fields and methods. Early research on first-class relationships was motivated by the observation that programmers are poorly served when trying to implement relationships in object-oriented programming languages. As those languages lack first-class support for relationships, relationships must be represented in terms of reference fields and auxiliary classes. This indirection leads to a distribution of relationship code across several classes, making the resulting program prone to error.

Figure 2 displays a simple Rumer program. The program models hierarchies between nodes. As illustrated by Figure 2, Rumer supports two kinds of programmer-definable types: *entities* and *relationships*. In the example, the entity `Node` and the relationship `Parent` are declared. Entities are similar to classes and abstract objects. Relationships, on the other hand, abstract the relationships between instances. To indicate the types of instances that a relationship instance relates, a relationship declaration includes a participants clause. According to its participants clause on line 9, relationship `Parent` relates instances of the entity `Node`. To disambiguate the position an instance takes in a relationship, identifiers are assigned to the type declarations in a participants clause, indicating the role an instance of the type plays in the relationship. In the example, a `Parent` relationship instance relates a `child` node to its corresponding `parent` node.

Figure 3 (a) provides a schematic illustration of the run-time instances that the Rumer program displayed in Figure 2 may produce. It represents entity and relationship instances as dark gray circles and light gray ellipses, respectively, and connects relationship instances to their participant instances by lines, which are labeled with the role identifiers of the relationship’s participants clause. For later reference, we mark entity and relationship instances with numbers and letters, respectively. In comparison to the object graph that would be produced by a corresponding class-based object-oriented implementation of the program shown in Figure 2, the run-time structure displayed in Figure 3 (a) differs in the existence of

explicit relationship instances and in the absence of references in nodes. Since relationships are bidirectional, Rumer facilitates access to the participating instances at either side of a relationship instance. As a result, the declaration of references in objects is no longer a prerequisite for navigating the object graph. The different nature of objects in Rumer is also the reason why we use the term “entity” for the type abstracting objects rather than the term “class”.

A further distinguishing feature of Rumer is the support of *extents*. An extent denotes a programmer-instantiable collection of instances. The term originates from the ODMG (Object Data Management Group) object model [16], where an extent of a type denotes the set of all instances of that type. Existing relationship-based programming languages maintain a single extent instance for each relationship declaration. Rumer builds on Nelson et al.’s suggestion to support multiple extent instances for first-class relationships [39] and provides extent instantiation not only for relationships but also for entities. For example, the following line of code instantiates an extent of type `Parent` and assigns the resulting extent instance to the variable `parents`:

```
Extent<Parent> parents = new Extent<Parent>();
```

As indicated by the argument type provided in angle brackets, the extent instance `parents` comprises `Parent` relationship instances. To distinguish the instances residing in an extent instance from the extent instance itself, we use the term *element instance* to refer to the former. Thus, the variable `parents` denotes a `Parent` extent instance that comprises `Parent` element instances.

Extent instances are explicitly populated and depopulated by programmers, and the Rumer type system guarantees that element instances always inhabit exactly one extent instance. To facilitate retrieval of element instances from an extent instance, Rumer supports a rich range of side-effect free queries in the spirit of LINQ (.NET Language-Integrated Query) [11, 12]. For example, given a `Node` element instance `p`, the following query returns all transitive children of `p`:

```
parents.tClosure().select(x: x.parent == p).child
```

The built-in query operator `tClosure()` builds the transitive closure of the relation described by the `parents` extent instance, and the built-in query operator `select()` reduces the resulting relation to the subset containing only `Parent` element instances that have `p` as an immediate parent or ancestor. The role projection operator `child` then projects the resulting subset of `Parent` element instances onto all `Node` element instances that participate as a `child` in those `Parent` element instances.

Extent instances in Rumer are not plain collections, but rather they can be equipped by the programmer with customized fields and methods. As a result, a Rumer type declaration may comprise field and method declarations both for element instances and extent instances of the type. Extent fields and methods are denoted by the *extent* keyword. For instance, entity `Node` declares the element field `info` and the element method `setInfo()` on line 2 and line 4 in Figure 2, respectively, and relationship `Parent` declares the extent method `link()` on line 11. The element method `setInfo()` sets the `info` field of the `Node` element instance that is the current receiver of the method. The keyword `this` refers to the current receiver instance of an element method. The extent method `link()` connects the argument `Node` element instance `c` as a `child` to the argument `Node` element instance `p`. The method invokes the built-in addition operator `add()` (line 13) on the `Parent` extent instance that is the current receiver of the method. The keyword `these` refers to the current receiver instance of an extent method. The choice of the keyword reflects the fact that an extent instance may contain several element instances. The ad-

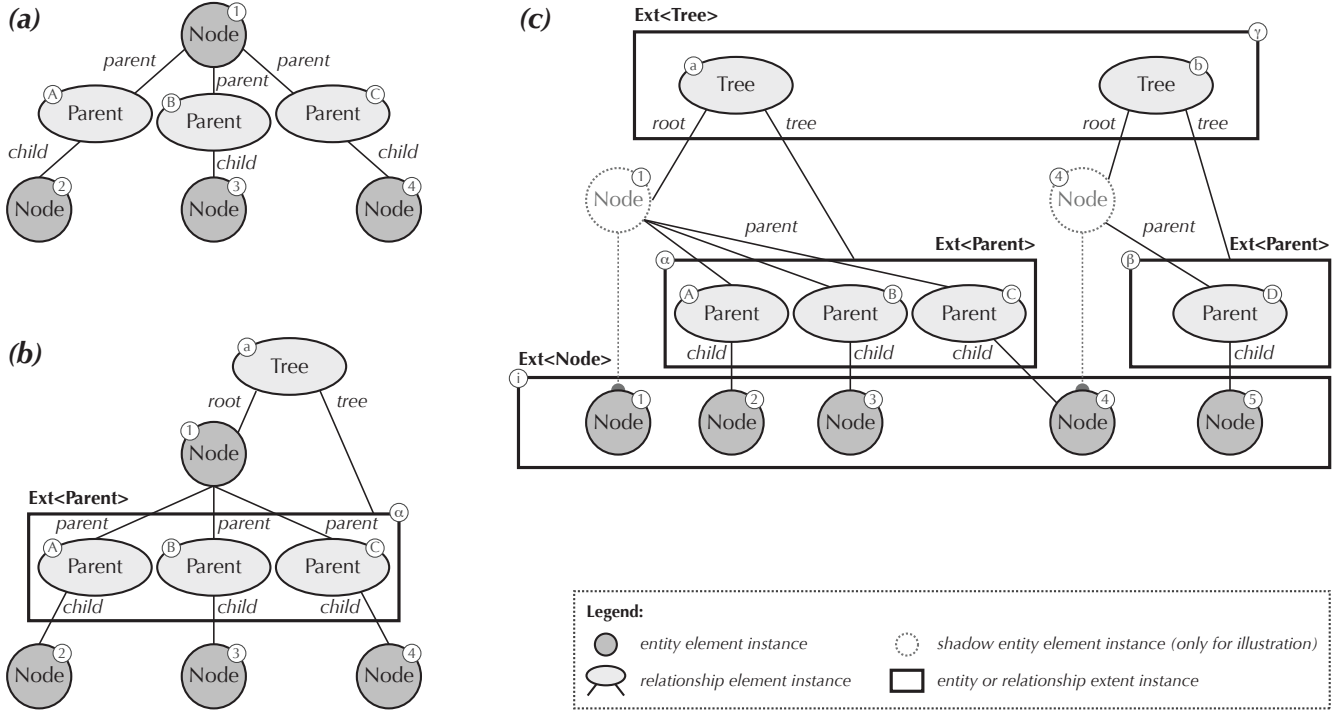


Figure 3. Schematic illustration of Rumer run-time instances: (a) several element instances of relationships declared in Figure 2; (b) one element instance instantiating relationship *Tree* declared in Figure 4; (c) complete heap for program declared in Figure 4 as well as its augmented version declared in Figure 6 that employs ownership.

dition operator instantiates a new *Parent* element instance that inhabits the current receiver extent instance.

3.2 Running example: tree

To illustrate the need for selective ownership, we extend the program shown in Figure 2 with a new relationship that allows us to model actual trees rather than node hierarchies. The new relationship *Tree* is shown in Figure 4. As indicated by its participants clause, a *Tree* element instance relates a *Node* element instance as its *root* to a *Parent* extent instance as its *tree*. In more abstract terms, a tree is thus represented by a tuple that has the root node of a tree as its left element and a relation describing the hierarchy between the tree’s nodes as its right element. Whereas typical object-oriented implementations use the same abstraction to describe a tree as well as its children sub-trees, we chose to separate the two notions. As we will see (see Section 3.3 and Section 3.4), the chosen representation facilitates a concise formulation of the tree properties in terms of an invariant.

The chosen tree representation also becomes apparent in Figure 3 (b), which provides a schematic illustration of a *Tree* element instance. Figure 3 (b) uses the same graphical notations as Figure 3 (a) but complements them with extent instances. Extent instances are represented as rectangular boxes. Figure 3 (b) thus displays the *Parent* extent instance “ α ”, which is the *tree* participant instance of the *Tree* element instance “a”. The matching labels used in Figure 3 (a) and Figure 3 (b) further indicate that the *Parent* extent instance “ α ” displayed in Figure 3 (b) exactly subsumes the relationship element instances displayed in Figure 3 (a). To keep Figure 3 (b) simple, we have chosen a *Tree* element instance with only one layer. However, the relationship declaration in Figure 4 also supports multi-layered trees as well.

Figure 3 (c) finally shows the complete view of a run-time heap that may be produced by instantiating the declarations listed in Figure 2 and in Figure 4. This heap comprises the two *Tree* element instances “a” and “b”. As indicated by the matching labels, the *Tree* element instance “a” exactly subsumes the run-time instances shown in Figure 3 (b). Since element instances are guaranteed to inhabit exactly one extent instance (see Section 3.1), Figure 3 (c) additionally displays the extent instances “i” and “ γ ” in which the *Node* element instances and *Tree* element instances, respectively, reside. To keep the graphical layout well-arranged, Figure 3 (c) makes use of “shadow” *Node* element instances. Those shadows are purely graphical copies of the instance they are connected to by a dotted line. For example, the *Node* element instance “4” is part of both displayed *Tree* element instances: it is a leaf node of the *Tree* element instance “a” and the root node of the *Tree* element instance “b”. *Node* element instance “4” also nicely motivates the need for selective ownership as it represents a run-time instance further down in the heap topology that is shared among and modified by both trees.

Relationship *Tree* declares various methods for tree manipulations, such as appending one tree to another one. In the following, we briefly explain the implementations of these methods for the interested reader:

The extent method `createTree()` (line 4) instantiates a new *Tree* element instance that inhabits the current receiver extent instance referred to by `these`. In terms of Figure 3 (c), `these` denotes the *Tree* extent instance “ γ ”. The newly created *Tree* element instance relates the argument *Node* element instance `r` as a root to an empty, newly created *Parent* extent instances as a tree.

The element method `appendTree()` (line 7) appends the *Tree* element instance `t` to the current receiver *Tree* element

```

1 relationship Tree participants (Node root, Extent<Parent> tree) {
2
3   // Instantiates a Tree element instance with 'root' r and a new empty 'tree' that inhabits these.
4   extent void createTree(Node r)
5   { these.add(new Tree(r, new Extent<Parent>())); }
6
7   void appendTree(Tree t, Node p) // Appends tree t to this as 'child' of p.
8   { this.appendNode(t.root, p); this.appendSubTree(t.tree, t.root); }
9
10  void appendSubTree(query Set<Parent> c, Node p) // Appends sub-tree c to this as 'child' of p.
11  { foreach (cp isElementOf c.select(x: x.parent == p)) {
12    this.appendNode(cp.child, cp.parent);
13    this.appendSubTree(c.select(x: x.child isElementOf
14      c.tClosure().select(y: y.parent == cp.child).child), cp.child); }}
15
16  void appendNode(Node c, Node p) // Appends node c to this as 'child' of p.
17  { this.tree.link(c, p); }
18 }

```

Figure 4. Relationship Tree. Extends program in Figure 2 to model trees.

instance as a child of the Node element instance p. Method `appendTree()` relies on the element methods `appendNode()` and `appendSubTree()`.

The element method `appendNode()` (line 16) appends the Node element instance c to the current receiver Tree element instance as a child of the Node element instance p. It relies on Parent's extent method `link()`, which it invokes on the current receiver instance's tree Parent extent instance.

The element method `appendSubTree()` (line 10), finally, appends the sub-tree denoted by the set of Parent element instances c to the current receiver Tree element instance as a child of the Node element instance p. The method is implemented recursively to append the sub-tree in a depth-first traversal order. In each recursive step (line 12), one Node element instance of the sub-tree (denoted by `cp.child`) is appended to its corresponding parent Node element instance in the current receiver instance's tree Parent extent instance (denoted by `cp.parent`). Recursion stops whenever the sub-tree c denotes the empty set. This is the case as soon as a leaf node has been inserted in the preceding recursive invocation.

3.3 Declaration of type ordering

A glimpse at Figure 3 (c) reveals that first-class relationships naturally give rise to an ordering on type declarations. Building on this observation, Rumer allows programmers to structure a program's heap by declaring relationships. The ordering on entity and relationships, in particular, is defined by a relationship's participants clause. In terms of Figure 1 (a), a relationship's participants clause defines two type order arrows such that each arrow points from the relationship to one of the relationship's participant types. For the running example defined in Figure 2 and Figure 4, the transitive reduction of the resulting type order thus is:

$$\{Tree \mapsto Parent, Tree \mapsto Node, Parent \mapsto Node\}$$

To meet the criterion of a type order (see Section 2.1), the transitive closure of the above relation must form a strict partial order. This requirement, in particular, expects relationship declarations to be acyclic. It is noteworthy that this requirement does not prevent the implementation of recursive data structures in Rumer since the "links" between the structure's data are represented by relationship instances, as opposed to data references.

In earlier work [4, 5], we introduced a visible-state verification technique that leverages the type order defined by relationship declarations. As outlined in Section 2.2, our technique prevents transi-

tive call-backs by requiring method invocations to either propagate in the direction of a program's relationship declarations or to dispatch on the current receiver instance. The built-in query operators in Rumer are not subject to this restriction as they are side-effect free.

To illustrate what kinds of invariants can be accommodated using the order prescribed by relationship declarations, we introduce an invariant to the running example. In particular, we declare an invariant for extent instances of relationship Parent to guarantee that a Parent extent instance indeed describes a hierarchy of nodes:

```

extent invariant // Parent's extent invariant
these.isPartialFunction() &
these.tClosure().isIrreflexive();

```

In addition to the built-in query operator `tClosure()` encountered earlier, the extent invariant uses the built-in query operators `isPartialFunction()` and `isIrreflexive()`. These operators evaluate to true if the set of relationship element instances on which the operator is invoked forms a partial function and irreflexive relation, respectively. The invariant thus guarantees that a Parent extent instance forms a forest of trees.

The extent invariant of Parent depends only on the state of the extent instance, but not on the states of the Node element instances since the links between the nodes are expressed as a relationship (as opposed to references stored in the Node element instances). Consequently, we can maintain the invariant without requiring the extent or the Tree relationship to own the nodes. The absence of ownership allows arbitrary instances to refer to and to modify the nodes - a setup that is not permitted in existing ownership-based verification techniques. Since Rumer enforces classical encapsulation (see Section 2.2) for its instances, the extent instance is the only instance that can write to its content. To maintain Parent's extent invariant, our verification technique thus imposes the proof obligation on any of Parent's extent methods to establish the current receiver's invariant in the final state of the method as well as before any invocations on the current receiver. An in-depth discussion of our verification technique, including a complete overview of its proof obligations as well as its soundness proof, can be found in [4].

3.4 Declaration of ownership

Thanks to relationship Parent's extent invariant, relationship Tree is guaranteed that its element instances' tree Parent extent instances describe node hierarchies. However, as exempli-

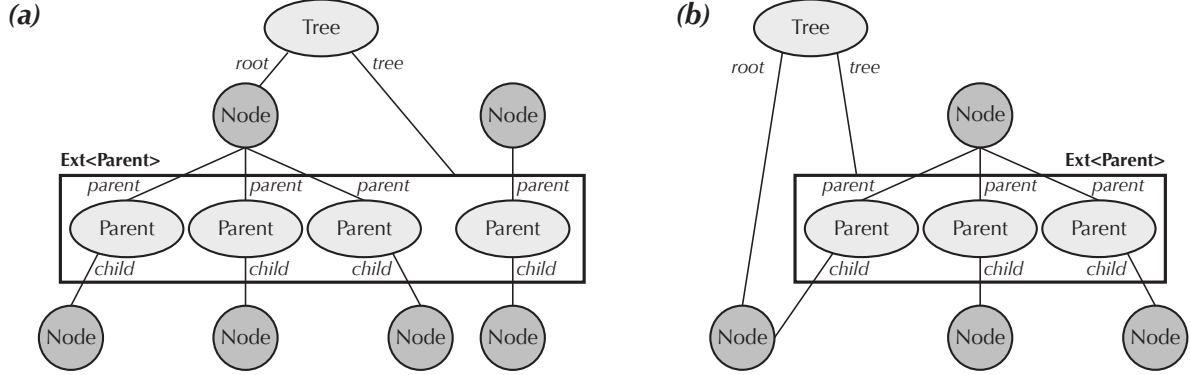


Figure 5. Possible instantiations of program declared in Figure 4. Both instantiations satisfy Parent’s extent invariant introduced in Section 3.3 but do not form proper trees: (a) the Tree element instance’s tree Parent extent instance represents a forest of trees, (b) the Tree element instance’s root Node element instance is not the topmost Node element instances of its tree Parent extent instance.

fied by Figure 5 (a) and Figure 5 (b), this invariant is not sufficient to guarantee that Tree element instances indeed form proper trees. For example, Figure 5 (a) displays a Tree element instance whose tree Parent extent instance represents a forest of trees, and Figure 5 (b) displays a Tree element instance whose root Node element instance is not the topmost Node element instance of its tree Parent extent instance.

To guarantee that Tree element instances actually form proper trees, we need to impose an appropriate invariant on element instances of relationship Tree. Intuitively, this invariant shall make sure, for any Tree element instance t , that t ’s root Node element instance is the same as the topmost Node element instance in t ’s tree Parent extent instance and, also, that there exists a topmost Node element instance in t ’s tree Parent extent instance. This invariant can be expressed in Rumer as an element invariant on relationship Tree as follows:

```
invariant // Tree's element invariant
!(this.root isElementOf this.tree.child) &
(!this.tree.isEmpty() => this.root isElementOf
  this.tree.parent) &
this.tree.tClosure().select(cp: cp.parent ==
  this.root).child == this.tree.child;
```

The first conjunct of the element invariant requires that a Tree element instance’s root Node element instance never appears as a child in the relation described by the Tree element instance’s tree Parent extent instance. The second conjunct requires that a Tree element instance’s root Node element instance appears as a parent in the relation described by the Tree element instance’s tree Parent extent instance, unless this relation is empty. The first and second conjunct thus rule out instantiations such as the one shown in Figure 5 (b). The third conjunct requires that a Tree element instance’s root Node element instance is the transitive parent of all children nodes of the relation described by the Tree element instance’s tree Parent extent instance. The third conjunct thus rules out instantiations such as the one shown in Figure 5 (a).

As opposed to Parent’s extent invariant, this invariant does not only depend on the state of the invariant’s Tree element instance but also on the state of its instance’s tree Parent extent instance. More specifically, the invariant depends on the content of its instance’s tree Parent extent instance and can thus be violated by the addition or removal of any Parent element instances to or from the instance’s tree Parent extent instance. As a result, the element invariant of relationship Tree cannot be

accommodated by our verification technique solely by leveraging type ordering.

However, our verification technique can accommodate the element invariant of relationship Tree by superimposing the type order prescribed by relationship declarations with ownership. More specifically, we must make a Tree element instance the owner of its tree Parent extent instance. In terms of Figure 3 (c), the ownership will guarantee that the Tree element instances “a” and “b” are the owners of their tree Parent extent instances “ α ” and “ β ”, respectively.

Figure 6 shows the result of superimposing ownership on the relationship Tree declared in Figure 4; the differences between the two versions are highlighted in Figure 6. The declarations of entity Node and relationship Parent are unaffected by the ownership declaration and are shown in Figure 2. As indicated by the participants clause of the new version of relationship Tree, we use type *modifiers* to express ownership of an instance relative to a current receiver instance. We distinguish the following selective ownership modifiers:

- **owned**: referred-to instance has the current receiver instance as its owner;
- **shared** (default modifier): referred-to instance does not have an owner;
- **readonly**: referred-to instance may or may not have an owner.

By annotating its tree participant type with the ownership modifier *owned*, relationship Tree guarantees that its element instances become the unique owner of their tree Parent extent instances. If a type declaration omits the ownership modifier, the default modifier *shared* is assumed. The use of type modifiers results in an ownership type system that is similarly lightweight as the Universe type system [20–23, 37]. However, unlike the ownership enforced by Universe types, the ownership enforced by selective ownership is not transitive. For example, the new version of relationship Tree in Figure 6 does neither affect the Parent element instances within an owned Parent extent instance nor the Node element instances related by those Parent element instances. As a result, the program heap depicted in Figure 3 (c) still amounts to a valid heap that can be produced by the new relationship declaration. The two Tree element instances “a” and “b” displayed in Figure 3 (c), in particular, are allowed to share and modify the Node element instance “4” while keeping their tree Parent extent instances “ α ” and “ β ”, respectively, separate.

To guarantee that the two ways of structuring a program’s heap offered by selective ownership — type order and instance ownership — nicely complement each other, the ownership relation must

```

1 // A Tree element instance owns its 'tree' Parent extent instance.
2 relationship Tree participants (Node root, owned Extent<Parent> tree) {
3
4     extent void createTree(Node r)
5         // New Tree element instance becomes owner of new Parent extent instance.
6         { these.add(new Tree(r, new owned Extent<Parent>())); }
7
8     void appendTree(Tree t, Node p)
9     { this.appendNode(t.root, p); this.appendSubTree(t.tree, t.root); }
10
11    void appendSubTree(query Set<Parent> c, Node p)
12    { foreach (cp isElementOf c.select(x: x.parent == p)) {
13        this.appendNode(cp.child, cp.parent);
14        this.appendSubTree(c.select(x: x.child isElementOf
15            c.tClosure().select(y: y.parent == cp.child).child), cp.child); }}
16
17    void appendNode(Node c, Node p)
18    { this.tree.link(c, p); } // Invocation admissible: this is owner of 'tree' Parent extent instance.
19 }

```

Figure 6. Relationship Tree augmented with ownership. Differences to version without ownership (see Figure 4) are highlighted.

be a subset of the type order. In Rumer, this requirement is met by allowing only relationships to impose ownership on instances of their participant types. Given this restriction, owners are guaranteed not to be re-entered (in a possibly inconsistent state) via method invocations from their owned instances.

Selective ownership modifiers constrain method invocations and thus strengthen the restrictions imposed on method invocations by the type order. In particular, selective ownership allows only owners to invoke methods on owned instances. For example, the invocation of method `link()` on the `Parent` extent instance referred-to by `this.tree` on line 18 in Figure 6 is admissible because the current receiver `Tree` element instance is the owner of its `tree` `Parent` extent instance. This regime guarantees that modifications of an owned instance’s fields are initiated by the instance’s owner, giving the owner a chance to re-establish the invariant upon modifications of the owned instance. Method invocations on shared instances, on the other hand, are not constrained by selective ownership. Furthermore, selective ownership permits the reading of fields of owned instances and the invocation of built-in query operators on owned instances. For example, the access `t.tree` on line 9 in Figure 6 is admissible because it is a read access. The selective ownership modifier `readonly`, lastly, forbids any method invocations on the referred-to instance but permits read accesses or invocations of built-in query operators.

4. Discussion

As exemplified by the program heap shown in Figure 3(c), selective ownership enables a *hybrid* ownership scheme in which owned and shared instances coexist. To permit such a scheme, type declarations must be formulated so as to allow possible callers to obtain either owned or shared instances of the type. As a result, only the caller-site knows about the ownership of an instance and can thus guard the instance against prohibited modifications or accidental leaking by establishing appropriate selective ownership modifiers. The callee-site, on the other hand, does not know whether a particular instance is owned or shared and may thus perceive an owned instance as “supposedly shared”. For example, relationship `Parent` in Figure 2 assumes the default selective ownership modifier `shared` for its extent instances and may thus accidentally leak a `Tree` element instance’s `tree` `Parent` extent instance.

In [4] we establish appropriate well-formedness conditions on a Rumer program to prevent the accidental leaking of “supposedly shared” instances. In particular, we prove that, given those well-

formedness conditions, “supposedly shared” instances cannot outlive the (possibly transitive) method executions within which they are produced. Furthermore, we prove a lemma that captures the effects of selective ownership on a program’s call stack. The lemma is stated from the perspective of an owned instance and guarantees that any owned instance residing on the call stack is preceded by its owner. As a result, the lemma guarantees that any modifications of an owned instance’s field are initiated by the instance’s owner. However, due to the hybrid nature of the underlying ownership system, this property does not hold for all the instances in a program’s heap, but only for those instances in a program’s heap that are owned. To contrast the discipline emerging from selective ownership with the “owner-as-modifier” discipline of the Universe type system [22, 24], we refer to our discipline as the “owned-called-by-owner” discipline.

The currently enforced well-formedness conditions to prevent the accidental leaking of “supposedly shared” instances restrict the type of fields entities and relationships can declare. In particular, they prevent an entity or relationship from defining fields that “point to” type instances of which the entity or relationship is a (possibly transitive) participant. As part of future work, we would like to investigate less restrictive mechanisms to prevent the accidental leaking of “supposedly shared” instances.

5. Related work

In this section, we focus on related work on Ownership type systems and, in particular, on ownership-based verification techniques. An extensive discussion of related work on relationship-based programming languages is given in [4].

Work on “traditional” forms of ownership can broadly be categorized into work on ownership types [17–19, 41] and work on Universe types [20–23, 37]. In both ownership schemes, all objects are owned and have exactly one owning object. The two schemes, however, differ in their applied encapsulation discipline. Whereas ownership types typically enforce the *owner-as-dominator* discipline, Universe types typically enforce the *owner-as-modifier* discipline [22, 24]. The owner-as-dominator discipline requires all reference chains to an object to pass through the object’s owner. The owner-as-modifier discipline, on the other hand, enforces a less stringent alias restriction and requires only modifications of an object to be initiated by the object’s owner.

Whereas ownership types allow owned objects to establish back-references to their owners, Universe types permit such ref-

erences only if they are read-only. This restriction makes the Universe type system attractive for program verification since it forbids call-backs into owners. The amenability of Universe types and similar systems for program verification has been shown in [30, 37, 38]. The benefits of ownership for program verification have also been demonstrated in the context of Oval [34], a variant of an ownership-type-based language. As opposed to other ownership type systems [17–19, 41], Oval’s types system enforces an owner-as-modifier discipline and employs effect annotations to deal with call-backs.

The presented verification technique is most closely related to the Universe-type-based verification technique [37, 38] since selective ownership is similarly lightweight as Universe types thanks to the use of ownership modifiers. However, selective ownership allows for unrestricted sharing of instances further “down” in the heap topology. This relaxation is due to the fact that selective ownership allows heap structure to be enforced either by type order alone or by type order combined with instance ownership. Selective ownership gives furthermore rise to a hybrid ownership scheme in which owned and shared instances coexist.

Our extent invariants are related to visibility-based invariants [30, 38] since they support certain multi-object invariants without requiring ownership. Extent invariants lead to simple proof obligations for extent methods, whereas visibility-based invariants require proof obligations that quantify over all objects that are possibly affected by a field update.

Leino et al. [31] use a programmer-declared type ordering to control class initialization and to verify static class invariants. In our system, the type ordering is used to prevent transitive call-backs. Instead of requiring explicit declarations, we infer the type ordering from the relationship declarations in a Rumer program.

Cameron et al. [15] propose a type system that supports multiple ownership. It enforces a DAG topology on the heap and, thus, permits several objects to own and modify owned objects. So far there is no verification technique that handles this expressiveness. In our approach, certain invariants can be stated as extent invariants rather than object invariants. Therefore, we can verify such invariants without requiring ownership.

In ownership domains [3], the objects owned by one owner can be grouped into several domains. A domain can be declared public; each client that may access an owner object may also own the objects in its public domain. For instance, a linked list structure may put all list nodes in a non-public domain and the list iterators in another, public domain. Each client with access to the list may then access its iterators, which in turn can be permitted to have access to the nodes. Therefore, ownership domains permit sharing of owned objects. Modular verification of invariants over such structures is difficult since owners do not have full control over the owned objects. For instance, it is unclear how to maintain the list’s invariant when the nodes are modified via an iterator. In our approach, we can formulate structural properties of the list as an extent invariant and are thus able to verify such invariants without imposing ownership on the nodes.

6. Conclusions

This paper introduces selective ownership, a flexible “mix and match” approach to giving structure to a program’s heap in two ways: by defining an ordering relation on a program’s type declarations and by imposing ownership on selected instances. We evaluate selective ownership for the purpose of program verification. As compared to existing ownership-based verification techniques, the heap topology enforced by selective ownership does not amount to a tree, but to a DAG with partial sub-trees. This scheme permits the modular verification of ownership-based invariants without restricting access to instances further “down” in the heap topology.

Although selective ownership is not tied to a particular host language, it seems to develop its full power in a language that supports first-class relationships. We illustrate selective ownership in the context of Rumer, for which we have developed selective ownership. First-class relationships in Rumer give naturally rise to an ordering on type declarations. Furthermore, the availability of an elaborate set of type abstractions — entity versus relationship as well as element instance versus extent instance — allows for a practical modularization of programs that facilitates the expression of invariants over heap topologies that subsume shared, modifiable sub-structures.

Acknowledgments

We are grateful to Jonathan Aldrich and the anonymous reviewers for their valuable feedback on this paper. We also thank Sophia Drossopoulou, Alexander J. Summers, and James Noble for stimulating discussions on selective ownership. This research was supported in part by the Swiss National Science Foundation through grant PBEZP2.140051 and in part by the Army Research Office under Contract W911NF-09-1-0273.

References

- [1] A. Albano, G. Ghelli, and R. Orsini. A relationship mechanism for a strongly typed object-oriented database programming language. In *17th International Conference on Very Large Data Bases (VLDB’91)*, pages 565–575. Morgan Kaufmann Publishers Inc., 1991.
- [2] J. Aldrich. *Using Types to Enforce Architectural Structure*. PhD thesis, University of Washington, 2003.
- [3] J. Aldrich and C. Chambers. Ownership domains: Separating aliasing policy from mechanism. In *18th European Conference on Object-Oriented Programming (ECOOP’04)*, volume 3086 of *Lecture Notes in Computer Science*, pages 1–25. Springer, 2004.
- [4] S. Balzer. *Rumer: a Programming Language and Modular Verification Technique Based on Relationships*. PhD thesis, 19851, ETH Zurich, 2011.
- [5] S. Balzer and T. R. Gross. Verifying multi-object invariants with relationships. In *25th European Conference on Object-Oriented Programming (ECOOP’11)*, volume 6813 of *Lecture Notes in Computer Science*, pages 358–382. Springer, 2011.
- [6] S. Balzer, T. R. Gross, and P. Eugster. A relational model of object collaborations and its use in reasoning about relationships. In *21st European Conference on Object-Oriented Programming (ECOOP’07)*, volume 4609 of *Lecture Notes in Computer Science*, pages 323–346. Springer, 2007.
- [7] M. Barnett and D. A. Naumann. Friends need a bit more: Maintaining invariants over shared state. In *7th International Conference on Mathematics of Program Construction (MPC’04)*, *Lecture Notes in Computer Science*, pages 54–84. Springer, 2004.
- [8] M. Barnett, R. DeLine, M. Fähndrich, K. R. M. Leino, and W. Schulte. Verification of object-oriented programs with invariants. *Journal of Object Technology (JOT)*, 3(6):27–56, 2004.
- [9] M. Barnett, K. R. M. Leino, and W. Schulte. The Spec[#] programming system: An overview. In *International Workshop on Construction and Analysis of Safe, Secure, and Interoperable Smart Devices (CASSIS’04)*, volume 3362 of *Lecture Notes in Computer Science*, pages 49–69. Springer, 2004.
- [10] G. M. Bierman and A. Wren. First-class relationships in an object-oriented language. In *19th European Conference on Object-Oriented Programming (ECOOP’05)*, volume 3586 of *Lecture Notes in Computer Science*, pages 262–286. Springer, 2005.
- [11] G. M. Bierman, E. Meijer, and M. Torgersen. Lost in translation: Formalizing proposed extensions to Spec[#]. In *22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA’07)*, pages 479–498. ACM, 2007.

- [12] D. Box and A. Hejlsberg. LINQ: .NET Language-Integrated Query. <http://msdn2.microsoft.com/en-us/library/bb308959.aspx>, February 2007.
- [13] C. Boyapati, R. Lee, and M. C. Rinard. Ownership types for safe programming: Preventing data races and deadlocks. In *17th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'02)*, pages 211–230, New York, NY, USA, 2002. ACM.
- [14] C. Boyapati, A. Salcianu, W. S. Beebe, and M. C. Rinard. Ownership types for safe region-based memory management in real-time Java. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '03)*, pages 324–337. ACM, 2003.
- [15] N. R. Cameron, S. Drossopoulou, J. Noble, and M. J. Smith. Multiple ownership. In *22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'07)*, pages 441–460. ACM, 2007.
- [16] R. G. Cattell and D. K. Barry. *The Object Data Standard: ODMG 3.0*. Morgan Kaufmann, 2000.
- [17] D. Clarke. *Object Ownership and Containment*. PhD thesis, University of New South Wales, October 2002.
- [18] D. Clarke and S. Drossopoulou. Ownership, encapsulation and the disjointness of type and effect. In *17th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'02)*, pages 292–310. ACM, 2002.
- [19] D. Clarke, J. Potter, and J. Noble. Ownership types for flexible alias protection. In *13th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'98)*, pages 48–64. ACM, 1998.
- [20] D. Cunningham, W. Dietl, S. Drossopoulou, A. Francalanza, P. Müller, and A. J. Summers. Universe Types for topology and encapsulation. In *6th International Symposium of Formal Methods for Components and Objects (FMCO'07)*, volume 5382 of *Lecture Notes in Computer Science*, pages 72–112. Springer, 2007.
- [21] W. Dietl. *Universe Types Topology, Encapsulation, Genericity, and Tools*. PhD thesis, ETH Zurich, 2009. 18522.
- [22] W. Dietl and P. Müller. Universes: Lightweight ownership for JML. *Journal of Object Technology (JOT)*, 4(8):5–32, 2005.
- [23] W. Dietl, S. Drossopoulou, and P. Müller. Generic Universe Types. In *21st European Conference on Object-Oriented Programming (ECOOP'07)*, volume 4609 of *Lecture Notes in Computer Science*, pages 28–53. Springer, 2007.
- [24] W. Dietl, S. Drossopoulou, and P. Müller. Separating ownership topology and encapsulation with generic universe types. *ACM Trans. Program. Lang. Syst.*, 33:20:1–20:62, 2011.
- [25] S. Drossopoulou, A. Francalanza, P. Müller, and A. Summers. A unified framework for verification techniques for object invariants. In *22nd European Conference on Object-Oriented Programming (ECOOP'08)*, volume 5142 of *Lecture Notes in Computer Science*, pages 412–437. Springer, 2008.
- [26] C. Hoare. Proof of correctness of data representations. *Acta Informatica*, 1(4):271–281, 1972.
- [27] K. Huizing and R. Kuiper. Verification of object oriented programs using class invariants. In *3rd International Conference on Fundamental Approaches to Software Engineering (FASE'00)*, volume 1783 of *Lecture Notes in Computer Science*, pages 208–221. Springer, 2000.
- [28] G. T. Leavens, A. L. Baker, and C. Ruby. Preliminary design of JML: A behavioral interface specification language for Java. Technical Report 98-06-rev29, Iowa State University, 2006.
- [29] G. T. Leavens, K. R. M. Leino, and P. Müller. Specification and verification challenges for sequential object-oriented programs. *Formal Aspects of Computing*, 19(2):159–189, 2007.
- [30] K. R. M. Leino and P. Müller. Object invariants in dynamic contexts. In *18th European Conference on Object-Oriented Programming (ECOOP'04)*, volume 3086 of *Lecture Notes in Computer Science*, pages 491–516. Springer, 2004.
- [31] K. R. M. Leino and P. Müller. Modular verification of static class invariants. In *International Symposium of Formal Methods Europe (FM'05)*, volume 3582 of *Lecture Notes in Computer Science*, pages 26–42. Springer, 2005.
- [32] K. R. M. Leino and W. Schulte. Using history invariants to verify observers. In *16th European Symposium on Programming (ESOP'07)*, *Lecture Notes in Computer Science*, pages 80–94. Springer, 2007.
- [33] Y. D. Liu and S. F. Smith. Interaction-based programming with Classages. In *20th ACM Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA'05)*, pages 191–209. ACM, 2005.
- [34] Y. Lu, J. Potter, and J. Xue. Validity invariants and effects. In *21st European Conference on Object-Oriented Programming (ECOOP'07)*, volume 4609 of *Lecture Notes in Computer Science*, pages 202–226. Springer, 2007.
- [35] B. Meyer. *Object-Oriented Software Construction*. Prentice Hall Professional Technical Reference, 1997.
- [36] R. Middelkoop, C. Huizing, R. Kuiper, and E. J. Luit. Invariants for non-hierarchical object structures. *Electronic Notes in Theoretical Computer Science*, 195:211–229, 2008.
- [37] P. Müller. *Modular Specification and Verification of Object-Oriented Programs*, volume 2262 of *Lecture Notes in Computer Science*. Springer, 2002.
- [38] P. Müller, A. Poetzsch-Heffter, and G. T. Leavens. Modular invariants for layered object structures. *Science of Computer Programming*, 62(3):253–286, 2006.
- [39] S. Nelson, D. J. Pearce, and J. Noble. First class relationships for OO languages. In *6th International Workshop on Multiparadigm Programming with Object-Oriented Languages (MPOOL'08)*, 2008.
- [40] A. Poetzsch-Heffter. *Specification and Verification of Object-Oriented Programs*. Habilitation thesis, Technical University of Munich, January 1997.
- [41] A. Potanin, J. Noble, D. Clarke, and R. Biddle. Generic ownership for generic Java. In *21th ACM Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA'06)*, pages 311–324. ACM, 2006.
- [42] J. Rumbaugh. Relations as semantic constructs in an object-oriented language. In *2nd ACM Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '87)*, pages 466–481. ACM, 1987.
- [43] A. J. Summers and S. Drossopoulou. Considerate reasoning and the Composite design pattern. In *11th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI 2010)*, volume 5944 of *Lecture Notes in Computer Science*, pages 328–344. Springer, 2010.
- [44] A. Wren. *Relationships for Object-oriented Programming Languages*. PhD thesis, University of Cambridge, November 2007.

Sheep Cloning with Ownership Types

Paley Li
Victoria University of
Wellington
New Zealand
lipale@ecs.vuw.ac.nz

Nicholas Cameron
Mozilla Corporation
ncameron@mozilla.com

James Noble
Victoria University of
Wellington
New Zealand
kjax@ecs.vuw.ac.nz

ABSTRACT

Object-oriented programmers often need to clone objects. Mainstream languages, such as C# and Java, typically default to shallow cloning, which copies just one object and aliases references from that object. Other languages, such as Eiffel, provide deep cloning. A deep clone is a copy of the entire object graph reachable from the cloned object, which could include many unnecessary objects. Alternatively, programmers can implement their own object cloning functions, however, this is often difficult and error prone.

Sheep Cloning is an automated cloning technique which uses ownership information to provide the benefits of both shallow and deep cloning without the costs. We describe, formalise, and prove soundness of Sheep Cloning in a minimal object-oriented calculus with ownership types.

Categories and Subject Descriptors

D.3.3 [Software]: Programming Languages—*Language Constructs and Features*

General Terms

Languages

Keywords

Ownership types, object cloning, type system

1. INTRODUCTION

Traditional object cloning techniques produce clones using either shallow cloning or deep cloning. In Java, an object can be shallow cloned by calling its `clone()` method, provided its class implements the `Cloneable` interface. A similar approach is taken in C# where the object's class is required to implement the `ICloneable` interface. To create deep clones in Java, programmers would need to overwrite the object's `clone()` method with an implementation of deep cloning themselves. This task is often daunting and challenging.

There are cases when it is not obvious which cloning technique would produce the more suitable clone, and there are even cases when neither technique is suitable. In all of these cases, languages tend to offer little support, forcing programmers to design and implement a custom cloning implementation themselves.

Ownership types enforce the heap into a hierarchically structured, by introducing an owner object for every object [10]. The term context is used to mean the formal set of objects owned by an object, and the term representation means the set of objects which are conceptually part of an object, therefore ownership types help describe an object's representation. Prescriptive ownership policies, like owners-as-dominators, can be incorporated on top of descriptive ownership types. The owners-as-dominators policy [7] ensures an object's representation can never be exposed outside of its enclosing owner, by forcing all reference paths to an object to pass through that object's owner.

Sheep Cloning [21] is an intuitive and automated ownership-based cloning technique. Sheep cloning clones the object's representation by copying an object's context and aliases the reachable objects not owned by the object. The owners-as-dominators policy and the hierarchical structure of the heap are key in constructing Sheep clones. This is because the decision to copy or alias an object is determined by the object's owner. A Sheep clone preserves owners-as-dominators and is structurally equivalent to the object it is a clone of. In this paper, we describe and formalise Sheep Cloning, prove our formalism is sound, and present its correctness properties.

The rest of this paper is organized as follows: in Sect. 2 we introduce object cloning and ownership types; in Sect. 3 we introduce Sheep Cloning; in Sect. 4 we describe Sheep Cloning formally, and show type soundness; in Sect. 5 we discuss possible future work; in Sect. 6 we discuss related work; and, in Sect. 7 we conclude.

2. BACKGROUND

Programs today are regularly required to be written defensively under the assumption they will interact with malicious code. Defensive copying and ownership types [10] are two mechanisms to reduce the possible harm caused by malicious code. Defensive copying is a programming discipline that aims to stop malicious code from unexpectedly retaining and mutating objects. This is achieved by requiring all method calls to pass in clones and for all method returns to

return clones [16]. Ownership types can statically restrict access to an object’s context, encapsulating the object’s behaviour and preventing unexpected mutation of the object.

2.1 Object Cloning

Object cloning is the process of copying objects [2, 22, 1]. Traditionally, there are two object cloning techniques. One is shallow cloning, which copies the single object to be cloned and aliases the fields of that object. The other is deep cloning, which copies the object to be cloned and every object reachable from it.

```
class Window {
  Document document;
  Database database;

  Window(Database database) {
    document = new Document(database);
    this.database = database;
  }
  ...
}
class Document {
  Database database;
  ...
}
```

Figure 1: Code of the display window.

In Fig. 1 and Fig. 2, we present an example of a display window, as code and a diagram respectively. The display window contains a document and a reference to a database. The window simply displays the items it retrieves from the database. The document has a reference to the same database as the window. This allows the document to reference items from the database, independent of window.

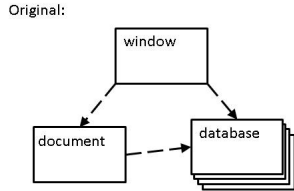


Figure 2: Diagram of the display window.

In Fig. 3 and Fig. 4, we present window’s shallow and deep clone respectively. We intend the clones (`windows` and `windowd`) to have the same structure and behaviour as the original object (`window`). Which means the clones should reference window’s database and they each should have their own document which also reference that database.

Shallow cloning window creates a new window (`windows`), which aliases the document and database of window. `windowd` is produced by deep cloning window, creating an entirely new document (`documentd`) and database (`databased`). A reference from `documentd` to `databased` is also created.

The shallow clone, `windows`, does not have the structure of window. As `windows` references the same document as window, any changes to the document would affect both win-

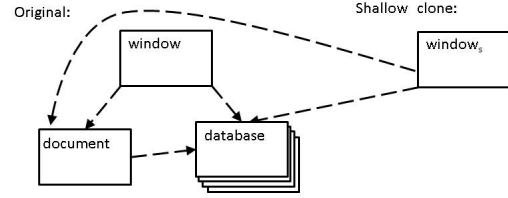


Figure 3: Shallow clone of the display window.

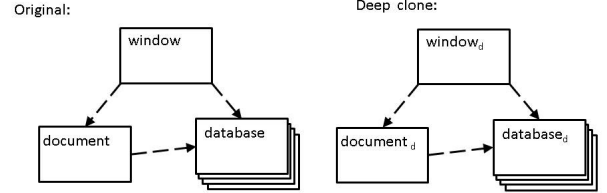


Figure 4: Deep clone of the display window.

dow and `windows`. Meanwhile, deep cloning copies database, which can be costly. The deep clone, `windowd`, also presents the problem that any changes to database will not be reflected in `databased`, and therefore can not be displayed in `windowd`.

2.2 Ownership Types

Ownership types were introduced in 1998 by Clarke et al [10]. This was followed by a variety of ownership systems, such as Ownership Domains [3], Universe Types [17, 18], Ownership with Disjointness of Types and Effects [9], External Uniqueness [8], and Ownership Generic Java [23]. The descriptive and/or prescriptive properties of these systems may differ, but the heap of all of these systems is structured hierarchically.

An ownership type is constructed from a class name and a list of owner parameters. In Fig. 5, we present the display window with ownership types. The ownership type for a document is `this:Document<dbowner>`. Document is the class name of the type, while `this` and `dbowner` are the owner parameters. `this` being the owner of the document. The owner `this` denotes a special owner, the current “this” object. The window object owns the document object, hence the owner of the document is this instance of the Window class. Declaring `dbowner` as the owner of the database permits the window to refer to the database. `dbowner` is also passed to the document, allowing the document to reference the same database as window. The workings of the owner parameters will be explained in greater detail in our formalism.

In Fig. 6, we present a diagram of the ownership typed display window. In this diagram, the boxes denote objects, and since objects are owners, the boxes are also owners. Objects


```

class Window<dbowner> {
  this:Document<dbowner> document;
  dbowner:Database<> database;
  ...
}
class Document<dbowner> {
  dbowner:Database<> database;
  ...
}

```

Figure 5: Code of ownership typed display window.

inside a box make up the context of the object that box represents. The `document` is owned by `window`, and therefore is inside `window`, while `database` is not part of `window`'s representation, and therefore is outside `window`. The dotted black arrows represents valid references.

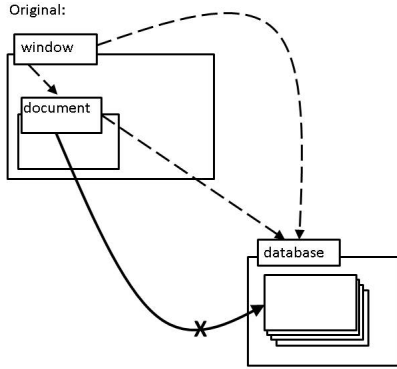


Figure 6: Diagram of ownership typed display window.

Owners-as-dominators, or deep ownership, ensures an object's representation can never be exposed outside its enclosing context [10, 7]. In practice this means all the references from a context must either go to its direct descendant, i.e., the objects it owns, its siblings, or up its ownership hierarchy. References are allowed up the ownership hierarchy because these references are pointing to representations that they are part of. In terms of the boxes, references can always go out of a box but never into a box. This is shown in Fig. 6. The `document` is permitted to reference the `database`. The `document`, however, is not permitted to reference the objects in the `database`'s context as shown by the cross on the solid black arrow. It is important to note that although ownership is transitive. An object can only reference the objects it owns directly, and never the objects that those objects own.

3. SHEEP CLONING

An ownership-based cloning operation was first proposed by Noble et al [21] who called this operation Sheep Cloning. Noble describes how Sheep Cloning clones an ownership-typed object by copying the objects it owns, while aliasing the references to external objects it doesn't own. They then discuss the need to maintain a map, to prevent objects from being copied more than once. Finally, they present an example where they Sheep clone a `Course`, which is represented by a linked list of `Students`. Sheep Cloning the linked list copies the nodes of the linked list, while aliasing the `Stu-`

`dents`. While Sheep Cloning the `Course` creates a replica of the entire linked list, with new copies of the `Students`.

Sheep Cloning incorporates aspects of both shallow and deep cloning. Like deep cloning, Sheep Cloning clones an object's representation by traversing references and copying every reachable object inside the context. Like shallow cloning, Sheep Cloning creates aliases after the essential object(s) is copied. Unlike deep cloning, however, Sheep Cloning uses the inside relation of ownership types to determine when it needs to stop copying, so no unnecessary objects are copied. Unlike shallow cloning, Sheep Cloning can recreate the entire representation of an object, instead of just copying a single object.

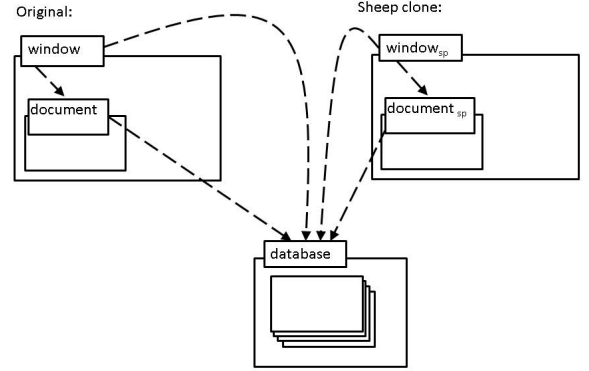


Figure 7: Sheep clone of the ownership typed display window.

In Fig. 7, we present the Sheep clone of the ownership-typed display `window`. Sheep Cloning `window` creates a new `window`, `windowsp`. In `windowsp`, a copy of `document` is created, `documentsp`, and `documentsp` contains a reference to the `database`. Finally, an alias of the reference to the `database` is created for `windowsp`.

The inside relation defines the ownership relation between two contexts. Sheep Cloning requires two variations of the same inside relation: a compile-time inside relation that ensures the owners-as-dominator property at compile-time, and a run-time inside relation that Sheep Cloning uses to determine whether to copy or alias an object. If an object has already been copied, then instead of copying this object twice, Sheep Cloning uses a map to refer to the copy that already exists.

Currently, Sheep Cloning requires owners-as-dominators. Systems with only descriptive ownership do not restrict access to an object's representation, which means there are objects that can not be reachable (directly or transitivity) by their owner. This is problematic for Sheep Cloning as it is expensive to locate every object in an object's representation, as a traversal over the entire heap is required. Owners-as-dominators guarantees every object inside an object's representation is reachable (directly or transitivity) from the owner.

4. FORMALISATION

In this section, we present our formalisation of Sheep Cloning, a calculus in the tradition of Featherweight Java [15]. The aim for our formalism is to present Sheep Cloning as a language feature that can be implemented in any ownership system that has owners-as-dominators. By formalising Sheep Cloning with the minimal amount of features it requires, we have lost some aspect of realism in our system, as our formalism is not Turing complete, since we do not support inheritance, and therefore cannot formalise conditionals. We have, however, maintained our aim, as most languages features required for Turing completeness are orthogonal to Sheep Cloning. Turing completeness is not required to show type safety, and we will present soundness of our formalism in a later section.

4.1 Static System

$Q ::= \text{class } C\langle o_l \preceq x \preceq o_u \rangle \{ \overline{N} f; \overline{M} \}$	<i>class declarations</i>
$M ::= N \mathbf{m}(N \mathbf{x}) \{ \text{return } e; \}$	<i>method declarations</i>
$T ::= N \mid \top$	<i>type</i>
$N ::= o : C\langle \overline{o} \rangle$	<i>class type</i>
$o ::= \gamma \mid \text{world} \mid \text{owner}$	<i>owners</i>
$e ::= \text{null} \mid \gamma \mid \gamma.f \mid \gamma.f = e \mid \gamma.m(e)$	<i>expressions</i>
$v ::= \text{null} \mid \iota$	<i>values</i>
$\gamma ::= x \mid \text{this} \mid \iota$	<i>expression variables and addresses</i>
$\Gamma ::= \gamma : T, \overline{o} : \top$	<i>variable environments</i>
$\mathcal{E} ::= \overline{o} \preceq \overline{o}$	<i>owners environments</i>
$\mathcal{H} ::= \iota \rightarrow \{ N, \overline{f} \rightarrow v \}$	<i>heaps</i>
$\text{map} ::= \{ \iota \rightarrow \iota \}$	<i>map</i>
$x \preceq o$	<i>owners relation</i>
x	<i>variables</i>
ι	<i>object address</i>
err	<i>errors</i>
null	<i>null expression</i>
f	<i>field names</i>
m	<i>method names</i>
C	<i>class names</i>

Figure 8: Syntax.

In Fig. 8, we present the syntax for our formalism. The syntax in grey is for our run-time model. Classes are parameterised with owner parameters. The formal owner parameters ($\overline{x}$) in the class declaration are bounded by a lower bound, o_l , and an upper bound, o_u . The valid owners of the system are: **world**, **owner**, **this**, and variables (x). **world** represents the top owner in the ownership hierarchy. Objects with **world** as their owner can be referenced from anywhere in the system. The **world** owner continues to exist at run-time. The owner parameter **owner** represents the owner of the current object, **this**. **owner** is only used statically, and at run-time it is substituted by the actual object it represents. The owner **this** represents the current object, **this**. At runtime, **this** is substituted by the instance of **this**. x is a variable representing a formal owner parameter within the class declaration.

Classes contain fields and methods. Fields are initialised to **null** when an object is created. Sub-classing is orthogonal to Sheep Cloning and is therefore omitted. Our method declaration is equivalent to those in Java. Method bodies consist

Well-formed owner: $\boxed{\mathcal{E}; \Gamma \vdash o \text{ OK}}$

$$\frac{\Gamma(\gamma) = \top}{\mathcal{E}; \Gamma \vdash \gamma \text{ OK}} \quad \text{(F-VAR)} \qquad \frac{}{\mathcal{E}; \Gamma \vdash \text{world} \text{ OK}} \quad \text{(F-WORLD)}$$

$$\frac{}{\mathcal{E}; \Gamma \vdash \text{owner} \text{ OK}} \quad \text{(F-OWNER)} \qquad \frac{}{\mathcal{E}; \Gamma \vdash \text{this} \text{ OK}} \quad \text{(F-THIS)}$$

Well-formed types: $\boxed{\mathcal{E}; \Gamma \vdash N \text{ OK}}$

$$\frac{\text{class } C\langle o_l \preceq x \preceq o_u \rangle \dots \quad \mathcal{E}; \Gamma \vdash o, \overline{o} \text{ OK} \quad \mathcal{E}; \Gamma \vdash [\overline{o}/\overline{x}] (\overline{o_l} \preceq \overline{o}) \quad \mathcal{E}; \Gamma \vdash [\overline{o}/\overline{x}] (\overline{o} \preceq \overline{o_u})}{\mathcal{E}; \Gamma \vdash o : C\langle \overline{o} \rangle \text{ OK}} \quad \text{(F-CLASS)}$$

Well-formed heap: $\boxed{\vdash \mathcal{H} \text{ OK}}$

$$\frac{\forall \iota \rightarrow \{ N; \overline{f} \rightarrow v \} \in \mathcal{H} : \quad \mathcal{H} \vdash N \text{ OK} \quad fType(\overline{f}, N) = N' \quad \mathcal{H} \vdash v : [\iota/\text{this}] N' \quad \forall v \in \overline{v} : v \neq \text{null} \Rightarrow v \in \text{dom}(\mathcal{H})}{\vdash \mathcal{H} \text{ OK}} \quad \text{(F-HEAP)}$$

Figure 9: Well-formed judgements.

$$\frac{\Gamma = \text{this} : \text{owner} : C\langle \overline{x} \rangle, \overline{x} : \top \quad \mathcal{E} = \overline{o_l} \preceq \overline{x}, \overline{x} \preceq \overline{o_u}, \text{owner} \preceq \text{world} \quad \mathcal{E}; \Gamma \vdash \text{owner} \preceq o_l \quad \mathcal{E}; \Gamma \vdash \overline{N}, \overline{M} \text{ OK}}{\vdash \text{class } C\langle o_l \preceq x \preceq o_u \rangle \{ \overline{N} f; \overline{M} \} \text{ OK}} \quad \text{(T-CLASS)}$$

$$\frac{\Gamma' = \Gamma, x : N' \quad \mathcal{E}; \Gamma \vdash N, N' \text{ OK} \quad \mathcal{E}; \Gamma' \vdash e : N}{\mathcal{E}; \Gamma \vdash N \mathbf{m}(N' \mathbf{x}) \{ \text{return } e; \} \text{ OK}} \quad \text{(T-METHOD)}$$

Figure 10: Classes and methods typing.

Static inside relation: $\boxed{\mathcal{E}; \Gamma \vdash o \preceq o'}$

$$\frac{o \preceq o' \in \mathcal{E}}{\mathcal{E}; \Gamma \vdash o \preceq o'} \quad \text{(IC-ENV)} \qquad \frac{\Gamma(\text{this}) = N}{\mathcal{E}; \Gamma \vdash \text{this} \preceq \text{owner}} \quad \text{(IC-THIS)}$$

$$\frac{\mathcal{E}; \Gamma \vdash o \text{ OK}}{\mathcal{E}; \Gamma \vdash o \preceq o} \quad \text{(IC-REFL)} \qquad \frac{\mathcal{E}; \Gamma \vdash o \preceq o'' \quad \mathcal{E}; \Gamma \vdash o'' \preceq o'}{\mathcal{E}; \Gamma \vdash o \preceq o'} \quad \text{(IC-TRANS)}$$

$$\frac{\mathcal{E}; \Gamma \vdash o \text{ OK}}{\mathcal{E}; \Gamma \vdash o \preceq \text{world}} \quad \text{(IC-WORLD)}$$

Figure 11: Inside relation.

of a return statement with an expression. Class types contain the class name, a single owner parameter (o), denoting

$$\begin{array}{c}
\frac{}{own_{\mathcal{H}}(\mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>) = \mathbf{o}} \\
\\
\frac{\mathcal{H}(\iota) = \{\iota' : \mathbf{C}<\overline{\mathbf{o}}>, \dots\}}{own_{\mathcal{H}}(\iota) = \iota'} \\
\\
\frac{\text{class } \mathbf{C}<\overline{\mathbf{o}}_l \preceq \mathbf{x} \preceq \overline{\mathbf{o}}_u> \{ \overline{N} \mathbf{f}; \overline{M} \}}{fields(\mathbf{C}) = \mathbf{f}} \\
\\
\frac{\text{class } \mathbf{C}<\overline{\mathbf{o}}_l \preceq \mathbf{x} \preceq \overline{\mathbf{o}}_u> \{ \overline{N} \mathbf{f}; \overline{M} \}}{fType(\mathbf{f}_i, \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>) = [\mathbf{o}/\mathbf{owner}, \mathbf{o}/\mathbf{x}] N_i} \\
\\
\frac{\text{class } \mathbf{C}<\overline{\mathbf{o}}_l \preceq \mathbf{x} \preceq \overline{\mathbf{o}}_u> \{ \overline{N} \mathbf{f}; \overline{M} \} \quad N \mathbf{m} (N' \mathbf{x}') \{ \mathbf{return} \ e; \} \in \overline{M}}{mBody(\mathbf{m}, \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>) = (x'; [\mathbf{o}/\mathbf{owner}, \mathbf{o}/\mathbf{x}] N')} \\
\\
\frac{\text{class } \mathbf{C}<\overline{\mathbf{o}}_l \preceq \mathbf{x} \preceq \overline{\mathbf{o}}_u> \{ \overline{N} \mathbf{f}; \overline{M} \} \quad N \mathbf{m} (N' \mathbf{x}') \{ \mathbf{return} \ e; \} \in \overline{M}}{mType(\mathbf{m}, \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>) = [\mathbf{o}/\mathbf{owner}, \mathbf{o}/\mathbf{x}] (N' \rightarrow N)}
\end{array}$$

Figure 12: Look up functions.

$$\begin{array}{c}
\text{Expression typing: } \boxed{\mathcal{E}; \Gamma \vdash e : N} \\
\\
\frac{\mathcal{E}; \Gamma \vdash e : N}{\mathcal{E}; \Gamma \vdash \text{sheep}(e) : N} \quad \text{(T-SHEEP)} \qquad \frac{\mathcal{E}; \Gamma \vdash e : N' \quad \mathcal{E}; \Gamma \vdash N' <: N}{\mathcal{E}; \Gamma \vdash e : N} \quad \text{(T-SUBS)} \\
\\
\frac{\mathcal{E}; \Gamma \vdash \gamma : \Gamma(\gamma)}{\mathcal{E}; \Gamma \vdash \gamma : \Gamma(\gamma)} \quad \text{(T-VAR)} \qquad \frac{\mathcal{E}; \Gamma \vdash \gamma : \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}> \quad fType(\mathbf{f}, \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>) = N' \quad \mathcal{E}; \Gamma \vdash e : N}{\mathcal{E}; \Gamma \vdash N <: [\gamma/\mathbf{this}] N'} \quad \text{(T-ASSIGN)} \\
\\
\frac{\mathcal{E}; \Gamma \vdash N \text{ OK}}{\mathcal{E}; \Gamma \vdash \text{null} : N} \quad \text{(T-NULL)} \qquad \frac{\mathcal{E}; \Gamma \vdash \gamma : \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}> \quad \mathcal{E}; \Gamma \vdash e : N'' \quad mType(\mathbf{m}, \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>) = N' \rightarrow N}{\mathcal{E}; \Gamma \vdash N'' <: [\gamma/\mathbf{this}] N'} \quad \text{(T-INVK)} \\
\\
\frac{\mathcal{E}; \Gamma \vdash \gamma : \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}> \quad fType(\mathbf{f}, \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>) = N}{\mathcal{E}; \Gamma \vdash \gamma.f : [\gamma/\mathbf{this}] N} \quad \text{(T-FIELD)} \qquad \frac{\mathcal{E}; \Gamma \vdash \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}> \text{ OK}}{\mathcal{E}; \Gamma \vdash \text{new } \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}> : \mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>} \quad \text{(T-NEW)}
\end{array}$$

Figure 13: Expression typing.

the owner of the type, and a set of owner parameters ($\overline{\mathbf{o}}$), denoting the actual owner parameters for the class declaration.

The owners-as-dominators policy is enforced by the bounds on the formal owner parameters ($\overline{\mathbf{x}}$) in the class declaration, and the premise in T-CLASS that states the lower bound ($\mathbf{o}_l$) is always outside **owner**. This ensures the owner parameters of a class must refer to classes that are outside the owner of this class.

Our judgments are decided under two environments: Γ and $\mathcal{E}$. Γ maps variables to their type, and $\mathcal{E}$ stores the static inside relations of the system. The variables in Γ are either expression variables or owner parameters. Owner parameters are distinguished by always having the top type ($\top$). At run-time, judgments are decided under the heap ($\mathcal{H}$). $\mathcal{H}$ is a set of mappings from address (ι) to object ($\{N; \mathbf{f} \rightarrow v\}$).

We elide presenting our sub-typing rules as they are trivially defined on reflexivity, transitivity, and top.

Well-formed owners, types, and heaps are defined in Fig. 9. An owner parameter is well-formed if it is in Γ . The owners **world**, **owner**, and **this** are variables and therefore are always well-formed. A class type is well-formed if a class declaration for that class exists, if its owner and actual owner parameters are well-formed, and if the upper and lower bounds are valid inside relations when the actual owners are substituted for the formal owner parameters. A heap is well-formed if every non-null object in the heap is well-formed.

In Fig. 10, we define class well-formedness (T-CLASS) and method well-formedness (T-METHOD). T-CLASS initialises Γ and $\mathcal{E}$, ensures the methods and types declared in the class are well-formed, and preserves the owner-as-dominator policy. T-METHOD ensures the method's return type and argument type are well-formed, and that the type of the expression in the method and the method's return type are the same.

In Fig. 11, we define the static inside relation. The inside relation defines the ordering of the owners, i.e., when a owner ($\mathbf{o}$) is inside another ($\mathbf{o}'$). Most valid inside relations are deduced from the relations in $\mathcal{E}$, this is reflected in IC-ENV, where a relation is valid if it is in $\mathcal{E}$. The owner **this** is only valid if it exists in Γ and is inside **owner**, as stated in IC-THIS. IC-REFL and IC-TRANS respectively define reflexivity and transitivity relations on owners. Finally, IC-WORLD denotes that all owners are inside **world**.

In Fig. 12, we present the look up functions in our formalism. The function $own_{\mathcal{H}}$ can either take a type or an address. If $own_{\mathcal{H}}$ is given a type, it returns the owner of that type. Otherwise if $own_{\mathcal{H}}$ is given an address (ι), it returns the owner of the object at ι in $\mathcal{H}$. The function $fields$ takes a class name and returns the names of the fields in that class. The function $fType$ takes a field ($\mathbf{f}_i$) and a type ($\mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>$) and returns the type of $\mathbf{f}_i$ in the class $\mathbf{C}$. The function $mBody$ takes a method ($\mathbf{m}$) and a type ($\mathbf{o}:\mathbf{C}<\overline{\mathbf{o}}>$) and returns the argument and expression of $\mathbf{m}$ in the class $\mathbf{C}$. Finally, the function $mType$ takes the same arguments as $mBody$ but it returns the type of the method, which is the argument type and return type of $\mathbf{m}$.

Finally in Fig. 13, we define expression typing. T-VAR ensures variables have the type as defined in Γ . T-NULL allows **null** to take any well-formed type. T-FIELD, T-ASSIGN, T-INVK, and T-NEW describe standard typing rules for field look up, field assignment, method invocation, and object creation. T-SUBS is our subsumption rule. The expression typing rule for Sheep Cloning (T-SHEEP) gives the new clone the same type as the expression being cloned.

4.2 Dynamic System

Our small step operational semantics for expressions are defined in Fig. 14. They are mostly standard: R-FIELD reduces a field look up expression to the value in that field. R-ASSIGN reduces a field assignment expression to the assigning value and updates the heap. R-NEW reduces an object creation expression to the address of the newly created object in the heap. R-INVK reduces a method invocation expression to the expression returned in the body of the method. Finally, R-SHEEP performs Sheep Cloning, which we describe in detail in the next section. We elide the congruence and error reduction rules.

Expression reduction:

$$\begin{array}{c}
\frac{\mathcal{H}(\iota) = \{N; \bar{f} \rightarrow v\}}{\iota.f_i; \mathcal{H} \rightsquigarrow v_i; \mathcal{H}} \\
\text{(R-FIELD)} \\
\\
\frac{\mathcal{H}(\iota) = \{N; \bar{f} \rightarrow v\} \quad \mathcal{H}' = \mathcal{H}[\iota \mapsto \{N; \bar{f} \rightarrow v[\bar{f}_i \mapsto v]\}]}{\iota.f_i = v; \mathcal{H} \rightsquigarrow v; \mathcal{H}'} \\
\text{(R-ASSIGN)} \\
\\
\frac{\mathcal{H}(\iota) \text{ undefined} \quad \text{fields}(C) = \bar{f} \quad \mathcal{H}' = \mathcal{H}, \iota \rightarrow \{o: C \langle \bar{o} \rangle; \bar{f} \rightarrow \text{null}\}}{\text{new } o: C \langle \bar{o} \rangle; \mathcal{H} \rightsquigarrow \iota; \mathcal{H}'} \\
\text{(R-NEW)} \\
\\
\frac{\mathcal{H}(\iota) = \{o: C \langle \bar{o} \rangle; \dots\} \quad \text{mBody}(m, o: C \langle \bar{o} \rangle) = (x; e)}{\iota.m(v); \mathcal{H} \rightsquigarrow [v/x, \iota/\text{this}, o/\text{owner}]; e; \mathcal{H}} \\
\text{(R-INVK)} \\
\\
\frac{\text{SheepAux}(v, v, \mathcal{H}, \emptyset) = v'; \mathcal{H}'; \{\iota \rightarrow \iota'\}}{\text{sheep}(v); \mathcal{H} \rightsquigarrow v'; \mathcal{H}'} \\
\text{(R-SHEEP)}
\end{array}$$

Figure 14: Expression reduction rules.

4.3 Sheep Cloning Semantics

The reduction for Sheep Cloning, R-SHEEP, reduces an expression passed to a Sheep clone by using the **SheepAux** function, given in Fig. 17. **SheepAux** clones a value by performing a graph traversal on the heap. The **SheepAux** function takes two values (v, v'), the heap ($\mathcal{H}$), and a **map** function. v is the object being cloned, and remains the same throughout the traversal. v' is the current object **SheepAux** has reached. The **map** is a mapping from objects to their clone, ($\iota \rightarrow \iota'$), and is used to ensure objects are copied at most once. The **SheepAux** function returns the Sheep clone of v' (v''), a heap ($\mathcal{H}'$), and a **map** function.

In Fig. 15, we present the dynamic variant of the inside relation. Both variants are reflexive, transitive, and contain a **world** case. The dynamic relation, however, uses the heap to derive the relation between an object and its owner (I-REC), while the static relation (IC-ENV) is deduced from the bounds in the class declaration and $\mathcal{E}$.

In Fig. 16, we present our well-formed map judgment and define **map**. **map** is a function that maps the object address (ι)

Dynamic inside relation:

$$\begin{array}{c}
\frac{}{\mathcal{H} \vdash \iota \preceq \iota} \\
\text{(I-REF)} \\
\\
\frac{\mathcal{H} \vdash \iota \preceq \iota'' \quad \mathcal{H} \vdash \iota'' \preceq \iota'}{\mathcal{H} \vdash \iota \preceq \iota'} \\
\text{(I-TRANS)} \\
\\
\frac{\mathcal{H}(\iota) = \{\iota': C \langle \bar{o} \rangle, \dots\}}{\mathcal{H} \vdash \iota \preceq \iota'} \\
\text{(I-REC)}
\end{array}$$

Figure 15: Dynamic inside relation.

Well-formed map and use of map:

$$\frac{\mathcal{H} \vdash \emptyset \text{ OK}}{\text{(F-EMPTYMAP)}} \quad \frac{\forall \iota: \iota \in \text{range}(\text{map}) \Rightarrow \iota \in \text{dom}(\mathcal{H})}{\mathcal{H} \vdash \text{map OK}} \\
\text{(F-MAP)}$$

Mapping of type:

$$\frac{\text{map} = \{\iota \mapsto \iota'\}}{\text{map}(N) = [\iota'/\iota]N} \\
\text{(M-TYPE)}$$

Figure 16: Map and mapping.

Auxiliary Sheep Clone Functions:

$$\begin{array}{c}
\mathcal{H}(\iota') = \{N; \bar{f} \rightarrow v\} \\
\mathcal{H} \vdash \iota' \preceq \iota \\
\text{map}(\iota') \text{ undefined} \\
\text{map}_1 = \text{map}, \iota' \mapsto \iota'' \\
\mathcal{H}(\iota'') \text{ undefined} \\
\mathcal{H}_1 = \mathcal{H}, \iota'' \mapsto \{\text{map}(N); \bar{f} \rightarrow \text{null}\} \\
n = |\{\bar{f} \rightarrow v\}| \\
\forall j: 1 \leq j \leq n: \{\text{SheepAux}(\iota, v_j, \mathcal{H}_j, \text{map}_j) = v'_j; \mathcal{H}_{j+1}; \text{map}_{j+1}\} \\
\mathcal{H}' = \mathcal{H}_{n+1}[\iota'' \mapsto \{\text{map}(N); \bar{f} \rightarrow v'\}] \\
\text{SheepAux}(\iota, \iota', \mathcal{H}, \text{map}) = \iota''; \mathcal{H}'; \text{map}_{n+1} \\
\text{(R-SHEEPINSIDE)} \\
\\
\mathcal{H} \not\vdash \iota' \preceq \iota \\
\text{map}(\iota') \text{ undefined} \\
\text{map}' = \text{map}, \iota' \mapsto \iota' \\
\text{SheepAux}(\iota, \iota', \mathcal{H}, \text{map}) = \iota'; \mathcal{H}; \text{map}' \\
\text{(R-SHEEPOUTSIDE)} \\
\\
\text{map}(\iota') = \iota'' \\
\text{SheepAux}(\iota, \iota', \mathcal{H}, \text{map}) = \iota''; \mathcal{H}; \text{map} \\
\text{(R-SHEEPREF)} \\
\\
\text{SheepAux}(v, \text{null}, \mathcal{H}, \text{map}) = \text{null}; \mathcal{H}; \text{map} \\
\text{(R-SHEEPNULL)}
\end{array}$$

Figure 17: Auxiliary sheep functions.

of an original object to the address of its clone (ι'). A **map** is well-formed when it is either empty (F-EMPTYMAP) or when every clone in the **map** is in the heap it is judged under (F-MAP). The mapping over types is crucial in defining the type of the Sheep clones. A mapping over a type (N) is when the owner parameters of N are applied with the mappings in **map** (M-TYPE). Applying an empty **map** over N will simply return N .

Next we discuss the cases of the **SheepAux** function in Fig. 17.

The inductive case, R-SHEEPINSIDE, constructs the Sheep clone (ι'') of the object in ι' , if ι' exists in the heap ($\mathcal{H}$) and if ι' is inside ι , as defined by the dynamic inside relation. The clone is created with a fresh address ι'' , where all its fields are initially set to **null**. A recursive call is made to **SheepAux** for each field in ι' . The returned values are assigned into the fields of ι'' once all the recursive calls have finished. A new heap ($\mathcal{H}'$) is constructed from the old heap ($\mathcal{H}$) with the addition of ι'' and any changes to the heap from the recursive calls on **SheepAux**. Similarly, the **map** is updated with the mapping from ι' to ι'' and any changes to the **map** from the recursive calls on **SheepAux**.

The case R-SHEEPOUTSIDE occurs when ι' is outside ι . In this case, **SheepAux** returns ι' as it would be aliased. The **map** is updated with a mapping from ι' to ι' , this shows that ι' is its own Sheep Clone. Owners-as-dominators ensures that ι' will not be encountered later in the context of an object that needs to be copied.

The case R-SHEEPPREF occurs when ι' already exists in the **map**. This indicates that ι' has already been cloned. **SheepAux** returns the Sheep clone (ι'') in the **map**, with no changes to the heap or the **map**.

Finally, the case R-SHEEPNULL occurs when **SheepAux** has to Sheep clone **null**. In this case **SheepAux** returns **null**, with no changes to the heap or the **map**.

4.4 Subject Reduction

In this subsection, we present subject reduction along with proofs for other properties of our formalism.

Theorem 1: Subject Reduction.

For all $\mathcal{H}, \mathcal{H}', e, v$, and N , if $\mathcal{H} \vdash e : N$ and $e; \mathcal{H} \rightsquigarrow v; \mathcal{H}'$ and $\vdash \mathcal{H} \text{ OK}$ then $\mathcal{H}' \vdash v : N$ and $\vdash \mathcal{H}' \text{ OK}$.

Subject reduction requires the system to show preservation of expression typing, heap well-formedness, and owners-as-dominators, for every expression reduction. We decided to state owners-as-dominators as a separate theorem away from subject reduction. The proof of subject reduction is by structural induction over the derivation of the expression reduction in Fig. 14. We have a large number of associated lemmas, mostly the standard weakening, inversion, well-formedness, and substitution lemmas. We state and prove some of the more interesting lemmas below.

The most interesting case of subject reduction is R-SHEEP, where $e = \text{Sheep}(v')$, for some v' . Intuitively, the proof for this case is to show that the Sheep clone has the same type as the cloned object. We use ‘cloned object’ to mean the object being Sheep cloned and ‘clone’ to refer to the newly created object which is a copy of the cloned object, that is ι and ι' respectively in the reduction $\text{sheep}(\iota); \mathcal{H} \rightsquigarrow \iota'; \mathcal{H}'$. We use the terms ‘inside’ and ‘outside’ to refer to our inside relation, which, defines the relation between two objects in the ownership hierarchy.

Subject reduction case: R-SHEEP.

For all $\mathcal{H}, \mathcal{H}', v, v'$, and N , if $\mathcal{H} \vdash \text{sheep}(v) : N$ and $\vdash \mathcal{H} \text{ OK}$ and $\text{sheep}(v); \mathcal{H} \rightsquigarrow v'; \mathcal{H}'$ then $\mathcal{H}' \vdash v' : N$ and $\vdash \mathcal{H}' \text{ OK}$.

Proof outline: The reduction of **sheep**(v) invokes the function **SheepAux**($v, v, \mathcal{H}, \emptyset$) by the premise of R-SHEEP. We then apply **Lemma 1** on this function, which returns $\vdash \mathcal{H}' \text{ OK}$ and $\mathcal{H}'(v') \downarrow_1 = \text{map}(\mathcal{H}(v) \downarrow_1)$. We can deduce that $\mathcal{H}'(v') \downarrow_1 = \text{map}(\mathcal{H}(v) \downarrow_1)$ as **map** is initially empty and that $v = v'$. Then by T-VAR, we get $\mathcal{H}' \vdash v' : N$.

The key to subject reduction is **Lemma 1**.

Lemma 1: Sheep Cloning preserves heap and map well-formedness, and type of the cloned object.

*For all $\mathcal{H}, \mathcal{H}', v, v', v'', \text{map}$, and map' , if $\vdash \mathcal{H} \text{ OK}$ and $\mathcal{H} \vdash \text{map} \text{ OK}$ and **SheepAux**($v, v', \mathcal{H}, \text{map}$) = $v''; \mathcal{H}'; \text{map}'$ then $\vdash \mathcal{H}' \text{ OK}$ and $\mathcal{H}' \vdash \text{map}' \text{ OK}$ and $\mathcal{H}'(v'') \downarrow_1 = \text{map}(\mathcal{H}(v') \downarrow_1)$.*

Proof outline: There are four cases to consider in the proof of **Lemma 1**, they are: R-SHEEPINSIDE; R-SHEEPOUTSIDE; R-SHEEPPREF; R-SHEEPNULL. The proof for R-SHEEPNULL is trivial as the heap and the map are unchanged, and by T-NUL the **null** expression can take any well-formed type.

For the case R-SHEEPOUTSIDE, we have $\mathcal{H} = \mathcal{H}'$ and $v' = v'' = \iota'$, while **map'** contains an identity mapping of ι in addition to the mappings in **map**. To show $\mathcal{H}' \vdash \text{map}' \text{ OK}$, we start by stating that $\iota' \in \mathcal{H}$, therefore $\iota' \in \mathcal{H}'$ as $\mathcal{H} = \mathcal{H}'$. Now with $\iota' \in \mathcal{H}'$, $\vdash \mathcal{H}' \text{ OK}$, the definition of **map**, and F-MAP we can state that $\mathcal{H}' \vdash \text{map}' \text{ OK}$. To show $\mathcal{H}'(\iota') \downarrow_1 = \text{map}(\mathcal{H}(\iota') \downarrow_1)$ we must show that the mappings in **map** does not apply through the type in $\mathcal{H}(\iota') \downarrow_1$. If we let $\mathcal{H}(\iota') \downarrow_1 = N'$, for some N' , then for all $\iota'' \in \text{dom}(\text{map})$ either $\mathcal{H} \not\vdash \iota' \preceq \iota''$ or $\iota'' \mapsto \iota'$. As either ι' is outside of ι'' , which means that ι'' is an object that has already been cloned, or that ι'' is an object that is also outside of ι . By **Theorem 2**, we can guarantee that N' does not have any owner parameters inside ι'' because the inside relation is transitive. This means that even if N' has owner parameters where ι'' is outside of ι , we still can ensure that N' remains unchanged as ι'' maps to itself in **map**. Therefore $\text{map}(\mathcal{H}(\iota') \downarrow_1) = N'$, and $\mathcal{H}'(\iota') \downarrow_1 = \text{map}(\mathcal{H}(\iota') \downarrow_1)$.

In the case R-SHEEPPREF, $\mathcal{H} = \mathcal{H}'$ and **map** = **map'**, which trivially proves $\vdash \mathcal{H}' \text{ OK}$ and $\mathcal{H}' \vdash \text{map}' \text{ OK}$ respectively. To show $\mathcal{H}(\iota'') \downarrow_1 = \text{map}(\mathcal{H}(\iota') \downarrow_1)$ we must consider how ι' was stored in **map**. If it is by R-SHEEPOUTSIDE then we can use the same reasoning as the case above, where $\iota' = \iota''$ and $\mathcal{H}(\iota') \downarrow_1 = \text{map}(\mathcal{H}(\iota') \downarrow_1)$. This is because either the mappings in **map** apply the owner parameters of $\mathcal{H}(v') \downarrow_1$ with an identity mapping or none at all. If ι' was added into **map** by R-SHEEPINSIDE then by definition ι'' is a Sheep clone. Therefore for some $\mathcal{H}_1$ and $\mathcal{H}_2$, and some **map''**, we have $\mathcal{H}_2(\iota'') \downarrow_1 = \text{map}''(\mathcal{H}_1(\iota') \downarrow_1)$. Then by the recursive nature of R-SHEEPINSIDE we know that $\mathcal{H}_1 \subseteq \mathcal{H}$ and $\mathcal{H}_2 \subseteq \mathcal{H}$. Therefore $\mathcal{H}(\iota'') \downarrow_1 = \text{map}''(\mathcal{H}(\iota') \downarrow_1)$, so now we must show $\text{map}''(\mathcal{H}(\iota') \downarrow_1) = \text{map}(\mathcal{H}(\iota') \downarrow_1)$. Again by the recursive nature of R-SHEEPINSIDE we know that $\text{map}'' \subseteq \text{map}$, hence **map** = **map''**, **map***, for some **map***. By **Theorem 2**, we can deduce that $\mathcal{H}(\iota') \downarrow_1$ has no formal owner parameters in $\text{dom}(\text{map}^*)$ because the formal owner parameters of ι' have

to be outside of ι' 's owner. Therefore $\text{map}''(\mathcal{H}(\iota') \downarrow_1) = \text{map}(\mathcal{H}(\iota') \downarrow_1)$, and finally, $\mathcal{H}(\iota'') \downarrow_1 = \text{map}(\mathcal{H}(\iota') \downarrow_1)$.

The proof for the case R-SHEEPINSIDE is far more complicated than the previous three cases. We will only outline the proof here. Please contact the authors for the full proof. The first step is to show $\vdash \mathcal{H}_1 \text{ OK}$, $\mathcal{H}_1 \vdash \text{map}_1 \text{ OK}$ and $\mathcal{H}_1(\iota'') \downarrow_1 = \text{map}(\mathcal{H}(\iota') \downarrow_1)$. This can be achieved by the premises of R-SHEEPINSIDE, along with **Lemma 2** and **Lemma 3**. Then we introduce a **sublemma** to show the recursive calls of **SheepAux** give $\vdash \mathcal{H}_j \text{ OK}$, $\mathcal{H}_j \vdash \text{map}_j \text{ OK}$ and $\mathcal{H}_j(v'_j) \downarrow_1 = \text{map}_{j-1}(\mathcal{H}(v_{j-1}) \downarrow_1)$. This **sublemma** is proved by numerical induction over j , where the base case is when $j = 1$. To prove the inductive case we invoke the induction hypothesis of the main lemma. Next we need to show $\vdash \mathcal{H}' \text{ OK}$ when $\mathcal{H}' = \mathcal{H}_{n+1}[\iota'' \mapsto \{\text{map}(N); \bar{f} \mapsto v'\}]$. For all the v' produced by the **sublemma** to be assigned into $\mathcal{H}_{n+1}$, we must first show that each v' has the same type as the **null** when the clone (ι'') was created. This is achieved by the correctness property that **SheepAux** does not change objects in $\mathcal{H}$, **Theorem 2**, and substitution principles. Then with the **Lemma: heap preserves expression typing on field assignment**, we show that the v 's have the correct types under $\mathcal{H}'$. This gives $\vdash \mathcal{H}' \text{ OK}$. Next is to show $\mathcal{H}' \vdash \text{map}_{n+1} \text{ OK}$. From the **sublemma** we have $\mathcal{H}_{n+1} \vdash \text{map}_{n+1} \text{ OK}$ and because **nulls** are not in $\text{dom}(\text{map}_{n+1})$ or $\text{range}(\text{map}_{n+1})$, by the definition of **map**, and F-MAP, and the definition of $\mathcal{H}'$, we can deduce that $\forall \iota : \iota \in \text{range}(\text{map}_{n+1}) \Rightarrow \iota \in \text{dom}(\mathcal{H}')$. Therefore $\mathcal{H}' \vdash \text{map}_{n+1} \text{ OK}$. Since we have already shown that $\mathcal{H}'(\iota'') \downarrow_1 = \text{map}(\mathcal{H}(\iota') \downarrow_1)$, we are done.

Lemma 2: Mapped types preserves well-formedness.

For all $\mathcal{H}$, **map**, and N , if $\vdash \mathcal{H} \text{ OK}$, $\mathcal{H} \vdash \text{map} \text{ OK}$, and $\mathcal{H} \vdash N \text{ OK}$ then $\mathcal{H} \vdash \text{map}(N) \text{ OK}$.

Proof outline: This lemma is proved by natural deduction on $\mathcal{H} \vdash N \text{ OK}$. Let $\text{map} = \{\iota \mapsto \iota'\}$, then by definition $\text{map}(N) = [\iota'/\iota]N$. $\mathcal{H} \vdash [\iota'/\iota]N \text{ OK}$ is proved by structural induction on the derivation of $\mathcal{H} \vdash N \text{ OK}$ when $N = \text{o} : \mathbb{C} < \bar{\text{o}} >$. By the premises of F-CLASS $\mathcal{H} \vdash \text{o}, \bar{\text{o}} \text{ OK}$, $\mathcal{H} \vdash [\bar{\text{o}}/\bar{x}](\bar{\text{o}}_l \preceq \bar{\text{o}}) \text{ OK}$, $\mathcal{H} \vdash [\bar{\text{o}}/\bar{x}](\bar{\text{o}} \preceq \bar{\text{o}}_u) \text{ OK}$, and there exists a class **class** $\mathbb{C} < \bar{\text{o}}_l \preceq \bar{x} \preceq \bar{\text{o}}_u > \dots$. Then we invoke the **Lemma: owner variable substitution preserves owner well-formedness** on $\mathcal{H} \vdash \text{o}, \bar{\text{o}} \text{ OK}$ with $\mathcal{H} \vdash \text{map} \text{ OK}$ and $\vdash \mathcal{H} \text{ OK}$ to get $\mathcal{H} \vdash [\iota'/\iota]\text{o}, [\iota'/\iota]\bar{\text{o}} \text{ OK}$. Similarly, we invoke the **Lemma: owner variable substitution preserves inside relation** on $\mathcal{H} \vdash [\bar{\text{o}}/\bar{x}](\bar{\text{o}}_l \preceq \bar{\text{o}}) \text{ OK}$ and $\mathcal{H} \vdash [\bar{\text{o}}/\bar{x}](\bar{\text{o}} \preceq \bar{\text{o}}_u) \text{ OK}$ with $\mathcal{H} \vdash \text{map} \text{ OK}$, and $\vdash \mathcal{H} \text{ OK}$. This gives $\mathcal{H} \vdash [\iota'/\iota](\bar{\text{o}}/\bar{x})(\bar{\text{o}}_l \preceq \bar{\text{o}}) \text{ OK}$ and $\mathcal{H} \vdash [\iota'/\iota](\bar{\text{o}}/\bar{x})(\bar{\text{o}} \preceq \bar{\text{o}}_u) \text{ OK}$. Finally we apply F-CLASS on $\mathcal{H} \vdash [\iota'/\iota]\text{o}, [\iota'/\iota]\bar{\text{o}} \text{ OK}$, $\mathcal{H} \vdash [\iota'/\iota](\bar{\text{o}}/\bar{x})(\bar{\text{o}}_l \preceq \bar{\text{o}}) \text{ OK}$, and $\mathcal{H} \vdash [\iota'/\iota](\bar{\text{o}}/\bar{x})(\bar{\text{o}} \preceq \bar{\text{o}}_u) \text{ OK}$ to get $\mathcal{H} \vdash [\iota'/\iota]N \text{ OK}$.

Lemma 3: Map preserves field type.

For all $\mathcal{H}$, **map**, $\mathbf{f}_i$, and N , if $\vdash \mathcal{H} \text{ OK}$, $\mathcal{H} \vdash \text{map} \text{ OK}$, $\mathcal{H} \vdash \text{map}(N) \text{ OK}$, and $\mathbf{fType}(\mathbf{f}_i, N) = N'$ then $\mathbf{fType}(\mathbf{f}_i, \text{map}(N)) = \text{map}(N')$.

Proof outline: This lemma is proved by natural deduction on $\mathbf{fType}(\mathbf{f}_i, N) = N'$. Let $\text{map} = \{\iota \mapsto \iota'\}$, then $\text{map}(N)$

$= [\iota'/\iota]N$, $\text{map}(N') = [\iota'/\iota]N'$, and $\mathbf{fType}(\mathbf{f}_i, [\iota'/\iota]N) = [\iota'/\iota]N'$. Let $N = \text{o} : \mathbb{C} < \bar{\text{o}} >$, then with $\mathcal{H} \vdash [\iota'/\iota](\text{o} : \mathbb{C} < \bar{\text{o}} >) \text{ OK}$ and **Lemma 2**, we can state that $\mathcal{H} \vdash [\iota'/\iota]\text{o}, [\iota'/\iota]\bar{\text{o}} \text{ OK}$, therefore $\mathcal{H} \vdash ([\iota'/\iota]\text{o}) : \mathbb{C} < [\iota'/\iota]\bar{\text{o}} > \text{ OK}$. Next, by applying the definition of **fType** on $\mathbf{fType}(\mathbf{f}_i, ([\iota'/\iota]\text{o}) : \mathbb{C} < [\iota'/\iota]\bar{\text{o}} >)$ and $\mathbf{fType}(\mathbf{f}_i, N) = N'$, along with substitution principles, we get $\mathbf{fType}(\mathbf{f}_i, ([\iota'/\iota]\text{o}) : \mathbb{C} < [\iota'/\iota]\bar{\text{o}} >) = [\iota'/\iota]N'$. This proof follows from the proof for the **Lemma: owner variable substitution preserves type well-formedness**.

Below we present the owners-as-dominators theorem. We are required to prove this theorem as part of subject reduction. This theorem states that for all well-formed heaps, all references to an object come from inside the owner (as defined by $\text{own}_{\mathcal{H}}$) of that object [4]. Intuitively this means all references to an object can only come from the object's owner, siblings of the object, or from inside the object's context.

Theorem 2: Owners-as-dominators.

For any $\mathcal{H}$, if $\vdash \mathcal{H} \text{ OK}$ then $\forall \iota \mapsto \{N; \{\bar{f} \mapsto v\}\} \in \mathcal{H}$ where $\forall \iota' \in \bar{v} : \mathcal{H} \vdash \iota \preceq \text{own}_{\mathcal{H}}(\iota')$.

This theorem is proved by showing every expression reduction preserves this property on the heap it produces. For the reduction R-SHEEP, the only interesting case is R-SHEEPINSIDE. The proof for owners-as-dominators on the heap ($\mathcal{H}'$) produced by R-SHEEPINSIDE is achieved in two parts. The first part shows the heap with the newly created clone (ι') preserves owners-as-dominators. This holds by the fact that the owner of the clone is inside the owner of the original cloned object, i.e., the owner of the object that initiated the Sheep Clone. The second part is to show all the values of ι' satisfy the owners-as-dominators property. This is achieved by the transitivity of two inside relations. The first inside relation is that the owner of the values is outside the owner of the fields they are assigned to. The second relation is that the owner of the field is outside ι' , the object which the fields belong to. The transitivity of these two inside relations gives owners-as-dominators for $\mathcal{H}'$.

Finally, we present our progress theorem. The proof for our progress theorem is standard and has been omitted.

Theorem 3: Progress.

For all $\mathcal{H}$, $\mathcal{H}'$, e , e' , and N , if $\mathcal{H} \vdash e : N$ and $\vdash \mathcal{H} \text{ OK}$ then $e; \mathcal{H} \rightsquigarrow e'; \mathcal{H}'$ or $\exists v : e = v$.

Progress is proved by structural induction on the derivation of expression typing. A case analysis is required for T-ASSIGN, T-INVK, and T-SHEEP, as the reduction for these expressions does not always reduce down to a value in a single step.

4.5 Correctness of Sheep Cloning

Below, we present seven correctness properties and their proofs to show correctness for Sheep Cloning.

The first property states that a new object must be created when Sheep Cloning an object and the newly created object must not be the same as the cloned object:

Correctness property 1: Sheep Cloning creates a new object.

For all $\mathcal{H}, \mathcal{H}', \iota$, and ι' , if $\vdash \mathcal{H}$ OK and $\text{sheep}(\iota)$; $\mathcal{H} \rightsquigarrow \iota'$; $\mathcal{H}'$ then $\iota' \notin \text{dom}(\mathcal{H})$ and $\iota \neq \iota'$.

Proof outline: This property is proved by case analysis on the premise of $\text{sheep}(\iota)$, which is the function $\text{SheepAux}(\iota, \iota, \mathcal{H}, \emptyset)$. The cases R-SHEEPNULL, R-SHEEPPREF, and R-SHEEPOUTSIDE are all not applicable for this particular SheepAux function. For the case R-SHEEPINSIDE, the premise states that $\mathcal{H}(\iota')$ undefined, hence $\iota' \notin \text{dom}(\mathcal{H})$. We can then deduce $\iota \neq \iota'$ by $\iota' \notin \text{dom}(\mathcal{H})$ and the semantics of SheepAux , where it states that $\iota \in \text{dom}(\mathcal{H})$.

The second property states the clones must preserve owners-as-dominators, as stated in **Theorem 2**:

Correctness property 2: Sheep Cloning preserves owners-as-dominators.

For all $\mathcal{H}, \mathcal{H}', \iota$, and ι' , if $\vdash \mathcal{H}$ OK and $\text{sheep}(\iota)$; $\mathcal{H} \rightsquigarrow \iota'$; $\mathcal{H}'$ and $\mathcal{H}$ preserves owners-as-dominators then $\mathcal{H}'$ preserves owners-as-dominators.

Proof outline: This property is proved by the proof of **Theorem 2**: *expression reduction preserves owners-as-dominators on heap*. The outline of the proof for **Theorem 2** is to show the heap preserves owners-as-dominators in each cases of the SheepAux function. This is trivial for R-SHEEPNULL, R-SHEEPPREF, and R-SHEEPOUTSIDE. For R-SHEEPINSIDE, we have already discussed its proof outline.

The third property states that Sheep Cloning creates a sub-heap that contains the new object:

Correctness property 3: Sheep Cloning creates a sub-heap that contains the new object.

For all $\mathcal{H}, \mathcal{H}', \mathcal{H}'', \iota$, and ι' , if $\vdash \mathcal{H}$ OK and $\text{sheep}(\iota)$; $\mathcal{H} \rightsquigarrow \iota'$; $\mathcal{H}'$ and $\iota' \neq \iota$ then $\exists \mathcal{H}''$ where $\mathcal{H}' = \mathcal{H}, \mathcal{H}''$ and $\iota' \in \text{dom}(\mathcal{H}'')$ and $\iota \in \text{dom}(\mathcal{H})$.

Proof outline: This property is proved by the same reasoning as the proof for **Lemma 1**. Once again R-SHEEPNULL, R-SHEEPPREF, and R-SHEEPOUTSIDE are not applicable. Making the only interesting case R-SHEEPINSIDE, by the premise of the case and **Lemma 1**, we know that if $\text{SheepAux}(\iota, \iota', \mathcal{H}, \text{map}) = \iota'', \mathcal{H}', \text{map}'$ then $\mathcal{H} \subseteq \mathcal{H}'$. This gives $\mathcal{H}_1 = \mathcal{H}, \iota' \mapsto \{\dots\}$, which implies $\mathcal{H} \subseteq \mathcal{H}_1$ and $\iota' \in \text{dom}(\mathcal{H}_1 \setminus \mathcal{H})$. Again by the premise of this case, we get $\mathcal{H}_1 \subseteq \mathcal{H}'$ and $\mathcal{H}_1 \setminus \mathcal{H} \subseteq \mathcal{H}''$. Which let us conclude that $\iota' \in \text{dom}(\mathcal{H}'')$.

The fourth property states that if a reference in a clone is pointing to an object (ι) in the original heap ($\mathcal{H}$), that does not contain the clone, then ι must be outside that clone.

Correctness property 4: Sheep Cloning does not introduce references to the cloned object's representation.

For all $\mathcal{H}, \mathcal{H}', \iota$, and ι' , if $\vdash \mathcal{H}$ OK and $\text{sheep}(\iota)$; $\mathcal{H} \rightsquigarrow \iota'$; $\mathcal{H}'$ where $\mathcal{H}' = \mathcal{H}, \mathcal{H}''$ and $\iota' \neq \iota$ and $\forall f \mapsto \iota'' \in \text{range}_{\downarrow 2}(\mathcal{H}'')$ where $\iota'' \in \text{dom}(\mathcal{H})$ then $\mathcal{H}' \vdash \iota \preceq \iota''$.

Proof outline: This property is proved by natural deduction on ways that ι'' can be added into the range of $\mathcal{H}'$. This is achieved by case analysis on the construction of the Sheep clone by the SheepAux function. The cases R-SHEEPNULL and R-SHEEPINSIDE are not applicable, as $\text{null} \notin \text{dom}(\mathcal{H})$ and $\iota'' \in \text{dom}(\mathcal{H})$ respectively. The case R-SHEEPPREF does not offer any insight into the relation between ι'' and ι . We must determine how ι'' was added into the map . For the case R-SHEEPOUTSIDE we have $\iota'' \in \text{dom}(\mathcal{H})$ if $f \mapsto \iota'' \in \text{range}_{\downarrow 2}(\mathcal{H}'')$ by the definition of R-SHEEPOUTSIDE. Then by the premise of R-SHEEPOUTSIDE, $\mathcal{H} \vdash \iota \preceq \iota''$, which then gives $\mathcal{H}' \vdash \iota \preceq \iota''$, since $\mathcal{H} \subseteq \mathcal{H}'$.

The fifth property states that a reference can point to objects in the cloned heap ($\mathcal{H}''$), if and only if those objects are inside the representation of a clone.

Correctness property 5: All new objects are in the representation of the clone, and all objects in that representation are new.

For all $\mathcal{H}, \mathcal{H}', \iota$, and ι' , if $\vdash \mathcal{H}$ OK and $\text{sheep}(\iota)$; $\mathcal{H} \rightsquigarrow \iota'$; $\mathcal{H}'$ where $\mathcal{H}' = \mathcal{H}, \mathcal{H}''$ and $\iota' \neq \iota$ then $\iota'' \in \text{dom}(\mathcal{H}'')$ if and only if $\mathcal{H}' \vdash \iota'' \preceq \iota'$.

Proof outline: This property is proved in two parts by case analysis on the SheepAux function. The first part is to show $\mathcal{H}' \vdash \iota'' \preceq \iota'$ when $\iota'' \in \text{dom}(\mathcal{H}'')$. This is only possible by R-SHEEPINSIDE. By the premises of R-SHEEPINSIDE, $\mathcal{H} \vdash \iota^* \preceq \iota$, where $\text{map}(\iota^*) = \iota''$. By **lemma: address mapping preserves inside relation**, we have $\mathcal{H} \vdash \text{map}(\iota^*) \preceq \text{map}(\iota)$, which gives $\mathcal{H} \vdash \iota'' \preceq \iota'$, and finally gives $\mathcal{H}' \vdash \iota'' \preceq \iota'$, as $\mathcal{H} \subseteq \mathcal{H}'$. The second part is to show $\iota'' \in \text{dom}(\mathcal{H}'')$ when $\mathcal{H}' \vdash \iota'' \preceq \iota'$. This part is also only possible in R-SHEEPINSIDE. By the same argument as the proof outlined for the fourth correctness property, we have $\mathcal{H} = \mathcal{H}, \iota' \mapsto \{N, \bar{f} \mapsto \bar{v}\}$ and $\mathcal{H}_1 \subseteq \mathcal{H}'$. Hence $\iota' \mapsto \{N, \bar{f} \mapsto \bar{v}\} \in \text{dom}(\mathcal{H}' \setminus \mathcal{H})$, which means $\iota' \mapsto \{N, \bar{f} \mapsto \bar{v}\} \in \text{dom}(\mathcal{H}'')$.

The sixth property states that all objects outside the cloned object are also outside the clone.

Correctness property 6: All objects outside the cloned object are outside the clone.

For all $\mathcal{H}, \mathcal{H}', \iota$, and ι' , if $\vdash \mathcal{H}$ OK and $\text{sheep}(\iota)$; $\mathcal{H} \rightsquigarrow \iota'$; $\mathcal{H}'$ where $\iota' \neq \iota$ and $\forall \iota'' \in \text{dom}(\mathcal{H})$ and $\mathcal{H}' \vdash \iota \preceq \iota''$ then $\mathcal{H}' \vdash \iota' \preceq \iota''$.

Proof outline: This property is proved by contradiction on the construction of the Sheep clone. Lets assume for some ι^* , where $\iota^* \in \text{dom}(\mathcal{H})$, $\iota^* \neq \iota$, and $\mathcal{H}' \vdash \iota \preceq \iota^*$, that $\mathcal{H}' \not\vdash \iota' \preceq \iota^*$. If $\iota^* = \iota$, then this property trivially holds. $\mathcal{H}' \not\vdash \iota' \preceq \iota^*$ could either mean $\mathcal{H}' \vdash \iota^* \preceq \iota'$ or there are no ownership relation between ι' and ι^* . The latter is not possible, because there must be an ownership relation between ι' and ι^* , as ι' and ι have the same owner and we know that ι is inside ι^* . By the definition of SheepAux , ι^* would have been applied by R-SHEEPINSIDE as $\mathcal{H}' \vdash \iota^* \preceq \iota'$. By the premise of R-SHEEPINSIDE we know that $\mathcal{H}' \vdash \iota^* \preceq \iota$, which contradicts $\mathcal{H}' \vdash \iota \preceq \iota^*$. Therefore $\mathcal{H}' \not\vdash \iota' \preceq \iota^*$ is not possible for some ι^* , where $\iota^* \in \text{dom}(\mathcal{H})$ and $\mathcal{H}' \vdash \iota \preceq \iota^*$.

The seventh property states that for each reference inside a clone pointing to objects that are outside the cloned object, there exists a corresponding reference from the cloned object pointing to those objects.

Correctness property 7: For all references from an object inside the clone to an object outside the clone, there is a reference to the same object from inside the cloned object.

For all $\mathcal{H}, \mathcal{H}', \iota$, and ι' , if $\vdash \mathcal{H}$ OK and $\text{sheep}(\iota)$; $\mathcal{H} \rightsquigarrow \iota'$; $\mathcal{H}'$ where $\mathcal{H}' = \mathcal{H}, \mathcal{H}''$ and $\iota' \neq \iota$ and $\forall \mathbf{f} \mapsto \iota'' \in \text{range}_{\downarrow 2}(\mathcal{H}'')$ and $\mathcal{H}' \vdash \iota' \preceq \iota''$ then $\exists \mathbf{f}' \mapsto \iota'' \in \text{range}_{\downarrow 2}(\mathcal{H})$.

Proof outline: This property is proved by natural deduction on the construction of Sheep clones by the **SheepAux** function. By $\mathcal{H}' \vdash \iota' \preceq \iota''$ we can deduce that $\iota'' \mapsto \iota'' \in \text{map}$ and $\mathcal{H}' \vdash \text{map}(\iota) \preceq \text{map}(\iota'')$, as $\text{map}(\iota') = \iota$. Hence by **lemma:** *address mapping preserves inside relation*, we have $\mathcal{H}' \vdash \iota \preceq \iota''$. Now we must consider how the field ($\mathbf{f}$) was constructed. The only possible way is when R-SHEEPINSIDE recursively traverse through the values of the object it is cloning. Hence, ι'' must be a value of a field of an object in the original heap.

5. FUTURE WORK

In this section, we explore some of our ideas for future work regarding Sheep Cloning.

5.1 Ownership Transfer with Sheep Cloning

Ownership transfer was first presented by Clarke and Wrigstad [8] using external uniqueness. External uniqueness ensures objects are only accessible via a single externally unique reference from outside the object's representation. No restrictions are placed on internal referencing from inside the object's representation. Ownership transfer is achieved by transferring the externally unique reference. Clarke and Wrigstad use movement bounds to designate the scope that each externally unique reference can be moved. A unique reference cannot be moved outside its movement bounds.

Müller et al. describe another implementation of ownership transfer with Universe Types [19]. Universe Types [18] enforce the owners-as-modifiers policy, where objects are freely aliased but only their owner is allowed to modify them. Ownership transfer in Universe Types is achieved by creating clusters of objects and moving a (externally) unique pointer between clusters. The external uniqueness property is achieved by allowing only one unique reference into each cluster, and similarly, internal referencing within a cluster is unrestricted.

Ownership transfer, with external uniqueness and Universe Types, illustrated the need for the owner of the object that would be transferred to enforce the external uniqueness property. By the semantics of Sheep Cloning in R-SHEEP and Fig. 17, we can deduce that all Sheep Clones inherit the external uniqueness property. This is because Sheep Cloning guarantees that the only reference into a newly created Sheep clone is from the owner of the Sheep clone. This means the Sheep clones possess the externally uniqueness property as the reference from its owner behaves as an external unique reference. By transferring this reference to

another owner, we can simulate the first part of ownership transfer. The second part is to disown and dereference the original object that has been cloned. Combining these two parts, we can simulate the behaviour of ownership transfer as shown in external uniqueness.

We believe there are advantages in supporting ownership transfer via Sheep Cloning. Any constraints that were on the object to allow its ownership to be transfer, would now be on its Sheep clone instead. For example, the movement bounds described by Clarke and Wrigstad [8] demand a trade-off between what an object can access and where it can be moved. An object with tight movement bounds has more restrictions on its movement but fewer restrictions on what it can access; whereas an object with loose movement bounds has fewer restrictions its movement but what it can access is severely limited. In a Sheep Cloning based ownership transfer system the movement bounds would only constrain the Sheep clones, and not the original object. The trade-off for movement bounds still exists, however, they would be generated when constructing the Sheep clones. We aim to formalise this style of ownership transfer in a formalism similar to the one presented in this paper.

5.2 Sheep Cloning without Owners as Dominators

Cheng and Drossopoulou present ideas to perform object cloning in an ownership system without owners-as-dominators [5]. Their system is build on top of the system developed by Drossopoulou and Noble [12]. Cheng and Drossopoulou identify a set of problematic cases. For example, when a reference path re-enters the representation of an object from outside the object's representation. It is also possible that this reference path is the only way to reach that particular part of the object's representation. Cheng and Drossopoulou offer two alternative solutions, either enforce owners-as-dominators and all possible problematic cases would cease to exist, or statically prevent cloning on these problematic cases.

It is possible for their system to determine these properties statically, however, Sheep Cloning operates at runtime. For a system without owners-as-dominators, Sheep Cloning must traverse the entire heap to determine which objects are in an object's representation. A simple solution is for Sheep Cloning to ignore all objects in the cloned object's representation that are not reachable from its owner without going out of its representation. However, this would mean Sheep Cloning would no longer clone every object in an object's representation.

Another issue is when a two-way reference exists between an object (A) inside a context and an object (B) outside that context, as shown in Fig. 18. Sheep Cloning the object (A) inside the context would create an object (A') that also has a reference to B, however, B would not know the existence of A'. Consider if the system has an invariant property where the purpose of A is to pass messages to B, and that B has to reply to any message passed by A. Then a clone of A would expect this bidirectional relationship with B. However, if A' passes a message to B, B would respond to A instead of A', as B does not know the existence of A', breaking the invariant of the system.

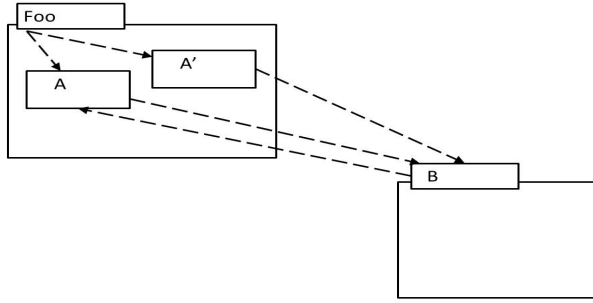


Figure 18: Sheep cloning with descriptive ownership types.

One of the benefits of Sheep Cloning is how it utilizes ownership types and the structures provided by owners-as-dominators. Which means we may need to consider a different set of semantics for Sheep Cloning in systems without owners-as-dominators.

6. RELATED WORK

Drossopoulou and Noble [12] propose a static object cloning implementation, inspired by ownership types. Every object has a cloning domain and objects are cloned by cloning their domain. Just as ownership types enforce a topological structure upon the heap, the cloning domain provides a hierarchical structure for the objects in the program. This is achieved by placing cloning annotations on every field of every class. Using these annotations the cloning paths for each field of a class are created. Objects can have paths to other objects that are not in their cloning domain. The decision to clone an object is determined by the cloning domain of the initial cloning object or the *originator*. Each `clone()` method explicitly states, through `Boolean` parameters which fields are in its cloning domain. The `clone()` method then recursively calls the `clone()` method of each field, passing in `Boolean` arguments set by the originator.

In Drossopoulou and Noble’s system, a parametric clone method is of the form `clone(Boolean s1, ..., Boolean sn, Map m)`. The variables `s1, ..., sn` in the arguments of a class’s `clone()` method are associated with the fields of that class. An object is cloned when that object’s `clone()` method is called, and fields are cloned only if `true` is passed into the cloning parameter (`si`). In contrast, the expression for Sheep Cloning is `sheep(ι)`, where *ι* is the object to be cloned.

Sheep Cloning is context free, which means the semantics of Sheep Cloning remain the same regardless of the class using it or the object (*ι*) passed in. Sheep Cloning uses a mapping between the original object and its clone, as does Drossopoulou and Nobles’ `map` in their `clone()` method. The mapping in Sheep Cloning, however, operates on the heap at run-time, hiding the implementation of Sheep Cloning from its users.

There are also several papers that discuss the need for ownership-based cloning. In “Exceptions in ownership types systems” [11], Dietl and Müller outline several possible solutions to exception handling for Universe Types. One solution is to

clone the exception object when it appears, then propagate the clone through the stack to the exception handler. They then explain that supporting exceptions by cloning requires no changes to their ownership system. In the end, they did not choose to handle exceptions with cloning. The reasons they cited were the need for every object in the system to be `cloneable` and the overhead cost of object cloning, especially if an exception is propagated multiple times before it is caught.

In “Minimal Ownership for Active Objects” [6], Clarke et al. develop active ownership, an ownership-based active object model for concurrency. An active object is an object that interacts with asynchronous methods while being controlled by a single thread. To guarantee safety and provide freedom from data races for the interaction between active objects, Clarke propose using unique references and immutable objects, and cloning the active object only when necessary. They then discuss three cases where they must clone the active objects, by using a “minimal clone operation”. The minimal clone operation determines whether an object’s fields are cloned or aliased based on their ownership annotation. This makes their operation very similar to Sheep Cloning, so much so that Clarke et al. mention how Sheep Cloning can be used in its place.

Aside from the work of Drossopoulou and Noble, we are aware of one other cloning implementation that is similar to Sheep Cloning. In Nienaltowski’s PhD. thesis [20], he reiterates the excessiveness of copying an object’s whole structure using `deep_import` (deep cloning) and the potential dangers introduced by shallow cloning. This inspired him to introduce a lightweight operation, object import, for Eiffel’s SCOOP (Simple Concurrent Object-Oriented Programming). Object import copies the objects of non-separate references while the objects from a separate reference are left alone. When cloning objects in SCOOP all non-separate references must be followed and the objects reached, are copied, whereas the objects of separate references are considered harmless. The policy of copying objects by distinguishing between separate references and non-separate references is similar to the policy of cloning objects by distinguishing between objects inside the representation and objects outside the representation. Sheep Cloning and object import, however, still have their differences. Sheep Cloning uses ownership types, a method to control the topology of objects on the heap, while object import uses separate types, a method to identify objects for the SCOOP processor.

Jensen et al [16] propose placing static cloning annotations on classes and methods to aid users in constructing their cloning methods. The annotations define the copy policy for each class, where the policies ensure the maximum sharing possible between the original object and its clones. All cloning applications of a class must adhere to their copy policy. The copy policy is checked statically by a type and effect system. The copy policy does not perform cloning functions or generate the cloning method, it is just a set of specifications for clones produced. This differs from Sheep Cloning as our formalism includes an actual algorithm for object cloning, and our proofs guarantee the clones produced are structurally equivalent (as defined in Sect. 4.5) to the original object.

One of the first papers to identify the confusion between the semantics and the implementation of the copy function was Grogono and Chalin [13]. They discuss how it is more important if the objects being cloned are immutable or mutable than if the object is a value or a reference. They also touched on the idea of object representation, and the need to distinguish semantics from efficiency when copying objects. They concluded that effect-like systems need to play a greater role in object copying.

Grogono and Sakkinen [14] present a technique to generate a cloning function. They discuss the issues surrounding copying objects and the difficulty in comparing objects. Grogono and Sakkinen also present a set of detailed examples of various cloning operations and type equality. They explore the copying and comparing features in several programming languages.

7. CONCLUSION

In this paper we have presented a formalism of Sheep Cloning, and its soundness proof. We motivated the need for Sheep Cloning by comparing Sheep Cloning against existing form of object cloning and showing that Sheep Cloning is preferable.

8. REFERENCES

- [1] Martín Abadi and Luca Cardelli. An imperative object calculus. In *Theory and Practice of Software Development (TAPSOFT)*. 1995.
- [2] Martín Abadi, Luca Cardelli, and Ramesh Viswanathan. An Interpretation of Objects and Object Types. In *Principles of Programming Languages (POPL)*, 1996.
- [3] Jonathan Aldrich and Craig Chambers. Ownership Domains: Separating Aliasing Policy from Mechanism. In *European Conference on Object Oriented Programming (ECOOP)*, 2004.
- [4] Nicholas Cameron. *Existential Types for Variance - Java Wildcards and Ownership Types*. PhD thesis, Department of Computing, Imperial College London, 2008.
- [5] Ka Wai Cheng and Sophia Drossopoulou. Types for deep/shallow cloning. Technical report, Imperial College London, 2012.
- [6] Dave Clarke, Tobias Wrigstad, Johan Östlund, and Einar Johnsen. Minimal ownership for active objects. In *Programming Languages and Systems*. 2008.
- [7] David Clarke. *Object Ownership and Containment*. PhD thesis, School of Computer Science and Engineering, The University of New South Wales, Sydney, Australia, 2001.
- [8] David Clarke and Tobias Wrigstad. External Uniqueness is Unique Enough. In *European Conference on Object Oriented Programming (ECOOP)*, 2003.
- [9] David G. Clarke and Sophia Drossopoulou. Ownership, Encapsulation and the Disjointness of Type and Effect. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2002.
- [10] David G. Clarke, John M. Potter, and James Noble. Ownership Types for Flexible Alias Protection. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 1998.
- [11] Werner Dietl and Peter Müller. Exceptions in Ownership Type Systems. In *Formal Techniques for Java-like Programs (FTfJP)*, 2004.
- [12] Sophia Drossopoulou and James Noble. Trust the clones. In *Formal Verification of Object-Oriented Software (FoVEOS)*, 2011.
- [13] Peter Grogono and Patrice Chalin. Copying, sharing, and aliasing. In *In Proceedings of the Colloquium on Object Orientation in Databases and Software Engineering (COODBSE'94)*, 1994.
- [14] Peter Grogono and Markku Sakkinen. Copying and comparing: Problems and solutions. In *European Conference on Object Oriented Programming (ECOOP)*. 2000.
- [15] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight Java: a Minimal Core Calculus For Java and GJ. *ACM Trans. Program. Lang. Syst.*, 2001.
- [16] Thomas Jensen, Florent Kirchner, and David Pichardie. Secure the clones: Static enforcement of policies for secure object copying. In *European Symposium on Programming (ESOP)*, 2011.
- [17] Peter Müller and Arnd Poetzsch-Heffter. Universes: A Type System for Controlling Representation Exposure. In *Programming Languages and Fundamentals of Programming*, 1999.
- [18] Peter Müller and Arnd Poetzsch-Heffter. Universes: A Type System for Alias and Dependency Control. Technical Report 279, Fernuniversität Hagen, 2001.
- [19] Peter Müller and Arsenii Rudich. Ownership transfer in universe types. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007.
- [20] Piotr Nienaltowski. *Practical framework for contract-based concurrent object-oriented programming*. PhD thesis, Department of Computer Science, ETH Zurich, 2007.
- [21] James Noble, David Clarke, and John Potter. Object ownership for dynamic alias protection. In *Proceedings of the 32nd International Conference on Technology of Object-Oriented Languages (TOOL)*, 1999.
- [22] John Plevyak and Andrew Chien. Type directed cloning for object-oriented programs. In *Languages and Compilers for Parallel Computing*. 1996.
- [23] Alex Potanin, James Noble, Dave Clarke, and Robert Biddle. Generic Ownership for Generic Java. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2006.

ParaSail: A Pointer-Free Path to Object-Oriented Parallel Programming

S. Tucker Taft

AdaCore
24 Muzzey Street
Lexington, MA 02421 USA
taft@adacore.com

Abstract

Pointers are ubiquitous in modern object-oriented programming languages, and many data structures such as trees, lists, graphs, hash tables, etc. depend on them heavily. Unfortunately, pointers can add significant complexity to programming. ParaSail, a new parallel object-oriented programming language, has adopted an alternative, pointer-free approach to defining data structures. Rather than using pointers, ParaSail supports flexible data structuring using *expandable* (and shrinkable) objects, along with generalized indexing. By eliminating pointers, ParaSail significantly reduces the complexity for the programmer, while also allowing ParaSail to provide pervasive, safe, object-oriented parallel programming.

Categories and Subject Descriptors D.3.2 [Programming Languages]: Language Classifications – concurrent, distributed, and parallel languages; object-oriented languages; D.3.3 [Programming Languages]: Language Constructs and Features – abstract data types, classes and objects, concurrent programming structures, polymorphism.

General Terms Algorithms, Design, Reliability, Languages, Theory, Verification.

Keywords pointer-free; region-based storage management; expandable objects; parallel programming.

1. Introduction

Pointers are ubiquitous in modern object-oriented programming languages, and many data structures such as trees, lists, graphs, hash tables, etc. depend on them heavily. Unfortunately, pointers can add significant complexity to programming. Pointers can make storage management more complex, pointers can make assignment and equality semantics more complex, pointers can increase the ways two different names (access paths) can designate the same object, pointers can make program analysis and proof more complex, and pointers can make it harder to *divide and conquer* a data structure for parallel processing.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
FOOL 2012 at SPLASH 2012 October 22, 2012, Tuscon, AZ, USA.
Copyright © 2012 ACM 1-59593-XXX-X/0X/000X...\$5.00.

Is there an alternative to using pointers? ParaSail[1], a new parallel object-oriented programming language, adopts a different paradigm for defining data structures. Rather than using pointers, ParaSail supports flexible data structuring using *expandable* (and shrinkable) objects, along with generalized indexing. By eliminating pointers, ParaSail significantly reduces the complexity for the programmer, while also allowing ParaSail to provide pervasive, safe, object-oriented parallel programming.

2. Expandable and Optional Objects

An *expandable* object is one that can grow without using pointers, much as a house can grow through additions. Where once there was a door to the back yard, a new screened-in porch can be added. Where once there was only one floor, a new floor can be added. The basic mechanism for expansion in ParaSail is that every type has one additional value, called **null**. A component can initially be null, and then be replaced by a non-null value, thereby expanding the enclosing object. At some later point the enclosing object could shrink, by replacing a non-null component with null.

Not every component of an object is allowed to be null. The component must be declared as **optional** if it is allowed to take on a null value. For example, a *Tree* structure might have a (non-optional) *Payload* component, and then two additional components, *Left* and *Right*, which are each declared as **optional** *Tree*. Similarly, a stand-alone object may be declared to be of a type *T*, or of a type **optional** *T*. Only if it is declared **optional** may it take on the null value. The value of an object *X* declared as **optional** may be tested for nullness using *X is null* or *X not null*.

Another example of a data structure using optional components would be a linked list, with each node having two components, one *Payload* component, and a *Tail* component of type **optional** *List*. There is also a built-in parameterized type, *Basic_Array<Component_Type>* which allows the *Component_Type* to be specified as **optional**. This allows, for example, the construction of a hash table with buckets represented as linked-lists, by declaring the *backbone* of the hash table as a *Basic_Array<optional List<Hash_Table_Item>>*. The components of the hash table would start out as **null**, but as

items are added to the hash table, one or more of the component lists would begin to grow.

2.1 Assignment, Move, and Swap Operations

Because there are no pointers, the semantics of assignment in ParaSail are very straightforward, namely the entire right-hand-side object is copied and assigned into the left-hand side, replacing whatever prior value was there. However, there are times when it is desirable to *move* a component from one object to another, or *swap* two components. Because implementing these on top of an assignment that uses copying might impose undue overhead, in ParaSail, *move* and *swap* are separate operations. The semantics of *move* are that the value of the left-hand-side is replaced with the value of the right-hand-side, and the right-hand-side ends up null. For *swap*, the values of the left- and right-hand-side are swapped. Syntactically, ParaSail uses "==" for (copying) assignment, "<==" for move, and "<=>" for swap. The ParaSail compiler is smart enough to automatically use *move* semantics when the right-hand-side is the result of a computation, rather than an object or component that persists after the assignment.

As an example of where *move* might be used, if our hash table grows to the point that it would be wise to lengthen the backbone, we could create a new `Basic_Array` twice as large (for example), and then *move* each list node from the old array into the new array in an appropriate spot, rebuilding each linked list, and then finally *move* the new array into the original hash-table object, replacing the old array. The *swap* operation is also useful in many contexts, for example when balancing a tree structure, or when sorting an array.

2.2 Binary Tree Example

The Appendix includes an example of a pointer-free tree-based map module implemented in ParaSail. This example illustrates the use of **optional** components, as well as "<==" (*move*), which is used as part of a Delete operation.

3. Cyclic Data Structures and Generalized Indexing

Expandable objects allow the construction of many kinds of data structures, but a general, possibly cyclic graph is not one of them. For this, ParaSail provides generalized indexing. The array-indexing syntax, "`A[I]`," is generalized in ParaSail to be usable with any container-like data structure, where `A` is the container and `I` is the key into that data structure. A directed graph in ParaSail could be represented as a table of Nodes, where the index into the table is a unique node `Id` of some sort, with edges represented as Predecessors and Successors components of each Node, where Predecessors and Successors are each sets of node-ids. See the second example in the Appendix for an illustration of using generalized indexing to represent a directed graph.

If edges in a directed graph are represented with pointers, it is possible for there to be an edge that refers to a deleted node, that is, a *dangling* reference. Such a dangling reference could result in a storage leak, because the target node could not be reclaimed, or it could lead to a potentially destructive reference to reclaimed storage. When edges are represented using node-ids, there is still the possibility of an edge referring to a deleted node or the wrong node, but there is no possibility for there to be associated storage leakage or destructive reference to reclaimed storage, as node-ids are only meaningful as keys into the associated container.

4. Region-Based Storage Management

Storage management without pointers is significantly simplified. All of the objects declared in a given scope are associated with a storage *region*, essentially a local heap. As an object grows, all new storage for it is allocated out of this region. As an object shrinks, the old storage can be immediately released back to this region. When a scope is exited, the entire region is reclaimed. There is no need for asynchronous garbage collection, as garbage never accumulates.

Every object identifies its region, and in addition, when a function is called, the region in which the result object should be allocated is passed as an implicit parameter. This *target* region is determined by how the function result is used. If it is a temporary, then it will be allocated out of a temporary region associated with the point of call. If it is assigned into a longer-lived object, then the function will be directed to allocate the result object out of the region associated with this longer-lived object. The net effect is that there is no copying at the call site upon function return, since the result object is already sitting in the correct region.

Note that pointers are still used *behind the scenes* in the current implementation of ParaSail, but eliminating them from the surface syntax and semantics eliminates essentially all of the complexity associated with pointers. That is, a semantic model of expandable and shrinkable objects, operating under (mutable) *value* semantics, rather than a semantic model of nodes connected with pointers, operating under *reference* semantics, provides a number of benefits, such as simpler storage management, simpler assignment semantics, easier analyzability, etc.

The move and swap operations have well-defined semantics independent of the region-based storage management, but they provide significant added efficiency when the objects named on the left and right-hand side are associated with the same region, because then their dynamic semantics can be accomplished simply by manipulating pointers. In some cases the programmer knows when declaring an object that it is intended to be moved into or swapped with another existing object. In that case, ParaSail allows the programmer to give a *hint* to that effect by specifying in the object's declaration that it is "**for** `X`" meaning that it should be associated with the same region as `X`. With region-based storage management, it is always safe to associate an object with a longer-lived region, but to avoid a storage leak, the ParaSail compiler will set the value of such an object to null on scope exit, as its storage would not otherwise be reclaimed until the longer-lived region is reclaimed. An optimizing compiler could automatically choose to allocate a local variable out of an outer region when it determines that its last use is a move or an assignment to an object from an outer region.

5. Parallel and Distributed Programming

In addition to removing pointers, certain other simplifications are made in ParaSail to ease parallel and distributed programming. In particular, there are no global variables; functions may only update objects passed to them as **var** (in-out) parameters. Furthermore, as part of passing an object as a **var** parameter, it is effectively *handed off* to the receiving function, and compile-time checks ensure that no further references are made to the object, until the function completes. In particular, the checks ensure that no part of the **var** parameter is passed to any other function, nor to

this same function as a separate parameter. This eliminates at compile-time the possibility of aliasing between a **var** parameter and any other object visible to the function. These two additional rules, coupled with the lack of pointers, mean that all parameter evaluation may happen in parallel (e.g. in "F(G(X), H(Y))", G(X) and H(Y) may be evaluated in parallel), and function calls may easily cross address-space boundaries, since the objects are self-contained (with no incoming or outgoing references), and only one function at a time can update a given object.

In the tree-based map example (given in the Appendix), the recursive routine `Count_Subtree` contains a pair of recursive calls which can be safely evaluated in parallel with each other, thanks in part to the ParaSail model which eliminates pointers and global variables.

6. Concurrent Objects

All of the above rules apply to objects that are *not* designed for concurrent access. ParaSail also supports the construction of concurrent objects, which allow lock-free, locked, and queued simultaneous access. These objects are *not* "handed off" as part of parameter passing; concurrent objects provide operations that synchronize any attempts at concurrent access. Three kinds of synchronization are supported. *Lock-free* synchronization relies on low-level hardware-supported operations such as atomic load and store, and compare-and-swap. *Locked* synchronization relies on automatic locking as part of calling a **locked** operation of a concurrent object, and automatic unlocking as part of returning from the operation. Finally, *queued* synchronization is provided, which evaluates a *dequeue condition* upon call (under a lock), and only if the condition is satisfied is the call allowed to proceed, still under the lock. A typical *dequeue condition* might be that a buffer is not full, or that a mailbox has at least one element in it. If the dequeue condition is not satisfied, then the caller is added to a queue. At the end of any operation on the concurrent object that might change the result of the dequeue condition for a queued caller, the dequeue condition is evaluated and if true, the operation requested by the queued caller is performed before the lock is released. If there are multiple queued callers, then they are serviced in turn until there are none with satisfied dequeue conditions.

See the third example in the Appendix, the `Locked_Box`, for an example of a concurrent module.

7. Related Work

There are very few pointer-free languages currently under active development. Fortran 77 [2] was the last of the Fortran series that restricted itself to a pointer-free model of programming. Algol 60 lacked pointers [3], but Algol 68 introduced them [4]. Early versions of Basic had no pointers [5], but modern versions of Basic use pointer assignment semantics for most complex objects [6]. The first versions of Pascal, Ada, Modula, C, and C++ all used pointers for objects that were explicitly allocated on the heap, while still supporting stack-based records and arrays; these languages also required manual heap storage reclamation. The first versions of Eiffel, Java, and C# provided little or no support for stack-based records and arrays, moving essentially all complex objects into the heap, with pointer semantics on assignment, and automatic garbage collection used for heap storage reclamation.

In many cases, languages that originally did not require heavy use of pointers, as they evolved to support object-oriented program-

ming, the use of pointers increased, often accompanied by a reliance on garbage collection for heap storage reclamation. For example, Modula-3 introduced *object* types, and all instances of such types were allocated explicitly on the heap, with pointer semantics on assignment, and automatic garbage collection for storage reclamation [7].

The Hermes language (and its predecessor NIL) was a language specifically designed for distributed processing [8]. The Hermes type system had high-level type constructors, which allowed them to eliminate pointers. As the designer of Hermes explained it, "pointers are useful constructs for implementing many different data structures, but they also introduce aliasing and increase the complexity of program analysis" [9]. Hermes pioneered the notion of *type state*, as well as *handoff* semantics for communication, both of which are relevant to ParaSail, where compile-time assertion checking depends on flow analysis, and handoff semantics are used for passing **var** parameters in a call on an operation.

The SPARK language, a high-integrity subset of Ada with added proof annotations, omits pointers from the subset [10]. No particular attempt was made to soften the effect of losing pointers, so designing semi-dynamic data structures such as trees and linked-lists in SPARK requires heavy use of arrays [11].

Whiley, a new language that has similar goals to ParaSail, has also chosen to avoid pointers and adopt a mutable value semantics [12]. Whiley does not support pervasive parallelism, but rather adopts an explicit *actor* model for concurrency, and allows parameter aliasing in certain contexts. Whiley provides high-level data structuring primitives such as maps, sets, and tuples, rather than adopting the general notion of expandable objects through the use of optional values.

Another relatively new language that is pointer-free is Composita, described in the 2007 Ph. D. thesis of Dr. Luc Bläser from ETH in Zurich [13]. Composita is a component-based language, which uses message passing between active components. Sequences of statements are identified as either *exclusive* or *shared* to provide synchronization between concurrent activities. Composita has the notion of *empty* and *installed* components, analogous to the notion of optional values in ParaSail.

Annotations that indicate an *ownership* relationship between a pointer and an object can provide some of the same benefits as eliminating pointers [14]. AliasJava [15] provides annotations for specifying ownership relationships, including the notion of a *unique* pointer to an object. Guava [16] is another Java-based language that adds *value* types which have no aliases, while still retaining normal *object* types for other purposes. Assignment of value types in Guava involves copying, but they also provide a *move* operation essentially equivalent to that in ParaSail. These approaches, by limiting the possibilities for aliasing, can significantly help in proving desirable properties about programs that use pointers. However, the additional programmer burden of choosing between multiple kinds of pointers or objects based on their aliasing behavior can increase the complexity of such approaches.

One reason given in these papers on aliasing control for not going entirely to a pointer-free, or unique-pointer approach for object-oriented programming, is that certain important object-oriented programming paradigms, such as the Observer pattern [17], depend on the use of pointers and aliasing. ParaSail attempts to

provide an existence proof to the contrary of that premise, as do other recent pointer-free languages such as Whiley and Composita. In general, a more loosely-coupled pointer-free approach using container data structures with indices of various sorts, allows the same problem to be solved, with fewer storage management and synchronization issues. For example, the Observer pattern, which is typically based on lists of pointers to observing objects, might be implemented using a pointer-free Publish-Subscribe pattern, which can provide better scalability and easier use of concurrency [18]. In general, pointers are not directly usable in distributed systems, so many of the algorithms adopted to solve problems in a distributed manner are naturally pointer-free, and hence are directly implementable in ParaSail.

Pure functional languages, such as Haskell [19], avoid many of the issues of pointers by adopting immutable objects, meaning that sharing of data creates no aliasing or race condition problems. However, *mostly* functional languages, such as those derived from the ML language [20], include references to mutable objects, thereby re-introducing most of the potential issues with aliasing and race conditions. Even Haskell has found it necessary to introduce special *monads* such as the IO monad to support applications where side-effects are essential to the operation of the program. In such cases, these side-effects need to be managed in the context of parallel programming [21].

Hoare in his 1975 paper on Recursive Data Structures [22] identified many of the problems with general pointers, and proposed a notation for defining and manipulating recursive data structures without the use of pointers at the language level, even though pointers were expected to be used at the implementation level. Language-level syntax and semantics reminiscent of this early proposal have appeared in functional languages, but have not been widely followed in languages with mutable values. Mostly-functional languages such as ML have also more followed the Algol 68 model of explicit references when defining mutable recursive data structures, despite Hoare's many good arguments favoring a pointer-free semantics at the language level. Hoare's notation did not introduce the notion of optional values, but instead relied on types defined by a tagged union of generators, at least one of which was required to not be recursive. ParaSail adopts the optional value approach and allows the set of generators that can be used to create objects to be open-ended, by relying on object-oriented polymorphism over interfaces.

Minimizing use of a global heap through the use of region-based storage management was proposed by Tofte and Talpin [23], implemented in the ML Kit with Regions [24], and refined further in the language Cyclone [25]. Cyclone was not a pointer-free language. Instead, every pointer was associated with a particular region at compile time, allowing compile-time detection of dangling references. A global, garbage-collected heap was available, but local dynamic regions provided a safe, more efficient alternative.

Many functional (or mostly functional) languages have a notion similar to ParaSail's *optional* objects. For example, in Haskell they are called *maybe* objects [19]. In ParaSail, because of its fundamental role in supporting recursive data structures, *optional* is a built-in property usable with every object, component, or type declaration, rather than being an additional level of type. In addition, this approach allows null-ness to be represented without a distinct null object, by ensuring that every type has at

least one bit pattern than can be recognizable as the null for that type.

8. Implementation Status and Evaluation

A prototype version of the ParaSail compiler front end and accompanying documentation is available for download [1]. The front end supports nearly all of the language (defined in an accompanying reference manual), and generates instructions for a *ParaSail Virtual Machine* (PSVM). A full multi-threaded interpreter for the PSVM instruction set is built into the front end, and includes a simple interactive Read-Eval-Print Loop for testing. A backend that translates from the PSVM instruction set to a compilable language is under development, with C, Ada, and LLVM assembly language as the initial targets.

The ParaSail front end automatically splits computations up into very light-weight *picothreads*, each representing a potentially parallel sub-computation. The PSVM includes special instructions for spawning and awaiting such picothreads. The PSVM interpreter uses the *work stealing* model [26] to execute the picothreads; work stealing incorporates heavier weight server processes which each service their own queue of picothreads (in a LIFO manner), stealing from another server's queue (in a FIFO manner) only when their own queue becomes empty.

ParaSail adopted a pointer-free model initially to enable easy and safe pervasively parallel programming. However, the early experience in programming in ParaSail with its pointer-free, mutable value semantics, has provided support for the view that pointers are an unnecessary burden on object-oriented programming. The availability of optional values allows the direct representation of tree structures, singly-linked lists, hash tables, and so on in much the same way they are represented with pointers, but without the added complexities of analysis, storage management, and parallelization associated with pointers.

As discussed above, data structures that inherently require multiple paths to the same object, such as doubly-linked lists or general graphs, can be implemented without pointers by using indexing into generalized container structures. Even without the restriction against pointers, it is not uncommon to represent directed graphs using indices rather than pointers, in part because the presence or absence of edges between nodes does not necessarily affect whether the node itself should exist.

A significant advantage of using a container such as a vector to represent a graph, is that partitioning of the graph for the purpose of a parallel divide-and-conquer computation over the graph can be simplified, by using a simple numeric range test on the index to determine whether a given node is within the subgraph associated with a particular sub-computation. As an example, see the *Boundary_Set* operation within the directed graph example in the Appendix. More generally, operations on indices tend to be easier to analyze than those on pointers, including, for example, a proof that two variables contain different indices, as would be needed for a proof of non-aliasing when the indices are used to index into a container.

In our early experiments, programmers familiar with Java or C# have not had trouble understanding and learning to program in ParaSail, thanks in part to its familiar class-and-interface object-oriented programming model. The notion of optional values matches quite directly how pointers work. The fact that assignment is by copy, and there is a separate *move* operation, is a bit of

a surprise, but once explained it seems to make sense. The ease of parallel programming and the lack of problems involving undesired aliasing are seen by these early users as valuable benefits of the shift. Perhaps the bigger challenge for some is the lack of global variables in ParaSail. Eliminating global variables seems to require more restructuring than does doing without pointers. It would be possible to allow global *concurrent* objects in ParaSail without interfering with easy parallelization, but these would add complexity to the language and its analysis in other ways.

The primary purpose of our eliminating pointers remains the support of easy, pervasive parallelism. From that point of view, ParaSail is a good showcase. ParaSail programs produce a great deal of parallelism without the programmer having to make any significant effort. Almost any algorithm that is structured as a recursive walk of a tree (see the `Count_Subtree` operation in the tree-based map example in the Appendix), or as a divide and conquer algorithm such as Quicksort (see the `Boundary_Set` operation in the directed-graph example in the Appendix for a similar divide-and-conquer approach), will by default have its recursive calls treated as potentially parallel sub-computations. Monitoring built into the ParaSail interpreter indicates the level of parallelism achieved, and it can be substantial for algorithms not

normally thought of as being *embarrassingly* parallel. We believe the implicit, safe, pervasive parallelism provided by ParaSail is one of its unique contributions, and this relies on the simplifications made possible by the elimination of pointers and other sources of hidden aliasing.

9. Pointer-Free Object-Oriented Parallel Programming

The pointer-free nature of ParaSail is not just an interesting quirk. Rather, we believe it represents a significantly simpler way to build large object-oriented systems. By itself it simplifies storage management, assignment semantics, and analyzability, and when combined with the elimination of global variables and parameter aliasing, it allows for the easy parallelization of all expression evaluation, and the easy distribution of computations across address spaces, while still supporting directly mutable data structures. As implied by Hoare in [22], pointers are effectively the “goto” of data structuring, and as such, eliminating their use at the language level can bring analogous benefits to data structures and object-oriented programming, as eliminating the “goto” brought to control structures and procedural programming.

Appendix

Pointer-free tree-based map

Here is an example of a pointer-free tree-based map module, showing both the **interface** for the module, and the **class** that defines its implementation.

```
// The following is the interface to a simple map module, which maps
// a key of type Key_Type to a value of type Element_Type.
// External clients of this module see only the interface declarations.
interface TMap<Key_Type is Ordered<>; Element_Type is Assignable<>> is
    op "[]"() -> TMap; // create an empty TMap

    func Insert(var TMap; Key : Key_Type; Value : Element_Type);
    func Find(TMap; Key : Key_Type) -> optional Element_Type;
    func Delete(var TMap; Key : Key_Type);
    func Count(TMap) -> Univ_Integer;
end interface TMap;

// Here is a possible implementation of this TMap module,
// based on a binary tree.
// The structure of each node of the binary tree
// is defined by the local (pointer-free) Binary_Node interface, which
// declares four components, a Left and Right optional Binary_Node,
// a Key component, and a Value component, which can be null if the Key
// has been (logically) deleted from the Map.
// The Count component of the Tree tracks the number of non-deleted keys
// in the map. The Count_Subtree operation is used to verify the correctness
// of the Count component, and is provided here to illustrate
// a parallel recursive operation.
// Declarations in a class preceding the exports keyword are private to the
// implementation.

class TMap is

    interface Binary_Node<> is
        // A simple "concrete" binary node structure used to implement a TMap
        var Left : optional Binary_Node;
        var Right : optional Binary_Node;
        const Key : Key_Type; // Key controls structure of tree
        var Value : optional Element_Type; // null means was deleted
    end interface Binary_Node;
```

```

// Root and Count are mutable components of an object defined by the TMap module
var Root : optional Binary_Node; // Root of the tree
var Count := 0;

// A private operation used to check the correctness of Count maintenance
func Count_Subtree(Subtree : optional Binary_Node)
-> Univ_Integer is
  if Subtree is null then
    return 0;
  else
    const SubCount := // these recursive calls are done in parallel
      Count_Subtree(Subtree.Left) + Count_Subtree(Subtree.Right);
    return (Subtree.Value is null? Subcount: Subcount + 1);
  end if;
end func Count_Subtree;

exports
op "[]"() -> TMap is // create an empty TMap
  return (Root => null, Count => 0);
end op "[]";

func Insert(var TMap; Key : Key_Type; Value : Element_Type) is
  // Insert Key => Value pair into TMap
  for M => TMap.Root loop
    if M is null then
      // Not already in the map; add it
      M := (Key => Key, Value => Value,
        Left => null, Right => null);
      TMap.Count += 1;
    else
      case Key =? M.Key of
        [#less] =>
          continue loop with M.Left;
        [#greater] =>
          continue loop with M.Right;
        [#equal] =>
          // Key already in the map
          if TMap.Value is null then
            TMap.Count += 1; // but had been deleted
          end if;
          // Overwrite the Value field
          M.Value := Value;
          return;
        end case;
      end if;
    end loop;
  end func Insert;

func Find(TMap; Key : Key_Type) -> optional Element_Type is
  // Find value associated with Key in the TMap; return null if not found
  for M => TMap.Root while M not null loop
    case Key =? M.Key of
      [#less] =>
        continue loop with M.Left;
      [#greater] =>
        continue loop with M.Right;
      [#equal] =>
        // Found it; return the value (which might be null)
        return M.Value;
      end case;
    end loop;
  // Not found in TMap; return null
  return null;
end func Find;

```



```

func Delete(var TMap; Key : Key_Type) is
  // Delete Key from the TMap
  for M => TMap.Root while M not null loop
    case Key =? M.Key of
      [#less] =>
        continue loop with M.Left;
      [#greater] =>
        continue loop with M.Right;
      [#equal] =>
        // Found it; if at most one subtree is non-null,
        // overwrite it; otherwise, set its value field
        // to null (to avoid a more complex re-balancing).
        if M.Value not null then
          TMap.Count -= 1; // Decrement unless already deleted.
        end if;
        if M.Left is null then
          // Move right subtree into M
          M <== M.Right;
        elsif M.Right is null then
          // Move left subtree into M
          M <== M.Left;
        else
          // Cannot immediately reclaim node;
          // set value field to null instead.
          M.Value := null;
        end if;
      end case;
    end loop;
  // Not found in the map
end func Delete;

func Count(TMap) -> Univ_Integer is
  // Return count of non-deleted keys
  // Verify that Count has been maintained properly
  // (ParaSail assertions use {})
  {Count_Subtree(TMap.Root) == TMap.Count}
  return TMap.Count;
end func Count;
end class TMap;

```

Pointer-Free Directed Graph

Here is an example of a pointer-free directed graph, represented as a vector of nodes, each with a Predecessor and Successor set of node-ids to represent edges of the graph. Preconditions, postconditions, and assertions are enclosed in braces ({}) in ParaSail, reminiscent of Hoare logic.

```

interface DGraph<Element is Assignable<>> is
  // Interface to a (pointer-free) Directed-Graph module
  type Node_Id is new Integer<1..10**6>;
  // A unique id for each node in the graph
  type Node_Set is Countable_Set<Node_Id>;
  // A set of nodes

  func Create() -> DGraph;
  // Create an empty graph
  func Add_Node(var DGraph; Element) -> Node_Id;
  // Add a node to a graph, and return its node id
  func Add_Edge(var DGraph; From, To : Node_Id)
    {From in DGraph.All_Nodes(); To in DGraph.All_Nodes()};
  // Add an edge in the graph
  op "indexing"(ref DGraph; Node_Id)
    {Node_Id in DGraph.All_Nodes()}
    -> ref Element;
  // Return a reference to an element of the graph

```

```

func Successors(ref const DGraph; Node_Id) -> ref const Node_Set
  {Node_Id in DGraph.All_Nodes()};
  // The set of successors of a given node
func Predecessors(ref const DGraph; Node_Id) -> ref const Node_Set
  {Node_Id in DGraph.All_Nodes()};
  // The set of predecessors of a given node

func All_Nodes(DGraph) -> Node_Set;
  // The set of all nodes
func Roots(DGraph) -> Node_Set;
  // The set of all nodes with no predecessor
func Leaves(DGraph) -> Node_Set;
  // The set of all nodes with no successor
end interface DGraph;

class DGraph is // Class defining the Directed-Graph module
  interface Node<> is
    // Local definition of Node structure
    var Elem : Element;
    var Succs : Node_Set;
    var Preds : Node_Set;
  end interface Node;

  var G : Vector<Node>;
  // The vector of nodes, indexed by Node_Id

  func Boundary_Set(DGraph; Nodes : Countable_Range<Node_Id>;
    Want_Roots : Boolean) -> Node_Set is
    // Recursive helper for exported Roots and Leaves functions
    const Len := Length(Nodes);
    case Len of
      [0] =>
        return [];
      [1] =>
        if Want_Roots?
          Is_Empty(Predecessors(DGraph, Nodes.First)):
          Is_Empty(Successors(DGraph, Nodes.First))
        then
          // This is on the desired boundary
          return [Nodes.First];
        else
          // This is not on the desired boundary
          return [];
        end if;
      [..] =>
        // Parallel recursive divide and conquer
        const Half_Way := Nodes.First + Len / 2;
        return
          Boundary_Set(DGraph,
            Nodes.First ..< Half_Way, Want_Roots) |
          Boundary_Set(DGraph,
            Half_Way .. Nodes.Last, Want_Roots);
    end case;
  end func Boundary_Set;

exports

  func Create() -> DGraph is
    // Create an empty graph
    return (G => []);
  end func Create;

  func Add_Node(var DGraph; Element) -> Node_Id is
    // Add a node to a graph, and return its node id
    DGraph.G |= (Elem => Element, Succs => [], Preds => []);

```

```

    return Length(DGraph.G);
end func Add_Node;

op "indexing"(ref DGraph; Node_Id) -> ref Element is
    // Return a reference to an element of the graph
    return DGraph.G[Node_Id].Elem;
end op "indexing";

func Add_Edge(var DGraph; From, To : Node_Id) is
    // Add an edge in the graph
    DGraph.G[From].Succs |= To;
    DGraph.G[To].Preds |= From;
end func Add_Edge;

func Successors(ref const DGraph; Node_Id) -> ref const Node_Set is
    // The set of successors of a given node
    return DGraph.G[Node_Id].Succs;
end func Successors;

func Predecessors(ref const DGraph; Node_Id) -> ref const Node_Set is
    // The set of predecessors of a given node
    return DGraph.G[Node_Id].Preds;
end func Predecessors;

func All_Nodes(DGraph) -> Node_Set is
    // The set of all nodes
    return 1 .. Length(DGraph.G);
end func All_Nodes;

func Roots(DGraph) -> Node_Set is
    // The set of all nodes with no predecessor
    return Boundary_Set
        (DGraph, 1 .. Length(DGraph.G), Want_Roots => #true);
end func Roots;

func Leaves(DGraph) -> Node_Set is
    // The set of all nodes with no successor
    return Boundary_Set
        (DGraph, 1 .. Length(DGraph.G), Want_Roots => #false);
end func Leaves;
end class DGraph;

```

Concurrent locked-box module

Here is an example of a concurrent module, using **locked** and **queued** operations. The **Locked_Box** contains a single component into which a value of type **Content_Type** may be stored. The **Set_Content** and **Content** operations provide simple, **locked** access to the content of the box, and allow for **null** values. The **Put** and **Get** operations deal only with non-null values, with the caller of **Put** being **queued** until the box is empty before allowing a new value to be stored, and the caller of **Get** being **queued** until the box has a non-null value, and then returning that value and setting the box back to empty (null).

```

concurrent interface Locked_Box<Content_Type is Assignable<>> is
    func Create(C : optional Content_Type) -> Locked_Box;
        // Create a box with the given content

    func Set_Content(locked var B : Locked_Box; C : optional Content_Type);
        // Set content of box

    func Content(locked B : Locked_Box) -> optional Content_Type;
        // Get a copy of current content

    func Put(queued var B : Locked_Box; C : Content_Type);
        // Wait for the box to be empty (i.e. null)
        // and then Put something into it.

    func Get(queued var B : Locked_Box) -> Content_Type;
        // Wait until content is non-null,
        // then return it, leaving it null.
end interface Locked_Box;

```

```

concurrent class Locked_Box is
  var Content : optional Content_Type; // Content might be null
exports
  func Create(C : optional Content_Type) -> Locked_Box is
    // Create a box with the given content
    return (Content => C);
  end func Create;

  func Set_Content(locked var B : Locked_Box; C : optional Content_Type) is
    // Set content of box
    B.Content := C;
  end func Set_Content;

  func Content(locked B : Locked_Box)
    -> optional Content_Type is
    // Get a copy of current content
    return B.Content;
  end func Content;

  func Put(queued var B : Locked_Box; C : Content_Type) is
    queued until B.Content is null then
      // Wait for the box to be empty (i.e. null)
      // and then Put something into it.
      B.Content := C;
    end func Put;

  func Get(queued var B : Locked_Box)
    -> Result : Content_Type is
    queued while B.Content is null then
      // Wait until content is non-null,
      // then return it, leaving it null.
      Result <= B.Content;
    end func Get;

end class Locked_Box;

```

References

- [1] S. Tucker Taft, *Designing ParaSail: A New Programming Language*, 2009, <http://parasail-programming-language.blogspot.com> (retrieved 8/10/2012).
- [2] Ansi x3.9-1978. *American National Standard – Programming Language FORTRAN*. American National Standards Institute, 1978, http://www.fortran.com/fortran/F77_std/rjcnf.html (retrieved 8/10/2012).
- [3] Peter Naur *et al*, *Revised Report on the Algorithmic Language Algol 60*, 1963 <http://www.masswerk.at/algol60/report.htm> (retrieved 8/10/2012)
- [4] C.H. Lindsey, “A History of Algol 68,” *Proceedings of HOPL-II: The second ACM SIGPLAN conference on History of programming languages*, pp 97-132, ACM New York, NY, 1993.
- [5] J.G. Kemeny and T.E. Kurtz, *BASIC, 4th Edition*, Trustees of Dartmouth College, 1968, http://www.bitsavers.org/pdf/dartmouth/BASIC_4th_Edition_Jan68.pdf (retrieved 8/10/2012).
- [6] Microsoft, *Visual Basic Concepts -- Programming with Objects*, <http://msdn.microsoft.com/en-us/library/aa716290.aspx> (retrieved 8/10/2012).
- [7] L. Cardelli *et al*, *Modula-3 Report (revised)*, <http://www.hpl.hp.com/techreports/Compaq-DEC/SRC-RR-52.pdf> (retrieved 8/10/2012).
- [8] R. Strom, “Hermes: An Integrated Language and System for Distributed Programming,” *Proceedings of the IEEE Workshop on Experimental Distributed Systems*, pp. 75--82, 1990 <http://researcher.ibm.com/files/us-bacon/Strom90Hermes.pdf> (retrieved 9/27/2012).
- [9] *ibid.*, p. 80.
- [10] R. Chapman and P. Amey, *SPARK-95 – The SPADE Ada Kernel (including RavenSPARK)*, Altran-Praxis, 2008 http://www.altran-praxis.com/downloads/SPARK/technicalReferences/SPARK95_RavenSPARK.pdf (retrieved 8/10/2012).
- [11] P. Thornley, *SPARKSure Data Structures*, 2009, http://www.sparksure.com/resources/SPARK_Data_Structures_10_0_9.zip (retrieved 8/10/2012).
- [12] D. Pearce, “While overview,” <http://whiley.org/about/overview/> (retrieved 9/28/2012).
- [13] L. Bläser, “A Component Language for Pointer-Free Programming,” Ph. D. Thesis, ETH Zurich, 2007 <http://e-collection.library.ethz.ch/eserv/eth:30090/eth-30090-02.pdf> (retrieved 9/28/2012)
- [14] D. Clarke. “Object Ownership & Containment,” Ph.D. Thesis, University of New South Wales, Australia, July 2001 <http://people.cs.kuleuven.be/~dave.clarke/papers/thesis.ps.gz> (retrieved 9/28/2012).
- [15] J. Aldrich *et al*, “Alias Annotations for Program Understanding,” *OOPSLA’02*, pp. 311-330, Nov. 4-8, 2002, Seattle, WA

- <http://archjava.fluid.cs.cmu.edu/papers/oopsla02.pdf> (retrieved 9/28/2012).
- [16] D. Bacon *et al*, “Guava: A Dialect of Java without Data Races,” *OOPSLA '00*, pp. 382-400, Minneapolis, MN, 2000 <http://researcher.watson.ibm.com/researcher/files/us-bacon/Bacon00Guava.pdf> (retrieved 9/30/2012).
- [17] E. Gamma, *et al*, *Design Patterns*, Addison-Wesley, 1994.
- [18] G. Hohpe *et al*, *Enterprise Integration Patterns* website, “JMS Publish/Subscribe Example,” 2003 <http://www.eaipatterns.com/ObserverJmsExample.html> (retrieved 9/28/2012).
- [19] S. Marlow, *Haskell 2010 Language Report*, 2010, <http://www.haskell.org/onlinereport/haskell2010/> (retrieved 8/10/2012).
- [20] R. Harper, *Programming in Standard ML*, Carnegie Mellon University, 2005, <http://www.cs.cmu.edu/~rwh/smlbook/book.pdf> (retrieved 8/10/2012).
- [21] S. Marlow *et al*, “A Monad for Deterministic Parallelism,” *Haskell '11 Proceedings of the 4th ACM symposium on Haskell*, pp. 71-82, ACM New York, NY, 2011.
- [22] C.A.R. Hoare, C.B. Jones (Ed.), “Recursive Data Structures,” *Essays in Computing Science*, pp. 217-244, Prentice-Hall International (UK), 1989 <http://dl.acm.org/citation.cfm?id=63445&bnc=1> (retrieved 9/30/2012).
- [23] M. Tofte and J.-P. Talpin, “Implementing the call-by-value lambda calculus using a stack of regions,” *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 188-201, ACM Press, 1994.
- [24] M. Tofte and J.-P. Talpin, “Region-based memory management,” *Information and Computation*, 132.2, pp. 109-176, 1997 <http://www.irisa.fr/prive/talpin/papers/ic97.pdf> (retrieved 9/27/2012).
- [25] D. Grossman *et al*, “Region-Based Memory Management in Cyclone,” *PLDI '02 Proceedings of the ACM SIGPLAN 2002 Conference on Programming language design and implementation*, pp. 282 – 293, ACM New York, NY, 2002 <http://www.eecs.harvard.edu/~greg/cyclone/papers/cyclone-regions.pdf>
- [26] R. Blumofe and C. Leiserson, “Scheduling Multithreaded Computations by Work Stealing,” *Journal of the ACM*, 46.5, pp. 720-748, Sep. 1999 http://supertech.csail.mit.edu/papers/ft_gateway.pdf (retrieved 9/29/2012).

Inferring AJ Types for Concurrent Libraries

Wei Huang Ana Milanova

Rensselaer Polytechnic Institute
Troy, NY, USA

{huangw5, milanova}@cs.rpi.edu

Abstract

Data-centric synchronization advocates data-based synchronization as opposed to control-based synchronization. It is more intuitive and can make correct concurrent programming easier. Dolby et al. [9] proposed AJ, a type system for data-centric synchronization, and showed that Java programs can be refactored into AJ. Unfortunately, programmers still have to add synchronization constructs manually (in the form of AJ type annotations), and the burden on programmers is high. In this paper we propose a type inference technique that infers AJ types for concurrent libraries. Our technique significantly reduces the amount of annotations.

Categories and Subject Descriptors D.3.3 [Programming Languages]: Language Constructs and Features; D.1.5 [Programming Techniques]: Object-oriented Programming

General Terms Languages, Theory

1. Introduction

Data races and *atomicity violations* are difficult to prevent in a multi-threaded program. Traditional approaches use synchronization for ordering instructions in order to prevent data races. These approaches are control-centric, because programmers have to protect all *accesses* to shared memory locations. Control-centric approaches are error-prone and inflexible. First, shared memory locations are not easy to identify because of the presence of aliasing in object-oriented programming. In addition, it is also hard to control granularity of synchronization. When adding or removing memory locations to be synchronized, a programmer has to carefully reorganize the instruction sequences.

Data-centric synchronization is a technique which advocates data-based synchronization as opposed to control-based synchronization. In short, programmers specify an *atomic set* of semantically-related locations; these locations must be synchronized consistently. Dolby et al. [9] proposed AJ, a type system for data-centric synchronization. AJ provides a correctness guarantee called atomic-set serializability, which prevents data races and other concurrency errors. Dolby et al. [9] show that Java programs can be refactored into AJ. Unfortunately, programmers still need to do significant work to add synchronization constructs in the form of AJ type annotations, and the overhead is relatively high.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

FOOL '12 October 22, 2012, Tucson, AZ, USA.

Copyright © 2012 ACM [to be supplied]. . . \$10.00

```
1 public class Counter {      7 Counter c = new Counter();
2   int val;                  8   c.inc();
3   int get() { return val; }  9   c.dec();
4   void dec { val--; }       10  ...
5   void inc { val++; }
6 }
```

Figure 1. A simple counter class example from [9]. We omit the `atomicset a` and `atomic(a)` annotations because we assume that every class has exactly one atomic set and all fields are protected by this atomic set.

We propose a technique that infers AJ types for concurrent libraries. Programmers specify a small number of AJ annotations to express the design decisions, from which our system would automatically infer the remaining ones and verify the inference result. Our approach reduces the annotation burden significantly. It requires 42 alias annotations for the 11826 LOC Java Collections library. In contrast, the approach from Dolby et al. [9] requires 370 alias annotations. Thus, our approach achieves almost 90% reduction.

2. Data-centric Synchronization with AJ

2.1 Overview

AJ [9] extends Java with annotations that support data-centric synchronization. The type system in our paper, called AJ-lite, differs from AJ [9] in two ways. First, AJ-lite assumes exactly one atomic set, named `a`, per class and all fields of the class are protected by this atomic set. The atomic set is retrieved by referencing a ghost field `a`, i.e. `this.a`. This is a simplification of AJ, but it is consistent with the implementation presented in [9]. Each atomic set has a logical lock protecting all fields of the object. The methods of the class are *units of work* which preserve the consistency of the atomic set. Second, AJ-lite replaces the *internal class* in AJ with *internal references*. In AJ, *internal* is an annotation on class declarations. AJ requires that every instance of an internal class is tracked by the type system, and not leaked outside of the object that constructed it. In contrast, in AJ-lite *internal* is an annotation on references; AJ-lite tracks *internal* references and disallows leaks outside of the object that constructed them. AJ-lite allows a class to have both internal and non-internal references. These two differences between AJ and AJ-lite do not violate the correctness property of AJ, i.e., the atomic set serializability guarantee. We justify this claim in Section 2.5.

Figure 1 shows a simple Counter class with atomic increment and decrement methods. Each Counter object has its own atomic set, protecting its only field `val`. Intuitively, when a thread accesses a Counter object, it must hold the logical lock associated with this atomic set. Thus, the accesses (increments and decrements of field `val`) are serialized and therefore consistent.

```

1 class PairCounter {
2   int diff;
3   A* Counter low = new A* Counter();
4   A* Counter high = new A* Counter();
5   void incHigh() {
6     high.inc();
7     diff = high.get() - low.get();
8   }
9   ...
10 }

```

(a) PairCounter with aliasing atomic sets.

```

1 class PairCounter {
2   int diff;
3   A! Counter low = new A! Counter();
4   A! Counter high = new A! Counter();
5   void incHigh() {
6     high.inc();
7     diff = high.get() - low.get();
8   }
9   ...
10 }

```

(b) PairCounter with internally aliasing atomic sets.

Figure 2. A pair counter class. The example is taken from [9].

In many cases, an atomic set must protect fields of more than one object. AJ supports merging atomic sets using *alias annotations*. Figure 2(a) shows a `PairCounter` class which has two integer counters and one method `incHigh` that updates the difference between counters. Merging is done by *aliasing* the atomic set of each `Counter` with the atomic set of the `PairCounter` object. AJ-lite uses and qualified type `A* Counter`. This corresponding to `|a = this.a| Counter` in AJ, where `a` refers to the atomic set of the `Counter` object and `this.a` refers to the atomic set of the enclosing `PairCounter` object. Intuitively, this means that at runtime, a thread that accesses `PairCounter`, and/or one of the `Counter` objects, must hold the logical lock of `PairCounter` as well as the locks of the two `Counter` objects. Note that the locks are only logical — an actual implementation can choose to map each logical lock to a distinct physical lock, merge aliased logical locks into a single physical lock, and so on.

If the low and high counter objects remain confined within the `PairCounter`, that is, all accesses to these counter objects go through their enclosing `PairCounter` object, then the `Counter` objects do not need locks because they are protected by the lock of `PairCounter`. Thus, if the programmer knows (or an analysis proves) that the counter objects are never exposed, then he/she may annotate references `low` and `high` with the internal alias qualifier `A!`. The typing of `PairCounter` will be as in Figure 2(b). This internal alias qualifier `A!` corresponds to the internal annotation on classes in AJ which we discussed earlier.

2.2 AJ-lite Qualifiers

We now formally introduce AJ-lite’s type qualifiers. There are three source-level qualifiers in AJ-lite, i.e., the universal set of qualifiers $U_{AJ\text{-}lite} = \{A^*, A^?, A!\}$:

- `A*`: The atomic set of the object referenced by an `A*` reference `x` is aliased with the atomic set of the current (i.e., `this`) object, i.e. `x.a = this.a`. `A*` corresponds to the `|a = this.a|` annotation in AJ.
- `A?`: The atomic set of the object that is referenced by an `A?` reference `x` may or may not be aliased to the set of the current

object. In other words, we do not know whether `this.a` and `x.a` are aliased or not. `A?` corresponds to the implicit default annotation in AJ.

- `A!`: The atomic set of an `A!` object is aliased to the current object. `A!` is different from `A*` because it forbids exposure of the object outside of the object that constructed it, i.e. the `A!` object is internal to the current object. In contrast, an `A*` object can be accessed by arbitrary objects. `A!` corresponds to the internal annotation on classes in AJ.

The qualifiers form the following subtyping hierarchy:

$$A^* <: A^?$$

Therefore, `A*` references can be assigned to `A?` ones. However, `A?` ones cannot be assigned to `A*` ones. This is consistent with AJ, which allows dropping the alias annotation but disallows adding an alias annotation. The difference between `A!` and `A*` is that `A!` is not a subtype of `A?` and therefore, `A!` references cannot be assigned to anything but `A!` references.

2.3 Viewpoint Adaptation

In AJ and AJ-lite, viewpoint adaptation is used when deciding the types of fields at field access, and the types of formal parameters and method returns at method call. Consider the field access `x.f` where both `x` and `f` are declared as `A*`. `x` being `A*` means that the `x` object and the current object have their atomic sets aliased. Similarly, the field `f` being `A*` means that the `x` object and the `f` object have their atomic sets aliased as well. Therefore, we can conclude that the type of `x.f` is `A*` as well. Consider another field access `y.g` where `y` is `A?` and `g` is `A*`. Because we cannot decide whether this’s atomic set is aliased to `g`’s atomic set, `y.g` is of type `A?`. Therefore, the types of fields at field access and the types of formal parameters and method returns at method call, depend on not only their declared types, but also the type of the *receiver*, which represents the access context.

Both AJ and AJ-lite encode this by means of *viewpoint adaptation*. Viewpoint adaptation is a concept from Universe Types [5, 7, 8], which can be adapted to Ownership Types [4]. Viewpoint adaptation of a type q from the point of view of another type q' , results in the adapted type q'' . This is written as $q' \triangleright q = q''$. Viewpoint adaptation in AJ-lite is defined as follows (Undefined adaptations would be considered as type errors):

$$\begin{aligned}
A! &\triangleright q &= q \\
A^* &\triangleright q &= q \\
A^? &\triangleright A^* &= A^? \\
A^? &\triangleright A^? &= A^?
\end{aligned}$$

The first two rules state that adapting any type q from the point of view of `A!` or `A*` results in q . Recall the field access `x.f` where both `x` and `f` are `A*`. We can decide the type `x.f` by using viewpoint adaptation: $A^* \triangleright A^* = A^*$. The last two adaptation rules state that adapting q ($q \neq A!$) from the point of view of `A?` results in `A?`. Recall the other field access `y.g` where `y` is `A?` and `g` is `A*`. We can decide the type of `y.f` is $A^? \triangleright A^* = A^?$. The reason that the rules forbid adapting `A!` from the point of view of `A?` is to guarantee that internal references would never escape to unknown context.

The above rules are consistent with the rules from Dolby et al. [9]:

$$\begin{aligned}
\text{adapt}(C, t) &= C \\
\text{adapt}(C[a = \text{this}.b], D[b = \text{this}.c]) &= C[a = \text{this}.c]
\end{aligned}$$

where $\text{adapt}(t, t')$ is the view of type t from the point of view of type t' . Here the first rule expresses that an `A?` type C adapted from any point of view, results in an `A?` type, as it is in our rules. The second rule states that if the adaptee type t is aliased (i.e., we have $C[a = \text{this}.b]$) then the adapter type t' must be aliased as well ($D[b = \text{this}.c]$), and the result of the adaptation is an aliased type

$cd ::= \text{class } C \text{ extends } D \{ \overline{fd} \ \overline{md} \}$ *class*
 $fd ::= t \ f$ *field*
 $md ::= t \ m(t \ x) \ q \{ \overline{t \ y} \ s; \text{return } y \}$ *method*
 $s ::= s; \ s \mid x = \text{new } t() \mid x = y$ *statement*
 $\mid x.f = y \mid x = y.f \mid x = y.m(z)$
 $t ::= q \ C$ *qualified type*
 $q ::= \mathcal{A}^* \mid \mathcal{A}^? \mid \mathcal{A}!$ *qualifier*

Figure 3. Syntax of a core OO language. C and D are class names, f is a field name, m is a method name, x, y and z are names of local variables and formal parameters, including implicit parameter this, and qualifier q is independent of the Java type. Qualifier q at the method declaration qualifies implicit parameter this.

$C[a = \text{this}.c]$. This seems different from AJ-lite because AJ-lite allows adapting $\mathcal{A}^*$ from the point of view of $\mathcal{A}^?$, but they are essentially the same. Remember that AJ allows dropping the alias annotation. Therefore, adapting $\mathcal{A}^*$ from the point of view of $\mathcal{A}^?$ in AJ-lite is essentially the same as dropping the alias annotation of $C[a = \text{this}.b]$ and apply the first adaptation rule of AJ, and the result is consistent — it is $\mathcal{A}^?$ in AJ-lite and C in AJ.

2.4 Typing Rules

Syntax For brevity, we restrict our formal attention to a core calculus in the style of Dolby et al. [9] whose syntax appears in Figure 3. The language models Java with a syntax in a “named form”, where the results of field accesses, method calls, and instantiations are immediately stored in a variable. Without loss of generality, we assume that methods have parameter this, and exactly one other formal parameter. Features not strictly necessary are omitted from the formalism, but they are handled correctly in the implementation. We write $\overline{t \ y}$ for a sequence of local variable declarations.

In contrast to a formalization of standard Java, a type t has two orthogonal components: type qualifier q (which expresses the alias annotation) and Java class type C . The AJ-lite type system is *orthogonal* to (i.e., independent of) the Java type system, which allows us to specify typing rules over type qualifiers q alone.

Typing rules The typing rules are shown in Figure 4. These rules are generic and enforce standard subtyping constraints with viewpoint adaptation at field access and method call. For example, at field write (TWRITE), f ’s type is adapted from the point of view of x ; the resulting adapted type must be a supertype of the type of the right-hand-side y of the assignment. The generic rules are part of an inference and checking framework for ownership-like type systems [14]. The framework takes as input (1) the universe of type qualifiers, in our case $U_{\text{AJ-lite}} = \{\mathcal{A}!, \mathcal{A}^*, \mathcal{A}^?\}$, (2) the subtyping hierarchy of type qualifiers, in our case $\mathcal{A}^* <: \mathcal{A}^?$, (3) the viewpoint adaptation function, in our case, as it was specified earlier (Section 2.3), and (4) the additional $\mathcal{B}$ constraints, which are constraints imposed by individual type systems.

In AJ no rules demand additional constraints. Therefore, all $\mathcal{B}$ sets are empty. We elaborate on the rule for (TCALL). Note that constraint $q_y <: q_y \triangleright q_{\text{this}}$ prevents a leak of an $\mathcal{A}!$, i.e., internally aliased reference, which is supposed to be encapsulated by its enclosing object. We also note here, that implicit parameters this can only be $\mathcal{A}!$ or $\mathcal{A}^*$. Implicit parameter this is always internally aliased or aliased as a result of our decision that every class has an atomic set. Intuitively, this is $\mathcal{A}^*$, except when the method is called on an internal receiver, in which case it must be $\mathcal{A}!$ in order to prevent a leak.

The above mentioned constraint enforces the notion of the internal class from [9]. Figure 5 shows an example. The return type of m in class C is $\mathcal{A}^?$ — m is public and can be invoked at arbitrary points. As a result, the return type of id and subsequently

$$\begin{array}{c}
\text{(TNEW)} \\
\frac{\Gamma(x) = q_x \quad q <: q_x \quad \mathcal{B}_{(\text{TNEW})}(q_x, q)}{\Gamma \vdash x = \text{new } q \ C} \\
\\
\text{(TASSIGN)} \\
\frac{\Gamma(x) = q_x \quad \Gamma(y) = q_y \quad q_y <: q_x \quad \mathcal{B}_{(\text{TASSIGN})}(q_x, q_y)}{\Gamma \vdash x = y} \\
\\
\text{(TWRITE)} \\
\frac{\Gamma(x) = q_x \quad \text{typeof}(f) = q_f \quad \Gamma(y) = q_y \quad q_y <: q_x \triangleright q_f \quad \mathcal{B}_{(\text{TWRITE})}(q_x, q_f, q_y)}{\Gamma \vdash x.f = y} \\
\\
\text{(TREAD)} \\
\frac{\Gamma(x) = q_x \quad \Gamma(y) = q_y \quad \text{typeof}(f) = q_f \quad q_y \triangleright q_f <: q_x \quad \mathcal{B}_{(\text{TREAD})}(q_y, q_f, q_x)}{\Gamma \vdash x = y.f} \\
\\
\text{(TCALL)} \\
\frac{\text{typeof}(m) = q_{\text{this}}, q \rightarrow q' \quad \Gamma(x) = q_x \quad \Gamma(y) = q_y \quad \Gamma(z) = q_z \quad q_z <: q_y \triangleright q \quad q_y \triangleright q' <: q_x \quad q_y <: q_y \triangleright q_{\text{this}} \quad \mathcal{B}_{(\text{TCALL})}(m, q_y, q_x)}{\Gamma \vdash x = y.m(z)}
\end{array}$$

Figure 4. Generic typing rules. The rules enforce standard subtyping constraints as well as additional constraints $\mathcal{B}$ that can be imposed by a concrete type system.

```

1  public class Id {
2      Id id() {
3          Id x = this;
4          return x;
5      }
6  }

7  class C {
8      public Id m() {
9           $\mathcal{A}!$  Id y; Id z;
10         y = new  $\mathcal{A}!$  Id();
11         z = y.id();
12         return z;
13     }
14 }

```

Figure 5. A leak of implicit parameter this. Example from [9].

x and this in id cannot be $\mathcal{A}!$. Suppose that the return of id and x are $\mathcal{A}^?$, and this of id is $\mathcal{A}^*$; thus, id type checks. The constraint $q_y <: q_y \triangleright q_{\text{this}}$ causes type checking at call $z = y.\text{id}()$ to fail: we have $\mathcal{A}!$ y but an $\mathcal{A}^*$ this, and $\mathcal{A}^? \triangleright \mathcal{A}!$ is undefined. This is the desired behavior because it disallows the leak of the internal id object.

Arrays are handled by specifying two types, one for the array element and one for the array object. For example, following Java 8 syntax [10]

$\mathcal{A}^?$ Object $\mathcal{A}!$ [] signers;

declares an $\mathcal{A}!$ array signers which stores $\mathcal{A}^?$ elements of type Object. The type of the array object in the above declaration is from the point of view of the declaring class, while the type of the element object is from the point of view of the array object. Given these two types (if the programmer chooses to specify these types), there exists a unique array field type. The array field type gives the type of the element from the point of view of the array object. In the above example, the array field type is $\mathcal{A}^?$. Thus, array accesses are checked as field accesses: For example, the statement $s = \text{signers}[i]$; generates the following constraint:

$$q_{\text{sig}} \triangleright q[] <: q_s$$


```

1 class Transfer {
2   void transfer(unitfor Counter from, unitfor Counter to) {
3     from.dec();
4     to.inc();
5   }
6 }

```

Figure 6. Adding atomic sets to a unit of work.

where q_s gives the type of s , q_{sig} gives the type of the signers array object, and $q_{[]}$ gives the array field type of that array object.

AJ [9] provides an additional annotation, `unitfor`, which is used to annotate formal parameters in bulk methods. `unitfor` unions the atomic set of the actual argument with the atomic set of the receiver of the method for the duration of the execution of the method. Consider the example in Figure 6. The `from` and `to` counters must be updated atomically. The `unitfor` construct ensures that atomic sets of the `from` and `to` objects are merged with the atomic set of the receiver of the `transfer` method which ensures the consistency of the update.

`unitfor` is a dynamic construct. It has no effect on the static type systems (it does not appear in the static typing rules for AJ in [9]). `unitfor` cannot be easily inferred because the consistency requirements of bulk methods are highly dependent on program semantics as it is in the `Transfer` example. In this paper, we assume that `unitfor` annotations are provided by the programmer and focus our inference effort on the alias annotations.

2.5 Correctness Argument

As discussed in Section 2.1, AJ-lite differs from AJ in two ways. We make an informal argument that these two differences do not violate the correctness property of AJ, namely, atomic set serializability.

The first difference is that AJ-lite has exactly one atomic set per class and all fields in the class are protected by this atomic set, while AJ allows more than one atomic set and some fields can be excluded from any atomic sets. In fact, AJ-lite captures a special case of AJ, and it is also consistent with AJ’s current implementation [9]. Therefore, AJ-lite’s simplification of AJ still has the correctness guarantee as proved in [9]. However, this simplification may hinder concurrent accesses to data structures designed for sharing. We will address this restriction in future work.

The second difference is that AJ-lite uses *internal references* instead of the *internal class* in AJ. The adaptation rules of AJ-lite enforce that an object referenced by an $\mathcal{A}!$ variable either (1) remains protected by the atomic set of its creating object, or (2) escapes to an object whose atomic set is aliased with its creating object. (1) is straightforward when an $\mathcal{A}!$ object is never exposed to the outside (this is exactly the same as ownership encapsulation [4]). (2) is enforced by the first two adaptation rules $\mathcal{A}! \triangleright q = q$ and $\mathcal{A}* \triangleright q = q$. An $\mathcal{A}!$ variable remains $\mathcal{A}!$ when it is adapted from the viewpoint of $\mathcal{A}!$ or $\mathcal{A}*$. For example, $y.f$ where y is $\mathcal{A}*$ and f is $\mathcal{A}!$ is of type $\mathcal{A}!$. Because y ’s atomic set is aliased with the current this’s atomic set, thus the $\mathcal{A}!$ f escapes to this and is protected by this’s atomic set. Also, the adaptation of $\mathcal{A}!$ from the viewpoint of $\mathcal{A}?$ is not allowed, thus an $\mathcal{A}!$ variable will not escape to an $\mathcal{A}?$ context. Because of (1) and (2), we know that an $\mathcal{A}!$ variable is always protected by the atomic set of its enclosing data structure and it behaves the same as an $\mathcal{A}*$ variable. AJ-lite’s extension on internal reference does not violate the atomic-set serializability property proved by [9]. Further, $\mathcal{A}!$ reference provides other optimization opportunities in the implementation. For example, if a class is always annotated $\mathcal{A}!$, we can conclude that this class can safely get rid of its atomic set, which could improve the performance of the program.

2.6 LinkedList Example

Figure 7 shows a `LinkedList` example with alias annotations as inferred by our analysis. Note that in general, AJ-lite would require at least some programmer annotations. Programmer annotations will denote semantically related objects that must belong to the same atomic set. Conversely, annotations could denote unrelated objects which must have separate atomic sets in order to increase parallelism. For example, the `LinkedList` and the `ListIter` objects are semantically related and must be aliased: a modification to the `LinkedList` while iteration is in progress will result in incorrect behavior of the iterator (e.g. we may get a `ConcurrentModificationException` in Java). Conversely, a new array created in the `toArray` method of a collection, is unrelated to the collection object; the programmer can annotate the array creation site as $\mathcal{A}?$.

The AJ example in Figure 7 requires no manual annotations. The `ListIter` object is inferred as $\mathcal{A}*$ (clearly, the `LinkedList` and its iterator are related, and must belong to the same atomic set). The `Entry` objects are inferred as $\mathcal{A}!$ because they are accessed only from the `LinkedList` and `ListIter` objects which belong to the same atomic set (thus, the `Entry` objects are internal to this atomic set).

3. Type Inference and Checking

Our type inference is phrased in the general framework for specification, inference and checking of ownership-like type systems [14]. Recall that the framework takes as input (1) the universe of type qualifiers, in our case $U_{AJ-lite} = \{\mathcal{A}!, \mathcal{A}*, \mathcal{A}?\}$, (2) the subtyping hierarchy of type qualifiers, in our case $\mathcal{A}* <: \mathcal{A}?$, (3) the viewpoint adaptation function, in our case, as it was specified earlier (Section 2.3), and (4) the additional $\mathcal{B}$ constraints, which are empty as argued earlier (Section 2.4).

Note that as with ownership type systems (e.g., Universe Types and Ownership Types) AJ-lite permits multiple valid typings. For example, all variables¹ in the program could be typed as $\mathcal{A}?$; the program will type check but will be unsafe in a sense that it will allow atomicity violations. Note that the correctness guarantee of AJ and AJ-lite, i.e., the atomic set serializability property, is with respect to the provided alias annotations. For example, if the programmer has missed to annotate as aliased objects `LinkedList` and `ListIter` in Figure 7, the program will type check, but it may throw an exception. Similarly, all variables can be typed $\mathcal{A}*$ which, again will type check, but will lose concurrency (as all objects will form one giant atomic set and the program will degenerate into a sequential program). Recall that we simplify the original AJ by assuming that every class has an atomic set and all fields belong to this atomic set.

In addition to the above parameters, which define the AJ-lite type system, the inference framework takes an additional parameter: an ordering of the qualifiers. The ordering expresses preference for typing of variables. The AJ-lite qualifiers are ordered $\mathcal{A}! > \mathcal{A}* > \mathcal{A}?$. This means (roughly) that if possible, a variable should be typed as $\mathcal{A}!$; in other words we prefer internally aliased. Otherwise (i.e., if $\mathcal{A}!$ is impossible), it should be typed as $\mathcal{A}*$. If neither $\mathcal{A}!$ or $\mathcal{A}*$ are possible, it should be typed as $\mathcal{A}?$. This ordering over qualifiers gives rise to an ordering over all valid typings: for two typings T_1, T_2 , we have $T_1 > T_2$ iff T_1 types more variables as $\mathcal{A}!$ than T_2 or T_1 and T_2 type the same number of variables as $\mathcal{A}!$ but T_1 types more variables as $\mathcal{A}*$ than T_2 . The highest ranked typing in this ordering is what we call the “best”, or most desirable typing. Our goal is to infer the “best” typing.

The inference initializes all annotated variables to the singleton set that contains the programmer-provided annotation, default initial

¹ Term “variable” is used to refer to (1) allocation sites, (2) local variables, including formal parameters, (3) fields, and (4) method returns

```

1  public abstract class AbsList {
2      int size;
3      public int size() A* {
4          return size;
5      }
6      public abstract A* ListIterator iterator() A*;
7      public abstract void add(Object o) A*;
8      public abstract boolean addAll(AbsList c) A*;
9      public abstract Object get(int i) A*;
10 }

12 class Entry {
13     Object elem;
14     A! Entry next;
15     A! Entry prev;
16     Entry(Object elem, A! Entry next,
17           A! Entry prev) A! {
18         this.elem = elem;
19         this.next = next;
20         this.prev = prev;
21     }
22 }

23 class LinkedList extends AbstractList {
24     A! Entry header;
25     public LinkedList() A* {
26         header = new A! Entry(null,null,null);
27         header.prev = header;
28         header.next = header;
29     }
30     public void add(Object o) A* {
31         A! Entry p = header.prev;
32         A! Entry newEntry = new A! Entry(o,header,p);
33         header.prev = newEntry;
34         p.next = newEntry;
35         size++;
36     }
37     public A* ListIterator iterator() A* {
38         ListIterator it = new A* Listltr(this,header);
39         return it;
40     }
41 }

43 class Listltr implements ListIterator {
44     final A* LinkedList list;
45     private A! Entry header;
46     Listltr(A* LinkedList l, A! Entry h) A* {
47         list = l;
48         header = h;
49     }
50     ...
51 }

```

Figure 7. The LinkedList example with annotations as inferred by our analysis. Unannotated variables are $A?$. The annotation after method declaration gives the type of implicit parameter this.

type assignments for special variables (discussed below), and all remaining variables to $U_{AJ-lite}$, the universal set of AJ-lite qualifiers. Then it repeatedly examines each program statement and applies the statement’s typing rule on the current set of qualifiers; it removes infeasible qualifiers from the sets until it reaches a fixpoint.

As an example, suppose that the iteration examines $x = y$ where x is mapping to $U_{AJ-lite} = \{A!, A*, A?\}$ and y is mapping to a singleton set $\{A?\}$. When the inference examines the statement, it removes qualifiers $A!$ and $A*$ from the set for x , because neither $A? <: A!$ or $A? <: A*$ holds (which the typing rule for $x = y$ requires as shown in Figure 4). The final result of the inference is a mapping from variables to *sets* of qualifiers. We derive a mapping from variables to AJ-lite types by mapping each variable to the maximal qualifier in its set. For AJ-lite, it is guaranteed that (1) this is a valid typing. This can be proven using a case-by-case analysis. In addition, it is verified by an independent type checker, which is part of our implementation. (2) this is the unique “best” typing according to the ordering on typings described in the previous paragraph. The proof of a general case, from which the above statement derives, is given in [14].

Our inference analysis uses programmer-provided annotations, default initial type assignments, and the above qualifier ordering in order to infer a desirable typing. Below, we describe the inference process.

Recall that in this work, we focus on the typing of concurrent libraries. We start from the following default initial type assignments:

1. All non-this parameters of *public* methods receive default type $\{A?\}$. It is expected that, in general, arguments will be unrelated to the current object and they will maintain their own atomic set. Note that default annotations take place only if there is no programmer-provided annotation; if it is needed that certain parameters are aliased, the programmer can annotate those

parameter, and the programmer-provided annotation would take precedence over the default.

2. All this parameters receive default $\{A!, A*\}$.
3. All return types of *public* methods receive default $\{A*, A?\}$. These methods can be called and return at arbitrary points.

All remaining variables are initialized to $\{A!, A*, A?\}$. Note that if a variable has a programmer-provided annotation, that annotation overrides the default.

The programmer must examine the allocation sites in the library and annotate as many allocation sites as $A?$ as possible. Some of the allocation sites are semantically related to the creating this object and must be aliased; others are unrelated and can have an independent atomic set. The analysis prefers $A*$ over $A?$ (as we discussed earlier). Thus, all unannotated allocation sites will be typed $A!$ or $A*$; more precisely, if a site cannot be typed $A!$, then it will be typed $A*$. Therefore, in order to increase parallelism, the programmer is encouraged to annotate as many allocation sites as $A?$ as the semantics of the program permits.

In addition to annotating allocation sites, the programmer may choose to annotate parameters in order to override the above-mentioned defaults. For example, in the commonly-used idiom `new X(this)`, the `this` object and the `new X` object are typically semantically related. However, if `X`’s constructor is public, the formal parameter to which `this` is assigned to will be typed as $A?$ by default. We would like to type this formal parameter as $A*$ because this will allow us to identify more $A!$ objects.

Next, we set the order over qualifiers as $A! > A* > A?$ and run the inference analysis. The optimality property holds for this system and ordering, and the inference produces the best AJ typing. We prefer $A*$ over $A?$ because it presents optimization opportunities.

4. Experiments

We have typed a subset of the Java Collections library: LinkedList, ArrayList, HashMap and all related classes. The subset amounts to 11826 LOC and includes 63 files.

We use the defaults outlined above. We manually added 3 $\mathcal{A}?$ annotations: one at the allocation site `Object[] result = new Object  $\mathcal{A}?$  [size()]` in `toArray` of `AbstractCollection`, one at `Object[] result = new Object  $\mathcal{A}?$  [size]` in `toArray` of `LinkedList`, and one at `Object[] result = new Object  $\mathcal{A}?$  [size]` in `toArray` of `ArrayList`. The newly created arrays can exist independently of their creating collection. In addition, we added $\mathcal{A}*$ annotations to parameters as follows: for every allocation site `new X(...,this,...)` where the constructor `X` was public, we annotated as $\mathcal{A}*$ the formal parameter to which `this` is assigned. For example, method `iterator` in class `AbstractList` contains statement `return (Iterator) new AbstractList.Itr(this)`. The `this` object and the newly created iterator objects must be aliased. However, the constructor `AbstractList.Itr` is public, and if not annotated, its formal parameter will be typed as $\mathcal{A}?$ by default. Therefore, we insert a manual annotation $\mathcal{A}*$ for its formal parameter. In addition, we provide several more manual annotations in order to override the public default: e.g., parameter `n` of public void `setNext( $\mathcal{A}*$  HashMap.Entry n)` { ... } must be annotated as $\mathcal{A}*$; if not annotated, `n` receives default type $\mathcal{A}?$ which forces all `HashMap.Entry` objects to be $\mathcal{A}?$. However, `HashMap.Entry` is semantically related to `AbstractList.Itr` and must be part of its atomic set (i.e., must be aliased). In total we added 32 $\mathcal{A}?$ and $\mathcal{A}*$ annotations.

We compared the results of our inference with the manually annotated Collections library used to report results in [9].² The majority of annotations were as in the manually annotated code. We observed several differences in `AbstractMap`. For example, the manually annotated code in [9] contains the following (we use our simplified annotations):

```

1  public boolean containsValue(Object value) {
2       $\mathcal{A}*$  Iterator i = entrySet().iterator();
3      if (value == null) {
4          while (i.hasNext()) {
5               $\mathcal{A}*$  Map.Entry e = ( $\mathcal{A}*$  Map.Entry)i.next();
6              if (e.getValue() == null)
7                  return true;
8          }
9      } else {
10         while (i.hasNext()) {
11              $\mathcal{A}*$  Map.Entry e = ( $\mathcal{A}*$  Map.Entry)i.next();
12             if (value.equals(e.getValue()))
13                 return true;
14         }
15     }
16     return false;
17 }
```

Our inference types the `e` at lines 5 and 11 as $\mathcal{A}?$. This is because we do not add a cast at the right-hand-side of the assignment. The return of public `Object next()` is $\mathcal{A}?$ (the object it returns is not part of the container's internal structure), and the $\mathcal{A}?$ annotation propagates to the `e`'s. The iterator `i` at line 2 is inferred as $\mathcal{A}*$, just as in the manually annotated code.

In order to get exactly the same set of alias annotations as the manually annotated Collection library we would need 10 downcast $\mathcal{A}*$ annotations (it is a design decision to not add these downcasts). Thus, in total, we will have 42 alias annotations. Compared with [9], 42 vs. 370 is almost 90% reduction. As mentioned earlier, we do not infer unitfor annotations. Those annotations are difficult to infer as it would require dynamic analysis. In order to achieve consistency

we would need to manually add about 53 unitfor annotations, as in [9].

Our implementation is part of the inference and checking framework described in [14]. The code for the framework is publicly available at <http://code.google.com/p/type-inference/>, including source.

5. Related Work

We briefly discuss related work on type systems for preventing data races and atomicity violations, and inference of pluggable types.

Abadi et al. [1] present a static race detection analysis for Java. The analysis is based on a type system that captures synchronization patterns. By checking programmer provided type annotations, the type system can guarantee the absences of data races if the synchronization and the type annotations are consistent. They also provide an inference algorithm to compute annotations automatically and a user interface to facilitate inspecting warnings generated by the checker. Abadi et al.'s inference algorithm and the inference algorithm used by Tip et al. [12, 18] are similar to our inference algorithm of AJ. These algorithms start with sets containing all possible answers and iteratively remove elements that are inconsistent with the typing rules. Our algorithm also uses a preference ranking over qualifiers to pick up the "best" typing for AJ.

Flanagan and Qadeer [11] present a type system for specifying and verifying the atomicity of methods for Java. The type system can check that the instructions of an *atomic* method are not interleaved with instructions from other threads for any arbitrary executions. They also implement an atomic type checker for Java and discover a number of atomicity violations in `java.lang.String` and `java.lang.StringBuffer`.

There are a number of works for inferring user-defined type qualifiers to reduce programmer's burden on annotations. Greenfieldboyce and Foster [13] present a framework called JQual for inferring user-defined type qualifiers in Java. JQual is effective for *source-sink* type systems, for which programmers need to add annotations to the sources and sinks and JQual infers the intermediate annotations for the rest of the program. Chin et al. [3] propose CLARITY for the inference of user-defined qualifiers for C programs based on user-defined rules, which can also be inferred given user-defined invariants. CLARITY infers several type qualifiers, including *pos* and *neg* for integers, *nonnull* for pointers, and *tainted* and *untainted* for strings.

There are also lots of works on inference of ownership types. Aldrich et al. [2] present an ownership type system and a type inference algorithm. Their inference creates equality, component and instantiation constraints and solves these constraints. Ma and Foster [16] propose Uno, a static analysis for automatically inferring ownership, uniqueness, and other aliasing and encapsulation properties in Java. Dietl et al. [6] present a tunable static inference for Generic Universe Types (GUT). Constraints of GUT are encoded as a boolean satisfiability problem, which is solved by a weighted Max-SAT solver. Milanova and Vitek [17] present a static dominance inference analysis, based on which they perform ownership type inference.

This work is closely related to our previous work on inference of ownership types [14], and reference immutability types [15]. All type systems are applications in our inference and checking framework.

6. Conclusions and Future Work

We presented an inference technique which infers AJ types for concurrent libraries. The technique reduces the number of alias annotations by approximately 90%. This result shows that our technique is feasible.

² We obtained the annotated library from Prof. Jan Vitek.

In the future we will expand our technique to infer types for whole programs in addition to libraries. Whole programs are harder than libraries because there is no easy way to assign defaults, the way we assign defaults in libraries. We will exploit opportunities for optimization due to internal aliasing and read-only. Our experience with ownership types suggests that there are many internally aliased objects and thus, significant opportunities for optimization.

In addition, our assumption that all fields belong to the single atomic set of a class may hinder concurrent access to data structures designed for sharing, e.g. immutable objects. We will improve the type system to address this restriction.

7. Acknowledgements

We thank Dr. Frank Tip, Dr. Mandana Vaziri and the anonymous FOOL reviewers for their detailed and extremely valuable comments on earlier versions of this paper.

References

- [1] M. Abadi, C. Flanagan, and S. N. Freund. Types for safe locking: Static race detection for Java. *ACM Transactions on Programming Languages and Systems*, 28(2):207–255, 2006.
- [2] J. Aldrich, V. Kostadinov, and C. Chambers. Alias annotations for program understanding. In *OOPSLA*, pages 311–330, 2002.
- [3] B. Chin, S. Markstrum, T. Millstein, and J. Palsberg. Inference of user-defined type qualifiers and qualifier rules. In *ESOP*, pages 264–278, 2006.
- [4] D. Clarke, J. M. Potter, and J. Noble. Ownership types for flexible alias protection. In *OOPSLA*, pages 48–64, 1998.
- [5] D. Cunningham, W. Dietl, S. Drossopoulou, A. Francalanza, P. Müller, and A. Summers. Universe types for topology and encapsulation. In *FMCO*, pages 72–112, 2008.
- [6] W. Dietl, M. D. Ernst, and P. Müller. Tunable static inference for generic universe types. In *ECOOP*, pages 333–357, 2011.
- [7] W. Dietl and P. Müller. Universes: Lightweight ownership for JML. *Journal of Object Technology*, 4(8):5–32, 2005.
- [8] W. Dietl and P. Müller. Runtime universe type inference. In *IWACO*, pages 72–80, 2007.
- [9] J. Dolby, C. Hammer, D. Marino, F. Tip, M. Vaziri, and J. Vitek. A data-centric approach to synchronization. *ACM Transactions on Programming Languages and Systems*, 34(1):1–48, Apr. 2012.
- [10] M. D. Ernst. Type Annotations specification (JSR 308). <http://types.cs.washington.edu/jsr308/>, 2012.
- [11] C. Flanagan and S. Qadeer. A type and effect system for atomicity. In *PLDI*, number 5, pages 338–349, 2003.
- [12] R. Fuhrer, F. Tip, A. Kiezun, J. Dolby, and M. Keller. Efficiently refactoring Java applications to use generic libraries. In *ECOOP*, pages 71–96, 2005.
- [13] D. Greenfieldboyce and J. S. Foster. Type qualifier inference for java. In *OOPSLA*, pages 321–336, 2007.
- [14] W. Huang, W. Dietl, A. Milanova, and M. D. Ernst. Inference and checking of object ownership. In *ECOOP*, pages 181–206, 2012.
- [15] W. Huang, A. Milanova, W. Dietl, and M. D. Ernst. ReIm & ReImInfer: Checking and inference of reference immutability and method purity. In *OOPSLA*, 2012.
- [16] K.-K. Ma and J. S. Foster. Inferring aliasing and encapsulation properties for java. In *OOPSLA*, pages 423–440, 2007.
- [17] A. Milanova and J. Vitek. Static dominance inference. In *TOOLS*, pages 211–227, 2011.
- [18] F. Tip, R. M. Fuhrer, A. Kiezun, M. D. Ernst, I. Balaban, and B. D. Sutter. Refactoring using type constraints. *ACM Transactions on Programming Languages and Systems*, 33(3):1–47, Apr. 2011.

Dataflow and Type-based Formulations for Reference Immutability

Ana Milanova Wei Huang

Rensselaer Polytechnic Institute
Troy, NY, USA

{milanova, huangw5}@cs.rpi.edu

Abstract

Reference immutability enforces the property that a reference cannot be used to mutate the referenced object. There are several type-based formulations for reference immutability in the literature. However, we are not aware of a dataflow formulation.

In this paper, we present a dataflow formulation for reference immutability using CFL-reachability, as well as a type-based formulation using viewpoint adaptation, a key concept in ownership types. We observe analogies between the dataflow formulation and the type-based formulation.

Categories and Subject Descriptors D.3.3 [Programming Languages]: Language Constructs and Features; D.1.5 [Programming Techniques]: Object-oriented Programming

General Terms Languages, Theory

1. Introduction

Reference immutability enforces the property that the state of an object, including its transitively reachable state, *cannot be mutated through an immutable reference*. Reference immutability is different from object immutability in that the former enforces constraints on references while the latter focuses on the object instance. For instance, in the following code, we cannot mutate the Date object by using the immutable reference rd, but we can mutate the same Date object through the mutable reference md:

```
Date md = new Date(); // mutable by default
readonly Date rd = md; // an immutable reference
md.setHours(1);        // OK, md is mutable
rd.setHours(1);        // error, rd is immutable
```

As a motivating example, consider the Class.getSigners method implemented in JDK 1.1.

```
class Class {
    private Object[] signers;
    Object[] getSigners() {
        return signers;
    }
}
```

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

FOOL '12 October 22, 2012, Tucson, AZ, USA.
Copyright © 2012 ACM [to be supplied]...\$10.00

This implementation is not safe because a malicious client can obtain and manipulate the signers array by invoking the getSigners method. A solution is to use reference immutability and annotate the return value of getSigners as readonly. (A readonly array of mutable objects is expressed, following Java 8 syntax [5], as Object readonly [].) As a result, mutations on the array after return will be disallowed:

```
Object readonly [] getSigners() {
    return signers;
}
...
Object readonly [] signers = getSigners();
signers[0] = null; // compile-time error
```

Reasoning about reference immutability has a number of benefits. It improves the expressiveness of interface design by specifying the immutability of parameters and return values; it helps prevent and it helps detect bugs or errors caused by unwanted object mutation; it facilitates reasoning about and proving other properties such as object immutability, method purity and object ownership. The problem has received abundant attention in the literature [1, 7, 12].

Reasoning about reference immutability entails partitioning the references in the program into three categories:

- **mutable**: A mutable reference can be used to mutate the referenced object. This is the implicit and only option in standard object-oriented languages.
- **readonly**: A readonly reference x cannot be used to mutate the referenced object nor anything it references. For example, all of the following are forbidden:
 - $x.f = z$
 - $x.setField(z)$ where $setField$ sets a field of its receiver
 - $y = id(x)$; $y.f = z$ where id is a function that returns its argument
 - $x.f.g = z$
 - $y = x.f$; $y.g = z$
- **polyread**: A polyread reference x cannot be used to mutate the object in the scope of the enclosing method m ; the object may be mutated after m returns. For example,
 - $x.f = y$ is not allowed, but
 - $z = id(y)$; $z.f = 0$, where id is polyread $X id(polyread X x) \{ return x; \}$, and z and y are mutable, is allowed.

polyread is useful because it allows for context sensitivity. Without polyread, the mutation of z in $z = id(y)$; $z.f = 0$ would force the formal parameter and return value of id to be mutable. Therefore, if id is called elsewhere, e.g., $z1 = id(y1)$,

$cd ::= \text{class } C \text{ extends } D \{ \overline{fd} \ \overline{md} \}$	<i>class</i>
$fd ::= t \ f$	<i>field</i>
$md ::= t \ m(t \ \text{this}, t \ x) \{ \overline{t} \ y \ s; \ \text{return } y \}$	<i>method</i>
$s ::= s; s \mid x = \text{new } t() \mid x = y$ $\mid x = y.f \mid x.f = y \mid x = y.m(z)$	<i>statement</i>
$t ::= q \ C$	<i>qualified type</i>
$q ::= \text{readonly} \mid \text{polyread} \mid \text{mutable}$	<i>qualifier</i>

Figure 1. Syntax. C and D are class names, f is a field name, m is a method name, and x , y , and z are names of local variables, formal parameters, or parameter `this`. As in the code examples, this is explicit. For simplicity, we assume all names are unique.

where $z1$ is readonly, the mutable formal parameter will cause $y1$ to be mutable, even though it is readonly.

Locals, parameters, and return variables in both instance and static methods can be polyread. Fields, either instance or static cannot be polyread. This essentially means that our reference immutability is context-sensitive in the call-transmitted dependences, but is approximate in the structure-transmitted (i.e., object-field-transmitted) dependences. This is necessitated by Reps’ undecidability result [11] which states that context-sensitive structure-transmitted data-dependence analysis is undecidable.

Reference immutability can be formulated in several ways. A dataflow formulation focuses on inference of mutable, readonly and polyread references. The goal is to prove as many references as readonly as possible. A type-based formulation provides a type system for reference immutability. It focuses on enforcement of immutability. Programmers specify readonly annotations on certain references and the system either proves the desired immutability or issues an error.

There are several type-based formulations of reference immutability in the literature, most notably Javari by Tschantz and Ernst [12], and more recently Relm by Huang et al. [7]. However, we are not aware of a dataflow formulation.

In this paper, we present a dataflow formulation of reference immutability using CFL-reachability. We argue the analogy between the dataflow and type-based formulations. In addition, we argue the analogy between context sensitivity in dataflow analysis and *viewpoint adaptation*, a key concept in ownership types [2–4]. The contributions of this paper are:

- A novel dataflow formulation of reference immutability using CFL-reachability.
- An observation on the analogy between context sensitivity in dataflow analysis and *viewpoint adaptation*.

The rest of the paper is organized as follows. Section 2 formalizes the program syntax. Section 3 describes the dataflow formulation of reference immutability. Section 4 describes the type-based formulation, argues the analogy with the dataflow one and describes the relationship between context sensitivity and *viewpoint adaptation*. Section 5 concludes the paper and outlines directions for future work.

2. Syntax

We restrict our formal attention to a core calculus in the style of Vaziri et al. [13] whose syntax appears in Figure 1. The language models Java with a syntax in a “named form”, where the results of field accesses, method calls, and instantiations are immediately stored in a variable. Without loss of generality, we assume that methods have parameter `this`, and exactly one other formal parameter.

Features not strictly necessary are omitted from the formalism, but they are handled correctly in the implementation. We write $\overline{t} \ y$ for a sequence of local variable declarations.

For the purposes of the type-based formulation, a type t has two orthogonal components: type qualifier q (which expresses reference immutability) and Java class type C . The immutability type system is *orthogonal* to (i.e., independent of) the Java type system, which allows us to specify typing rules over type qualifiers q alone. In the dataflow formulation a type t has only one component, the Java class type C .

3. Dataflow Formulation

We can formulate the problem as a CFL-reachability computation [11] over a directed graph G . The nodes in G are the *references* in the program, and the edges show the dependences between these references. The term *reference* denotes (1) fields, (2) local variables and formal parameters, (3) method return values, and (4) objects, denoted by allocation sites.

The goal of the analysis is to infer mutable, polyread and readonly references maximizing the number of readonly references and minimizing the number of mutable ones. Section 3.1 describes the construction of dependence graph G , Section 3.2 describes the CFL-reachability computation. Section 3.3 and Section 3.4 elaborate on the handling of context sensitivity.

3.1 Dependence Graph

The edges in dependence graph G are constructed according to the rules in Figure 2. Initially, the graph is empty. Each rule takes as input current graph G and adds edges to it to produce graph G' . Note that there is no need to iterate: each rule adds a constant set of edges, regardless of input G . Rule (ASSIGN) adds an edge from y , the right-hand-side of the assignment to x , the left-hand-side of the assignment. The edge expresses a dependence of y on x , i.e., that the mutability of x affects y . If x is mutated in statement $x.f = z$, i.e. x is mutable, then y is mutable as well, because x obtained its value through the assignment $x = y$. Rule (READ) creates dependences from reference y to left-hand-side x , and from field f to x . The intuition is that a mutation of x affects y , because x refers to parts of the y ’s object, and x obtained its value through y . The mutation of x affects f as well. Rule (WRITE) adds an edge from y to f .

Rule (CALL) demands a more detailed explanation. Auxiliary function $\text{target}(i)$ retrieves the compile-time target m at call site i , namely $\text{this}_m, p \rightarrow \text{ret}_m$. this_m is m ’s implicit parameter `this`, p is m ’s formal parameter, and ret_m is m ’s return variable. For simplicity, we assume no inheritance, that is, there is a single target at each call; again, the general case can be handled easily¹.

Rule (CALL) creates labeled *procedure entry edges* from actual receiver y to parameter this_m , and from actual argument z to formal parameter p . As it is customary in CFL-reachability, the label is an open parenthesis suffixed with the unique label at the call: $(i$. The rule creates labeled *procedure exit edges* from return variable ret_m to the left-hand-side x at the call assignment. The label is a close parenthesis suffixed with the label at the call: $)i$. The rule transmits dependences at method calls. The role of the labels is to transmit mutations only along valid paths and avoid polluting references as mutable when said references are readonly.

Consider the code example below. Recall that our syntax makes the receiver `this` an explicit parameter.

¹ We note that in Java, when there is no inheritance, method overloading does not lead to multiple targets. In Java, the compile-time target at the call is still decided at compile time, even in the presence of overloading. The run-time target is dispatched at run-time based on the type of the receiver. Under our simplifying assumption, there will be a single run-time receiver type, and therefore a single target.

(NEW)	$j: x = \text{new } C()$	$\Rightarrow$	$G' = G \cup j \rightarrow x$
(ASSIGN)	$x = y$	$\Rightarrow$	$G' = G \cup y \rightarrow x$
(READ)	$x = y.f$	$\Rightarrow$	$G' = G \cup y \rightarrow x \cup f \rightarrow x$
(WRITE)	$x.f = y$	$\Rightarrow$	$G' = G \cup y \rightarrow f$
(CALL)	$i: x = y.m(z)$	$\Rightarrow$	$\text{let } \text{this}_m, p \rightarrow \text{ret}_m = \text{target}(i) \text{ in}$ $G' = G \cup y \xrightarrow{(i)} \text{this}_m \cup z \xrightarrow{(i)} p \cup \text{ret}_m \xrightarrow{(i)} x$

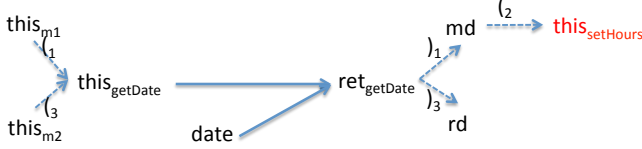
Figure 2. Rules for construction of G . Rules are defined over the "named form" syntax in Figure 1.

```

class DateCell {
  Date date;
  Date getDate(Date this) { return this.date; }
  void m1(Date this) {
    1: Date md = this.getDate();
    2: md.setHours(1); // md is mutated
  }
  void m2(Date this) {
    3: Date rd = this.getDate();
    4: int hour = rd.getHours();
  }
}

```

The dependence graph for this example is as follows:



Consider statement `return this.date;` as an example. It is treated as an assignment `ret = this.date`, and results in two edges: $\text{this}_{\text{getDate}} \rightarrow \text{ret}_{\text{getDate}}$ and $\text{date} \rightarrow \text{ret}_{\text{getDate}}$ (we denote reference variables by their name suffixed with the name of their enclosing method). Also, the call at 1 results in labeled procedure entry edge $\text{this}_{m1} \xrightarrow{(1)} \text{this}_{\text{getDate}}$ and labeled procedure exit edge $\text{ret}_{\text{getDate}} \xrightarrow{(1)} \text{md}$. The labeled edges are dashed in the graph.

We use the graph to propagate direct mutations, backwards, towards affected references. In the above example, only $\text{this}_{\text{setHours}}$ is mutated directly (the mutation is not shown in the code). $\text{this}_{\text{setHours}}$ is shown in red in the graph.

The mutation of $\text{this}_{\text{setHours}}$ makes `md` mutable. The mutation of `md` is transmitted via call

```
Date md = this.getDate();
```

back to this_{m1} . However, it should not be transmitted to this_{m2} , because the path from this_{m2} to `md` is not a valid path as we shall explain shortly in Section 3.2.

As a more involved example, consider the code in Figure 3 and its corresponding dependence graph in Figure 4. For the rest of this paper, we will use this code and its graph as a running example.

3.2 Reachability Computation

This section describes the CFL-reachability computation.

A path $x \rightarrow^* y$ in G is a *same-level path* from x to y if all procedure exits *match* the corresponding procedure entries. More formally, path $x \rightarrow^* y$ is a same-level path the labels on its edges form a well-formed string in the language defined by the following context-free grammar:

$$SLP \rightarrow (i \ SLP)_i \mid SLP \ SLP \mid \epsilon$$

For example, path $\text{this}_{\text{get}} \xrightarrow{(1)} \text{this}_{\text{getX}} \rightarrow \text{x}_{\text{getX}} \rightarrow \text{ret}_{\text{getX}} \xrightarrow{(1)} \text{x}_{\text{get}}$ is a same-level path from this_{get} to x_{get} .

```

class A {
  X f;
  X get(Y y) {
    ... = y.h;
    1: X x = this.getX();
    return x;
  }
  X getX() {
    X x = this.f;
    return x;
  }
}

void m1() {
  A a = ...
  Y y = ...
  2: X x = a.get(y);
  x.g = null;
}

void m2() {
  A a = ...
  Y y = ...
  3: X x = a.get(y);
  ... = x.g;
}

```

Figure 3. Example program

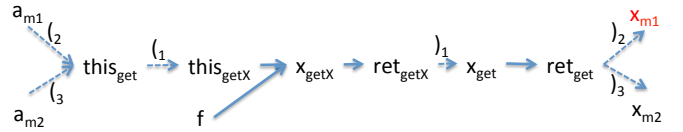


Figure 4. Dependence graph for example program. Labeled edges are dashed. Directly mutated variable x in `m1` is shown in red.

A path $x \rightarrow^* y$ in G is a *call path* from x to y if all the procedure exits *match* the procedure entries but it is possible that some procedures are entered and not yet exited. The following grammar defines same-level paths:

$$CP \rightarrow (i \ SLP \mid CP \ CP \mid SLP$$

For example, path $a_{m1} \xrightarrow{(2)} \text{this}_{\text{get}} \xrightarrow{(1)} \text{this}_{\text{getX}} \rightarrow \text{x}_{\text{getX}} \rightarrow \text{ret}_{\text{getX}} \xrightarrow{(1)} \text{x}_{\text{get}}$ is a call path. Note that a same-level path is also a call path.

Similarly, a path $x \rightarrow^* z$ in G is a *return path* from x to z if procedure exits *match* the procedure entries but there are at least some procedure exits whose corresponding entries are not on the path. The grammar that describes return paths is as follows:

$$RP \rightarrow SLP)_i \ SLP \mid RP \ RP$$

For example, path $\text{this}_{\text{get}} \xrightarrow{(1)} \text{this}_{\text{getX}} \rightarrow \text{x}_{\text{getX}} \rightarrow \text{ret}_{\text{getX}} \xrightarrow{(1)} \text{x}_{\text{get}} \rightarrow \text{ret}_{\text{get}} \xrightarrow{(2)} \text{x}_{m1}$ is a return path from this_{get} to x_{m1} . Note that a same-level path is not a return path.

We require that nodes on same-level, call and return paths *cannot be fields*. Dependences transmitted through fields are special and will be explained shortly.

The computation of reference immutability is as follows:

- Reference x in $x.f = y$ is marked mutable. Clearly, if x is the receiver in a field write $x.f = y$, then x must be mutable.
- Reference x is marked mutable if there exists a call path $x \rightarrow^* y$ in G such that y is marked mutable. In our running example (Figure 3 and Figure 4), a_{m1} is mutable because there is a call path from a_{m1} to x_{m1} and x_{m1} is mutable. Note that in this case, the call path is a same-level path. Reference a_{m2} is not mutable however, because there is no call path to the mutable

x_{m1} ; there is a path of course, but it is not a valid call path because procedure entries and exits do not match.

- Reference x is marked polyread if there exist a return path $x \rightarrow^* z$ in G such that z is mutable. A reference can be marked as both mutable and polyread. A reference variable is marked as polyread when a mutation is reached after the return of the variable's enclosing method. For example, x_{getX} is polyread because x_{m1} , which is mutated, is reachable on a return path. Intuitively, x_{getX} is polymorphic. It is not mutated in its enclosing method `getX`. However, the object it refers to is mutated in one of the contexts of invocation of `getX`, after the return of `getX`; the object is not mutated in the other context.
- Field f is marked mutable if there is an edge $f \rightarrow y$ in G such that y is marked mutable or polyread. A field f is mutable if it is assigned to a mutable or polyread reference. As mentioned in the introduction, a field cannot be polyread. We elaborate on this restriction shortly.
- Reference x is marked mutable if there is an edge $x \rightarrow f$ in G such that field f is marked mutable. This rule marks a reference as mutable, if it is assigned to a mutable field.

The computation applies the above rules repeatedly, until it reaches a fixpoint — that is, no more references are marked mutable or polyread. At the end, references marked as mutable are inferred as mutable. References marked as polyread but not mutable, are inferred as polyread. The remaining references are inferred as readonly.

The final result in our running example is the following:

mutable: a_{m1} , x_{m1} and f .

polyread: this_{get} , ret_{get} , x_{get} , $\text{this}_{\text{getX}}$, ret_{getX} , x_{getX} .

readonly: a_{m2} and x_{m2} .

3.3 Call-transmitted Dependences

At this point, readers have noticed that our analysis is context-sensitive in the call-transmitted dependences. Clearly, it follows only valid call and return paths (i.e., paths with matching procedure entry and procedure exit edges). As mentioned earlier, the analysis does not propagate the mutation at x_{m1} back to a_{m2} because the path from a_{m2} to x_{m1} is not a valid call path. As a result, a_{m2} can be proven readonly.

3.4 Structure-transmitted Dependences

Structure-transmitted dependences are dependences that arise due to flow through object fields. Readers have likely noticed that our analysis is *approximate* in the structure-transmitted dependences. It merges the mutability of fields across all objects (recall that fields are either mutable or readonly but not polyread). In other words, the analysis handles imprecisely the case when there are two different objects of the same class, where one object has a mutable f field, but the other object has a readonly f field. Consider the example:

```
x = new C(); i
x.f = new D();
y = x.f;
y.g = 0;
...
x2 = new C(); j
x2.f = y2;
```

The mutation of y will be propagated through f to $y2$ and $y2$ will be inferred as mutable even though it is not mutated. This is because the analysis cannot distinguish that the x in $y = x.f$ and the $x2$ in $x2.f = y2$ refer to two distinct objects, i and j , and therefore the mutation of y cannot affect $y2$ at runtime.

The approximation is necessitated by Reps' undecidability result, which states that context-sensitive structure-transmitted data-dependence analysis is undecidable [11]. Thus, analysis designers must approximate in at least one of the two dimensions, at calls or at fields, and there is a wide variety of ways to approximate. In the analysis above, we use a straightforward approximation where every object (i.e., structure) is abstracted by its class. A more precise approximation is to abstract an object by its allocation site, an even more precise one is to use a combination of the object's allocation site and the allocation site of its creating object, and so on.

As an example, if objects are distinguished by their allocation sites, there we will use i to abstract the first Cobject and j to abstract the second. When constructing graph G we will create an edge $i.f \rightarrow y$ at field read $y = x.f$ and an edge $y2 \rightarrow j.f$ at field write $x2.f = y2$. Thus, the write of y will not propagate to $y2$.

4. Type-based Formulation

The type-based formulation of reference immutability uses the same type qualifiers with the same meaning. As with the dataflow formulation polyread cannot be applied to fields. The subtyping relation between the qualifiers is

mutable <: polyread <: readonly

Thus, it is allowed to assign a mutable reference to a polyread or readonly one, but it is not allowed to assign a readonly reference to a polyread or mutable one.

In previous work we presented a type system for reference immutability called Relm [7]. The type system presented in this paper, which we call Relm', differs slightly from Relm. Relm' better illustrates the analogy between the dataflow formulation and the type based formulation. We will elaborate on the differences shortly. Relm' is not a contribution over Relm.

4.1 Viewpoint Adaptation

Viewpoint adaptation is a concept from Universe types [3, 4], which applies to other ownership and ownership-like type systems as well [2, 13]. For example, the type of $x.f$ is not just the declared type of field f — it is the type of f adapted from the point of view of x . For example, in Universe types, $\text{rep } x$ denotes that the current this object is the owner of the object o_x referenced by x . If field f has type peer , this means that the object o_x and the object o_f referenced by field f have the same owner. Thus, the type of $x.f$, or the type of f adapted from the point of view of x , is rep — the object o_f 's owner is the current this object as well.

Ownership type systems make use of a *single* viewpoint adaptation operation. This viewpoint adaptation operation is performed at both field accesses and method calls. It is written $q \triangleright q'$, which denotes that type q' is adapted from the point of view of type q to the viewpoint of the current object this. Viewpoint adaptation adapts the type of a field, formal parameter, or return type, from the viewpoint of the *receiver* at the corresponding field access or method call to the viewpoint of the current object this. In other words, the context of adaptation at both field access and method call, is the receiver object.

One key point of this paper is to illustrate and explore the interesting relationship between context sensitivity in dataflow analysis and viewpoint adaptation. We argue that the role of viewpoint adaptation is to transmit dependences at structures (by adapting fields), and at calls (by adapting formal parameters and return types).

In this spirit, we propose a generalization of traditional viewpoint adaptation. First, we allow for two different viewpoint adaptation operations, one applied at fields, and the other one applied at calls. Effectively, this separates the handling of dependences at fields, from the handling of dependences at calls. Second, we allow for

$$\begin{array}{c}
\text{(TNEW)} \\
\frac{\Gamma(x) = q_x \quad q <: q_x}{\Gamma \vdash x = \text{new } q \text{ } C()} \\
\\
\text{(TASSIGN)} \\
\frac{\Gamma(x) = q_x \quad \Gamma(y) = q_y \quad q_y <: q_x}{\Gamma \vdash x = y} \\
\\
\text{(TWRITE)} \\
\frac{\Gamma(x) = \text{mutable} \quad \text{typeof}(f) = q_f \quad \Gamma(y) = q_y \quad q_y <: \text{mutable} \triangleright_f q_f}{\Gamma \vdash x.f = y} \\
\\
\text{(TREAD)} \\
\frac{\Gamma(y) = q_y \quad \text{typeof}(f) = q_f \quad \Gamma(x) = q_x \quad q_y \triangleright_f q_f <: q_x}{\Gamma \vdash x = y.f} \\
\\
\text{(TCALL)} \\
\frac{\Gamma(y) = q_y \quad \text{typeof}(m) = q_{\text{this}}, q \rightarrow q_{\text{ret}}, m \quad \Gamma(x) = q_x \quad \Gamma(z) = q_z \quad q_y <: q_x \triangleright_m q_{\text{this}}, q_z <: q_x \triangleright_m q \quad q_x \triangleright_m q_{\text{ret}}, q <: q_x}{\Gamma \vdash x = y.m(z)}
\end{array}$$

Figure 5. Typing rules. Function *typeof* retrieves the immutability types of fields and methods. Γ is a type environment that maps references to immutability qualifiers.

adaptation from different viewpoints, not only from the viewpoint of the receiver. This allows for different kinds of context sensitivity.

We now return to reference immutability and explain the viewpoint adaptation that it needs.

Viewpoint adaptation operation $q \triangleright_f q_f$ is applied at field accesses. It adapts field type q_f from the point of view of receiver type q . We define $\triangleright_f$ for field access:

$$\begin{array}{ll}
_ \triangleright_f \text{readonly} &= \text{readonly} \\
q \triangleright_f \text{mutable} &= q
\end{array}$$

The underscore denotes a “don’t care” value. Consider field access $y.f$. If the type of receiver y is *readonly* and the type of field f is *mutable*, then the type of $y.f$ is *readonly* $\triangleright_f \text{mutable} = \text{readonly}$. A field access $y.f$ is *mutable* if and only if both the receiver y and field f are *mutable*. If the receiver or the field is *readonly*, $y.f$ is *readonly*. It is important to note that the adapted type at $y.f$ is the least upper bound of the types of y and f .

Viewpoint adaptation operation $q_x \triangleright_m q$ is applied at method calls $x = y.m(z)$. It adapts q , the type of a formal parameter/return value of m , from the point of view of q_x , the *context* of the call. $\triangleright_m$ is defined as follows:

$$\begin{array}{ll}
_ \triangleright_m \text{mutable} &= \text{mutable} \\
_ \triangleright_m \text{readonly} &= \text{readonly} \\
q \triangleright_m \text{polyread} &= q
\end{array}$$

If a formal parameter/return value is *readonly* or *mutable*, its adapted value remains the same regardless of q_x . However, if q is *polyread*, the adapted value depends on q_x — it becomes q_x (i.e., the *polyread* type is instantiated to q_x).

4.2 Typing Rules

The typing rules are presented in Figure 5. Rule (TASSIGN) is straightforward. They require that the left-hand-side is a supertype of the right-hand-side. Observe the analogy with the dataflow formulation: rule (ASSIGN) creates edge $y \rightarrow x$ in G . More generally,

we conjecture that we have $y <: x$, including transitive subtyping, if and only if there is a same-level path from y to x in G .

Rule (TWRITE) requires $\Gamma(x)$ to be *mutable* because x ’s field is updated in the statement. The viewpoint adaptation operation for field access is used in both (TWRITE) and (TREAD). Intuitively, $\triangleright_f$ combined with rules (TWRITE) and (TREAD) handles structure-transmitted dependences, in the same fashion, as the edges through fields f in dependence graph G do. Consider a field write $x.f = y$ and a field read $z = w.f$. Rule (TWRITE) enforces $q_y <: q_f$ and (TREAD) enforces $q_f <: q_z$. Thus, a mutation on z will force f to be *mutable* and this in turn will force y to be *mutable*. This is analogous to the dataflow formulation in Section 3. $x.f = y$ results in edge $y \rightarrow f$ in G and $z = w.f$ results in edge $f \rightarrow z$. A mutation on z forces f to be *mutable*, and the *mutable* f forces y to be *mutable* as well. The handling of structure-transmitted dependences in the type-based formulation is analogous to the handling in the dataflow formulation.

Rule (TCALL) handles calls and demands detailed explanation. This rule, along with $\triangleright_m$ handles call-transmitted dependences. Function *typeof* retrieves the type of m . q_{this} is the type of implicit parameter *this*, q is the type of the formal parameter, and q_{ret} is the type of the return value. Rule (TCALL) requires $q_x \triangleright_m q_{\text{ret}} <: q_x$. This constraint disallows the return value of m from being *readonly* when there is a call to m , $x = y.m(z)$, where left-hand-side x of the assignment is *mutable*. Only if the left-hand-sides of all call assignments to m are *readonly*, can the return type of m be *readonly*; otherwise, it is *polyread*. A programmer can annotate the return type of m as *mutable*. However, this typing is pointless, because it unnecessarily forces local variables and parameters in m to become *mutable* when they can be *polyread*.

In addition, the rule requires $q_y <: q_x \triangleright_m q_{\text{this}}$. When q_{this} is *readonly* or *mutable*, its adapted value is the same according to the adaptation rules of $\triangleright_m$. Thus, when q_{this} is *mutable* (due to *this.f* = 0 in m , for example),

$$q_y <: q_x \triangleright_m q_{\text{this}} \text{ becomes } q_y <: \text{mutable}$$

which disallows q_y from being anything but *mutable*, as expected. In Section 3 this is handled by call paths. In the case described above, there is a call path $y \xrightarrow{c_i}$ *this* which forces y to be *mutable*.

The most interesting case arises when q_{this} is *polyread*. A *polyread* parameter *this* is *readonly* within the enclosing method, but there could be a dependence between *this* and *ret* such as

$$X \ m() \{ z = \text{this}.f; w = z.g; \text{return } w; \}$$

Thus, the *this* object can be modified in caller context, after m ’s return. Well-formedness in Relm' guarantees that whenever there is dependence between *this* and *ret*, as in the above example, the following subtyping constraint holds:

$$q_{\text{this}} <: q_{\text{ret}}$$

Recall that when there exists a context where the left-hand-side of the call assignment x is mutated, q_{ret} must be *polyread*. Therefore, constraint $q_{\text{this}} <: q_{\text{ret}}$ forces q_{this} to be *polyread* (let us assume that *this* is not mutated in its enclosing method).

The role of viewpoint adaptation is to transfer the dependence between *this* and *ret* in m , into a dependence between actual receiver y and left-hand-side x in the call assignment.

In the above example, there is a dependence between *this* and the return *ret*. Thus, we also have a dependence between y and x in the call $x = y.m()$ — that is, a mutation of x makes y *mutable* as well. Function $\triangleright_m$ does exactly that. Rule (TCALL) requires

$$q_y <: q_x \triangleright_m q_{\text{this}}$$

When there is a dependence between this and ret, q_{this} is polyread, and the above constraint becomes

$$q_y <: q_x$$

This is exactly the constraint we need. If x is mutated, y becomes mutable as well. In contrast, if x is readonly, y remains unconstrained.

Note the analogy with the analysis in Section 3. In the example

```
X m() { z = this.f; w = z.g; return w; }
```

there is a same-level path between this and ret. Thus, call $x = y.m()$ generates a same-level path from y to x (the entry and exit parentheses will balance out) and a mutation of x propagates to y along this path. Again, as with structure-transmitted dependences, the handling of call-transmitted dependences in the type-based formulation is analogous to the handling in the dataflow formulation. Viewpoint adaptation helps achieve the desired behavior.

The typed DateCell class from Section 3 is as follows.

```
class DateCell {
  mutable Date date;
  polyread Date getDate(polyread Date this) { return this.date; }
  void m1(mutable Date this) {
    mutable Date md = this.getDate();
    md.setHours(1); // md is mutated
  }
  void m2(readonly Date this) {
    readonly Date rd = this.getDate();
    int hour = rd.getHours();
  }
}
```

Field date is mutable because it is mutated indirectly in method m1. Because the return value of getDate is polyread, it is instantiated to mutable in m1 as follows:

$$q_{md} \triangleright_m q_{ret} = \text{mutable} \triangleright_m \text{polyread} = \text{mutable}$$

It is instantiated to readonly in m2:

$$q_{rd} \triangleright_m q_{ret} = \text{readonly} \triangleright_m \text{polyread} = \text{readonly}$$

Thus, this_{m2} can be typed readonly.

We conclude this section with a brief discussion. Allowing for adaptation from *different viewpoints*, not only from the point of view of the receiver, enables different kinds of context sensitivity. For example, adapting from the viewpoint of the receiver, as it is customary in ownership type systems, can be interpreted as *object sensitivity* [9]. Adapting from the viewpoint of the context of invocation, as it is necessary for reference immutability, can be interpreted as *call-site context sensitivity*.

Differentiation of viewpoint adaptation at fields from viewpoint adaptation at methods allows us to implement the handling of structure-transmitted dependences *differently* from the handling of the call-transmitted dependences. For reference immutability, we handled transmission through fields approximately by merging flow through a field across all objects. We handled transmission through calls precisely, by matching calls and returns. We envision further generalization, where one can implement other, more interesting approximations.

The difference between Relm' and Relm is that Relm allows only readonly and polyread fields, where a readonly field has the same semantics as a readonly field in Relm', and a polyread field has exactly the same semantics as a mutable field in Relm'. Relm uses a single viewpoint adaptation operation applied at both field accesses and method calls:

$$\begin{aligned} _ \triangleright \text{mutable} &= \text{mutable} \\ _ \triangleright \text{readonly} &= \text{readonly} \\ q \triangleright \text{polyread} &= q \end{aligned}$$

instead of the two different operations in Relm'. Relm treats context as in Relm', the context of adaptation at field access is the receiver, and at method calls is the left-hand-side of the call assignment.

We chose to use separate the viewpoint adaptation operations in Relm' in order to emphasize the analogy with dataflow analysis (even though reference immutability can be formulated using a single operation as in Relm). Separate operations differentiate the handling of dependences at field access from dependences at method calls. Thus, we used two separate operations: $\triangleright_f$ is used to handle transmission of dependences at field access, and $\triangleright_m$ is used to handle transmission at calls. These operations can be instantiated in different ways in order to accommodate different approximations in data transmission. In the future, we plan to investigate different kinds of approximations and their handling using viewpoint adaptation.

4.3 Type Inference

In previous work [6, 7] we propose an inference algorithm, which infers the “best” (i.e., most desirable) typing for reference immutability. Roughly, this is the typing with a maximal number of readonly references and a minimal number of mutable references. We conjecture that when there are no programmer-provided annotations, this “best” typing is equivalent to the inference result obtained by the dataflow formulation in Section 3.

The analogy between the dataflow formulation and the type-based formulation is interesting because we can use dataflow (CFL-reachability) machinery to study and solve the type inference problem, or conversely, we can use type inference to solve the dataflow problem. The complexity bound of CFL-reachability in the general case is $O(|N \cup T|^3 n^3)$, where N is the set of nonterminals in the CFG grammar, T is the set of terminals in the grammar and n is the number of nodes in G , or roughly the number of reference variables in the program. For the special case of context-free language that we consider here, known as *Dyck language*, the bound can be improved to $O(kn^3)$ [8] where k is the number of different kinds of parentheses, or roughly, the number of call sites in the program. Thus, the complexity of the CFL-reachability formulation for reference immutability is $O(S^4)$ where S is the size of the program. Interestingly, the complexity of type inference for the type-based formulation is $O(S^2)$. Our hope is that other dataflow analyses can be found analogous to type inference, and therefore, type inference machinery can solve these problems.

5. Conclusions and Future Work

We have described a dataflow formulation of reference immutability using CFL-reachability, and an analogy between this dataflow formulation and a type-based formulation. We believe that the analogy between context sensitivity in dataflow analysis, and viewpoint adaptation in ownership types is a promising direction of future research.

In future work, we will continue to study the relationship between context sensitivity and viewpoint adaptation and more generally, the relationship between context-sensitive dataflow analysis (e.g., slicing, points-to analysis) and context-sensitive type systems such as Relm, Universe Types, Ownership types, and others. We conjecture that problems in dataflow analysis (e.g., points-to analysis) can be formulated as type-based analysis and solved using type inference. We plan to explore this direction in the future.

Acknowledgments

We thank the anonymous FOOL reviewers for their valuable comments on this paper.

References

- [1] S. Artzi, A. Kiezun, J. Quinonez, and M. D. Ernst. Parameter reference immutability: formal definition, inference tool, and comparison. *Automated Software Engineering*, 16(1):145–192, Dec. 2009.
- [2] D. Clarke, J. M. Potter, and J. Noble. Ownership types for flexible alias protection. In *OOPSLA*, pages 48–64, 1998.
- [3] D. Cunningham, W. Dietl, S. Drossopoulou, A. Francalanza, P. Müller, and A. J. Summers. Universe Types for topology and encapsulation. In *FMO*, 2008.
- [4] W. Dietl and P. Müller. Universes: Lightweight ownership for JML. *Journal of Object Technology*, 4:5–32, 2005.
- [5] M. D. Ernst. Type Annotations specification (JSR 308). <http://types.cs.washington.edu/jsr308/>, July 3, 2012.
- [6] W. Huang, W. Dietl, A. Milanova, and M. D. Ernst. Inference and checking of object ownership. In *ECOOP*, pages 181–206, 2012.
- [7] W. Huang, A. Milanova, W. Dietl, and M. D. Ernst. ReIm and ReImInfer: Checking and inference of reference immutability and method purity. In *OOPSLA*, 2012.
- [8] J. Kodumal and A. Aiken. The set constraint/cfl reachability connection in practice. In *PLDI*, pages 207–218, 2004.
- [9] A. Milanova, A. Rountev, and B. G. Ryder. Parameterized object sensitivity for points-to analysis for Java. *ACM Transactions on Software Engineering and Methodology*, 14(1):1–41, Jan. 2005.
- [10] J. Quinonez, M. S. Tschantz, and M. D. Ernst. Inference of reference immutability. In *ECOOP*, pages 616–641, 2008.
- [11] T. Reps. Undecidability of context-sensitive data-independence analysis. *ACM Transactions on Programming Languages and Systems*, 22: 162–186, 2000.
- [12] M. S. Tschantz and M. D. Ernst. Javari: Adding reference immutability to Java. In *OOPSLA*, pages 211–230, 2005.
- [13] M. Vaziri, F. Tip, J. Dolby, C. Hammer, and J. Vitek. A type system for data-centric synchronization. In *ECOOP*, pages 304–328, 2010.

SAFE: Formal Specification and Implementation of a Scalable Analysis Framework for ECMAScript

Hongki Lee

KAIST

petitkan@kaist.ac.kr

Sooncheol Won

KAIST

wonsch@kaist.ac.kr

Joonho Jin

KAIST

myfriend12@kaist.ac.kr

Junhee Cho

KAIST

ssaljaluk@kaist.ac.kr

Sukyoung Ryu

KAIST

sryu.cs@kaist.ac.kr

Abstract

The prevalent uses of JavaScript in web programming have revealed security vulnerability issues of JavaScript applications, which emphasizes the need for JavaScript analyzers to detect such issues. Recently, researchers have proposed several analyzers of JavaScript programs and some web service companies have developed various JavaScript engines. However, unfortunately, most of the tools are not documented well, thus it is very hard to understand and modify them. Or, such tools are often not open to the public.

In this paper, we present formal specification and implementation of SAFE, a scalable analysis framework for ECMAScript, developed for the JavaScript research community. This is the very first attempt to provide both formal specification and its open-source implementation for JavaScript, compared to the existing approaches focused on only one of them. To make it more amenable for other researchers to use our framework, we formally define three kinds of intermediate representations for JavaScript used in the framework, and we provide formal specifications of translations between them. To be adaptable for adventurous future research including modifications in the original JavaScript syntax, we actively use open-source tools to automatically generate parsers and some intermediate representations. To support a variety of program analyses in various compilation phases, we design the framework to be as flexible, scalable, and pluggable as possible. Finally, our framework is publicly available, and some collaborative research using the framework are in progress.

Categories and Subject Descriptors D.3.3 [Programming Languages]: Language Constructs and Features

General Terms Languages, Formalization

Keywords JavaScript, ECMAScript 5.0, formal semantics, formal specification, compiler, interpreter

1. Introduction

JavaScript is now the language of choice for client-side web programming, which enables dynamic interactions between users and web pages. By embedding JavaScript code that use event handlers such as `onMouseOver` and `onClick`, static HTML web pages become “Dynamic HTML” [12] web pages. JavaScript is originally developed in Netscape, released in the Netscape Navigator 2.0 browser under the name LiveScript in September 1995, and renamed as JavaScript in December 1995. After Microsoft releases

```
1 function Wheel4() { this.wheel = 4 }
2 function Car() { this.maxspeed = 200 }
3 Car.prototype = new Wheel4;
4 var modernCar = new Car;
5
6 var beforeModern =
7     modernCar instanceof Car; // true
8
9 function Wheel6() { this.wheel = 6 }
10 Car.prototype = new Wheel6;
11 var afterModern =
12     modernCar instanceof Car; // false
13 var truck = new Car;
14 var aftertruck =
15     truck instanceof Car; // true
```

Figure 1. Unintuitive behavior of JavaScript prototypes

its own implementation of the language, JScript, in the Internet Explorer 3.0 browser in 1996, Ecma International develops the standardized version of the language named ECMAScript [8, 9]. JavaScript was first envisioned as a simple scripting language, but with the advent of Dynamic HTML, Web 2.0 [28], and most recently HTML5 [1], JavaScript is now being used on a much larger scale than intended. All the top 100 most popular web sites according to the Alexa list [2] use JavaScript and its use outside web pages is rapidly growing.

As Brendan Eich, the inventor of JavaScript, says [7]:

“Dynamic languages are popular in large part because programmers can keep types latent in the code, with type checking done *imperfectly* (yet often more quickly and expressively) in the programmers’ heads and unit tests, and therefore programmers can do more with less code writing in a dynamic language than they could using a static language.”

By sacrificing strong static checking, JavaScript enjoys aggressively dynamic features such as run-time code generation using `eval` and dynamic scoping using `with`. In addition, JavaScript provides quite different semantics from conventional programming languages like C [22] and Java [4]. For example, JavaScript allows programmers to use variables and functions before defining them, and to assign values to new properties of an object even before declaring them in the object. Also, JavaScript allows users to access the global object of a web page via interactions with the DOM (Document Object Model) without requiring any permissions. JavaScript provides “prototype-based” inheritance instead of classes.

Consider the code example in Figure 1. Unlike conventional programming languages, the inheritance hierarchy may be changed after creation of objects. When `modernCar` is constructed at line 4, it is an instance of the `Car` object. However, because the prototype

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

FOOL '12 October 22, 2012, Tucson, AZ, USA.

Copyright © 2012 ACM [to be supplied]...\$10.00

of `Car` is changed from `Wheel4` to `Wheel6` at line 10, `modernCar` is not an instance of `Car` any more at line 12. In JavaScript, when some properties of a constructor change, the objects constructed before and after the change may be considered different instances even though they are constructed from the same constructor.

Due to such quirky semantics, understanding and analyzing JavaScript applications are well known to be difficult, and they are often targets of security attacks [19]. Because of the crude control by the same-origin policy of HTML, once a web page trusts third-party code it permits subsequent contents from the same origin, which often allows malicious scripts to sneak in. Such code injections can easily allow attackers to get high access permissions to secure contents including session cookies and unprotected personal information. This security problem known as XSS (cross-site scripting) shows up often in web pages and web applications. To resolve the problem, web service companies have developed several defense mechanisms such as cookie-based security policies and filtering out string inputs that may contain malicious scripts, but their functionalities are very limited. More robust approach might be using a presumably *safe subset* of JavaScript: Yahoo! ADsafe [6], Facebook FBJS [10], and Google Caja [13]. While they are intended to be safe subsets of JavaScript, none of them has been shown safe. Rather, researchers have reported security vulnerabilities with ADsafe and FBJS [24, 25, 31].

Clearly, better analyses of JavaScript applications for developing more reliable programs become indispensable. As more fundamental solutions to the security vulnerability problems of JavaScript, researchers recently have proposed formal specifications [11, 17, 23], type systems [3, 18, 33], static analyses [16, 20], and combinations of static and dynamic analyses [5, 24] for JavaScript. Web-service companies also have fertilized the JavaScript research community by open sourcing their JavaScript engines such as Rhino [26] and SpiderMonkey [27] from Mozilla, and V8 [14] from Google. While each of them contributes various aspects to solve the problem of JavaScript vulnerability issues, they are yet unsatisfactory in several reasons. First, most of them do not have a well-defined specification or a document to describe them; it is hard for other researchers to understand them and utilize them for their own further research. Secondly, they are not designed and developed for general research but often tightly coupled with their underlying browsers; it is quite challenging to integrate new ideas and new analyzers to existing systems. Thirdly, it is almost impossible to change or extend the existing implementations: most of them do not have any implementations yet, they do not make the implementations available to the public, or the design of the hand-written parsers and Abstract Syntax Tree (AST) nodes are not well-suited to extension and experimentation for researchers since they are with full of undocumented optimizations. Finally, even though the 5th edition of ECMA-262 [9] is released in 2009, most of them deal with the 3rd edition of ECMA-262 [8] released in 1999.

In this paper, we present formal specification and implementation of SAFE (Scalable Analysis Framework for ECMAScript) [30], developed for the JavaScript research community. Based on our own struggles and experiences, the first principles of our framework are formal specification, flexible, scalable, and pluggable framework design, open-source implementation, and aggressive use of various tools for automatic generation. Unlike most of the existing approaches, our framework deals with the 5th edition of ECMA-262 (hereafter called the ECMAScript specification). To help other researchers to understand our framework more easily, we specify every intermediate representation used in the framework formally, and we try to narrow the gaps between the specification and the corresponding implementation. To allow adventurous research ideas to be realized on top of our framework, we use automatically gen-

erated parsers and AST nodes from high-level, brief descriptions thanks to various third-party open-source tools. To support a variety of analyses on various compilation phases, we provide three levels of intermediate representations and well-defined translation mechanisms between them. Using SAFE, some collaborative research on JavaScript such as clone detection and code structure analysis are in progress with both academia and industry.

In short, our contributions are as follows:

- SAFE is the very first attempt to support both formal specification and its implementation for JavaScript.
- SAFE is based on the 5th edition of the ECMAScript specification.
- SAFE formally defines every intermediate representation used in the framework and provides formal specifications of the translations between them.
- SAFE describes the formal semantics of its Intermediate Representation (IR) with the descriptions of the corresponding language constructs in the ECMAScript specification.
- SAFE consists of formally defined components that are adequate for pluggable analysis extensions.
- SAFE makes its implementation available to the public for the research community:

<http://plrg.kaist.ac.kr/research/safe>

2. SAFE

Before describing the formal specification and the implementation of SAFE in detail in Sections 3 and 4, we describe the motivation of our work and a big picture of the framework.

2.1 Motivation

We encountered several obstacles while using existing tools in our previous research. Recently, we have worked on JavaScript-related topics: 1) adding modules to the existing JavaScript language via desugaring [21] and 2) removing the `with` statement in JavaScript applications [29]. For 1), we designed a module system for JavaScript and devised a desugaring mechanism from JavaScript extended with the module system to a slightly modified λ_{JS} [17]. Following the tradition of λ_{JS} , we extended the implementation of λ_{JS} and its desugaring mechanism to handle our module system. We have been very grateful for the authors to open source their implementation but the paper does not describe the desugaring process in detail, the implementation in multiple languages including Haskell and Scheme is not well documented, and the big semantic gap between JavaScript and λ_{JS} is not helpful to reason about the original JavaScript applications. For 2), we tried three open-source JavaScript parsers and engines: `PluginForJS`¹ in C#, `Caja`² in Java, and `Rhino` in Java. `PluginForJS` does not cover the entire JavaScript language, `Caja` supports a dialect of JavaScript, and `Rhino` uses a hand-written parser with undocumented optimizations and a set of simplified AST nodes. Finally, all of them deal with the 3rd edition instead of the 5th edition of the ECMAScript specification.

Based on our own struggles, we design and develop SAFE, a scalable ECMAScript analysis framework for the JavaScript research community. We present formal specifications of intermediate representations and translations between them for other researchers to understand our framework as easily and quickly as possible. Many parts of the formal specifications of SAFE describe

¹<https://jslexerparser.codeplex.com>

²<http://code.google.com/p/google-caja>

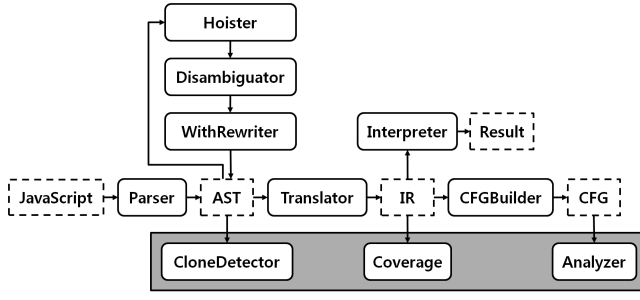


Figure 2. SAFE flow graph

the corresponding sections in the ECMAScript specification to help the readers to consult with the specification. To allow aggressive modifications even on the syntax of JavaScript, we actively use automated generation tools such as Rats! [15] for parsers and AST-Gen [32] for intermediate representations. We make our framework open to the public so that any JavaScript research groups can save their work on developing a series of routine compilation phases. At the same time, the framework is modularly designed and developed so that new research ideas can be easily realized and tested by developing a pluggable module on top of our framework.

2.2 Big Picture

Figure 2 describes the overall structure of SAFE. Dashed boxes denote data and solid boxes denote modules that transform data. The framework takes a JavaScript program; Parser parses the program and translates it into an AST; a series of compilation phases—Hoister, Disambiguator, and WithRewriter—transforms an AST to a simplified version in AST to make it easier to analyze and evaluate in later phases; Translator translates an AST into yet another intermediate representation, Intermediate Representation (IR); finally, Interpreter evaluates an IR and produces a result, or CFGBuilder constructs a Control Flow Graph (CFG) from an IR to analyze the program. As we describe in later sections, AST, IR, CFG, Translator, and CFGBuilder are formally specified and their implementations are publicly available.

The shaded box shows additional pluggable components to the framework. Taking advantage of our framework, several collaborative research with academia and industry are in progress: CloneDetector detects possible clones among multiple JavaScript applications, Coverage calculates the degree to which the JavaScript code has been tested, and Analyzer performs a simple type-based analysis of JavaScript programs. Note that each component operates on a different intermediate representation. CloneDetector traverses AST nodes, Coverage works closely with Interpreter on IR, and Analyzer scans CFGs for various analyses.

3. Formal Specifications

The ECMAScript specification [9] describes the syntax and semantics of JavaScript in prose. The voluminous and informal specification makes it difficult to formally reason about JavaScript applications. While the 258-page specification describes JavaScript in very much detail, it is not rigorous enough: it does not specify every possible case exhaustively, it does not provide a high-level description of various ways to achieve the same behavior, and it includes a plenty of implementation-dependent features. For example, Figure 3 shows the description of the typeof operator in the ECMAScript specification, which does not specify the case when evaluating *UnaryExpression* results in an error. Also, JavaScript provides several ways to create function objects, but the specification does not describe them collectively in one place but men-

11.4.3 The typeof Operator

The production *UnaryExpression* : **typeof** *UnaryExpression* is evaluated as follows:

1. Let *val* be the result of evaluating *UnaryExpression*.
2. If *Type(val)* is Reference, then
 - a. If *IsUnresolvableReference(val)* is true, return "undefined".
 - b. Let *val* be *GetValue(val)*.
3. Return a String determined by *Type(val)* according to Table 20.

Table 20 — typeof Operator Results

Type of <i>val</i>	Result
Undefined	"undefined"
Null	"object"
Boolean	"boolean"
Number	"number"
String	"string"
Object (native and does not implement <i>[[Call]]</i>)	"object"
Object (native or host and does implement <i>[[Call]]</i>)	"function"
Object (host and does not implement <i>[[Call]]</i>)	Implementation-defined except may not be "undefined", "boolean", "number", or "string".

Figure 3. The typeof operator in the ECMAScript specification

tions them sporadically throughout the specification³. The under-specified, implementation-dependent, and implementation-defined features result in incompatible JavaScript engines producing different results for the same JavaScript program.

In this section, we present the formal specifications of the major components of SAFE.

3.1 Intermediate Representations

SAFE provides three levels of intermediate representations: AST, IR, and CFG. The highest level among them is AST, which is very close to the JavaScript concrete syntax; thus, it is the most applicable to source-level analyses such as clone detection. Lower than AST but still higher than machine-level code is IR, which is appropriate for evaluation by an interpreter. IR could be even more compiled down to a lower-level representation for better performance with aggressive code optimizations, and SAFE is open for such a future extension. CFG is the best representation for tracing control flows of a program; most program analyses perform on CFGs. SAFE provides formal specification and implementation of each intermediate representation⁴. Due to space limitations, we describe only IR in this paper and we refer interested readers to the formal specifications of AST and CFG in our open-source repository [30].

Figure 4 presents the syntax of IR. A program *p* in IR is a sequence of IR statements, which consists of function declarations, variable declarations, and IR statements. An IR statement *s* is a simplified version of a corresponding AST statement, and an IR expression *e* is an operator application, a property access, a literal, or an identifier, which does not have any side effects. An IR mem-

³ The specification describes five ways to create function objects: Section 13.2 describes creating function objects by function declarations and function expressions, Section 15.3.2.1 describes the cases by function constructors as functions and as part of new expressions, and Section 15.3.4.5 describes the case by the bind method of function objects.

⁴ Formal specifications are available at:
<http://plrg.kaist.ac.kr/redmine/projects/jsf/repository/revisions/master/entry/doc>
 and the implementations are available at:
<http://plrg.kaist.ac.kr/redmine/projects/jsf/repository/revisions/master/entry/astgen>
<http://plrg.kaist.ac.kr/redmine/projects/jsf/repository/revisions/master/entry/src/kr/ac/kaist/jsaf/analysis/cfg>

$p ::= s^*$
 $s ::= \underline{x} = \underline{e}$
 $\quad | \underline{x} = \text{delete } \underline{x}$
 $\quad | \underline{x} = \text{delete } \underline{x}[\underline{x}]$
 $\quad | \underline{x} = \{(\underline{m},)^*\}$
 $\quad | \underline{x} = [(\underline{e},)^*]$
 $\quad | \underline{x} = \underline{x}(\underline{x}, \underline{x})^?$
 $\quad | \underline{x} = \text{new } \underline{x}((\underline{x},)^*)$
 $\quad | \underline{x} = \text{function } \underline{f}(\underline{x}, \underline{x}) \{s^*\}$
 $\quad | \text{function } \underline{f}(\underline{x}, \underline{x}) \{s^*\}$
 $\quad | \underline{x} = \text{eval}(\underline{e})$
 $\quad | \underline{x}[\underline{x}] = \underline{e}$
 $\quad | \text{break } \underline{x}$
 $\quad | \text{return } \underline{e}^?$
 $\quad | \text{with } (\underline{x}) \underline{s}$
 $\quad | \underline{x} : \{ \underline{s} \}$
 $\quad | \text{var } \underline{x}$
 $\quad | \text{throw } \underline{e}$
 $\quad | s^*$
 $\quad | \text{if } (\underline{e}) \text{ then } \underline{s} \text{ (else } \underline{s})^?$
 $\quad | \text{while } (\underline{e}) \underline{s}$
 $\quad | \text{try } \{ \underline{s} \} (\text{catch } (\underline{x}) \{ \underline{s} \})^? (\text{finally } \{ \underline{s} \})^?$
 $\quad | \langle \underline{s}^* \rangle$
 $\underline{e} ::= \underline{e} \otimes \underline{e}$
 $\quad | \ominus \underline{e}$
 $\quad | \underline{x}[\underline{e}]$
 $\quad | \underline{x}$
 $\quad | \diamond \underline{x}$
 $\quad | \text{this}$
 $\quad | \underline{num}$
 $\quad | \underline{str}$
 $\quad | \text{true}$
 $\quad | \text{false}$
 $\quad | \text{undefined}$
 $\quad | \text{null}$
 $\underline{m} ::= \underline{x} : \underline{x}$
 $\quad | \text{get } \underline{f}(\underline{x}, \underline{x}) \{s^*\}$
 $\quad | \text{set } \underline{f}(\underline{x}, \underline{x}) \{s^*\}$
 $\otimes ::= | \mid \& \mid ^ \mid \ll \mid \gg \mid \ggg \mid + \mid - \mid * \mid / \mid \%$
 $\quad | == \mid != \mid === \mid !== \mid < \mid > \mid <= \mid >=$
 $\quad | \text{instanceof} \mid \text{in}$
 $\ominus ::= \sim \mid ! \mid + \mid - \mid \text{void} \mid \text{typeof}$

Figure 4. Syntax of the JavaScript IR

ber $\underline{m}$ is either a data property or an accessor property, which is introduced in the 5th edition of the ECMAScript specification. To capture the function call semantics correctly as described in the ECMAScript specification, every function takes exactly two parameters: the first parameter denotes *ThisBinding*, the value associated with the `this` keyword within the function body, and the second parameter denotes an array of the actual arguments.

$(H, A, tb) \in \text{Heap} \times \text{Env} \times \text{ThisBinding}$
 $H \in \text{Heap} = \text{Loc} \xrightarrow{\text{fin}} \text{Object}$
 $A \in \text{Env} ::= \# \text{Global}$
 $\quad | \text{er} :: A$
 $\text{er} \in \text{EnvRec} = \text{DeclEnvRec} \cup \text{ObjEnvRec}$
 $\sigma \in \text{DeclEnvRec} = \text{Var} \xrightarrow{\text{fin}} \text{StoreValue}$
 $l \in \text{ObjEnvRec} = \text{Loc}$
 $tb \in \text{ThisBinding} = \text{Loc}$

Figure 5. Execution contexts and other domains

$ct \in \text{Completion} ::= nc$
 $\quad | ac$
 $nc \in \text{NormalCompletion} ::= \text{Normal}(vt)$
 $ac \in \text{AbruptCompletion} ::= \text{Break}(vt, x)$
 $\quad | \text{Return}(v)$
 $\quad | \text{Throw}(ve)$
 $vt \in \text{Val} \cup \{\text{empty}\}$
 $ve \in \text{ValError} = \text{Val} \cup \text{Error}$

Figure 6. Completion specification type

Execution Context: Heap, Environment, and ThisBinding As the ECMAScript specification describes, when an interpreter evaluates an ECMAScript executable code, it evaluates the code in an *execution context*. We represent an execution context by a triple of a heap, an environment, and a ThisBinding: (H, A, tb) .

Figure 5 presents a partial set of domain definitions. A heap maps locations to their corresponding objects; an environment is a list of environment records ending with the global object environment record, `#Global`. An environment record is either a declarative environment record or an object environment record: a declarative environment record maps variables to their values, and an object environment record itself is an object. This environment structure is one of the major differences from the 3rd edition of the ECMAScript specification.

Completion Specification Type Under an execution context, evaluating a statement may change the given heap and environment, and it always produces a completion value. As Figure 6 describes, a completion specification type is either a normal completion or an abrupt completion; a normal completion denotes producing a JavaScript value v or nothing (`empty`), and an abrupt completion denotes either diverting the program control via the `break` statement with a value vt and a label x , returning from a function call with a value v , or throwing an exception ve . For example, the semantics of the `break` statement is specified as follows:

$$(H, A, tb), \text{break } \underline{x} \rightarrow_{\underline{s}} (H, A), \text{Break}(\text{empty}, \underline{x})$$

Under an executable context (H, A, tb) , evaluating the `break` statement with a label $\underline{x}$ does not change the heap nor the environment (H, A) , and it produces the `Break` completion specification type without any value (`empty`) but with the target label $\underline{x}$.

Recovering from an Abrupt Completion When evaluating a statement results in an abrupt completion, the abrupt completion propagates back to its enclosing statements until a statement recovers the abrupt completion. For example, a `Break` completion with a target label $\underline{x}$ becomes a normal completion when it reaches an enclosing statement labelled with $\underline{x}$:

$$\frac{(H, A, tb), \underline{s} \rightarrow_{\underline{s}} (H', A'), \text{Break}(v, \underline{x})}{(H, A, tb), \underline{x} : \{ \underline{s} \} \rightarrow_{\underline{s}} (H', A'), \text{Normal}(v)}$$

When evaluating a statement s labelled with a label x results in a **Break** completion with a value v and the same label x , the labelled statement recovers the abrupt completion and produces a normal completion with the value v . Similarly, a **Return** completion may become a normal completion by a function call, and a **Throw** completion may become a normal completion by a **try** statement.

The typeof Operator Now, let us present the operational semantics rules for the **typeof** operator in our IR semantics, which corresponds to the ECMAScript description in Figure 3. Using the following helper function, *TypeTag*, which corresponds to Table 20 in Figure 3:

$$\text{TypeTag}(H, v) = \begin{cases} \text{"undefined"} & \text{if } v = \text{undefined} \\ \text{"object"} & \text{if } v = \text{null} \\ \text{"boolean"} & \text{if } v \in \text{Bool} \\ \text{"number"} & \text{if } v \in \text{Num} \\ \text{"string"} & \text{if } v \in \text{Str} \\ \text{"object"} & \text{if } v \in \text{Loc} \wedge \neg \text{IsCallable}(H, v) \\ \text{"function"} & \text{if } v \in \text{Loc} \wedge \text{IsCallable}(H, v) \end{cases}$$

we formally specify the operational semantics of the **typeof** operator as follows:

$$\frac{(H, A, tb), e \rightarrow_e v}{(H, A, tb), \text{typeof } e \rightarrow_e \text{TypeTag}(H, v)}$$

$$\frac{(H, A, tb), e \rightarrow_e \text{err}}{(H, A, tb), \text{typeof } e \rightarrow_e \text{undefined}}$$

Unlike the informal description in the ECMAScript specification, our formal specification exhaustively covers all the cases for evaluating the **typeof** operator. The first rule describes that when evaluation of e produces a value v , evaluation of the **typeof** operator produces a value by using the *TypeTag* helper function. The second rule describes that when evaluation of e results in an error, evaluation of the **typeof** operator produces **undefined** as most browsers do.

3.2 Translations between Intermediate Representations

In addition to the formal specifications of intermediate representations, SAFE also provides formal specification and implementation of translations between them⁵. Due to space limitations, we describe only several cases of the translation from AST to IR in this paper and we refer interested readers to the formal specification of CFG construction from IR in our open-source repository [30].

Translation from AST to IR consists of translation functions as partially shown in Figure 7. The translation functions maintain an environment Σ to handle the names of temporary variables and labels created during translation. Translation of a single AST statement may produce a list of IR statements; we use angle brackets $\langle \rangle$ to denote a list, and semicolons to denote concatenation of IR statements as a single list. The translation functions use internal names prefixed by $\diamond$: a variable name prefixed by $\diamond$ such as $\diamond \text{obj}$ denotes a temporary variable created during translation, and a function name prefixed by $\diamond$ such as $\diamond \text{getBase}$ denotes an internal function defined by the IR semantics.

⁵ Formal specifications are available at: <http://plrg.kaist.ac.kr/redmine/projects/jsf/repository/revisions/master/entry/doc> and the implementations are available at: <http://plrg.kaist.ac.kr/redmine/projects/jsf/repository/revisions/master/entry/src/kr/ac/kaist/jsaf/compiler>

12.6 Iteration Statements

Syntax

```
IterationStatement :
do Statement while ( Expression ) ;
while ( Expression ) Statement
for ( ExpressionNoInOpt ; ExpressionOpt ; ExpressionOpt ) Statement
for ( var VariableDeclarationListNoIn ; ExpressionOpt ; ExpressionOpt ) Statement
for ( LeftHandSideExpression in Expression ) Statement
for ( var VariableDeclarationNoIn in Expression ) Statement
```

Figure 8. Syntax of iteration statements

The translation function $\text{ast2ir}_p[p]$ takes a program in AST and produces a program in IR by invoking translation functions on the components of the program: $\text{ast2ir}_{fd}[fd]$ for function declarations, $\text{ast2ir}_{vd}[vd]$ for variable declarations, and $\text{ast2ir}_s[s]$ for statements. As we describe in Section 4.2.1, **Hoister** already reorganized lists of source elements in a program and function bodies in AST so that function declarations and variable declarations appear before statements. Because **Hoister** separates variable declarations from variable initializations, the translation of a variable declaration in AST to IR, $\text{ast2ir}_{vd}[vd]$, is very simple. The translation of a function declaration is similar to that of a program. The function declaration in IR takes only two parameters; the first parameter denotes the **this** binding for a function call and the second parameter denotes an array of the arguments given at a function call. Accordingly, a function call is translated to take two arguments: the base value of the function reference to denote its **this** binding and an array of the arguments given at the call.

While IR provides a single iteration statement, **while**, JavaScript supports six statements for iteration as shown in Figure 8: **DoWhile**, **While**, **For**, **ForVar**, **ForIn**, and **ForVarIn**. Among them, **ForVar** and **ForVarIn** are already desugared away by **Hoister** and the others are translated using the IR **while** statement by **Translator**. The translation of **While** is conventional but that of **ForIn** deserves more attention. Because **ForIn** enumerates the properties of an object and iterates its body until no property remains unvisited, we introduce three internal helper functions: $\diamond \text{iteratorInit}$ creates an iterator object for a given object, $\diamond \text{iteratorHasNext}$ checks whether any property remains unvisited, and $\diamond \text{iteratorNext}$ returns a property name to be visited next. Finally, the translation of a **switch** statement consists of several subsequent translation functions to handle a default clause, if any, and fall through cases.

3.3 Example Translation

To illustrate what we have described so far, consider the following JavaScript code:

```
var sum = 0;
for(var i = 1; i <= 10; i++)
  sum += i;
_<>_print(sum);
```

To show how each part is translated to intermediate representations, we color the corresponding parts in the JavaScript source code and the translated intermediate representations in the same color. Because AST is very similar to the source code, we show only the translated IR and CFG. The code first initializes the variable **sum** to 0 (in orange), and iteratively adds **i** to **sum** (in purple) where **i** is incremented by 1 (in brown) from 1 (in blue) to 10 (in green). To provide a debugging facility for our development, we add a special debugging function $\diamond \text{print}$. The code ends by printing **sum** (in red).

The following code is a simplified version of the translated IR from the above JavaScript code:

$ast2ir_p \llbracket fd^* \text{ vd}^* s^* \rrbracket$	$= \langle (ast2ir_{fd} \llbracket fd \rrbracket (\langle \rangle))^* (ast2ir_{vd} \llbracket vd \rrbracket (\langle \rangle))^* (ast2ir_s \llbracket s \rrbracket (\langle \rangle))^* \rangle$
$ast2ir_{fd} \llbracket \text{function } f((x,)^*) \{fd^* \text{ vd}^* s^*\} \rrbracket (\Sigma)$	$= \text{function } f(\diamond \text{this}, \diamond \text{arguments}) \{$ $(ast2ir_{fd} \llbracket fd \rrbracket (\Sigma))^*$ $(\text{var } x_i)^*$ $(ast2ir_{vd} \llbracket vd \rrbracket \Sigma)^*$ $(x_i = \diamond \text{arguments}["i"])^*$ $(ast2ir_s \llbracket s \rrbracket (\Sigma; \diamond \text{this}; \diamond \text{arguments}))^* \}$
$ast2ir_{vd} \llbracket \text{var } x \rrbracket (\Sigma)$	$= \text{var } x$
$ast2ir_{lhs} \llbracket f((e,)^*) \rrbracket (\Sigma)(\underline{x})$	$= \text{LET } ((\underline{s}^*, \underline{e}) = ast2ir_e \llbracket e \rrbracket (\Sigma)(\diamond y))^*$ $\text{IN } (\diamond \text{obj} = \diamond \text{toObject}(f); (\underline{s}^*; \diamond y = \underline{e}))^*;$ $\diamond \text{arguments} = [(\diamond y_i,)^*];$ $\diamond \text{fun} = \diamond \text{getBase}(f);$ $\underline{x} = \diamond \text{obj}(\diamond \text{fun}, \diamond \text{arguments}), \underline{x})$
$ast2ir_s \llbracket \text{while } (e) s \rrbracket (\Sigma)$	$= \text{LET } (\underline{s}^*, \underline{e}) = ast2ir_e \llbracket e \rrbracket (\Sigma)(\diamond \text{new}_1)$ $\text{IN } \langle \diamond \text{break} : \{$ $\underline{s}^*;$ $\text{while } (\underline{e}) \{$ $\diamond \text{continue} : \{ast2ir_s \llbracket s \rrbracket (\Sigma; \diamond \text{break}; \diamond \text{continue})\};$ $\underline{s}^*;$ $\}$ $\}\rangle$
$ast2ir_s \llbracket \text{for } (lhs \text{ in } e) s \rrbracket (\Sigma)$	$= \text{LET } (\underline{s}^*, \underline{e}) = ast2ir_e \llbracket e \rrbracket (\Sigma)(\diamond \text{new}_1)$ $\text{IN } \langle \diamond \text{break} : \{$ $\underline{s}^*;$ $\diamond \text{obj} = \diamond \text{toObject}(\underline{e});$ $\diamond \text{iterator} = \diamond \text{iteratorInit}(\diamond \text{obj});$ $\diamond \text{cond}_1 = \diamond \text{iteratorHasNext}(\diamond \text{obj}, \diamond \text{iterator});$ $\text{while } (\diamond \text{cond}_1) \{$ $\diamond \text{key} = \diamond \text{iteratorNext}(\diamond \text{obj}, \diamond \text{iterator});$ $ast2ir_{lval} \llbracket lhs \rrbracket (\Sigma)(\diamond \text{key})(\text{false}).1;$ $\diamond \text{continue} : \{ast2ir_s \llbracket s \rrbracket (\Sigma; \diamond \text{break}; \diamond \text{continue})\};$ $\diamond \text{cond}_1 = \diamond \text{iteratorHasNext}(\diamond \text{obj}, \diamond \text{iterator});$ $\}$ $\}\rangle$
$ast2ir_s \llbracket \text{switch } (e) \{cc_1^* (\text{default}: s^*)^? cc_2^*\} \rrbracket (\Sigma)$	$= \text{LET } (\underline{s}^*, \underline{e}) = ast2ir_e \llbracket e \rrbracket (\Sigma)(\diamond \text{val})$ $\text{IN } \langle \diamond \text{break} : \{$ $\underline{s}^*; \boxed{\diamond \text{val} = \underline{e};}$ $ast2ir_{case} \llbracket (\text{rev } cc_2^*)(s^*)^? (\text{rev } cc_1^*) \rrbracket (\Sigma; \diamond \text{break}; \diamond \text{val}) \}\rangle$
$ast2ir_{case} \llbracket (\text{case } e : s_1^* :: cc_2^* (s_2^*)^? cc_1^*) \rrbracket (\Sigma)(c^*)$	$= \langle \diamond \text{label} : \{ast2ir_{case} \llbracket cc_2^* (s_2^*)^? cc_1^* \rrbracket (\Sigma)((e, \diamond \text{label}) :: c^*)\};$ $(ast2ir_s \llbracket s_1 \rrbracket (\Sigma))^* \}$
$ast2ir_{case} \llbracket () (s^*)^? cc_1^* \rrbracket (\Sigma)(c^*)$	$= \langle \diamond \text{label} : \{ast2ir_{case} \llbracket () () cc_1^* \rrbracket (\Sigma)(c^* @ [(), \diamond \text{label}])\};$ $((ast2ir_s \llbracket s \rrbracket (\Sigma))^*)^? \}$
$ast2ir_{case} \llbracket () () (\text{case } e : s^* :: cc_1^*) \rrbracket (\Sigma)(c^*)$	$= \langle \diamond \text{label} : \{ast2ir_{case} \llbracket () () cc_1^* \rrbracket (\Sigma)((e, \diamond \text{label}) :: c^*)\};$ $(ast2ir_s \llbracket s \rrbracket (\Sigma))^* \}$
$ast2ir_{case} \llbracket () () \rrbracket (\Sigma)((e, l)^*)$	$= \langle ast2ir_{scond} \llbracket (e, l)^* \rrbracket (\Sigma);$ $\text{break } \Sigma(\diamond \text{break}) \rangle$
$ast2ir_{scond} \llbracket (e, l) :: (c^*) \rrbracket (\Sigma)$	$= \text{LET } (\underline{s}^*, \underline{e}) = ast2ir_e \llbracket e \rrbracket (\Sigma)(\diamond \text{cond})$ $\text{IN } \langle \underline{s}^*;$ $\text{if } (\Sigma(\diamond \text{val}) === \underline{e}) \text{ then break } l \text{ else } ast2ir_{scond} \llbracket c^* \rrbracket (\Sigma) \rangle$
$ast2ir_{scond} \llbracket [(), l] \rrbracket (\Sigma)$	$= \langle \text{break } l \rangle$
$ast2ir_{scond} \llbracket () \rrbracket (\Sigma)$	$= \langle \rangle$
Where c is either (e, l) or $((), l)$.	

Figure 7. An excerpt of the translation rules from AST to IR

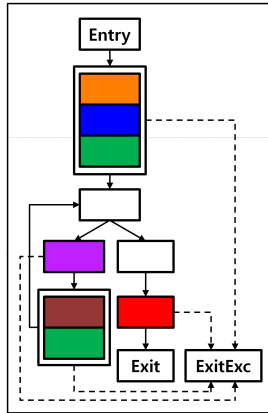
```

var sum
var i
<>_one_1 = 1
{
  <>_old_2 = sum
  <>_temp_3 = 0
  sum = <>_temp_3
  <>_ignore_4 = sum
}
{
  {
    <>_old_5 = i
    <>_temp_6 = 1
    i = <>_temp_6
    <>_ignore_4 = i
  }
  <>_break_7 : {
    {
      <>_temp_8 = i
      <>_temp_9 = 10
      <>_temp_10 = <>_temp_8 <= <>_temp_9
    }
    <>_temp_11 = <>toBoolean(<>_temp_10)
    while(<>_temp_11)
    {
      <>_temp_12 : {
        <>_old_13 = sum
        <>_temp_14 = i
        <>_new_15 = <>_old_13 + <>_temp_14
        sum = <>_new_15
      }
      <>_temp_20 = sum
      <>_ignore_4 = <>print(<>_temp_20)
    }
  }
}
<>_old_16 = i
<>_new_17 = <>toNumber(<>_old_16)
<>_new_18 = <>_new_17 + <>_one_1
i = <>_new_18
<>_temp_19 = i
<>_temp_19 = <>_new_17
}
<>_temp_8 = i
<>_temp_9 = 10
<>_temp_10 = <>_temp_8 <= <>_temp_9
<>_temp_11 = <>toBoolean(<>_temp_10)

```

Because the translated IR includes verbose information such as source location, we cleaned up such information to clearly show the correspondence between the original JavaScript source code and the generated IR code. Note that the conditional expression to check whether *i* is less than equal to 10 (in green) shows up twice, before the while statement and inside the while statement.

Now, the following graph presents the generated CFG from the above IR code:



Each colored box denotes a sequence of instructions corresponding to the IR code segment with the same color; several colored boxes constitute a basic block. The **Entry** node denotes the beginning of the program, three consecutive nodes next to **Entry** denote the initialization part of the loop, the left branch with three nodes (of the colors purple, brown, and green) denotes the loop body, and the right branch ending with the **Exit** node denotes the control flow after the loop. Note that the dashed lines to the **ExitExc** node from various basic blocks denote possible exception flows.

4. Implementation

In this section, we describe how we realized the formal specifications of SAFE described in the previous section.

4.1 Why Yet Another Parser for JavaScript?

As we briefly mentioned in Section 2.1, our previous research on a JavaScript module system required modifications of a JavaScript parser and its AST nodes to extend the syntax to support modules.

```

/**
 * SourceElement ::= Stmt
 */
abstract Stmt();
/**
 * Stmt ::= do Stmt while ( Expr );
 */
DoWhile(Stmt body, Expr cond);
/**
 * Stmt ::= while ( Expr ) Stmt
 */
While(Expr cond, Stmt body);
/**
 * Stmt ::= for ( Expr? ; Expr? ; Expr? ) Stmt
 */
For(Option<Expr> init, Option<Expr> cond,
    Option<Expr> action, Stmt body);
/**
 * Stmt ::= for ( lhs in Expr ) Stmt
 */
ForIn(LHS lhs, Expr expr, Stmt body);
/**
 * Stmt ::= for ( var VarDecl(, VarDecl)* ;
 *              Expr? ; Expr? ) Stmt
 */
ForVar(List<VarDecl> vars, Option<Expr> cond,
    Option<Expr> action, Stmt body);
/**
 * Stmt ::= for ( var VarDecl in Expr ) Stmt
 */
ForVarIn(VarDecl var, Expr expr, Stmt body);

```

Figure 9. An excerpt of the high-level AST specification

We considered various parsers and AST structures from academia and industry including ANTLR⁶, Scala’s parser combinators⁷, Rhino, SpiderMonkey, Closure Tools⁸, JSTest⁹, and JSure¹⁰, but they were not satisfactory to us. Most of them do not cover the entire JavaScript language and their AST structures do not reflect the JavaScript syntax well. Even though the Rhino parser written in Java is very powerful, because it is a ported version of the hand-written SpiderMonkey parser in C++, we excluded it from consideration for productivity reason.

We actively use open-source tools to automatically generate parsers and intermediate representations. We provide a high-level description of the AST node hierarchy as partially shown in Figure 9 where the indentation denotes a subclassing relationship. Then, ASTGen [32] reads the description and generates Java classes for the AST nodes with some utility methods such as getters, setters, equals, and hashCode. Similarly, we provide a BNF-style grammar and the corresponding action code for the grammar productions, then Rats! [15] generates a JavaScript parser in Java.

The implementation languages of our framework are Scala and Java, where most of the hand-written code are written in Scala. We use both languages to take advantage of the abundant libraries and tools in Java such as ASTGen and Rats!, and to get benefits from pattern matching and higher-order functions in Scala. Also, because both languages are compiled into Java bytecode, we enjoy the seamless interoperability between them.

⁶<http://www.antlr.org>

⁷<http://www.scala-lang.org/api/current/scala/util/parsing/combinator/Parsers.html>

⁸<https://developers.google.com/closure>

⁹<https://github.com/heidegger/JSTest>

¹⁰<https://github.com/berke/jsure>

4.2 AST Transformations due to the Quirky Semantics

Because of the quirky semantics of JavaScript, SAFE transforms an AST to a simplified version to make it easier to analyze and evaluate in later phases via the following phases:

- **Hoister** lifts the declarations of functions and variables inside programs and functions up to the beginning of them;
- **Disambiguator** checks some static restrictions and renames identifiers to unique names; and
- **WithRewriter** rewrites the `with` statements that do not include any dynamic code generation such as `eval` into other statements without using the `with` statement but with the same semantics.

Because all the intermediate representations preserve backward-mapping information such as source locations, any errors detected on intermediate representations are reported back to the users in terms of JavaScript source locations.

4.2.1 Hoister

When JavaScript code is evaluated, functions and variables declared in the code are first bound in the environment of the running execution context before evaluating the code. In effect, the functions and variables can be used before their textual declarations. For example, the following code:

```
1  x;
2  var x = f();
3  function f() { return 42; }
```

shows that `x` on line 1 is used before its declaration on line 2. Because every compilation phase or an analysis of JavaScript programs should take this feature into account, SAFE *hoists* the declarations of functions and variables to the beginning of the enclosing programs and functions, which preserves the original semantics. By taking care of this feature once at an early phase of compilation, the tasks of the later phases become less burdensome.

After hoisting, the above code is transformed as follows:

```
1  function f() { return 42; }
2  var x;
3  x;
4  x = f();
```

The function declaration is hoisted as it is, but the variable declaration of `x` is split into a declaration without initialization on line 2 and an assignment on line 4.

4.2.2 Disambiguator

Like compilers for conventional programming languages, SAFE renames identifiers in JavaScript code to unique names via the Disambiguator phase. By assigning unique names, identifiers with the same name but in different scopes become explicitly distinct, which makes the tasks of the later phases easier. For example, the following code:

```
1  function f() {
2    var x;
3    function() { var x; };
4  }
5  function h() { var x; }
6  var x;
```

is transformed as follows:

```
1  function f() {
2    var <>_x_1;
3    function() { var <>_x_2; };
```

```
abstract class Completion()
case class Normal(v:Option[Val]) extends Completion
abstract class Abrupt() extends Completion
case class Break(v:Option[Val],
               l:IRId) extends Abrupt
case class Return(v: Val) extends Abrupt
case class Throw(e: ValError) extends Abrupt
```

Figure 10. Completion specification type in the interpreter

```
4  }
5  function h() { var <>_x_3; }
6  var x;
```

Also, Disambiguator checks some static restrictions such as the following:

- A `return` statement should be within a function body.
- A `continue` statement should occur inside an iterator statement.
- A labelled statement should not be enclosed by another labelled statement with the same label.

4.2.3 with Rewriter

The `with` statement in JavaScript makes static analysis of JavaScript applications difficult by introducing a new scope at run time and thus invalidating lexical scoping. Thus, many static approaches to JavaScript program analysis simply disallow the `with` statement. Instead of simply avoiding the uses of the `with` statement, our previous research [29] investigates the usage patterns of the `with` statement in real-world JavaScript applications currently used in the 944 most popular web sites, and it shows that we can rewrite all the static occurrences of the `with` statement that does not have any dynamic code generating functions.

Using the previous research, SAFE provides the `WithRewriter` phase to statically eliminate the `with` statement in AST as long as it is possible. Because evaluating the `with` statement creates an object environment record, the properties of the given `with` object are added to the current lexical environment for the body. Therefore, if the `with` object has a property with the same name as an identifier in the body, the identifier becomes referring to the property; otherwise, it refers to a name bound in its enclosing environment record. The strategy of `with` rewriting is to replace each identifier occurrence in the `with` body with a ternary expression to access the correct entity depending on the existence of a property with the same name in the `with` object. For example, the following `with` statement:

```
with (expr) { a + b; }
```

is rewritten as follows:

```
var $f = toObject(expr);
("a" in $f ? $f.a : a) + ("b" in $f ? $f.b : b);
```

More detailed description of the `with` rewriting is available in our previous work [29].

4.3 AST Translations and IR Interpretation

As we discussed in Section 3.2, `Translator` translates an AST into an IR, `CFGBuilder` constructs a CFG from the IR, and `Interpreter` evaluates the IR. Our implementation follows the corresponding formal specifications as closely as possible so that they can be mutually beneficial. For example, Figure 10 presents the corresponding implementation of the completion specification type in Figure 6, which is very close to each other.

12.6.2 The while Statement

The production *IterationStatement* : **while** (*Expression*) *Statement* is evaluated as follows:

1. Let $V = \text{empty}$.
2. Repeat
 - a. Let *exprRef* be the result of evaluating *Expression*.
 - b. If *ToBoolean*(*GetValue*(*exprRef*)) is **false**, return (normal, V , empty).
 - c. Let *stmt* be the result of evaluating *Statement*.
 - d. If *stmt.value* is not empty, let $V = \text{stmt.value}$.
 - e. If *stmt.type* is not **continue** || *stmt.target* is not in the current label set, then
 - i. If *stmt.type* is **break** and *stmt.target* is in the current label set, then
 1. Return (normal, V , empty).
 - ii. If *stmt* is an abrupt completion, return *stmt*.

Figure 11. The while statement in the specification

12.6.2 The while Statement

$$\begin{array}{c}
 \frac{(H, A, tb), \underline{e} \rightarrow \underline{e} \text{ err}}{(H, A, tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H, A), \text{Throw}(\text{err})} \\
 \\
 \frac{(H, A, tb), \underline{e} \rightarrow \underline{e} v \quad \text{ToBoolean}(v) = \text{false}}{(H, A, tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H, A), \text{Normal}(\text{empty})} \\
 \\
 \frac{(H, A, tb), \underline{e} \rightarrow \underline{e} v \quad \text{ToBoolean}(v) = \text{true} \quad (H, A, tb), \underline{s} \rightarrow \underline{s} (H', A'), ac}{(H, A, tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H', A'), ac} \\
 \\
 \frac{(H, A, tb), \underline{e} \rightarrow \underline{e} v \quad \text{ToBoolean}(v) = \text{true} \quad (H, A, tb), \underline{s} \rightarrow \underline{s} (H', A'), nc \quad (H', A', tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H'', A''), ac}{(H, A, tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H'', A''), ac} \\
 \\
 \frac{(H, A, tb), \underline{e} \rightarrow \underline{e} v \quad \text{ToBoolean}(v) = \text{true} \quad (H, A, tb), \underline{s} \rightarrow \underline{s} (H', A'), nc \quad (H', A', tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H'', A''), \text{Normal}(\text{empty})}{(H, A, tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H'', A''), nc} \\
 \\
 \frac{(H, A, tb), \underline{e} \rightarrow \underline{e} v \quad \text{ToBoolean}(v) = \text{true} \quad (H, A, tb), \underline{s} \rightarrow \underline{s} (H', A'), nc \quad (H', A', tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H'', A''), \text{Normal}(v)}{(H, A, tb), \text{while } (\underline{e}) \underline{s} \rightarrow \underline{s} (H'', A''), \text{Normal}(v)}
 \end{array}$$

Figure 12. The while statement in the IR semantics

By developing both the formal specification and the implementation in parallel, we often found that one form reveals missing cases in the other form, and vice versa. For example, consider the description of the **while** statement in the ECMAScript specification presented in Figure 11. For each construct in the ECMAScript specification, we proceed to describe its operational semantics formally as in Figure 12, and we simultaneously implement its semantics as in Figure 13. The implementation in Scala often matches the specification closely, or the initial implementation looks like a direct translation of the formal specification and we optimize them for performance improvement. In such cases, we keep the original implementation in comment for documentation purpose. While it is often unclear whether the informal prose in the ECMAScript specification covers every possible case, the operational semantics rules are well equipped to cover the cases exhaustively. At the same time, while the operational semantics rules have many duplicated antecedents which make the rules bulky, the concise implementation of the interpreter complements the rules and the pattern match-

```

/*
 * 12.6.2 The while Statement
 */
case SIRWhile(info, cond, body) =>
  var V: Option[Val] = None
  var AC: Option[Abrupt] = None
  var cont: Boolean = true
  while (cont) {
    walkExpr(cond) match {
      case v: Val => cont = IH.toBoolean(v)
      case err: JSError =>
        AC = Some(throwErr(err, info))
        cont = false
    }
    if (cont) {
      walkExpr(body) match {
        case Normal(v) => if (v.isDefined) V = v
        case ac: Abrupt =>
          AC = Some(ac)
          cont = false
      }
    }
  }
  AC match {
    case Some(ac) => ac
    case None => Normal(V)
  }

```

Figure 13. The while statement in the IR interpreter

ing mechanism of Scala checks the exhaustiveness and redundancy of the cases.

The implementation of the interpreter is still in progress. Currently, it does not support the **eval** code, and it partially supports the **with** statement, the strict mode, and the standard built-in objects. We are actively developing the missing pieces, and we are validating the interpreter implementation using regression testing.

4.4 Experiences with the Quirky Semantics

Because the ECMAScript specification does not provide a high-level overview of the quirky semantics of JavaScript in one place, we have encountered various inconsistencies and under-specified behaviors sporadically. We share some of such experiences here.

Unlike conventional programming languages where the value of a non-empty sequence of statements is the value of the last statement of the sequence regardless of the value, in JavaScript, it is the value of the last statement that produces a non-empty value, if any; otherwise, the value of the sequence is **empty**. For example, in the following code, an excerpt from the ECMAScript specification [9]:

```

eval("1; ; ; ;")
eval("1; {}")
eval("1; var a;")

```

the calls to the **eval** function all return the value 1. If we do not consider this semantics and translate an AST to an IR as we do for the conventional languages, the translation will not preserve the semantics. At the same time, if we apply such a semantics to get the value of a sequence of IR statements, it will not preserve the semantics either because a single AST statement often gets translated into a sequence of IR statements; a single AST statement of the value **empty** can be translated into a sequence of IR statements whose values are not all **empty**. For example, while the following JavaScript code:

```
if (false) { 42; }
```

produces the value `empty`, the translated IR from the code:

```
<>_temp_1 = false;
if (<>_temp_1) { <>_temp_2 = 42; };
```

consists of IR statements whose values are `false` and `empty`, in order. Thus, we should treat a sequence of IR statements translated from a single AST statement differently from a sequence of IR statements translated from a sequence of AST statements. For this reason, we introduce the `IRStmtUnit` node in addition to the `IRSeq` node: we wrap a sequence of IR statements translated from a single AST statement with `IRStmtUnit`, and we use `IRSeq` to wrap a sequence of IR statements translated from a sequence of AST statements. Thus, the value of a sequence of IR statements wrapped by `IRStmtUnit` is the value of the last statement in the sequence, while the value of a sequence of IR statements wrapped by `IRSeq` is the last non-`empty` value of the sequence.

The flexibility of JavaScript allows the number of arguments to a function call to be different from the number of the function's parameters. When the number of the arguments is smaller than the number of the parameters, the parameters whose corresponding arguments do not exist are mapped to `undefined`. More specifically, evaluation of the arguments at a function call creates an `Arguments` object, which contains properties of the names from "0" to the maximum number of the number of the arguments, say n_a , and the number of the parameters, say n_p . Each of the properties maps to the corresponding values of the arguments, except that when n_a is smaller than n_p the properties of the names from n_a to $n_p - 1$ map to `undefined`.

In JavaScript, many standard built-in objects are functions in the sense that they can be invoked with arguments, and at the same time, some of them are constructors in the sense that they can be used with the `new` operator. The issue here is that the semantics of some built-in objects are very different depending on whether they are used as functions or as constructors. For example, when a `Date` object is used as a function [9, Section 15.9.2], it ignores its arguments and behaves as it is called as `(new Date()).toString()` returning a string value, while it creates a `Date` object when it is used as a constructor [9, Section 15.9.3]. To handle this semantics correctly, our AST-to-IR translation creates separate IR nodes for function calls and object constructions even though their semantics are very similar.

Because JavaScript provides prototype-based inheritance, every object has an internal property called `[[Prototype]]`. Also, an object may have a prototype of name "prototype"; in particular, every function automatically has a `prototype` property so that it can be used as a constructor [9, Section 8.6.2]. A somewhat confusing part is that while `[[Prototype]]` and `prototype` are quite different, they are used in an intermingled way. Because `[[Prototype]]` is an internal property, users can not access it explicitly and the value of the property does not change once it is set when the object is constructed. However, users can explicitly access and change the value of `prototype` of an object freely. In spite of such differences, they are used in an intermixed way which may lead to some confusion. For instance, a `instanceof` `b` denotes whether `b` has `a` as its instance, and the ECMAScript specification defines its semantics as the result of calling the `[[HasInstance]]` internal method of `b` with the argument of `a`, which checks whether the value of the "prototype" property of `b` exists in the `[[Prototype]]` chain from the `[[Prototype]]` internal property of `a`. Because users can change the value of the "prototype" property of `b` any time, the result of the `instanceof` operation can be surprising as we have shown in Figure 1 in Section 1.

Another issue is *implementation-dependent* and *implementation-defined* features in the ECMAScript specification. For example, the description of `Array.prototype.sort` [9, Section 15.4.4.11] includes several implementation-dependent semantics. When an `Array` object includes some missing elements, it is said to be *sparse*, and the semantics of `Array.prototype.sort` is full of implementation-dependent behavior for sparse arrays. While the evaluation of the `Array.prototype.sort` method involves evaluating the `[[Get]]`, `[[Put]]`, and `[[Delete]]` internal methods, it is often impossible to evaluate them for sparse arrays. If a sparse array has the `[[Extensible]]` internal property of `false`, it is impossible to put a non-existing array index property, and if a sparse array has an index property with the `[[Configurable]]` internal property of `false`, it is impossible to delete the property. Such implementation-dependent and implementation-defined semantics make it difficult to analyze JavaScript programs.

5. Related Work

The first thorough approach to formally specify a significant (if not all) portion of the JavaScript programming language is an operational semantics [23] that directly translates the ECMAScript specification. It intends to faithfully capture the various aspects of JavaScript and translate the informal ECMAScript specification into a formal semantics. Therefore, it is well suited for analyzing existing JavaScript programs and applying the analysis results back to the JavaScript programs, though reasoning about the programs and proving their properties using the semantics are not conventional and complicated [11].

Another approach for a formal specification of JavaScript is to devise a core calculus, λ_{JS} [17], which has a very different semantics from the original JavaScript semantics but more conventional. By providing a set of desugaring (or rather, compilation) rules from JavaScript to λ_{JS} , it desugars away the quirky features of JavaScript and translates them to a standard core calculus with a conventional semantics. Thus, it suits well for extending and reasoning about the JavaScript language thanks to its conventional calculus, though extra work is necessary to scale the analysis results back to the full JavaScript language.

Jensen *et al.* [20] present a static analysis framework for JavaScript using type analysis for the full JavaScript language. Their framework is originally based on the T. J. Watson Libraries for Analysis (WALA) [34], which is a collection of Java libraries to support static analyses of Java bytecode and JavaScript. Because WALA originally supported only Java programs, its support for JavaScript is yet premature. It parses JavaScript programs using the Rhino parser, and it translates the parsed JavaScript AST in Rhino into the WALA common AST. Thus, it has the same problem as Rhino regarding supporting any research that modifies the JavaScript syntax. Instead, Jensen *et al.* developed their own framework to support the peculiar features of JavaScript more directly.

Recent research on JavaScript covers various aspects of the JavaScript language such as its formal specifications [11, 17, 23], type systems [3, 18, 33], static analyses [16, 20], and combinations of static and dynamic analyses [5, 24] for JavaScript. Implementation of such research results often requires extensive modifications of an existing framework. Implementing a proposed JavaScript module system [21] requires modifications in the parser and the AST structure, for example. Based on our experience, the hand-written Rhino parser is too complicated to be applicable, and the big semantic gap between JavaScript and λ_{JS} and poor performance of the λ_{JS} interpreter are obstacles to be usable. We believe that our framework would be useful for easily implementing future research on JavaScript.

6. Conclusion

Motivated by our own struggles with the existing approaches, we present SAFE, a scalable analysis framework for ECMAScript, the very first attempt to support both formal specification and implementation of JavaScript, developed open for the research community. To make the framework more usable to other researchers, SAFE provides not only the formal specification of its various components but also the implementation of them. Designing and developing the formal specification and implementation of the JavaScript language simultaneously has been a worthwhile approach; the operational semantics rules in our formal specification and the pattern matching mechanism in our implementation checked by the Scala compiler have been mutually beneficial to catch missing cases in each other. Also, thorough investigation of the ECMAScript specification revealed several issues and error-prone semantics of JavaScript, which opens up possibilities for future research in detecting erroneous programs. In order for the framework to be as flexible, scalable, and pluggable as possible, SAFE supports three levels of intermediate representations: AST, the closest to the JavaScript source, IR for evaluation, and CFG for flow-based analyses. Last but not least, we actively use third-party open-source tools to automatically generate parsers and intermediate representations, which enhances the productivity of aggressive research and development on JavaScript.

Currently, we have been finalizing the implementation of the interpreter with extensive testing, and we have been collaborating with researchers in academia and industry. As we have shown in Figure 2 in Section 2.2, such collaborative research includes CloneDetector, Coverage, and Analyzer. Each research topic performs on different intermediate representations: CloneDetector detects possible code clones among multiple JavaScript programs via AST comparisons, Coverage calculates code coverage of JavaScript programs via IR evaluation, and Analyzer estimates type information via CFG traversal. We plan to improve our current analyzer and add more analyses to the framework, and we more than welcome other researchers using our framework for their own research. As a theoretical foundation of our JavaScript semantics specification, we're working on mechanizing the formal semantics in Coq to guarantee some properties. While we're supporting most of the built-in objects both in the interpreter implementation and an accompanying analysis implementation, we do not intend to formally describe built-in objects and functions exhaustively.

Acknowledgments

The authors sincerely thank the S-Core team for the fruitful discussions and their tremendous contributions to the framework. This work is supported in part by the Mid-career Researcher Program of Korea Ministry of Education, Science and Technology(MEST) / National Research Foundation of Korea(NRF) (Grant 2012-0005256), the Engineering Research Center of Excellence Program of MEST / NRF (Grant 2012-0000469), Microsoft Research Asia, Samsung Electronics, and S-Core.

References

- [1] HTML5: A vocabulary and associated APIs for HTML and XHTML. <http://www.w3.org/TR/html5/>.
- [2] Alexa the Web Information Company. <http://www.alexa.com/>, 2011.
- [3] Christopher Anderson, Paola Giannini, and Sophia Drossopoulou. Towards type inference for JavaScript. In *Proceedings of the 19th European Conference on Object-Oriented Programming*, 2005.
- [4] Gilad Bracha, Guy Steele, Bill Joy, and James Gosling. *Java™ Language Specification, The 3rd Edition (Java Series)*. Addison-Wesley Professional, 2005.
- [5] Ravi Chugh, Jeffrey A. Meister, Ranjit Jhala, and Sorin Lerner. Staged information flow for JavaScript. In *Proceedings of the 2009 ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2009.
- [6] Douglas Crockford. ADsafe. <http://www.adsafe.org/>.
- [7] Brendan Eich. Static analysis FTW. <http://brendaneich.com/2010/08/static-analysis-ftw/>, 2010.
- [8] European Association for Standardizing Information and Communication Systems (ECMA). ECMA-262: ECMAScript Language Specification. Third edition., 1999.
- [9] European Association for Standardizing Information and Communication Systems (ECMA). ECMA-262: ECMAScript Language Specification. Fifth edition., 2009.
- [10] Facebook. FBJS. <http://developers.facebook.com/docs/fbjs/>.
- [11] Philippa Anne Gardner, Sergio Maffeis, and Gareth David Smith. Towards a program logic for javascript. In *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2012.
- [12] Danny Goodman. *Dynamic HTML: The Definitive Reference*. O'Reilly Media, 1998.
- [13] Google. Caja. <http://code.google.com/p/google-caja/>.
- [14] Google. V8 javascript engine. <http://code.google.com/p/v8/>.
- [15] Robert Grimm. Rats! – An Easily Extensible Parser Generator. <http://cs.nyu.edu/~rgrimm/xtc/rats.html>.
- [16] Arjun Guha, Shriram Krishnamurthi, and Trevor Jim. Using static analysis for Ajax intrusion detection. In *Proceedings of the 18th International Conference on World Wide Web*, 2009.
- [17] Arjun Guha, Claudiu Saftoiu, and Shriram Krishnamurthi. The essence of JavaScript. In *Proceedings of the 24th European Conference on Object-Oriented Programming*, 2010.
- [18] Phillip Heidegger and Peter Thiemann. Recency types for analyzing scripting languages. In *Proceedings of the 24th European Conference on Object-Oriented Programming*, 2010.
- [19] Dongseok Jang, Ranjit Jhala, Sorin Lerner, and Hovav Shacham. An empirical study of privacy-violating information flows in JavaScript web applications. In *Proceedings of the 17th ACM Conference on Computer and Communications Security*, 2010.
- [20] Simon Holm Jensen, Anders Møller, and Peter Thiemann. Type analysis for JavaScript. In *International Static Analysis Symposium*, 2009.
- [21] Seonghoon Kang and Sukyoung Ryu. Formal specification of a JavaScript module system. In *Proceedings of the 2012 ACM International Conference on Object Oriented Programming, Systems, Languages, and Applications (OOPSLA '12)*, 2012.
- [22] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988.
- [23] Sergio Maffeis, John C. Mitchell, and Ankur Taly. An operational semantics for JavaScript. In *Proceedings of the 6th Asian Symposium on Programming Languages and Systems*, 2008.
- [24] Sergio Maffeis, John C. Mitchell, and Ankur Taly. Isolating JavaScript with filters, rewriting, and wrappers. In *14th European Symposium on Research in Computer Security*, 2009.
- [25] Sergio Maffeis, John C. Mitchell, and Ankur Taly. Object capabilities and isolation of untrusted web applications. In *IEEE Symposium on Security and Privacy*, 2010.
- [26] Mozilla. Rhino: Javascript for java. <https://developer.mozilla.org/en-US/docs/Rhino>.
- [27] Mozilla. Spidermonkey. <https://developer.mozilla.org/en-US/docs/SpiderMonkey>.
- [28] Tim O'Reilly. What Is Web 2.0. <http://oreilly.com/web2/archive/what-is-web-20.html>.

- [29] Changhee Park, Hongki Lee, and Sukyoung Ryu. An empirical study on the rewritability of the with statement in JavaScript. In *2011 International Workshop on Foundations of Object-Oriented Languages (FOOL'11)*, 2011.
- [30] KAIST PLRG. JSAF: JavaScript Analysis Framework. <http://plrg.kaist.ac.kr/research/safe>.
- [31] Joe Gibbs Politz, Spiridon Aristides Eliopoulos, Arjun Guha, and Shriram Krishnamurthi. Type-based verification of JavaScript sandboxing. In *Proceedings of the 20th Conference on USENIX Security Symposium*, 2011.
- [32] Brian R. Stoler, Eric Allen, and Dan Smith. ASTGen. <http://sourceforge.net/projects/astgen>.
- [33] Peter Thiemann. Towards a type system for analyzing JavaScript programs. In *Proceedings of the 14th European Symposium on Programming*, 2005.
- [34] T.J. Watson Libraries for Analysis (WALA). <http://wala.sf.net>.